

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
«_____» _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА СИСТЕМЫ ПОДДЕРЖКИ АНАЛИЗА КОМПЬЮТЕРНЫХ
ИГР ДЛЯ ИХ РАНЖИРОВАНИЯ И СОСТАВЛЕНИЯ РЕКОМЕНДАЦИЙ
ПО УЛУЧШЕНИЮ

по направлению подготовки 09.03.03 Прикладная информатика

Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникационные технологии

Выполнил
студент гр. 5130903/10302

А.А. Лихачева

Руководитель
ст. преподаватель ВШ ПИ

В.А. Пархоменко

Консультант
по нормоконтролю

Е.Е. Андрианова

Санкт-Петербург
2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Лихачевой Анастасии Алексеевне, гр. 5130903/10302

1. Тема работы: «Разработка системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению».
2. Срок сдачи студентом законченной работы: 17.05.2025.
3. Исходные данные по работе: обзорная статья [3.2], база данных пользовательских обзоров компьютерных игр с платформы Steam [3.1], а также база данных о продажах компьютерных игр на данной платформе [3.3].
 - 3.1. Список пользовательских обзоров. — URL: <https://partner.steamgames.com/doc/store/getreviews> (дата обращения: 08.01.2025).
 - 3.2. *Marchand A., Hennig-Thurau T.* Value Creation in the Video Game Industry: Industry Economics, Consumer Benefits, and Research Opportunities // Journal of Interactive Marketing. — 2013. — Т. 27. — С. 141—157. — URL: <https://doi.org/10.1016/j.intmar.2013.05.001> (дата обращения: 19.11.2024).
 - 3.3. SteamSpy - All the data and stats about Steam games. — URL: <https://steamspy.com/> (visited on 16.02.2025).
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1. Исследование методов и средств оценки и ранжирования компьютерных игр.
 - 4.1.1. Обзор существующих наборов данных о компьютерных играх.
 - 4.1.2. Обзор средств анализа текстовой информации.
 - 4.1.3. Обзор средств ранжирования компьютерных игр.

- 4.2. Проектирование системы поддержки анализа компьютерных игр.
- 4.3. Реализация системы поддержки анализа компьютерных игр.
- 4.4. Тестирование и апробация системы поддержки анализа компьютерных игр.
- 5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Диаграмма случаев использования веб-ориентированной системы оценки конкурентоспособности компьютерных игр.
 - 5.2. Диаграмма алгоритма оценки и рейтингования компьютерных игр.
- 6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):
 - 6.1. Система управления базами данных MongoDB.
 - 6.2. Библиотека React.
 - 6.3. Библиотека Paspailleur.
- 7. Консультанты по работе:
 - 7.1. Ст. преподаватель ВШ ПИ, Е.Е. Андрианова (нормоконтроль).
- 8. Дата выдачи задания: 22.12.2024.

Руководитель ВКР _____ В.А. Пархоменко

Задание принял к исполнению 22.12.2024

Студент _____ А.А. Лихачева

РЕФЕРАТ

На 148 с., 18 рис., 21 табл., 49 прил.

Ключевые слова: ранжирование, компьютерные игры, Steam, React

Тема выпускной квалификационной работы: «Разработка системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению».

Данная работа посвящена разработке подхода к анализу и ранжированию компьютерных игр на основе их конкурентных характеристик. Целью исследования является разработка инструмента, поддерживающего принятие решений в индустрии компьютерных игр – со стороны разработчиков, издателей и инвесторов. Объектом исследования являются методы и средства многокритериального ранжирования. В качестве методологической основы использован гибридный подход, включающий методы многокритериальной оптимизации, а также качественную и количественную оценку значимости признаков. Для анализа сходства между игровыми проектами применены методы кластерного анализа и сопоставления по ключевым параметрам: жанрам, мета-данным и показателям популярности. Система была разработана с использованием MongoDB в качестве системы управления базами данных и библиотеки React. В результате реализован прототип веб-приложения, позволяющий ранжировать игры, находить успешные аналоги и формировать рекомендации по улучшению проектов. Разработанная система может найти применение в игровой индустрии и быть использована при планировании новых игр и оценке их доли продаж в сегменте похожих игр. Сделаны выводы о высокой эффективности предложенного решения при сравнительном анализе с существующими системами (Steam, STEAMer, DeepFM).

ABSTRACT

On 148 pages, 18 figures, 21 tables, 49 appendices

Keywords: ranking, computer games, Steam, React

The subject of the graduate qualification work is «Development of a computer game rating system to recommend product improvements».

The present paper is dedicated to the development of an approach to analyzing and ranking computer games based on their competitive characteristics. The objective

of the present study is to devise a mechanism that will facilitate decision-making processes within the domain of the video game industry, particularly among developers, publishers, and investors. The object of the study is methods and tools of multi-criteria ranking. A methodological framework is employed that incorporates a hybrid approach, which integrates multi-criteria optimization methods and qualitative and quantitative assessments of feature significance. In order to facilitate an analysis of the similarities between game projects, a set of methodologies was employed, including cluster analysis and a comparison by key parameters. The parameters in question included genres, metadata, and popularity indicators. The system was developed using MongoDB as the database management system and the React library. Consequently, a prototype web application was developed that facilitates the evaluation of games based on the identification of successful analogies, and the formulation of recommendations for enhancing project quality. The developed system can find application in the game industry and be used in planning new games and assessing their market potential. The following conclusions are drawn regarding the comparative analysis of the efficiency of the proposed solution with existing systems (Steam, STEAMer, DeepFM).

СОДЕРЖАНИЕ

Введение	11
1 Исследование методов и средств ранжирования компьютерных игр.....	16
1.1 Обзор существующих наборов данных о компьютерных играх	16
1.1.1 Обзор платформ цифровой дистрибуции	16
1.1.2 Особенности выбранной платформы.....	18
1.1.3 Выбор исходных данных	20
1.2 Обзор средств анализа текстовой информации.....	21
1.2.1 Особенности анализа игровых отзывов.....	21
1.2.2 Методы векторного представления текстов.....	22
1.2.3 Перспективы расширения анализа	25
1.3 Обзор методов ранжирования компьютерных игр	27
1.3.1 Общее понятие и цели ранжирования игр.....	27
1.3.2 Принципы и методы построения рейтингов компьютерных игр.....	28
1.3.3 Методы свёртки критериев.....	30
1.4 Обзор существующих средств ранжирования	33
1.4.1 Система Steam.....	33
1.4.2 Система STEAMer	35
1.4.3 Система DRGame	36
1.4.4 Система DeepFM.....	37
1.4.5 Система D-VSMR.....	38
1.4.6 Сравнительный анализ моделей ранжирования	39
1.5 Выводы	41
2 Проектирование системы поддержки анализа компьютерных игр.....	42
2.1 Функциональная модель системы	42
2.1.1 Use-case диаграмма и основные сценарии.....	42
2.1.2 Входные и выходные данные системы.....	45
2.1.3 Этапы работы системы анализа и рекомендаций	47
2.2 Архитектурное проектирование системы	48
2.2.1 Архитектурные паттерны и их сравнение	48
2.2.2 Выбранная архитектура и её компоненты	50
2.3 Технологический стек	52
2.3.1 Критерии выбора технологий	52
2.3.2 Выбор языка программирования	52
2.3.3 Выбор фреймворка для веб-разработки.....	54

2.3.4 Выбор серверной платформы	55
2.3.5 Выбор системы управления базой данных	57
2.4 Проектирование диаграммы сущность-связь	57
2.5 Математические модели и алгоритмы	58
2.5.1 Предобработка данных и формирование признаков	58
2.5.2 Методы анализа сходства игр	63
2.5.3 Методы ранжирования компьютерных игр	66
2.5.4 Метод последовательного уточнения предпочтений	73
2.6 Формирование рекомендательных решений	74
2.6.1 Выявление уникальных характеристик	74
2.6.2 Механизм генерации рекомендаций	75
2.6.3 Математическая модель анализа	76
2.7 Пример работы системы	77
2.8 Выводы	79
3 Реализация системы поддержки анализа компьютерных игр	80
3.1 Реализация основных модулей системы	81
3.1.1 Диаграмма классов	81
3.1.2 Класс DataRepository	81
3.1.3 Класс FeatureExtractor	83
3.1.4 Класс BazeAnalyzer	84
3.1.5 Класс GameClusterer	84
3.1.6 Класс GameCompetitivenessAnalyzer	85
3.1.7 Класс GameSimilarityAnalyzer	87
3.1.8 Класс GameUniqueFeaturesAnalyzer	87
3.1.9 Класс GameRecommendationAnalyzer	89
3.1.10 Класс GameAnalysisPipeline	91
3.1.11 Класс ReviewAnalyzer	93
3.1.12 Класс CompetitivenessRanker	94
3.1.13 Реализация API	95
3.2 Реализация пользовательского интерфейса	98
3.2.1 Структура веб-интерфейса	98
3.2.2 Компонент отображения результатов	99
3.2.3 Компонент отображения рейтинга игр	100
3.2.4 Компонент настройки важности критериев	102
3.2.5 Техническая реализация	103
3.3 Выводы	106

4 Тестирование и апробация системы поддержки анализа компьютерных игр	107
4.1 План тестирования.....	107
4.2 Тестирование компонентов системы	108
4.2.1 Модульное тестирование backend.....	108
4.2.2 Интеграционное тестирование	113
4.2.3 Анализ покрытия кода	119
4.2.4 Профилирование времени выполнения и использования памяти.....	120
4.3 Тестирование пользовательского интерфейса.....	121
4.3.1 Функциональное тестирование интерфейса.....	121
4.3.2 Проверка отзывчивости интерфейса	126
4.4 Экспериментальное сравнение с существующими решениями.....	128
4.4.1 Система рекомендаций Steam.....	128
4.4.2 Сравнительный анализ рейтинговой системы	129
4.4.3 Анализ полученных рейтингов	130
4.4.4 Анализ полученных схожих игр и формирование рекомендаций.....	133
4.4.5 Выводы на основе экспериментального сравнения.....	135
4.5 Апробация на конференции	136
4.6 Выводы	138
Заключение	140
Список сокращений и условных обозначений	142
Список использованных источников.....	143
Приложение 1 Диаграмма компонентов системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению	149
Приложение 2 Диаграмма случаев использования системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению	150
Приложение 3 Архитектура системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению.....	151
Приложение 4 Логическая модель документоориентированной БД GamesAnalyzer с цветовой маркировкой источников данных	152
Приложение 5 Диаграмма классов системы GamesAnalyze	153
Приложение 6 Основные параметры игр из Steam и SteamSpy	154
Приложение 7 Основные параметры отзывов пользователей.....	155
Приложение 8 Сравнение количества игр в Steam и на других игровых платформах для ПК.....	156
Приложение 9 Сравнительный анализ методов векторизации текстовых данных.....	157
Приложение 10 Определение оптимального числа кластеров методом локтя	158

Приложение 11 Объектно-признаковая таблица моделей ранжирования....	159
Приложение 12 Сравнительный анализ подходов к ранжированию игр	160
Приложение 13 Сравнение инструментальных средств для разработки веб–приложения	161
Приложение 14 Производительность ключевых методов системы	162
Приложение 15 Определение оптимального числа кластеров методом локтя	163
Приложение 16 Определение оптимального числа кластеров методом локтя	164
Приложение 17 t-SNE визуализация кластеров компьютерных игр.....	165
Приложение 18 Блок-схема выполнения одного из возможных сценариев работы системы.....	166
Приложение 19 Пример POST-запроса к API анализа игры через Postman .	167
Приложение 20 Форма загрузки данных пользовательской игры при светлой теме	168
Приложение 21 Программный код модуля BaseAnalyzer	169
Приложение 22 Программный код модуля CompetitivenessRanker	181
Приложение 23 Программный код модуля GameClusterer	189
Приложение 24 Программный код модуля FeatureExtractor	200
Приложение 25 Программный код модуля GameAnalysisPipeline.....	208
Приложение 26 Программный код модуля GameAnalyzer.....	220
Приложение 27 Программный код модуля GameCompetitivenessAnalyzer ..	223
Приложение 28 Программный код модуля GameRecommendationAnalyzer.	236
Приложение 29 Программный код модуля GameRecommendationAnalyzer.	245
Приложение 30 Программный код модуля GameSimilarityAnalyzer	252
Приложение 31 Программный код модуля GameUniqueFeaturesAnalyzer....	259
Приложение 32 Программный код компонента GameAnalysisForm	266
Приложение 33 Программный код компонента GameIdForm.....	271
Приложение 34 Программный код компонента GameRatingDisplay	273
Приложение 35 Программный код компонента ResultsDisplay	280
Приложение 36 Программный код компонента SettingsDialog	286
Приложение 37 Программный код тестов модуля FeatureExtractor.....	293
Приложение 38 Программный код тестов модуля FeatureExtractor для HTML	298
Приложение 39 Программный код тестов модуля GameClusterer	300
Приложение 40 Программный код тестов стабильности GameClusterer	302
Приложение 41 Программный код тестов модуля GameCompetitivenessAnalyzer	305
Приложение 42 Программный код тестов обработки пропущенных данных в GameCompetitivenessAnalyzer	310
Приложение 43 Программный код тестов модуля GameUniqueFeaturesAnalyzer	315

Приложение 44 Программный код тестов конвейера анализа игр	323
Приложение 45 Программный код тестов модуля формирования рекоменда- ций игр	326
Приложение 46 Программный код тестов компонента gameIdForm	332
Приложение 47 Программный код тестов компонента GameAnalysisForm .	335
Приложение 48 Программный код тестов компонента ResultsDisplay	339
Приложение 49 Программный код тестов компонента HomePage	343

ВВЕДЕНИЕ

Индустрия компьютерных игр, объем мирового рынка которой по данным исследования Entertainment Software Association [52] на 2023 год превысили 220 миллиардов долларов, представляет собой один из самых динамичных и быстро развивающихся секторов развлечений. Темпы создания продуктов высоки – каждый день в Steam выходит более 30 новых игр, что подчеркивает большую конкуренцию и постоянный приток пользователей. Несмотря на такой рост, оценка доли продаж игры в сегменте похожих игр остается сложной задачей из-за разнообразия показателей успеха [18; 43]. Финансовые показатели, количество удержанных пользователей и отзывы критиков дают частичное представление об игре, но ни один стандартный метод не объединяет эти разнообразные параметры в единую аналитическую систему.

Традиционные системы рейтингов, такие как Steam Top Sellers [75] или Metacritic [76], часто опираются на отдельные агрегированные показатели, которые не отражают нюансы различий между жанрами или стилями игры, являясь коллекцией отзывов и балльных оценок пользователей. Такие платформы, как правило, не учитывают взаимосвязь между финансовыми результатами и вовлеченностью пользователей, что ограничивает их применимость для разработчиков и издателей, стремящихся к более глубокому стратегическому анализу. Более того, отсутствие инструментов, способных оценивать проекты в сравнительном аспекте – с учетом позиционирования на рынке, предпочтений аудитории и особенностей геймплея, – создает значительный пробел для лиц, принимающих решения как в контексте разработки, так и в контексте инвестиций.

Актуальность исследования обусловлена решением проблемы отсутствия специализированных инструментов для многокритериального ранжирования игр [61], поскольку предлагает аналитическую структуру, способную выявлять конкурентные преимущества и проводить параллели с эталонными играми. Ранжирование в данном контексте служит не только средством организации больших объемов данных, связанных с играми, но и основой для принятия обоснованных решений в области разработки, маркетинга и финансирования игр. Систематизируя разнообразные параметры в понятные иерархии, такие модели позволяют проводить более объективную оценку продукта на ранней стадии разработки.

Особенно актуально это для независимых студий, которые часто не имеют доступа к передовым инструментам оценки, способных помочь им более эффективно

позиционировать свои продукты на мировом рынке. В отличие от систем ранжирования, ориентированных на потребителя, предлагаемый подход предназначен для поддержки принятия стратегических решений разработчиками, издателями и инвесторами.

Объектом данного исследования являются методы и средства многокритериального ранжирования.

Предметом исследования является автоматизация ранжирования компьютерных игр, а также составление рекомендаций по их улучшению.

Цель исследования — разработать систему поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению.

В соответствии с целью исследования, логически определяются следующие **задачи работы**):

1. Провести обзор существующих наборов данных о компьютерных играх и их характеристиках.
2. Исследовать методы обработки текстовой информации (включая методы векторизации).
3. Провести сравнительный анализ существующих методов ранжирования в игровой индустрии.
4. Спроектировать систему поддержки анализа и ранжирования компьютерных игр.
5. Реализовать программные модули оценки доли продаж игр в своем сегменте.
6. Провести тестирование и апробацию разработанной системы поддержки анализа и ранжирования компьютерных игр.

Теоретическую основу исследования составили работы отечественных и зарубежных авторов, изучающих методы обработки данных, систем анализа игровых потребительских оценок и поведения. Среди них ключевые публикации:

1. Эмпирические исследования о влиянии отзывов на восприятие видеоигр [13];
2. Идеи и методы теории важности критериев в многокритериальных задачах принятия решений [7].

Исследование основано на **методологической базе**, которая сочетает в себе элементы науки о данных, теории принятия решений и вычислительных экспериментов. Центральным элементом этого подхода является применение алгоритмов кластеризации, таких как k-means, для определения групп похожих

компьютерных игр на основе многомерных векторов характеристик. Эти кластеры служат основой для более глубокого сравнительного анализа, подкрепленного теоретико-множественными операциями (объединение, пересечение и различие), которые выявляют отличительные признаки отдельных названий по сравнению с их аналогами. Для оценки и ранжирования игр в исследовании используются методы многокритериального анализа решений, дополненные регулируемые весовыми коэффициентами, что позволяет гибко настраивать их в соответствии с конкретными условиями оценки.

Новизна работы заключается в разработке комплексного подхода к оценке и рейтингу игр, который преследует следующие цели:

- определение оценки доли продаж в своем сегменте игр с помощью числовых показателей (доля игроков на рынке, выручка и т.д.);
- разработку гибридного алгоритма ранжирования, сочетающего Парето-оптимизацию для первичного отсева доминируемых вариантов, адаптивную аддитивную свёртку с динамической коррекцией весов и кластерный анализ для нишевого позиционирования игр;
- разработку интерактивного интерфейса, который позволяет регулировать важность критериев ранжирования в реальном времени и генерировать аналитические отчеты в PDF-формате;
- реализацию механизма анализа для прогнозирования оценки доли продаж в своем сегменте новых игр и выявления их уникальных характеристик через операции над множествами.

Как в отечественных, так и в международных исследованиях академическое изучение систем ранжирования игр проходило в три этапа. В 2000–2010 годах внимание исследователей было сосредоточено преимущественно на однокритериальных оценках — таких, как объем продаж или пользовательские рейтинги. В течение следующего десятилетия начали применяться более сложные многопараметрические подходы, примером которых стали алгоритмы рекомендаций платформы Steam. С 2020 года наблюдается рост интереса к гибридным методам, сочетающим кластеризацию и адаптивное взвешивание параметров, что подтверждают работы [19; 45]. Однако, несмотря на достижения, остаются нерешёнными две задачи: слабая интеграция методов в оценку доли продаж игр и ограниченные возможности настройки критериев ранжирования в существующих системах. Настоящая работа предлагает решения этих проблем — через применение теоре-

тико-множественных операций к кластеризованным данным с учётом специфики игровой индустрии.

Система, разработанная в рамках этого исследования, представляет ощутимую **практическую значимость** на различных этапах разработки игры. На стадии проектирования она позволяет заранее оценить рыночные перспективы будущей игры, сопоставляя её предполагаемые характеристики с существующими рыночными кластерами. В ходе стратегического планирования издатели могут моделировать различные рыночные условия, настраивая приоритетность факторов – от жанровой принадлежности до параметров монетизации. Возможность экспорта результатов в формате PDF делает систему удобным инструментом для работы с инвесторами и проведения анализа игры.

Основные положения исследования были представлены на Международной научной конференции «Современные технологии в теории и практике программирования» (2025 г.), где обсуждались ключевые аспекты применения методов многокритериального анализа в игровой индустрии.

Теоретическая значимость работы заключается в расширении существующей базы знаний в двух направлениях:

- предложена новая модель сравнения игр, опирающаяся на кластерный анализ и сопоставление как количественных, так и качественных признаков с использованием операций над множествами;
- систематизированы методы многокритериальной оптимизации применительно к задачам ранжирования игровых продуктов.

Эти результаты создают основу для будущих исследований в области адаптивных систем рекомендаций и прогнозного анализа рынка.

Полная реализация, включая механизм ранжирования, модуль кластеризации и систему рекомендаций, представлена в виде программного обеспечения с открытым исходным кодом.

Результаты будут полезны в первую очередь *разработчикам и издателям игр*, заинтересованным в улучшении понимания своей аудитории и совершенствовании рекламных стратегий. *Исследователи и аналитики* смогут подробнее изучить взаимосвязь между качественными характеристиками игр и их рыночным успехом. *Инвесторы и маркетологи* получают возможность принимать более взвешенные решения относительно инвестиций в проекты начинающих компаний.

Основной гипотезой является то, что с помощью использования математически корректных методов ранжирования можно получить ранжировки компьютер-

ных игр, сопоставимые (близкие по метрикам сравнения рейтингов) с рейтингами выбранных БД (Steam Top, Metacritic) с играми.

1 ИССЛЕДОВАНИЕ МЕТОДОВ И СРЕДСТВ РАНЖИРОВАНИЯ КОМПЬЮТЕРНЫХ ИГР

Настоящая глава преследует две основные задачи: познакомить с предметом исследования и представить обзор существующих исследований в выбранной области.

Параграф 1.1 содержит общий анализ платформ для цифровой дистрибуции и предоставляет сведения о платформе Steam, которая выступает в качестве основного источника данных. В параграфе 1.2 проводится анализ используемых методов векторизации текстовых характеристик компьютерных игр и в 1.3 дается более подробное представление о построении систем ранжирования, что позволяет выбрать наиболее подходящую модель для системы, разрабатываемой в дальнейшем.

1.1 Обзор существующих наборов данных о компьютерных играх

1.1.1 Обзор платформ цифровой дистрибуции

Компьютерные игры – быстро развивающаяся категория приложений, мировой рынок которых демонстрирует устойчивый рост, который связан с расширением аудитории (особенно в сегменте мобильных устройств и ПК) [52], инновациями в технологиях и увеличением разнообразия жанров. Игры сегодня распространяются преимущественно через *платформы цифровой дистрибуции*, служащие посредниками между разработчиками и конечными пользователями, упрощая покупку, установку и обновление продуктов.

Таблица 1.1

Сравнение количества игр в Steam и на других игровых платформах для ПК (по состоянию на 11 января 2025 года)

Название платформы	Количество компьютерных игр
Game Jolt [57]	1,644,492
Steam	~3,000
Good Old Games (GOG) [59]	9,685
GamersGate [58]	9,090
VK Play [77]	7,072
Epic Games Store [53]	406
Origin [63]	144

На рынке представлено множество платформ, специализирующихся на продаже игр. Каждая из них предлагает свои уникальные особенности, ориентированные на потребности разработчиков и игроков (см. табл. 1.1):

1. Game Jolt [57] – платформа, сосредоточенная на поддержке инди-игр и позволяющая разработчикам бесплатно размещать свои проекты. Благодаря этому база доступных игр насчитывает более 1,6 миллиона продуктов – это больше, чем у других платформ. Однако работа с ней затрудняется отсутствием официального интерфейса прикладного программирования (Application Programming Interface, далее — API) и системы отзывов, а большинство игр представляют собой слишком разные по содержанию некоммерческие проекты, порой без описательных характеристик.
2. Good Old Games (GOG) [59] – преимущественно ориентирована на распространение ретро-игр. Несмотря на наличие системы оценивания и API, доступные метрики ограничены и не позволяют проводить глубокий анализ взаимодействия пользователей с играми (например, временные ряды активности или отзывы недоступны для получения).
3. GamersGate [58] – это бесплатный сервис, который не требует от пользователей входа в систему для того, чтобы играть в купленные игры. Как и в случае с GOG, выбор игр на платформе крайне мал (всего около 9 тысяч единиц). Большинство игр относятся или к инди-категориям, или к относительно устаревшим проектам. Кроме того, у платформы отсутствуют официальные методы сбора аналитики.
4. VK Play [77] – отечественная площадка дистрибуции видеоигр, которая выделяется на фоне других платформ за счет выпуска эксклюзивных продуктов. На площадке можно оставлять отзывы к купленным продуктам, ставить им оценку (от 0 до 10), отслеживать время, проведенное в играх, однако данный сервис не предоставляет официального API, что осложняет процесс получения данных для изучения.
5. Epic Games Store [53] – активно использует эксклюзивные контракты с крупными игровыми студиями и предоставляет бесплатные игры для привлечения пользователей. По состоянию на январь 2025 года в каталоге EGS имеется всего 406 игр, что значительно уступает более крупным цифровым платформам, а пользователи не имеют возможности оценивать купленные игры и оставлять к ним отзывы.

6. Steam [68] – платформа имеет метрики об активности пользователей (данные о сыгранных часах, количество онлайн-игроков и т.д.), развитую систему классификации видеоигр и инструменты рецензирования, которые, однако, позволяют оценить игру только как «рекомендуемую», либо «нерекомендуемую». Но несмотря на этот недостаток, платформа также имеет API, предоставляющий доступ к текстовому содержанию отзывов, что является большим преимуществом Steam.

Представим определение *оценки доли продаж игры* как возможность компьютерной игры занимать значительную долю рынка в своём сегменте, а также привлекать целевую аудиторию, сохраняя высокий уровень её вовлечённости. Этот показатель может оцениваться через численные показатели, такие как процентная доля пользователей в соответствующем жанре, и финансовые результаты (количество покупок или доход от внутриигровых транзакций).

Для анализа оценки игр важно выбрать платформу, которая предоставляет доступ к наиболее *подробным данным*, имеет *встроенные механизмы взаимодействия* пользователей с играми (например, отзывы или теги) и облегчает сбор *статистики* по широкому спектру продуктов. Среди всех цифровых платформ *Steam* выделяется уникальными характеристиками, такими как большое количество тегов и доступных метаданных, высокая активность пользователей и неограниченный доступ к нужным данным через API.

1.1.2 Особенности выбранной платформы

На протяжении многих десятилетий игры распространялись на физических носителях, для чего покупателям приходилось посещать магазины, но в 2003 году компания Valve Corporation выпустила *Steam* [68], которая в 2023 году являлась крупнейшей платформой цифровой дистрибуции игр [34]. Она и сейчас остается одной из крупнейших (см.табл.1.1), не считая платформы, на которых размещаются некоммерческие продукты: по состоянию на январь 2025 года в ее каталоге можно найти более 73 тысяч видеоигр [70] для покупки. Одна из ключевых особенностей *Steam* заключается в их развитой системе взаимодействия пользователей с игрой, в рамках которой игроки могут:

- поставить игре оценку «Рекомендуется» (т.е. положительный отзыв), либо «Не рекомендуется» (т.е. отрицательный отзыв);

- добавлять текстовые комментарии об опыте прохождения игры, делясь впечатлениями с другими пользователями;
- создавать пользовательские метки и альтернативные системы классификации игр (теги).

Рецензии в Steam может прочитать любой желающий, но Valve Corporation также создала API, который позволяет собирать их [10]. Отзывы можно проанализировать с помощью нескольких методов, чтобы понять опыт игроков [27] или просто оценить их полезность [21; 35; 46; 47].

Можно заметить, что преобладающим методом распределения компьютерных игр является *классификация по жанрам* [29], которой придерживаются разработчики на платформе Steam. Пользовательские же теги предоставляют альтернативное понимание жанров.

Кроме того, согласно документации Steamworks [4], в дополнение к жанровой классификации игры могут быть разделены на *супержанры* и *поджанры*. В вышеупомянутой документации говорится о существовании 11 основных жанров (TLG или top-level genres), такие как симуляторы, RPG, стратегии, приключения и т.д., 68 жанров (например, платформер) и 61 поджанра (например, платформер на точность) – для каждой игры они описываются в разделе «Categories». Примечательно, что на платформе Steam каждой игре присваивается как минимум один супержанр, жанр и поджанр. В данном исследовании мы сосредоточимся на *жанрах, категориях и тегах*, поскольку они способны вызвать выраженные различия между играми наравне с их сходствами.

В метках разработчик игры может указать также *визуальные характеристики* (измерения, расположение камеры, визуальный стиль), определить *тему* (например, космос, зомби и т.д.) и *атмосферу* игры (расслабляющая, смешная, мрачная и т.д.), а также особенности игры: её *игровую механику* (решения с последствиями, обмен и т.д.), *элементы дизайна* (физика, процедурная генерация и т.д.) и *игровые возможности* (мореплавание, шахты, хакерство и т.д.). С недавних пор стало обязательным добавление 5 меток к каждому продукту, который будет выпущен в Steam, но разработчики могут добавлять и больше. Данные метки также могут быть учтены в процессе анализа оценки игры, однако перечисленные ранее характеристики (жанры, теги и категории) уже дают достаточно полное представление о продукте.

Сочетая данные о *взаимодействии игроков с играми* – (отзывы и пользовательские теги,) – со *статистическими показателями*, такими как стоимость игры,

количество активных пользователей, жанровая принадлежность и объем продаж, можно создать *прогностическую модель*, способную учитывать наиболее значимые факторы, влияющие на оценку игры и служить инструментом для анализа её доли продаж на рынке. Определение и обоснование подобных факторов подробно рассматривается в следующем параграфе.

1.1.3 Выбор исходных данных

Игровая индустрия представлена как *крупными проектами* (с огромными бюджетами на разработку и маркетинг), так и *небольшими инди-студиями*, ориентированными на инновации и оригинальность, а компьютерные игры, в свою очередь, сегодня сопровождаются огромным объемом как структурированных, так и неструктурированных данных: от текстовых отзывов на платформах вроде Steam до количественных метрик, таких как число проданных копий, время игры или соотношение положительных и отрицательных комментариев. Анализ этих данных позволяет выявить наиболее похожие продукты, определить целевую аудиторию, а также спланировать эффективные стратегии продвижения.

Стоит учесть, что в данном исследовании мы постараемся сосредоточиться на изучении способов получения наибольшей прибыли от игр с фиксированной ценой, независимо от наличия в продукте внутриигровой рекламы, обеспечивающей дополнительный доход, поскольку большинство игр Steam придерживаются именно такой политики [32]. То есть в оценке игры не будут учитываться внутренние покупки игроков.

Исследование [13] показало, что основной фактор, влияющий на рост продаж и популярности игры, – это *пользовательские отзывы*. В своем исследовании о 6224 играх, 98 823 обновлениях новостей, 37 613 заметках о релизах авторы считают отзывы не просто демонстрацией удовлетворенности или неудовольствия игрой, но и важным источником данных для анализа ключевых характеристик: качества геймплея, стабильности программного обеспечения и интереса к игровым механикам. Это также даёт нам понять, что отзывы можно использовать для извлечения полезной информации об игре, если это требуется.

Основные выводы, которые можно сделать из исследований [13; 23; 43]:

1. *Отзывы пользователей* являются важным источником данных для анализа предпочтений игроков и выявления сильных и слабых сторон игр.

Они дают представление о качестве игрового процесса, технических характеристиках, уровне вовлеченности и удовлетворенности игроков.

2. Комбинация анализа текстовых отзывов (например, через Sentiment Analysis или тематическое моделирование) и агрегированных оценок пользователей может позволить *более точно классифицировать игры* по их популярности и доли продаж в сегменте.
3. В Steam иногда случаются ошибки в классификации. Это может быть потенциальной причиной того, что фактор жанра не оказывает влияния [23]. *Цена игры* является наиболее важным фактором. Год публикации и жанры менее важны.
4. *Негативные отзывы* часто подсвечивают технические проблемы (например, проблемы с оптимизацией) даже у популярных игр, что дает объективную информацию о реальных потребностях аудитории. В 29% положительных отзывов обсуждаются недостатки игр, в 7% — ошибки.
5. На платформе Steam 96% игр собирают в среднем менее 10 отзывов в день, что свидетельствует о важности *долгосрочного маркетингового анализа*. С другой стороны, количество отзывов напрямую коррелирует с охватом игры и ее вовлечением (чем больше игроков, тем больше отзывов).

Для разработчиков, особенно малых студий, умение анализировать характеристики других игр становится ключевым конкурентным преимуществом. Комбинация анализа текстов отзывов, агрегированных оценок и маркетинговых метрик позволяет не только быстрее выявить сильные и слабые стороны своей игры, но и спроецировать ее перспективы на фоне аналогов.

1.2 Обзор средств анализа текстовой информации

1.2.1 Особенности анализа игровых отзывов

В исследовании [24] с помощью методов обработки естественного языка (Natural Language Processing — далее NLP) был проведен анализ 35,983,481 обзора Steam с целью изучения опыта и эмоциональных реакций игроков на основные жанры TLG. Были отобраны 11 различных жанров для более детального изучения. В результате был проведен сравнительный анализ отзывов, оставленных этими игроками, и получены следующие выводы:

1. Анализ текстовых отзывов с использованием методов *NLP* помогает выявить *эмоции и поведение игроков*. Понимание того, какие эмоции вызывают игры определённых жанров, может быть полезным для построения рейтинговых систем, учитывающих эмоциональный отклик.
2. Несмотря на преобладание негативных настроений в отзывах, первоначальные эмоциональные реакции, как правило, положительные. По мере того как люди формулируют и отправляют отзывы, они часто испытывают негативные эмоции, что приводит к снижению эмоциональной валентности по ходу повествования.
3. Анализ эмоций в тексте показывает, что гоночные и спортивные игры наиболее схожи в плане отрицательных рецензий. Это говорит о *различиях в эмоциональных паттернах и характеристиках отзывов для разных жанров игр*. Эти данные могут быть полезны, если модель оценки будет учитывать жанровую специфику игр.
4. По содержанию рецензий можно понять *уровень управления и взаимодействия с игрой* [30], а также то, *какие факторы делают игру популярной* [11]. Этот факт очень важен, так как текстовые отзывы обычно содержат настроения и имеют большое значение, и могут выступать в качестве *обзора* продукта [14; 39].

Понимание эмоциональных откликов игроков в зависимости от класса (жанра) помогает разработчикам создавать более привлекательные и успешные игры, а значит интеграция анализа эмоциональной насыщенности текстовых отзывов игроков действительно может позволить *более точно прогнозировать успех* компьютерных игр на рынке.

1.2.2 Методы векторного представления текстов

В данной работе анализ текстовой информации осуществляется в контексте контентной векторизации описаний компьютерных игр, с целью их последующего сравнения и группировки по семантической близости.

1.2.2.1 Алгоритм TF-IDF

Алгоритм TF-IDF используется для определения веса ключевого слова в любом документе (например, в описании игры) и присваивания ему значение в зависимости от частоты его появления в документе; более высокий показатель

TF-IDF (вес) указывает на то, что фраза встречается реже и является *более значимой*, и наоборот.

Частота термина (Term Frequency, далее – TF) определяется как общее количество раз, когда слово встречается в текущем документе. Чем больше вес слово, тем более высокая частота. Чтобы обеспечить достоверность результатов, необходимо нормализовать данные с помощью деления на длину документа. Формула для этого выглядит следующим образом:

$$TF(T_{x,y}) = \frac{N(T_x, D_y)}{N(P_y)} \quad (1.1)$$

где $N(T_x, D_y)$ – общее количество раз, когда слово T_x встречается в документе D_y , а $N(P_y)$ – общее количество терминов в документе.

Обратная частота документа (Inverse Document Frequency, далее — IDF) определяется как суммарное количество записей, содержащих слово x . Поскольку данная характеристика уменьшается, когда слово встречается в тексте, она указывает на *редкость слова*. Таким образом, чем больше IDF тем больше значимость слова во всей совокупности документов. Показатель можно рассчитать по следующей формуле:

$$IDF(T_x) = \log \left(\frac{N(D)}{N(D, T_x)} \right) \quad (1.2)$$

где $N(D)$ – общее количество документов, а $N(D, T_x)$ – количество документов, содержащих термин x .

По сути, TF-IDF — это мера нормализации, используемая для оценки важности слова для документа в базе документов. Формула для расчета TF-IDF выглядит следующим образом:

$$TF - IDF(T_{x,y}) = TF(T_{x,y}) \times \lg \left(\frac{N(D)}{N(D, T_i)} \right) \quad (1.3)$$

Для расчета сходства между данными пользователя (описанием игры, названием, жанрами) или отзывами и векторной пространственной моделью предметов можно использовать метод косинусного сходства, как показано в уравнении 1.4. Этот подход позволяет выявить предметы, аналогичные введённым данным, и создать рекомендацию. Затем полученные результаты могут быть подвержены классификации.

$$\cos(\Theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \quad (1.4)$$

1.2.2.2 Хеширование признаков

Для признаков с большим числом уникальных значений (например, теги или категории игр) применяется хеширование признаков (feature hashing). Этот метод позволяет сократить размерность пространства признаков без явного хранения словаря. Формально, хеширование можно описать как:

$$h : V \rightarrow \{1, \dots, k\}, \quad \varphi_i = \sum_{w: h(w)=i} f_w, \quad (1.5)$$

где V – множество уникальных значений признака (например, тегов), k – размер хеш-пространства, f_w – частота или вес признака w .

Этот метод позволяет работать с огромным числом уникальных значений (например, миллионами тегов), не храня их явно, но при этом размерность вектора признаков остается постоянной. Но есть главный минус: разные признаки могут попадать в одну и ту же ячейку хеш-таблицы, что может снижать точность. Хотя на практике коллизии часто незначительно влияют на качество модели.

Категории с небольшим числом вариантов (например, жанр игры: Action, RPG, Strategy) удобно представлять *бинарной (one-hot) кодировкой*. Такой вектор длины примерно равной числу жанров (нескольких десятков) и содержит 1 в позициях, соответствующих жанрам данной игры. Каждая позиция представляет собой конкретный жанр. Для многожанровых игр используют *мульти-hot кодирование* (несколько единиц). При малом числе категорий разреженность невысока и вычисления легки.

Однако это не отражает отношений между категориями (жанры «Экшен» и «Шутер» друг другу не ближе по вектору; все категории ортогональны). Если появится новый жанр (очень редко), нужно расширять вектор. Для очень большого числа категорий one-hot неудобен.

Кроме того, можно адаптировать под нашу задачу методы хеширования, представленные в [28]. Для тегов и категорий можно использовать хеширование признаков для сокращения размерности и применить алгоритмы типа Locality-Sensitive Hashing (LSH) для эффективного поиска похожих игр в хеш-пространстве. Для жанров же оставить бинарную кодировку, так как их количество невелико. В будущем можно рассмотреть бинарные вложения (binary embeddings), если потребуется учесть семантическую близость жанров.

1.2.2.3 Семантическое кодирование с использованием Sentence-BERT

Современные модели обработки естественного языка (NLP) позволяют получать *семантически значимые векторные представления* текстов. В частности, модель *Sentence-BERT* (S-BERT) [37] основана на архитектуре BERT, адаптированной для задач семантического сопоставления, и позволяет кодировать тексты (в нашем случае описания игр) в компактные векторы фиксированной размерности, отражающие смысл текста.

Sentence-BERT обучается на парах предложений и используется в задачах поиска схожих документов, кластеризации и построения рекомендаций. Один из наиболее эффективных предобученных вариантов этой модели — all-MiniLM-L6-v2, объединяющий точность и производительность.

При этом модель предлагает следующие функции:

- учитывает контекст и порядок слов;
- эффективно работает с небольшими и средними по длине текстами (что часто встречается среди описаний игр);
- поддерживает сравнение смысловой близости между описаниями;
- устойчива к синонимам и перефразированию текста.

Поэтому подобные семантические эмбединги могут использоваться в системе для сравнения и кластеризации игр, а также расчета $S_{\text{контент}}$ в многокритериальной оценке доли продаж игры в своем сегменте.

1.2.3 Перспективы расширения анализа

Sentence-BERT является наиболее удачным решением для обработки текстовых описаний игр по следующим причинам:

- ключевое требование – сохранение смысловых связей между описаниями игр (например, "космический симулятор" и "игра про космос" должны иметь близкие векторы);
- предобученные модели обеспечивают высокое качество при минимальных вычислительных затратах;
- фиксированная размерность векторов (384) упрощает их интеграцию в систему ранжирования;
- наличие готовых инструментов для косинусного сравнения (см. уравнение 1.4).

Таблица 1.2

Сравнительный анализ методов векторизации текстовых данных

Метод	Преимущества	Недостатки	Применимость
TF-IDF	– простота реализации; – низкие вычислительные затраты.	– игнорирует контекст; – плохо работает с синонимами.	Устарел для семантического анализа [6]
Bag-of-Words	– быстрая обработка; – подходит для простых задач.	– высокая размерность; – потеря семантики.	Не учитывает смысл описаний
Sentence-BERT	– учитывает контекст; – компактные векторы; – многоязычность.	– требует GPU; – сложная настройка.	Основной метод для векторизации текстовых описаний [6]

Полученные эмбединги можно также использовать в других задачах анализа текстовой информации. Например, с их помощью можно обучить классификатор (например, на базе LSTM, BiLSTM или GRU) для определения тональности отзывов пользователей. Такой подход позволяет:

- учитывать не только семантику отдельных слов, но и их контекст (настроение, полярность высказывания);
- в будущем – использовать оценки эмоционального фона отзывов как один из факторов формирования доли продаж игры;
- выполнять тематическую группировку и фильтрацию контента по настроению аудитории.

Интеграция таких методов анализа тональности рассматривается как потенциальное направление расширения системы в рамках дальнейших исследований и улучшения качества рекомендаций.

1.3 Обзор методов ранжирования компьютерных игр

Данный раздел включает обзор существующей литературы по теме исследования с целью выявления основных подходов и методологий, которые использовались другими исследователями при анализе, оценке и продвижении компьютерных игр или похожих продуктов.

1.3.1 *Общее понятие и цели ранжирования игр*

Ранжирование (от англ. *Ranking*)— это процесс упорядочивания элементов в соответствии с определёнными критериями качества или релевантности. На основе любых коллекции можно составить рейтинги. В области информационных технологий ранжирование широко применяется для организации информации по степени важности или полезности, например, в поисковых или рекомендательных системах, системах оценки и т.п. Данный процесс выходит далеко за рамки простой сортировки: ранжирование может учитывать множество гетерогенных факторов и подразумевает построение весовой функции, агрегирующей их в единое значение [40].

Классическими примерами служат алгоритмы PageRank, HITS и SALSA, разработанные для оценки важности веб-страниц с учётом ссылочной структуры и контекста [40]. Эти методы показывают, что релевантность и авторитетность – разные, но взаимосвязанные понятия, и построение эффективной модели ранжирования требует как структурного анализа, так и учёта пользовательского поведения.

В индустрии компьютерных игр ранжирование играет важную практическую роль. Каждую неделю на платформах вроде Steam появляются десятки новых проектов, и пользователи сталкиваются с серьёзной проблемой выбора. В условиях такого перенасыщения информация теряет структуру – популярные и качественные игры могут быть незаметны на фоне множества менее релевантных предложений. Чтобы помочь игроку ориентироваться в этом большом объёме данных и найти действительно интересные ему проекты, необходим механизм ранжирования – система, способная отсортировать игры по уровню их актуальности, качества и доле продаж на основе их характеристик. Чем более релевантна игра для определенных параметров и чем выше её оценка доли продаж в сегменте, тем выше

вероятность того, что она будет находиться в топе рейтинга. Такая система может быть использована:

- для формирования *персонализированных рекомендаций*;
- для *оценки доли продаж в сегменте* новой игры на фоне аналогов;
- как *аналитический инструмент* при исследовании предпочтений аудитории и рыночных трендов.

Кроме того, система должна учитывать многокритериальность задачи. Например, в модели 4PEG [15] для образовательных игр оценка проводится по четырём опорам: геймплей, образовательный контент, поддержка преподавателя и баланс между игровыми и учебными элементами. Хотя эта модель рассчитана на обучающие игры, её подход к многоаспектной оценке релевантен и для коммерческих проектов, что будет полезно для нашей системы.

Таким образом, ранжирование в игровой индустрии может помочь не только при маркетинговом анализе, но и при поддержке проектных решений и разработке. Правильная реализация такой системы требует как сбора и нормализации данных, так и применения методов машинного обучения, что делает задачу особенно актуальной в условиях роста объёмов данных на игровых платформах вроде Steam.

1.3.2 Принципы и методы построения рейтингов компьютерных игр

Ранжирование компьютерных игр по степени их коммерческой и пользовательской привлекательности представляет собой многокритериальную задачу. Каждый вариант $x \in X$ (игра) описывается вектором количественных признаков (рыночных и контентных), которые далее обозначаются как $f_1(x), f_2(x), \dots, f_n(x)$. Для корректной агрегации эти признаки предварительно нормализованы по шкале $[0; 1]$ посредством линейной или квантильной нормализации:

$$f_i^{\text{norm}}(x) = \frac{f_i(x) - \min f_i}{\max f_i - \min f_i}. \quad (1.6)$$

1.3.2.1 Однопризнаковое ранжирование

Наиболее простой метод заключается в сортировке вариантов по значению одного признака $f_k(x)$. В игровых платформах часто используются такие индикаторы, как общее количество продаж, рейтинг Metacritic или средняя оценка пользователей. Однако, как показывает практика [40], подобная стратегия игнорирует сложную взаимосвязь между метриками, в частности – между коммерческой и качественной составляющей продукта.

1.3.2.2 Многокритериальное ранжирование

Многокритериальные методы принятия решений (MCDM), такие как АНР, TOPSIS и VIKOR, позволяют агрегировать разнородные показатели в единый индекс. Например, в работе [12] предложен ансамблевый подход, комбинирующий несколько MCDM-методов для выбора оптимального плана обслуживания энергосистем. Адаптация подобных методов для игровой индустрии может включать взвешивание критериев:

- рыночных показателей, по типу количества продаж, активности игроков, доли рынка и т.п;
- конкретных характеристик, включающих в себя оценку геймплея, графики или сюжета через анализ отзывов [36].
- социального влияния, в том числе упоминания в соцсетях и рейтинги платформ.

Ключевая сложность данного метода – нормализация данных и определение весовых коэффициентов. Исследование [42] демонстрирует, что tree-модели (LightGBM, Random Forest) эффективны для прогнозирования рейтингов на Steam, минимизируя ошибку за счёт учёта нелинейных зависимостей.

Таким образом, при большом количестве данных и требовании к повышенной интерпретируемости, перспективным направлением является применение многокритериального метода, при котором конечная оценка доли продаж игры в своем сегменте $S(x)$ определяется через свёртку всех нормализованных признаков $f_i^{\text{norm}}(x)$ с учётом весов w_i , отражающих их важность:

$$S(x) = \sum_{i=1}^n w_i \cdot f_i^{\text{norm}}(x), \quad \sum_{i=1}^n w_i = 1, \quad w_i \geq 0. \quad (1.7)$$

Такая *аддитивная свёртка* (или взвешенная сумма критериев) отличается прозрачностью, объяснимостью и адаптируемостью. Она особенно удобна для задач, в которых каждый критерий можно рассматривать как частично компенсируемый. Однако у неё есть свои минусы, представленные в работе [7]. Основными недостатками аддитивной свертки являются её теоретическая некорректность при работе с порядковыми критериями, неявное допущение компенсации между критериями, что может привести к нелогичным выводам, а также зависимость результатов от произвольного назначения весов и нормирующих функций без учёта интервалов изменения значений критериев. Например, линейная свертка исключает сбалансированные варианты, даже если они обладают преимуществами,

как показано в [7], где вариант с равными значениями критериев никогда не выбирается. Кроме того, метод нарушает аксиому независимости от посторонних альтернатив, демонстрируя нестабильность результатов при удалении доминируемых вариантов [7]. Эти ограничения подчёркивают, что аддитивная свёртка, несмотря на её популярность, часто применяется без должного теоретического обоснования, что снижает её надёжность в практических задачах.

1.3.3 Методы свёртки критериев

Как уже было сказано, одним из наиболее интуитивно понятных и широко используемых способов объединения нескольких показателей в единую итоговую оценку является *аддитивная свёртка*. Этот метод предполагает, что *каждый критерий вносит вклад в общую оценку пропорционально заданному весу*.

В отличие от «чёрных ящиков» машинного обучения, такая модель позволяет явно видеть, как каждый параметр влияет на итоговую оценку. Это особенно важно в системах, ориентированных на объяснимость и возможность ручной корректировки (например, в образовательных или экспертных рекомендационных системах). Поэтому среди преимуществ метода:

- прозрачность и простота реализации;
- гибкость при выборе и настройке критериев;
- возможность оперативной перенастройки весов под конкретные задачи.

К недостаткам можно отнести *чувствительность к масштабу* входных данных (что требует обязательной нормализации) и *невозможность отразить сложные зависимости между признаками*. Тем не менее, при должной предварительной обработке данных и экспертной настройке весов, аддитивная свёртка остаётся эффективным инструментом, особенно в условиях ограниченного объёма обучающей выборки или отсутствия пользовательской истории.

Существует и альтернативная стратегия – *мультипликативная свёртка*, определяемая как:

$$S(x) = \prod_{i=1}^n (f_i^{\text{norm}}(x))^{w_i}. \quad (1.8)$$

Мультипликативная свёртка используется в задачах, где важно учесть *взаимозависимость* факторов и недопустимость нулевых значений. Такой метод усиливает влияние слабых показателей: если один из критериев равен нулю, итоговая оценка также становится равной нулю, независимо от остальных значений. Это делает её особенно чувствительной к «узким местам» объекта оценки. Ниже

приведена сравнительная таблица табл.1.3, демонстрирующая различия между этими подходами применительно к задаче ранжирования игр:

Таблица 1.3

Сравнение методов свёртки критериев

Критерий	Аддитивная	Мультипликативная
Интерпретируемость	Высокая	Низкая
Учёт сбалансированных решений	Подходит при Парето-фильтрации	Уязвим к дисбалансу
Чувствительность к нулям	Не критична	Критична
Интеграция разнородных данных	Гибкая	Сложная
Научное обоснование [7]	Да	Нет

Как отмечает В.В. Подиновский, метод взвешенной суммы критически зависит от корректной нормализации и адекватного задания весов [7]. Несмотря на критику метода анализа иерархий (АНР) в случае порядковых шкал, наша задача использует *количественные шкалы*, а потому допущения аддитивной модели допустимы.

Для целей настоящего исследования была выбрана *аддитивная модель свёртки*, поскольку она обеспечивает:

- прозрачность вычислений и понятность для конечного пользователя;
- гибкость в настройке весов в зависимости от задачи – будь то прогнозирование, сравнение или улучшение характеристик игр;
- устойчивость к отсутствующим или слабым признакам, в отличие от мультипликативной модели, где нулевое значение критически обнуляет весь результат.

При этом рыночная оценка может формироваться на основе:

- показателей общей популярности игры: количестве активных игроков, рейтинге (Metacritic), числе продаж;
- сходства с успешными top-N играми посредством сравнения ключевых характеристик, таких как описание, жанры и прочие метки;
- индекса привлекательности, рассчитанного как соотношение позитивных отзывов и вовлеченности.

Такой подход позволяет не только упорядочить игры по степени их привлекательности, но и – при необходимости – выдать обоснованные рекомендации по улучшению их характеристик на основе анализа конкурентов.

Однако чистая свёртка (даже при корректных весах) не способна корректно обработать «сбалансированные» варианты [7]. Известен контрпример: пусть

$$x^1 = (1, 0), \quad x^2 = (0, 1), \quad x^3 = (0.45, 0.45). \quad (1.9)$$

При любой взвешенной свёртке (любох $w_1 + w_2 = 1$) вариант x^3 уступает как x^1 , так и x^2 . Хотя x^3 представляет собой сбалансированный выбор, он оказывается исключённым – что противоречит интуиции. Подиновский [7] называет такие ситуации эффектом «парадокса доминирования».

Для решения этой задачи мы будем использовать двухэтапную стратегию:

1. По принципу *Парето-оптимизации* формируется множество недоминируемых решений $X_P \subseteq X$, т.е. таких x , для которых не существует x' , доминирующего по всем K_i . Это устраняет заранее «слабые» игры.
2. Аддитивная свёртка по оставшимся вариантам:

$$S(x) = w_1 \cdot S_{\text{рынок}}(x) + w_2 \cdot S_{\text{контент}}(x), \quad (1.10)$$

где каждая компонента заранее нормализована в диапазоне $[0; 1]$.

Такой подход позволяет сохранить важные «промежуточные» решения, а затем упорядочить их по гибкому критерию.

Для примера рассмотрим две популярные Steam-игры табл.1.4:

Таблица 1.4

Сравнительные показатели игр		
Показатель	<i>The Sims 4</i>	<i>Cities: Skylines</i>
Доля игроков в жанре (%)	18.7	12.3
Годовая выручка (\$ млн)	120	85
Доля положительных отзывов (%)	78	92
Новизна (лет с релиза)	9	5
Оценка отзывов (0-10)	7.2	8.9

Обе игры проходят Парето-фильтр:

- *The Sims 4* доминирует по: выручке, доле игроков;
- *Cities: Skylines* доминирует по: оценке отзывов, новизне;
- Оба варианта являются *недоминируемыми* ($X_P = \{x^{\text{Sims}}, x^{\text{Cities}}\}$).

Нормируем показатели (линейно к $[0; 1]$) и применяем аддитивную свёртку. Допустим, что *The Sims 4* лидирует по рыночным показателям, однако сбалан-

сированность *Cities: Skylines* обеспечивает преимущество в итоговом рейтинге ($0.82 > 0.79$), что устраняет «парадокс доминирования».

Согласно теории важности критериев (далее – ТВК) В.В. Подиновского [7], относительная значимость K_i может быть определена на двух уровнях:

1. *Качественная важность* заключается в упорядочении критериев ($K_1 \succ K_2 \sim K_3$) на основе экспертных предпочтений;
2. *Количественная коррекция* опирается на численные веса w_i , уточняемые, например, через исторический анализ успешных игр (регрессия, А/В-тесты).

Подиновский подчёркивает, что разделение этих этапов снижает субъективность и позволяет соблюдать строгость моделей. В нашей модели можно подключить:

- экспертное ранжирование метрик по значимости (например, $K_{\text{продажи}} > K_{\text{рейтинг}} > K_{\text{жанр}}$);
- последующую коррекцию на основе DASS-системы (см. [7]) или анализа чувствительности.

Таким образом, гибридный метод, основанный на Парето-оптимизации и аддитивной свёртке, с разделением качественной или количественной важности критериев, обеспечит надёжную, масштабируемую и интерпретируемую стратегию ранжирования игр, ориентированную на принятие обоснованных решений в условиях конкуренции.

1.4 Обзор существующих средств ранжирования

В данном разделе представлен обзор наиболее известных моделей ранжирования, используемых в индустрии компьютерных игр. Мы рассмотрим их ключевые признаки, применяемые методы, а также преимущества и недостатки.

1.4.1 Система Steam

Платформа Steam [68] предоставляет несколько публичных рейтинговых списков, таких как *Top Sellers*, *Most Played*, *New Releases*, *Trending* и другие. Эти чарты автоматически формируются на основе внутренней аналитики платформы, однако детали их построения не раскрываются публично. Согласно документации Steamworks [4], ключевыми метриками при формировании списков являются:

- совокупная выручка от игры (включая продажи DLC и внутриигровых покупок);
- игровая активность (общее и среднее количество игроков, время, проведённое в игре);
- внутренняя аналитика пользовательского поведения (установки, добавления в «желаемое», частота запуска).

Можно заметить, что основным критерием ранжирования на платформе выступает реальная *ценность игры для пользователя*, измеряемая через вовлечённость и коммерческий успех, а не только формальная «популярность» в виде отзывов или просмотров. Это отличает Steam от систем, где приоритет отдается только субъективным оценкам.

Помимо статических рейтингов, в 2014 году в Steam была внедрена персонализированная система рекомендаций под названием *Discovery Queue*, которая с тех пор активно развивается [67]. Она использует методы машинного обучения и ориентируется, прежде всего, на:

- поведение пользователя (время, проведённое в играх, частота запусков, пройденные достижения);
- историю взаимодействий (оценки, установки, «избранное», покупки);
- предпочтения социальной сети Steam (список друзей, их интересы, игры, обсуждения);

На основе этих факторов формируется индивидуальный профиль, с которым сопоставляются другие игры из базы данных. Рекомендации выдаются на основе схожести с уже опробованными проектами, что делает механизм Steam примером *коллаборативной фильтрации*.

Тем не менее, у существующего алгоритма есть ряд ограничений:

- алгоритм рекомендаций ориентирован исключительно на *историю взаимодействий* конкретного пользователя и почти не использует семантический анализ описаний игр;
- система игнорирует метаинформацию о продуктах (жанры, описание, особенности механик) как источник для обоснованных рекомендаций;
- фокус на популярных играх снижает вероятность попадания в рекомендации менее известных или инди-проектов;
- сам алгоритм закрыт, а принципы работы полностью не раскрываются, что делает невозможной его внешнюю верификацию.

Исследования показывают, что около 68% пользователей считают рекомендации Steam не соответствующими их интересам [44], а 81% редко используют данную систему.

Таким образом, несмотря на продвинутые методы сбора данных и персонализации, алгоритмы Steam ориентированы, в первую очередь, на коммерческие показатели и поведенческие паттерны, не учитывая контентные особенности игр. Это открывает пространство для разработки альтернативных систем, в которых ранжирование будет учитывать более широкий набор признаков — от семантической близости до внешней рыночной аналитики.

1.4.2 Система STEAMer

Система *STEAMer*, предложенная в исследовании [44], представляет собой гибридный подход к ранжированию игр на платформе Steam, сочетающий методы глубокого обучения и многокритериального анализа. STEAMer использует *глубокий автоэнкодер (DAE)* для обработки комплексных признаков, включая жанры игр, даты их релиза, статистику пользователя, наличие игры у друзей и т.д.

Архитектура *глубокого автоэнкодера* состоит из 5-слоёв (256-128-256 нейронов) с ReLU-активацией в скрытых слоях и сигмойдой на выходе. Это позволяет выявлять скрытые зависимости между признаками, проявлять устойчивость к шуму и работать с разреженными данными. В то же время, такая система имеет риск переобучения при малом объеме данных.

Многокритериальный анализ включал в себя обработку *контентных* (жанр, разработчик, Metacritic score), *поведенческих* (время игры, активность друзей) и *социальных* (принадлежность к Steam-группам) признаков. Для ранжирования использовалась *ранговая агрегация* предсказанных значений DAE с весовыми коэффициентами, аналогично формуле:

$$S_{\text{рейтинг}} = w_1 \cdot \text{Playtime} + w_2 \cdot \text{Metacritic} + w_3 \cdot \text{Friend_Ownership}. \quad (1.11)$$

Работа показала, что:

1. Система может надежно и эффективно обслуживать активное количество пользователей до 20,000 человек, поддерживая необходимые показатели производительности, такие как скорость обработки и время отклика.
2. Добавление уникальных пользовательских данных, может улучшить производительность.

Данные, используемые в STEAMer, были собраны и опубликованы в 2016 году О’Нил и др. в [18]. Система достигла лучших результатов по точности и релевантности рекомендаций, а также улучшила их разнообразие, но за этим следовало небольшое *снижение новизны* из-за популярности рекомендуемых игр и рост риска переобучения из-за большого объема данных.

В перспективе можно интегрировать в систему анализ текстовых обзоров через BERT для учета субъективных оценок (ср. [36]), адаптацию коэффициентов через reinforcement learning и использование генеративных моделей для синтеза «идеального» профиля игры. Однако есть некоторые ограничения, которые делают данную систему нерелевантной для нашей работы:

1. STEAMer требует массивных данных о поведении пользователей (игровое время, социальные связи), что неприменимо для новых игр без сложившейся аудитории. В нашем случае фокус смещен на *контентные характеристики* (описания, жанры, метаданные), что позволяет оценивать игры до их релиза.
2. Глубокие автоэнкодеры требуют вычислительных ресурсов для обработки. В нашей системе можно использовать более легковесные методы (например, семантические эмбединги для текстовых обзоров), сохраняя точность при меньших затратах.
3. Вместо анализа игрового времени мы предлагаем *семантический анализ отзывов или описаний* через методы типа BERT (как в [36]) и *сравнение с другими играми* через векторные представления описаний. Это позволяет выявлять скрытые правила (например, «игры с атмосферой как у Dark Souls») без отслеживания поведения пользователей.

1.4.3 Система DRGame

DRGame [19] предлагает баланс между предпочтениями пользователей и разнообразием рекомендаций. Система использует *кластеризацию данных и графовые нейронные сети* (GNN) для учета различных категорий игр и интересов пользователей.

Система DRGame применяет следующие методы для обеспечения разнообразия рекомендаций:

- пользователи и игры группируются в кластеры на основе их категориальных предпочтений. Для этого используется матрица игровых категорий Q и

преобразованная матрица взаимодействий R . Для каждого кластера выбирается наиболее репрезентативный узел (игра или пользователь) на основе сбалансированных неявных предпочтений H , за счет чего уменьшается перекос в сторону популярных игр и обеспечивается более равномерное покрытие категорий;

- в отличие от традиционных GNN, которые симметрично агрегируют информацию от всех соседей, DRGame использует индивидуальную агрегацию для пользователей и игр. Агрегация происходит по следующим формулам:

$$e_i^{(l+1)} = \sum_{u \in \mathcal{N}_i} \frac{\mathbb{I}((u, i) \in \mathcal{G}^{U \rightarrow I})}{\sqrt{|\mathcal{N}_i|} e_u^{(l)}}. \quad (1.12)$$

- для минимизации влияния популярных категорий и повышения разнообразия вводится вес w_i для каждой игры, который обратно пропорционален частоте её категорий. Это позволяет увеличить представленность редких категорий в рекомендациях.

Несмотря на все преимущества подхода, DRGame имеет несколько ограничений. Во-первых, кластеризация и асимметричная агрегация требуют значительных вычислительных ресурсов, в которых ограничена наша система. Во-вторых, эффективность метода сильно зависит от качества категоризации игр – некорректно назначенные категории ухудшают рекомендации. Для нашей системы целесообразно взять наиболее эффективные элементы DRGame: кластеризацию игр по категориям для повышения разнообразия, а также механизм весов для редких категорий. При этом стоит оптимизировать вычислительные процессы, например, за счет приближенных методов кластеризации.

1.4.4 Система DeepFM

Модель DeepFM, предложенная в исследовании [17], представляет собой гибридный подход, сочетающий факторизационные машины (Factorization Machines) и глубокие нейронные сети (Deep Neural Networks). В основе системы лежит анализ игровой активности пользователей, включая общее время игры (playtime_forever) и активность за последние две недели (playtime_2weeks). Для реализации алгоритмов использовалась библиотека Surprise в Python, что обеспечило простоту развертывания и сравнительный анализ различных методов.

Алгоритмы, такие как SlopeOne и SVD++, показывают отличные результаты в точности рекомендаций. В остальном результаты исследования демонстриру-

ют отсутствие существенной разницы между исследуемыми алгоритмами для используемого набора данных. Кроме того, модель позволяет преобразовывать время игры для получения явных оценок для составления рекомендаций в области видеоигр, в частности на платформе Steam.

Ранжирование в DeepFM осуществляется через комбинацию двух компонент:

- *линейная часть* (FM) вычисляет попарные взаимодействия признаков;
- *нейросетевая часть* (DNN) выявляет сложные нелинейные зависимости.

Финальная оценка формируется как сумма выходов обоих компонент: $\hat{y}(x) = y_{FM}(x) + y_{DNN}(x)$. Для обучения используется логистическая функция потерь с L2-регуляризацией.

Но основным недостатком подобной системы является сильная зависимость от единственного фактора – времени игры, что может привести к некорректным оценкам. Кроме того, пользователи с разными стилями игры (любители долгих RPG против фанатов коротких сессий) будут оцениваться одинаково, что противоречит цели системы персонализированных рекомендации. В нашей модели мы планируем преодолеть эти ограничения за счет учета категориальной принадлежности игр и ведения поправочных коэффициентов для разных признаков.

1.4.5 Система D-VSMR

Модель D-VSMR[25] использует визуальные семантические признаки, такие как объекты и сцены в играх, для создания профиля пользователя. Применение *сверточных нейронных сетей* позволяет системе работать даже при недостатке данных о взаимодействиях (проблема холодного старта).

Визуальные признаки включают сцены, объекты и тематические элементы игр (например, «огонь», «автомобиль», «оружие»), которые представляют отдельные жанры (шутер, гонки и т.д.).

Итак, интеграция визуальных признаков с данными текста описаний и метаданными может сделать систему оценки более точной и гибкой. Тем не менее, если первоначальные данные о визуальных особенностях игры, например, скриншоты или видео, не очень качественны, качество определения модели будет снижено. Кроме того, поскольку одна и та же игра может принадлежать к нескольким жанрам или ее визуальный контент варьируется значительно, модель может неправильно определить атрибут.

Визуальные атрибуты в комбинации с текстовыми описаниями и метаданными могли бы дать более полное представление о предпочтениях пользователя. Даже если это новая вещь без истории, как люди с ней взаимодействуют, визуально-семантические атрибуты могут дать правильный старт рекомендуемой системы.

1.4.6 Сравнительный анализ моделей ранжирования

Для систематизации существующих подходов воспользуемся двумя таблицами. Первая посвящена признакам систем (см.табл.1.5), вторая – методам реализации ранжирования (см.табл.1.6).

Таблица 1.5

Объектно-признаковая таблица моделей ранжирования

Признак	Оценка доли продаж	Диверсификация	Мульти-модальность	Кластеризация	Рекомендации по улучшению
Steam	-	-	-	-	-
STEAMer [44]	-	+	+	-	-
DRGame [19]	-	+	-	+	-
DeepFM [17]	-	-	-	-	-
D-VSMR [25]	-	-	+	-	-
Разрабатываемая система	+	+	+	+	+

Ключевые критерии сравнения включают:

1. *Диверсификацию* – способность обеспечивать разнообразие рекомендаций;
2. *Мультимодальность* – использование различных типов данных (визуальные, текстовые, поведенческие);
3. *Кластеризация* – группировка пользователей/игр по схожим характеристикам;
4. *Рекомендации по улучшению* – формирование советов для повышения рыночной привлекательности продукта на основе его описательных характеристик.

Как видно из таблицы табл.1.5, разрабатываемая система будет включать все ключевые аспекты: просчет доли продаж игры в своем сегменте, улучшенную диверсификацию, использование различных типов данных, кластеризацию для поиска похожих игр и алгоритм для составления рекомендации по улучшению продукта пользователя. При разработке нашей системы ранжирования важно,

чтобы получаемые результаты были не только релевантными, но и более персонализированными.

Таблица 1.6

Сравнительный анализ подходов к ранжированию игр

Система / метод	Тип данных	Метод свёртки критериев	Контентный анализ	Оценка доли продаж
Steam (официальные чарты)	Коммерческие и поведенческие метрики	Не раскрыт	–	–
STEAMer	Поведенческие, социальные и контентные признаки	Взвешенная сумма (агрегация)	Частично (жанры, разработчик)	–
DRGame	Графы взаимодействий и категории	Неявная агрегация в GNN	Категории (кластеры)	–
DeepFM	Поведенческие признаки (время игры)	FM + DNN (суммирование)	–	–
D-VSMR	Визуальные и текстовые признаки	Векторная агрегация	+	–
Разрабатываемая система	Контентные и рыночные признаки	Аддитивная свёртка + метод АНР	+	+

Таблица 1.6 демонстрирует ключевые отличия между существующими системами ранжирования компьютерных игр. Как видно, большинство современных решений сосредоточено на поведенческих данных пользователей (время игры, друзья, предпочтения), что делает их непригодными для анализа новых игр. В отличие от них, предлагаемая система основывается на контентных и рыночных признаках, позволяет объективно оценивать долю продаж в своем сегменте как выпущенных, так и разрабатываемых игр и предоставляет гибкий механизм многокритериальной агрегации на основе метода аналитической иерархии (АНР). Это делает её универсальным инструментом поддержки принятия решений в игровой индустрии.

Кроме того, рекомендуется изучить использование методов, которые ранее не применялись в задачах ранжирования видеоигр, но обладают значительным потенциалом. Текущая работа включает в себя разработку метода, который будет учитывать следующие аспекты:

- расчёт расстояний по методикам [41];

- объединение рейтингов игроков может быть эффективным, если применить теорию важности критериев ([7]), которая позволяет оценивать значимость множества факторов;
- возможное использование аналитической иерархии [5] для определения весов факторов, влияющих на рекомендацию игр.

Поскольку методы, основанные на подходах [5; 7] или других разработках в области многокритериального принятия решений, пока не применялись для построения рекомендательных систем в области видеоигр, они являются перспективной областью для дальнейших исследований.

1.5 Выводы

Данная глава закладывает теоретические основы для разработки системы, сочетающей многокритериальный анализ с особенностями продвижения продуктов в игровой индустрии. Среди всех заключений, сделанных при исследовании, можно выделить следующие:

1. Платформа Steam [68] определена как оптимальный источник данных, предоставляющий метаинформацию об играх (жанры, теги, описания, среднее количество часов на прохождение, категории и т.д.), объективные метрики (активность игроков и количество пользователей) и текстовые отзывы [21; 35; 46; 47].
2. Выявлен недостаток существующих систем: фокус на поведенческих данных (игровое время, социальные связи) делает их непригодными для оценки новых игр.
3. Для текстовой информации выбраны Sentence-BERT (семантическое кодирование описаний) и хеширование с целью эффективного сокращения размерности данных [37].
4. Аддитивная свёртка критериев демонстрирует наилучший баланс между гибкостью настройки и устойчивостью к нулевым значениям признаков. В отличие от мультипликативной модели, она позволяет сохранять сбалансированные альтернативы, не исключая их из рассмотрения. При этом предварительная Парето-оптимизация обеспечивает отсеивание доминируемых (заведомо слабых) игр и тем самым снижает риск искажения итогового ранжирования. Выбор весов в модели предлагается осуществ-

лять на основе принципов теории важности критериев [7] – с упором на качественное упорядочивание.

5. Существующие системы ранжирования (Steam, STEAMer и DeepFM) ориентированы на персонализацию, но игнорируют конкретные характеристики игр (например, семантику описаний) и не предоставляют возможность оценки доли продаж игры до её релиза. Предлагаемый метод устраняет эти ограничения за счёт анализа рыночных и контентный (жанры, описания) признаков, а также стратегии ранжирования.
6. С точки зрения научной новизны мы адаптируем теорию важности критериев [5; 7; 41] для ранжирования игр, реализуя качественную оценку критериев и количественную коррекцию весов на основе полученных данных, а также интегрируем кластерный анализ для выявления схожих игр.

Полученные выводы в дальнейшем помогут реализовать систему, способную ранжировать игры по объективным и контентным показателям, формулировать рекомендации по улучшению (например, "добавить элементы жанра X") и оценивать долю продаж новых проектов до их выхода на рынок.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ ПОДДЕРЖКИ АНАЛИЗА КОМПЬЮТЕРНЫХ ИГР

В данной главе рассматриваются особенности проектирования и реализации системы поддержки анализа и рейтингования компьютерных игр, включая описание функционала системы выбранной архитектуры 2.1, используемых математических алгоритмов 2.2 и процесс формирования рекомендаций 2.3.

2.1 Функциональная модель системы

2.1.1 Use-case диаграмма и основные сценарии

В предыдущей главе определено, что разрабатываемая система предполагает различные варианты использования, включая составление рейтингов игр и получение рекомендаций на основе выявленных уникальных черт вводимой компьютерной игры. В процессе доработки модели были уточнены функции системы, представ-

ленные на рисунке рис.2.1. Предполагаемые сценарии использования могут быть выполнены следующими категориями потенциальных пользователей:

1. *Разработчиками игр*, желающими определить уникальные черты своего продукта ещё на этапе его разработки, получить рекомендации по его улучшению и найти похожие по введённым параметрам компьютерные игры;
2. *Маркетологами и издателями*, которые могут спрогнозировать оценку доли продаж разрабатываемой компьютерной игры и внести изменения в стратегию её продвижения на основе рейтинга похожих продуктов, а также оценки их коммерческого успеха;
3. *Инвесторами*, использующими приложение с целью нахождения потенциальной выгоды инвестирования, опираясь на анализ трендов и схожесть с конкурентами.

Все данные о компьютерных играх, включая отзывы, теги, жанры, количество игроков, описание, цены и другие метаданные, собираются через API *SteamSpy* и *Steam*.

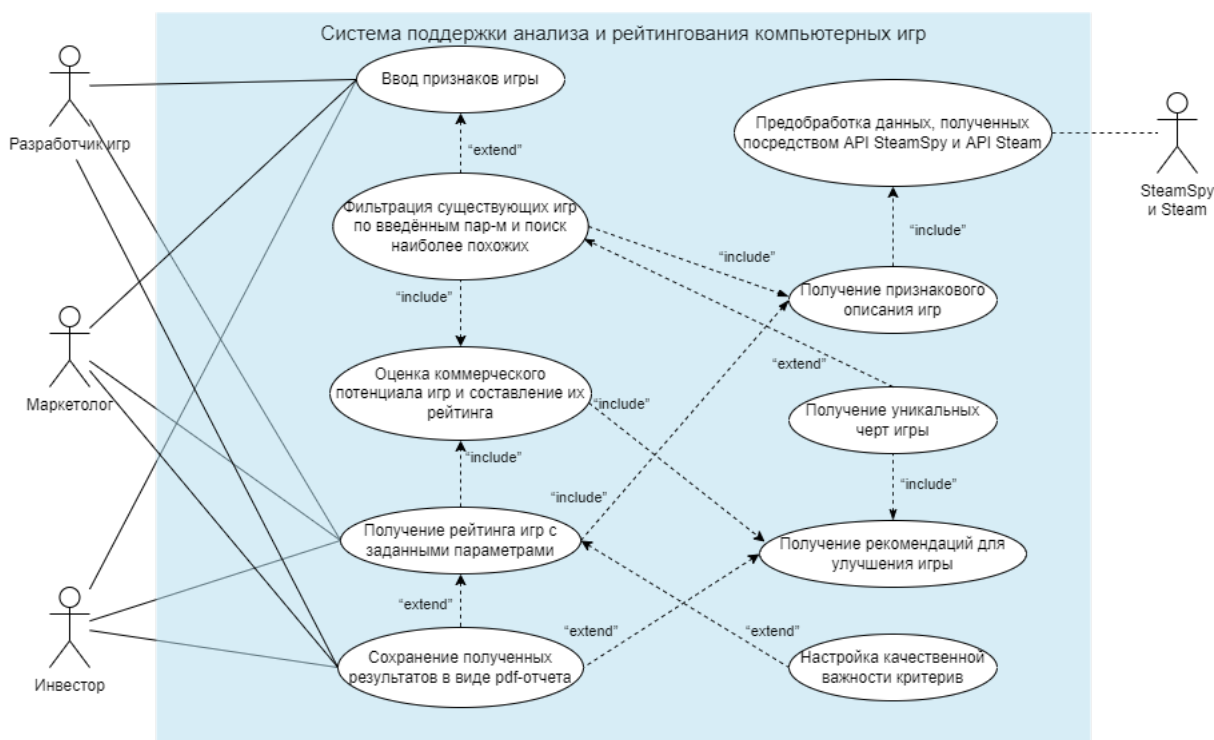


Рис.2.1. Диаграмма случаев использования системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

В ходе проектирования функциональные особенности системы подвергались изменениям и пересматривались с целью улучшения пользовательского опыта

взаимодействия с системой и повышения точности результатов. Рассмотрим сценарии использования приложения подробнее:

- 1) *Предобработка данных, полученных посредством API SteamSpy и API Steam* – процесс получения и первичной обработки данных, необходимых для последующего анализа. На данном этапе данные, заносятся в MongoDB и подготавливаются для дальнейшего использования в системе.
- 2) *Получение признакового описания игр* – извлечение ключевых характеристик игр, таких как жанры, теги, описание, отзывы и стоимость из подготовленных данных. Этот процесс служит основой для дальнейшего анализа и сопоставления игр и включается в предобработку данных.
- 3) *Ввод параметров выбранной игры* – пользователь вводит ключевые характеристики, включая название, жанры, теги, категории, описание и возможную стоимость, которые затем используются системой для анализа и формирования рекомендаций. Также имеется возможность использовать данные уже существующих игр с платформы Steam.
- 4) *Получение рекомендаций для улучшения игры* – процесс формирования советов по улучшению игры. Анализ проводится на основе уникальных черт продукта. Результаты предоставляются пользователю в удобной и структурированной форме.
- 5) *Получение уникальных черт игры* включается в предыдущий процесс и заключается в поиске различий и сходств введённой игры с похожими продуктами. На основе этих данных система предлагает недостающие теги, механики и фрагменты описания, которые могут улучшить уникальность и повысить долю продаж игры.
- 6) *Фильтрация существующих игр по введённым пар-м и поиск наиболее похожих* – выбор наиболее близкого к заданным характеристикам кластера игр, схожих между собой по описанию, стоимости, жанрам и тегам. Обязательно производится при оценке доли продаж рассматриваемых в ходе анализа игр.
- 7) *Получение рейтинга игр с заданными параметрами* – процесс составления ранжированного списка компьютерных игр, наиболее близких по содержанию к введённым характеристикам.
- 8) *Настройка качественной важности критериев* – возможность регулировать значимость критериев ранжирования (выручка/отзывы/новизна/активность) с помощью интерфейса.

- 9) *Оценка доли продаж игр и составление их рейтинга* – анализ характеристик компьютерных игр, попавших в один кластер с введённой игрой, с целью расчёта оценки их продаваемости. Этот показатель строится на основе доли продаж среди похожих продуктов, отношения положительных отзывов к отрицательным и доли пользователей на всём рынке.
- 10) *Сохранение полученных результатов в виде pdf-отчета* – возможность для конечных пользователей скачать предложенные системой рекомендации и рейтинг похожих игр в формате pdf.

2.1.2 Входные и выходные данные системы

Для создания системы анализа и оценки компьютерных игр необходимо собрать и обработать широкий спектр данных, отражающих основные аспекты игровой индустрии. Основным источником информации для формирования набора данных выступают платформы Steam API[10] и SteamSpy API[71].

Steam API преимущественно предназначен для получения *основной информации о играх*, включая их *стоимость, описание, основные жанры и пользовательские теги* в формате json. Мы также можем получить данные о минимальных и рекомендованных требованиях, возрастном рейтинге и обложке, но, как показывают исследования, подобные характеристики не делают предложения рекомендательных систем более точными и эффективными. Кроме прочего, данное API предоставляет данные о пользователях, включая их *текстовые отзывы* к играм и *время*, проведенное в играх.

SteamSpy, в свою очередь, предлагает *дополнительные аналитические данные*, такие как *количество положительных и отрицательных рецензий, объём продаж* и других данных о динамике популярности игр.

Для повышения качества анализа планируется *объединение данных* из Steam API и SteamSpy. Такой подход обеспечит комплексное исследование: структурированные данные будут использоваться для *количественного анализа*, а текстовые и поведенческие метрики дополнят его *качественными оценками*. Совместное использование обеспечит разнообразие и актуальность данных, что позволит реализовать точные оценки доли продаж компьютерных игр в определенном сегменте. Примеры входных и выходных данных представлены в таблице табл.2.1.

Таблица 2.1

Пример преобразования данных игры

Входные данные (Steam/SteamSpy API)	Выходные данные (обработанные)
<pre>{ "data": { "detailed_description": "<p>Great racing game!</p>", "categories": [{"description": "Multi-player"}, {"description": "VR Supported"}], }, "genre": "Car, Racing", "tags": ["Driving", "Simulation"], "price": "19.99", "median_forever": 240, "owners": "100000..200000", "positive": 1200, "negative": 200 }</pre>	<pre>{ "description": "Great racing game!", "genres": ["Racing"], "tags": ["Driving", "Simulation"], "price": 19.99, "categories": ["Multi-player", "VR Supported"], "median_forever": 240, "is_vr": 1, "is_multiplayer": 1, "owners": 150000, "positive_ratio": 0.857 }</pre>

В рамках проекта реализовано получение данных через API с помощью языка программирования Python. Для взаимодействия используются следующие методы:

- получение детальной информации об игре с помощью Steam Store API [68];
- извлечение статистики по играм через SteamSpy API [71];
- загрузка отзывов пользователей через Steam Reviews API [10].

Примеры функций взаимодействия с внешними источниками приведены ниже:

- `get_game_data(game_id)` — получение подробной информации об игре по её идентификатору;
- `get_steamspy_data(app_id)` — получение статистических данных об игре из SteamSpy по её идентификатору;
- `get_top_games(num_games=100)` — получение списка наиболее популярных игр за последние две недели (указывается количество игр);

- `get_steam_reviews(app_id, num_reviews=50)` — получение отзывов пользователей о выбранной игре (указывается количество отзывов).

Для быстрой работы с вышеприведенными источниками также был использован структурированный датасет *Steam Games Dataset 2025* [31], содержащий предварительно очищенные данные о 72,000 игр. Все обязательные параметры (идентификаторы, названия, жанры) заполнены, аномалии отсутствуют и идентификаторы `steam_appid` используются для кросс-платформенной выгрузки дополнительных данных через Steam/SteamSpy API.

2.1.3 Этапы работы системы анализа и рекомендаций

Каждый этап работы системы анализа и рекомендаций играет важную роль в формировании результатов, как показано на диаграмме 2.2.



Рис.2.2. Обобщенная архитектура системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

Первым этапом обработки данных является *преобразование признаков игр в числовое представление*, пригодное для машинного обучения. Текстовые описания игр векторизуются с помощью модели *SentenceTransformer*, которая преобразует их в семантические эмбединги (плотные векторные представления). Категориальные признаки, такие как жанры и категории, кодируются с помощью *MultiLabelBinarizer*, создавая разреженные бинарные векторы. Теги игр дополнительно нормализуются с помощью *L2-нормализации* для обеспечения сравнимых масштабов значений. Числовые характеристики (цена, рейтинг Metacritic, среднее время игры) приводятся к единому масштабу путём нормализации относительно максимального значения. Полученное многомерное признаковое пространство позволяет эффек-

тивно сравнивать игры между собой и находить схожие экземпляры в процессе кластеризации.

Ключевым этапом работы системы является комплексный анализ конкурентного окружения целевой игры. На этом шаге применяется *кластеризация игр по их векторным представлениям*, что позволяет выделить семантически близкие группы продуктов. Для каждого продукта из этого кластера затем вычисляется оценка доли продаж игры в своем сегменте как *взвешенная сумма ключевых метрик*: активности игроков, соотношения положительных отзывов к общему числу отзывов, доля рынка и т.д. На основе этого показателя *формируется рейтинг игр с наибольшими оценками* внутри кластера.

Для целевой игры затем рассчитывается *её собственная доля продаж* в сегменте похожих игр по взвешенной формуле, а также мера схожести с наиболее успешными играми кластера через косинусное сходство векторных представлений.

На заключительном этапе система выявляет *уникальные характеристики игры* путём сравнения её признаков с наиболее похожими конкурентами (общие черты), с наименее похожими играми кластера (дифференцирующие особенности). На основе этого анализа формируются рекомендации, включающие выявленные преимущества продукта и потенциальные направления развития, отсутствующие у аналогов. Таким образом, после анализа *пользователь получает полезную информацию*, которая помогает оптимизировать продукт и принимать более обоснованные решения.

2.2 Архитектурное проектирование системы

2.2.1 Архитектурные паттерны и их сравнение

Для определения наиболее подходящей архитектуры проведено сравнение различных архитектур (см. табл. 2.2), включая клиент-серверную, модульную, Model-View-Controller (далее – MVC) и сервис-ориентированную архитектуры (далее – SOA). Сравнение проводилось по нескольким критериям:

- 1) *Гибкость* – способность архитектуры адаптироваться к изменениям требований, добавлению новых компонентов или функциональности без значительных изменений в кодовой базе. Важна для нашей системы и должна быть не ниже среднего уровня, поскольку система может в дальнейшем интегрироваться с другими платформами, предоставляющими

данные об играх, с другими признаками, при этом требуя добавления дополнительного функционала;

- 2) *Масштабируемость* – возможность системы обрабатывать увеличивающийся объём данных и нагрузки без существенного снижения производительности, в основном путём добавления новых сервисов или увеличения ресурсов на сервере. Тесно связана с предыдущей способностью системы и предоставляет возможность расширения функционала без перегрузки. При этом не требуется высокий уровень масштабируемости, так как приложение имеет ограниченный функционал;
- 3) *Сложность разработки* – уровень сложности при проектировании, реализации и поддержке системы. Текущий проект разрабатывается в соответствии с ограниченными ресурсами и сроками. Более сложные архитектуры требуют могут увеличивать затраты на разработку, поэтому сложность должна быть не выше средней;
- 4) *Производительность* – способность системы при заданных условиях обеспечивать необходимую работоспособность по отношению к выделяемым для этого ресурсам. Включает в себя задержки, пропускную способность и скорость обработки запросов пользователей. Особенно важна для быстрой генерации рекомендаций и возможности обработки данных в реальном времени, поэтому должна быть не ниже средней;
- 5) *Распределение нагрузки* – способность архитектуры эффективно распределять нагрузку между компонентами системы, предотвращая узкие места и точки отказа. Помогает регулировать нагрузку в пиковые моменты работы нашей системы, обрабатывающей большие объёмы данных, а также гарантирует доступность основных функций. При этом приложение рассчитано на среднюю нагрузку, так как предполагает небольшое количество пользователей;
- 6) *Тестируемость* – лёгкость написания тестов для компонентов системы, изоляция компонентов для модульного тестирования и поддержка интеграционных тестов. В системе должно обеспечиваться безопасное изменение в кодовой базе и быстрое выявление аналитических ошибок, однако оптимален умеренный уровень тестируемости, при котором основное внимание уделяется интеграционным тестам, критические функции покрыты модульными тестами, а ресурсы распределены пропорционально важности компонентов.

Таблица 2.2

Сравнение архитектурных парадигм

Критерий	Монолит	Микро-сервисы	Клиент-сервер	SAO	MVC
Гибкость	Низкая	Высокая	Средняя	Низкая	Высокая
Масштабируемость	Ограниченная	Высокая	Умеренная	Ограниченная	Средняя
Сложность разработки	Низкая	Высокая	Средняя	Низкая	Средняя
Производительность	Высокая	Средняя	Высокая	Высокая	Высокая
Распределение нагрузки	Централиз.	Распределённое	Сбалансир.	Централиз.	Сбалансир.
Тестируемость	Низкая	Высокая	Средняя	Низкая	Высокая

Выбор микросервисной или сервис-ориентированной архитектуры для проектируемой системы нецелесообразен ввиду их избыточной сложности и неоправданных затрат на управление большим числом сервисов [38], что не соответствует относительно небольшому объёму функционала разрабатываемого решения. Монолитная архитектура, несмотря на высокую производительность и простоту реализации, ограничивает возможности масштабирования и затрудняет последующие модификации системы. Модель MVC также менее предпочтительна из-за потенциальных трудностей с распределением нагрузки на контроллер и избыточной сложности в сравнении с логикой взаимодействия в системе.

Клиент-серверная архитектура является наиболее оптимальным вариантом решения между требованиями к производительности, модульности, масштабируемости и скоростью разработки [1].

2.2.2 Выбранная архитектура и её компоненты

На основе требований к системе и анализа альтернативных подходов была выбрана клиент-серверная архитектура, а необходимость разделения приложения, на котором находится сервер, и базы данных обеспечивается в трехуровневой архитектуре.

Подобная архитектура включает в себя следующие компоненты:

1. *Клиентский уровень* – интерфейс пользователя, реализованный в виде веб-приложения на базе React [65], выбранного в ходе предыдущего исследования. Данный уровень отвечает за сбор параметров запроса от

пользователя, их отправку на сервер и отображение полученных результатов.

2. *Серверный уровень* – сервер приложений на базе FastAPI [54], отвечающий за обработку запросов пользователя, выполнение бизнес-логики, а также за взаимодействие с базой данных.
3. *Уровень данных* – база данных MongoDB [72], размещённая на отдельном сервере. Она обеспечивает хранение неструктурированной информации о компьютерных играх, полученных через API с платформ Steam и SteamSpy [68; 71].

Такая система позволит чётко *разграничить функциональные обязанности между уровнями системы* (клиентский уровень отвечает только за взаимодействие с пользователем и визуализацию данных, серверный - только за бизнес-логику приложения), обеспечить возможность *параллельной разработки уровней* с использованием *различных технологий* и при необходимости реализовать *вертикальное масштабирование серверной части и горизонтальное - базы данных*.

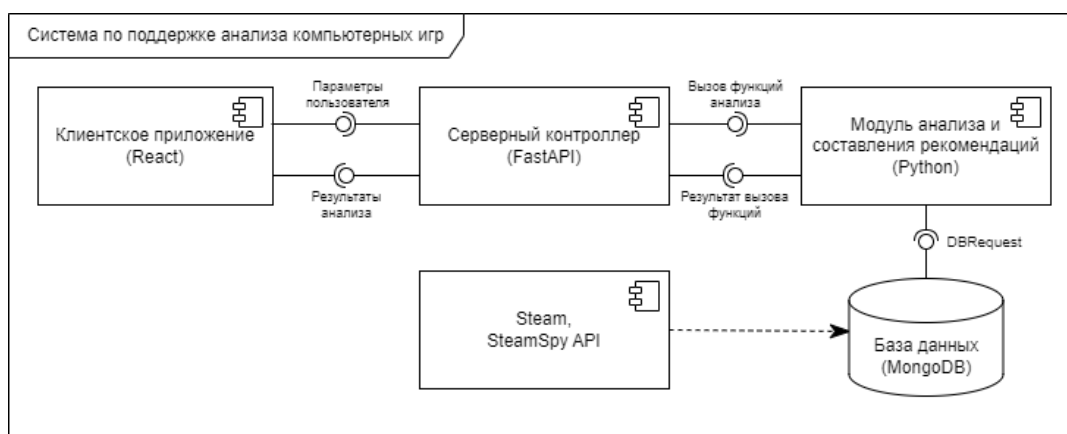


Рис.2.3. Диаграмма компонентов системы по поддержке анализа видеоигр

На диаграмме 2.3 показано взаимодействие компонентов проектируемой системы в рамках трёхуровневой архитектуры. Клиентское приложение отправляет параметры через REST API на сервер, где производится обработка полученных данных о вводимой игре и взаимодействие с базой данных, получаемой информацией внешними API (Steam и SteamSpy) [68; 71]. После обработки результатов сервер возвращает ответ в виде составленных рекомендаций по улучшению игрового продукта клиенту для визуализации. Подробнее о том, как работает серверный контроллер и модуль анализа, говорится в следующих разделах исследования.

2.3 Технологический стек

2.3.1 Критерии выбора технологий

Проектируемая система рейтингования компьютерных игр ориентирована в первую очередь на скорость отклика и простоту интерфейса для удержания пользователя, поэтому она должна соответствовать следующим требованиям:

- 1) *Высокая производительность и отзывчивость*, которые заключаются в минимальном времени отклика, оптимизированной обработке данных на сервере и возможностью работы в условиях ограниченных ресурсов;
- 2) *Удобство использования и привлекательность сервиса для пользователей*. Это включает минимальное количество шагов для получения результата и отсутствие необходимости регистрации;
- 3) *Масштабируемость компонентов* с целью возможного расширения функционала, подключения новых алгоритмов и поддержки различных форматов входных данных;
- 4) *Поддержка современных ML-алгоритмов*, что подразумевает интеграцию библиотек машинного обучения (scikit-learn) и возможность обработки потоковых данных в реальном времени;
- 5) *Совместимость* со стандартными API (REST), СУБД и интеграция с платформами цифровой дистрибуции (Steam API);
- 6) *Экономическая эффективность*, включающая невысокую стоимость хостинга и обслуживания и энергоэффективность решений.

На основе данных требований производится дальнейший выбор технологического стека, архитектурного паттерна и системы управления базами данных.

2.3.2 Выбор языка программирования

Для реализации системы рейтингования компьютерных игр был выбран язык *Python* [56], как наиболее полно соответствующий техническим, аналитическим и эксплуатационным требованиям проекта.

С точки зрения *производительности и отзывчивости*, Python позволяет реализовать высокоэффективные алгоритмы обработки данных благодаря оптимизированным библиотекам NumPy, Pandas, scikit-learn, которые используют нативный C/C++-код. Это обеспечивает быструю обработку больших массивов данных без необходимости вручную управлять памятью, как в C++.

Система должна обеспечивать *поддержку современных ML-алгоритмов*, в том числе кластеризации, рекомендательных и ранжирующих моделей. Python является отраслевым стандартом в области машинного обучения: он поддерживает ключевые фреймворки `scikit-learn`, `XGBoost`, `LightGBM`, `TensorFlow` и `PyTorch`. Кроме того, возможности Python по работе с текстом (через `spaCy`, `NLTK`, `Sentence Transformers`) делают его оптимальным выбором в случае анализа семантики и построения признаков по текстовым описаниям игр.

Важным требованием является *масштабируемость компонентов* – Python хорошо интегрируется с REST API, поддерживает микросервисную архитектуру через такие инструменты, как `FastAPI`, `Flask`, `Django`, и легко встраивается в облачные и серверные инфраструктуры (например, через `Docker` и `Gunicorn`).

Этот язык также обеспечивает *экономическую эффективность* разработки: он позволяет быстро создавать MVP, повторно использовать готовые модули, а также разворачивать системы на недорогих VPS-хостингах (например, с помощью `uWSGI` и `Nginx`). Наличие обширного открытого сообщества значительно снижает затраты на сопровождение и поиск решений при возникновении технических проблем.

В сравнении с альтернативами, Java предлагает устойчивую экосистему и строгую типизацию, но требует большего объема кода и ресурсов на разработку и развёртывание системы. C++ обеспечивает максимальную производительность, однако сложность отладки, отсутствие высокоуровневых ML-библиотек и высокая стоимость поддержки делают его мало пригодным для задач, связанных с быстрым прототипированием и анализом данных.

Таким образом, выбор Python обусловлен его:

- широкой поддержкой в области анализа данных и машинного обучения;
- простотой интеграции с внешними API и цифровыми платформами (например, `Steam Web API`);
- возможностью быстрой разработки и отладки прототипов;
- гибкостью архитектурных решений и удобством для масштабирования.

Такой выбор обеспечивает баланс между вычислительной эффективностью, простотой сопровождения и качеством пользовательского взаимодействия, что критически важно для достижения целей проекта.

2.3.3 Выбор фреймворка для веб-разработки

Обратим особое внимание на слабые стороны Vue.js и Angular, а также на ключевые различия между этими фреймворками (см. табл.2.3), то можно раскрыть следующие аспекты:

- в *производительности* фреймворк Vue.js уступает своим аналогам, поскольку в сложных и динамичных приложениях может возникнуть эффект перенагрузки. Angular иногда требует больше ресурсов, что может замедлить производительность, особенно на мобильных устройствах;
- для React и Angular доступно больше *документации*, руководств и готовых решений для распространенных проблем. Vue.js, хотя и пользуется популярностью, все еще меньше по сравнению со своими аналогами;
- React и Angular показывают схожие результаты в *интеграции с сервером*, позволяя разработчикам использовать различные подходы, такие как REST и GraphQL. Реализация аутентификации и обработки данных с сервера в Vue.js требует больше ручной работы;
- React и Vue.js предлагают высокую *гибкость в архитектуре приложений*. Разработчики могут самостоятельно выбирать технологии, добавлять в функциональность веб-приложения индивидуальные модули и разные визуальные компоненты;
- Vue.js и React считаются более гибкими в вопросах модульности и повторного использования *компонентов*. Это особенно полезно для проектов, в которых есть много взаимосвязанных объектов. Angular же требует разработки больших модулей с использованием реактивного подхода, что иногда становится затруднительным для небольших проектов. Тем не менее, для крупных и сложных приложений Angular может значительно упростить управление зависимостями, используя Dependency Injection[51];
- React традиционно имеет *большую экосистему библиотек* и инструментов, которые упрощают различных аспекты разработки, от состояния до работы с роутингом (переключение между страницами);
- как React, так и Angular предлагают возможности для *мобильной разработки*. Однако React Native является более зрелым инструментом для создания приложений для iOS и Android. Vue.js, в свою очередь, не имеет похожего решения на уровне фреймворка.

Таблица 2.3

Сравнение инструментальных средств для разработки веб-приложения

Название инструментального средства	React[65]	Angular[48]	Vue.js[73]
Производительность	+	+-	+-
Сообщество и поддержка	+	+	+
Интеграция с сервером	+	+	+-
Гибкость	+	-	+
Модульность и повторное использование компонентов	+	-	+
Наличие библиотек и инструментов	+	+-	+
Поддержка мобильной разработки	+	+	-

Таким образом, на клиентской стороне основным выбором является библиотека *React*[65], как это подтверждается результатами сравнения. Данный инструмент предоставляет динамическую маршрутизацию через *React Router*, управление состоянием компонентов через хуки и возможность построения современного пользовательского интерфейса с использованием *CSS-модулей*[20]. Она также является оптимальным инструментом в случае необходимости разработки мобильного приложения.

2.3.4 Выбор серверной платформы

При выборе серверной платформы для реализации *REST API* необходимо учитывать требования к производительности, масштабируемости, удобству интеграции с модулями анализа данных, а также совместимость с выбранным языком программирования (в данном случае – *Python*).

Наиболее подходящими решениями для серверной разработки являются:

1. *Node.js* [62] – событийно-ориентированная среда исполнения *JavaScript*, широко используемая в высоконагруженных веб-приложениях. Обеспечивает высокую производительность и имеет богатую экосистему (*Express.js*, *NestJS*). Однако, несмотря на преимущества в обработке асинхронных запросов, *Node.js* требует дополнительных усилий при интеграции с *Python*-библиотеками анализа данных, что снижает его применимость в данном проекте.

2. *Django (Python)* [3] – полнофункциональный фреймворк с встроенной ORM, системой авторизации и административной панелью. Отличается надёжностью и модульностью, но ориентирован скорее на разработку традиционных веб-сайтов и менее удобен для реализации легковесных REST API и асинхронной обработки. В нашей же системе система авторизации отсутствует.
3. *Spring Boot (Java)* [66] – промышленный фреймворк для разработки REST API и микросервисов. Предоставляет высокую масштабируемость и богатую экосистему, но требует значительных усилий по настройке и разработке, а также не соответствует выбранному стеку на Python.

С учётом требований проекта (см. подраздел 2.3), в частности необходимости высокой скорости отклика, асинхронной обработки запросов, простоты интеграции с Python-алгоритмами анализа данных и поддержки REST API, оптимальным выбором является фреймворк *FastAPI* [54].

Среди преимуществ FastAPI:

- *асинхронная архитектура*, основанная на `async/await`, которая позволяет обрабатывать множество параллельных запросов с минимальной задержкой;
- *автоматическая генерация OpenAPI-документации*, что упрощает отладку, поддержку и интеграцию с внешними клиентами;
- *совместимость с машинным обучением*, благодаря работе в одной среде с Python-алгоритмами (например, `scikit-learn`, `NumPy`, `spaCy` и др.), без необходимости сериализации/десериализации данных между языками;
- *высокая производительность* – FastAPI по скорости обработки не уступает `Node.js` и `Go` в задачах REST API, согласно независимым бенчмаркам;
- *низкий порог входа* (в контексте ресурсов), особенно важный при ограниченных ресурсах разработки.

Таким образом, выбор FastAPI как серверного фреймворка обеспечивает баланс между производительностью, удобством разработки и интеграцией с аналитическим ядром системы. Он позволяет реализовать масштабируемый и отзывчивый backend для сервиса ранжирования игр, соответствующий целям и архитектуре проекта.

2.3.5 Выбор системы управления базой данных

Для хранения структурированных данных может быть рассмотрена СУБД *PostgreSQL*[74], которая, помимо прочего, поддерживает сложные реляционные запросы. В качестве дополнительного инструмента для повышения эффективности поиска схожих объектов возможно использование *Elasticsearch*, который оптимизирован для полнотекстового поиска и обработки больших массивов информации.

Однако для хранения данных системы был выбран документоориентированный подход с использованием *MongoDB*[72], что обусловлено спецификой игровых данных и следующими ключевыми факторами:

- возможность хранить большие объемы неструктурированных и полуструктурированных данных (описания игр, отзывы пользователей в формате JSON);
- простое добавление новых полей без изменения схемы, что особенно полезно при добавлении новых характеристик игр;
- поддержка вложенных документов для комплексного представления игровых характеристик;
- встроенная поддержка горизонтального масштабирования (шардирование);
- отсутствие необходимости в сложных JOIN-операциях;

Особенности проектирования базы данных при этом включают в себя следующие аспекты:

- отсутствие сложных транзакций (операции атомарны в пределах документа);
- максимальный размер документа: 16MB;
- глубина вложенности: не более 3 уровней;
- частые поля вынесены на верхний уровень.

2.4 Проектирование диаграммы сущность-связь

Хранилище реализовано на *MongoDB* (версия 6.0)[72] с документоориентированным подходом. Основные коллекции:

- 1) *GameStoreData* — хранит базовые метаданные игр (название, разработчик, цена), примерно 50 тыс. документов;

- 2) *SteamSpyData* — содержит статистику игровой активности для каждой игры. Предполагается еженежельное обновление данных для более точных рейтингов;
- 3) *GameReviews* — хранит отзывы пользователей о играх;
- 4) *EncodingSchema* — справочник по всем собранным данным (теги, категории, какие вообще есть и тд);
- 5) *GameDataCluster* – содержит информацию о кластерах, обновляется при появлении новых документов;
- 6) *GameFeatures* – содержит предобработанные ML-признаки, включая векторизованные теги, жанры, описания, а также нормализованные числовые данные.

Структура записей в коллекциях соответствует исходному формату JSON, без строгой схемы (schema-less), что позволяет эффективно работать с данными, отличающимися по структуре. Для документоориентированной СУБД применяется *дениормализованная* схема (см.2.4) с контролируемым дублированием данных, что обусловлено отсутствием необходимости в сложных JOIN-операциях, требованиями к скорости чтения и частыми изменениями структуры данных.

Схема отражает гибридный подход: синие документы содержат сырые данные из внешних источников (Steam, SteamSpy API), красные - результаты аналитической обработки. Отсутствие строгих связей между коллекциями характерно для документоориентированного подхода MongoDB.

В таблице 2.4 представлены основные параметры, получаемые из Steam и SteamSpy для анализа игр:

2.5 Математические модели и алгоритмы

2.5.1 Предобработка данных и формирование признаков

Для обеспечения корректной работы алгоритмов составления рейтинга и оценки доли продаж игр необходима тщательная предобработка исходных данных.

Текстовые данные, полученные с платформы Steam, часто содержат HTML-разметку и лишние спецсимволы, что затрудняет их дальнейшую обработку. Для удаления HTML-тегов используется парсер *BeautifulSoup*, оставляющий только чистый текст описания игр. Дополнительно текст приводится к нижнему регистру

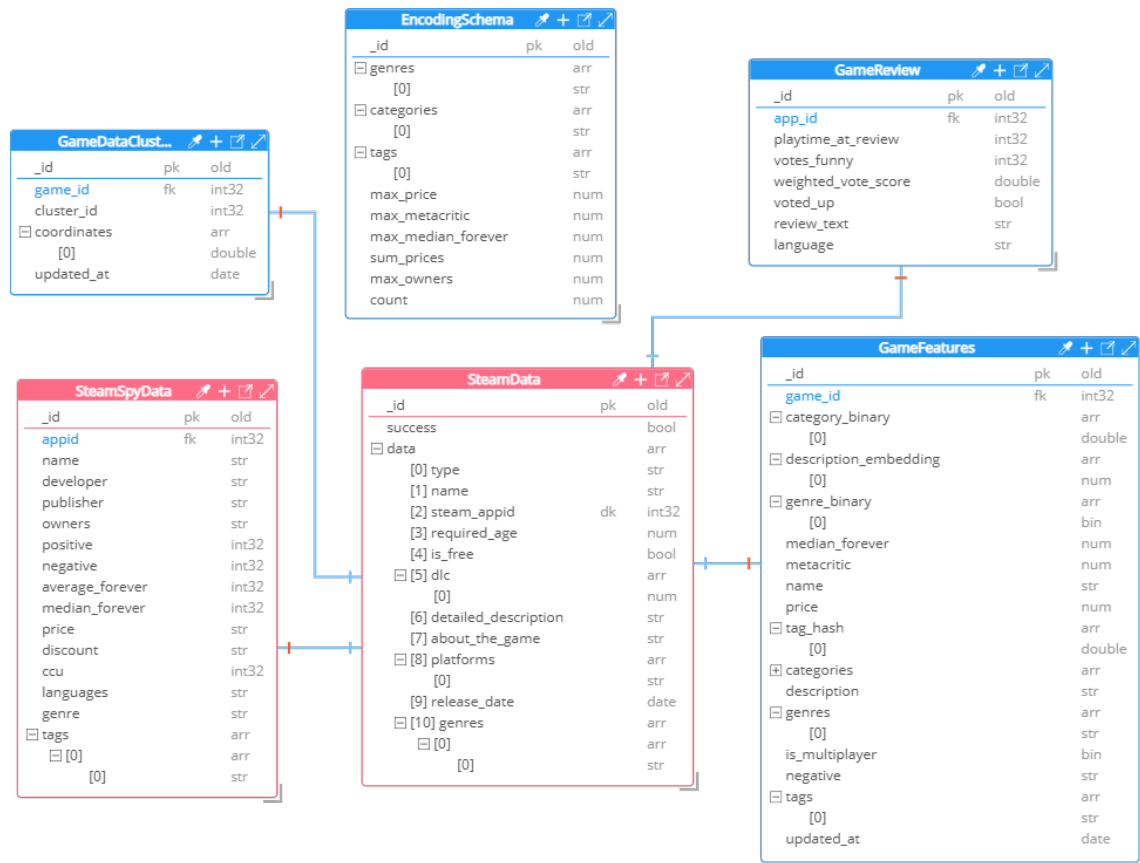


Рис.2.4. Логическая модель документоориентированной БД GamesAnalyzer с цветовой маркировкой источников данных

и очищается от стоп-слов с использованием параметра `stop_words='english'` в методах векторизации.

Поскольку различные игры могут иметь схожие жанры, которые обозначаются разными словами (например, "Racing" и "Driving"), введена процедура приведения жанров к унифицированным категориям с помощью словаря синонимов.

Числовые характеристики, такие как цена (*price*), оценка Metacritic (*metacritic*) и среднее время прохождения (*median_forever*), могут отличаться друг от друга на несколько порядков. Поэтому, для обеспечения их сопоставимости, применяется минимаксная нормализация по формуле:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}, \quad (2.1)$$

где x – исходное значение признака, а $\min(x)$ и $\max(x)$ – минимальное и максимальное значения признака в выборке соответственно. Отдельно нормализуются также бинарные признаки, такие как поддержка VR (*is_vr*) и наличие многопользовательского режима (*is_multiplayer*), которые остаются в формате 0 или 1. Для показателя

Таблица 2.4

Основные параметры игр из Steam и SteamSpy

Данные из Steam	Описание	Тип данных
<i>detailed_description</i>	Описание выбранной игры	string
<i>demos</i>	Наличие демо-версии игры, ID и описание демо	list или string
<i>metacritic</i>	Оценка metacritic (от 0 до 100)	integer или string
<i>categories</i>	Категории предоставленной игры	list
<i>release_date</i>	Дата выпуска игры (формат: "DD MMM, YYYY")	string
<i>positive</i>	Количество позитивных отзывов	integer
<i>negative</i>	Количество негативных отзывов	integer
<i>owners</i>	Количество владельцев игры (например, "1000000..2000000")	string
<i>price</i>	Текущая стоимость игры (\$)	integer или string
<i>tags</i>	Пользовательские теги игры	list (строки)
<i>median_forever</i>	Среднее время игры за всё время	integer (в минутах)

отношения положительных отзывов к общему количеству (*positive_ratio*) нормализация не требуется, так как он уже находится в диапазоне $[0; 1]$. После нормализации числовые признаки обрабатываются с помощью стандартизации через *StandardScaler*, что приводит данные к нулевому среднему и единичной дисперсии при необходимости более сложной обработки.

При отсутствии числовых данных соответствующие признаки заполняются нулями, что эквивалентно минимаксному значению в рамках проводимой нормализации. Для текстовых данных в подобном случае используется пустая строка.

Для преобразования текстовых атрибутов компьютерных игр в числовые представления, пригодные для машинного обучения, разработан комплексный метод, включающий несколько этапов обработки.

Векторизация *текстовых описаний игр* осуществляется с использованием архитектуры трансформеров. Выбор пал на предобученную модель all-MiniLM-L6-v2, что обусловлено оптимальным соотношением между размерностью пространства признаков и качеством семантического представления, а также

способностью модели учитывать контекстные взаимосвязи между словами – это помогает определять более точно похожие игры.

Для жанров и категорий применяется метод мульти-хот кодирования (Multi-Hot Encoding), формально описываемый следующим образом:

$$f_{MLE} : G \rightarrow \{0,1\}^k, \quad (2.2)$$

где G – множество всех возможных жанров, k – мощность словаря уникальных жанров. Каждая компонента выходного вектора соответствует наличию (1) или отсутствию (0) конкретного жанра у игры.

Особенность обработки тегов заключается в учете их частотной значимости. Формально процесс включает:

1. Построение словаря тегов по всем играм выборки (поле tags в коллекции EncodingSchema).
2. Формирование частотного вектора для каждой игры:

$$\mathbf{v}_i = (f_{t_1}^i, \dots, f_{t_m}^i), \quad (2.3)$$

где f_{t_i} – частота встречаемости тега t_j для игры i .

3. Нормализацию векторов по L2-норме:

$$\mathbf{v}_i^{norm} = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2}, \quad (2.4)$$

где $\|\mathbf{v}\|_2$ – евклидова норма вектора $\mathbf{v}$ с координатами $v_1, v_2, \dots, v_n$, вычисляемая как:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2} \quad (2.5)$$

Для повышения качества оценки игр дополнительно анализируются отзывы пользователей. Параметры отзывов приведены в таблице 2.5.

2.5.1.1 Обоснование выбора методов

Предложенные выше методы обеспечивают сохранение семантических отношений между текстами в описаниях игр и учёт популярности и значимости отдельных характеристик, в отличие от векторизации TF-IDF, учитывающую только частоту встречаемости тех или иных слов в документах (таких, как описание или теги).

Анализ представленных методов векторизации (см. таб. 2.6) демонстрирует соответствие их характеристик текущим задачам. Рассмотрим критерии выбора методов векторизации и их значимость:

Таблица 2.5

Основные параметры отзывов пользователей

Параметр отзыва	Описание	Тип данных
<i>voted_up</i>	Отметка "Рекомендую"	bool
<i>playtime_at_review</i>	Время игры на момент написания отзыва (минуты)	integer
<i>votes_funny</i>	Количество пользователей, сочтящих отзыв забавным	integer
<i>weighted_vote_score</i>	Оценка полезности отзыва (от 0 до 1)	float
<i>review_text</i>	Текст отзыва	string
<i>language</i>	Язык отзыва	string

Таблица 2.6

Сравнение методов векторизации

Метод	Размерность	Сохранение семантики	Обработка множеств. категорий
Sentence Transformer	384	Высокое	Нет
Multi-Hot Encoding	Переменная	Нет	Да
Частотные векторы	Переменная	Частичное	Да

- фиксированная размерность упрощает интеграцию и обработку данных, а постоянный размер векторов гарантирует совместимость между разными компонентами системы,
- сохранение семантики критично в основном для описания игр, поскольку позволяет выявлять смысловую близость даже при разном словесном оформлении (например, "шутер от первого лица" достаточно близок к "Action"),
- обработка множественных категорий обеспечивает корректное представление игр, сочетающих несколько характеристик (например, жанры "RPG" и "Action" теги "Открытый мир" и "Кооператив"). Этот подход повышает точность рекомендаций за счёт анализа пересечений категорий.

Для текстовых описаний важным является баланс между сохранением семантических отношений и вычислительной эффективностью, что достигается использованием *Sentence Transformer* с фиксированной размерностью 384. В случае жанров и категорий применяется бинарное кодирование, специально адаптированное для работы с множественными категориальными признаками, где принципиальное

значение имеет факт принадлежности к категории, а не степень выраженности. *Частотный метод для тегов*, в свою очередь, учитывает весовые коэффициенты популярности, что обеспечивает более тонкую градацию характеристик игр. Подобное комбинирование подходов позволяет максимально полно учитывать специфику каждого типа признаков при формировании итоговых векторных представлений.

Таким образом, на выходе этапа предобработки формируются матрица векторизованных текстовых данных и отдельно матрица нормализованных числовых признаков, которые далее используются для расчета сходства между играми и оценки их доли продаж.

2.5.2 Методы анализа сходства игр

2.5.2.1 Парные метрики близости

Для текстовых признаков применяется косинусное сходство между векторизованными представлениями, для числовых – нормализация и вычисление абсолютной разницы. Далее все частные оценки объединяются в *итоговую оценку схожести с помощью свертки критериев* и использования следующих метрик:

- А. Для текстовых признаков (описания, теги, жанры) применяется *косинусное сходство*:

$$\text{similarity}_{\text{text}} = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| |\vec{B}|} \quad (2.6)$$

- В. Для числовых характеристик (цена, рейтинг) используется:

– *Евклидово расстояние*:

$$d_E(\vec{x}, \vec{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.7)$$

Программная реализация предусматривает расчёт следующих частных метрик:

- косинусное сходство описаний;
- косинусное сходство жанров, тегов и категорий;
- разницу нормализованных числовых признаков.

2.5.2.2 Кластерный анализ игр

Игры с похожими тегами, жанрами и описаниями далеко не всегда оказываются действительно схожими по восприятию пользователями, что делает задачу их

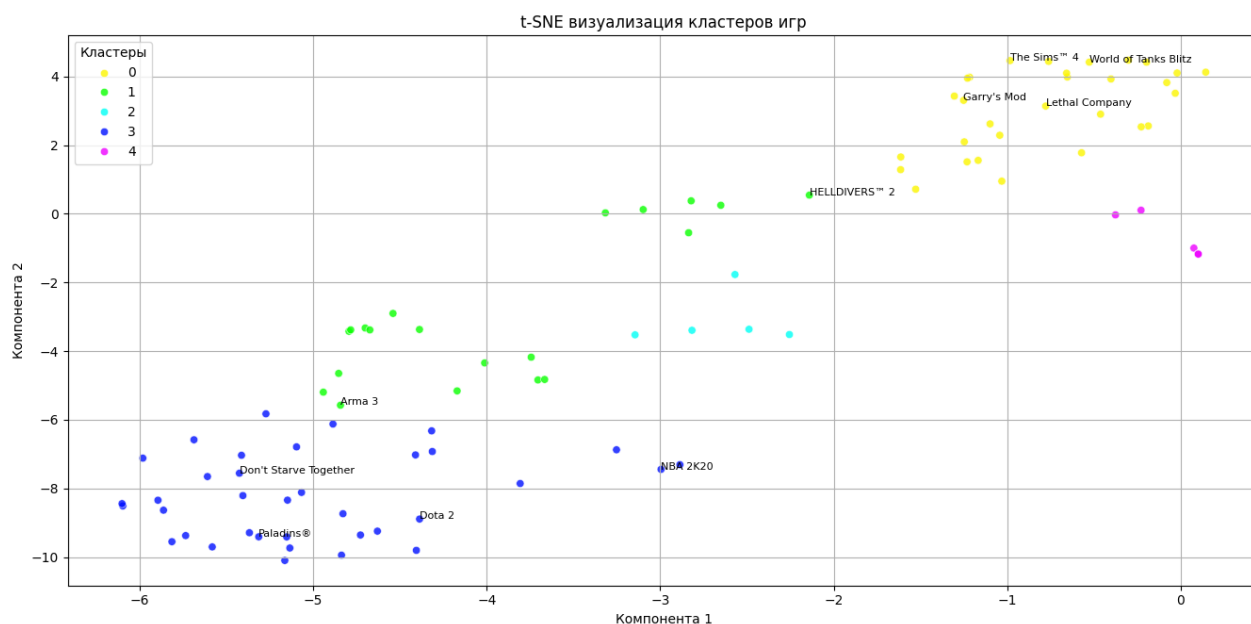


Рис.2.5. t-SNE визуализация кластеров компьютерных игр

априорной категоризации сложной. В силу отсутствия заранее известного фиксированного количества классов, на которые можно корректно разделить игры в нашей базе данных, в качестве одного из этапов анализа была выбрана кластеризация. Она помогает *выявить естественные группы игр*, основываясь на их признаках без обучения на заранее размеченных классах.

Подход к кластеризации компьютерных игр также основывается на объединённых текстовых и числовых характеристиках, а именно: на векторизованных описаниях, жанрах, тегах и нормализованных ценах, оценках Metacritic и медианном времени игры.

Для кластеризации была выбрана модель *K-means++*. Основными причинами стали её простота в реализации, высокая скорость работы на относительно больших объёмах данных и минимальная потребность в подборе дополнительных гиперпараметров по сравнению с альтернативными методами (например, DBSCAN требует выбора чувствительных к масштабу параметров ϵ и минимального числа точек) [45; 60]. Кроме того, *K-means++* позволяет повысить стабильность итоговой кластеризации за счёт более удачного выбора начальных положений центров кластеров по сравнению со случайным размещением.

Например, на экспериментальной выборке из 100 игр оптимальным оказалось разделение на 5 кластеров. Для визуализации результатов кластеризации была проведена двумерная проекция высокоразмерного признакового пространства с использованием метода *t-SNE*, что позволило наглядно оценить плотность и распределение группировок игр (рис. 2.5).

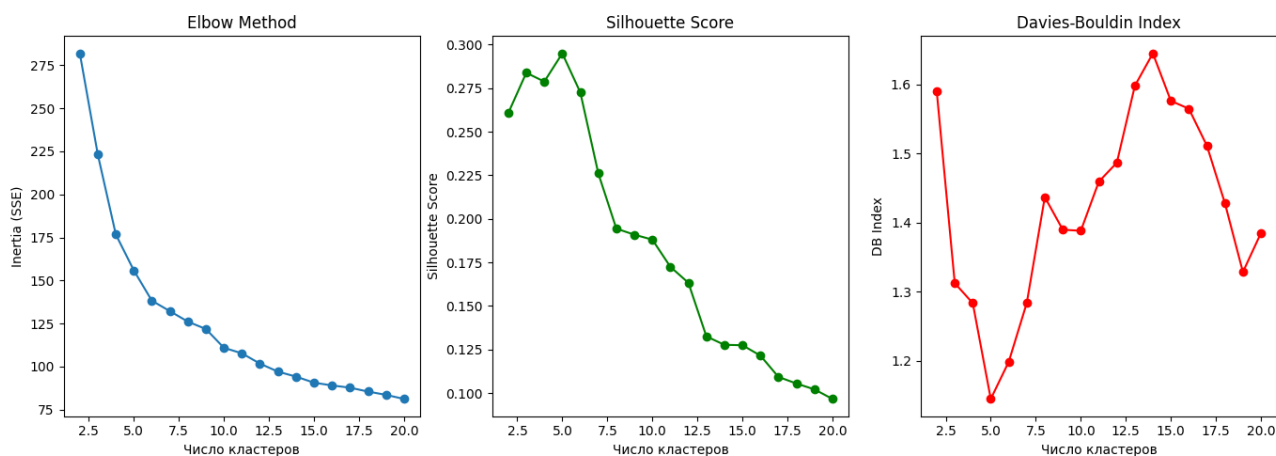


Рис.2.6. Определение оптимального числа кластеров методом локтя

При вводе новой игры в систему её признаки векторизуются аналогично этапу кластеризации, после чего определяется кластер, к которому наиболее вероятно принадлежит данная игра. Таким образом, дальнейшая работа системы ведётся исключительно с играми из этого кластера, что позволяет существенно снизить вычислительную нагрузку и сосредоточиться на более релевантных объектах. Этот подход обеспечивает возможность быстрого поиска наиболее схожих игр и оценки их доли продаж в сегменте этих игр.

2.5.2.3 Актуализация и использование кластерного анализа

Важным аспектом разработки системы кластерного анализа является обеспечение её актуальности при изменении данных. Кластеризация выполняется не при каждом запуске системы, а лишь в следующих случаях:

- в базе данных появились новые игры, которые не были распределены по кластерам ранее;
- с момента последней кластеризации прошло более 7 дней.

В случае обнаружения новых данных или истечения срока актуальности кластеров запускается процедура повторной кластеризации. При этом, прежде чем распределить данные по кластерам, система повторно оценивает оптимальное количество кластеров с помощью *метода локтя* и коэффициента силуэта (*Silhouette Score*), а также индекса Дэвиса-Болдина (*Davies-Bouldin Index*) (рис. 2.6). Данный подход позволяет учесть возможные изменения в структуре данных и обновить конфигурацию кластеров.

После выполнения кластеризации данные о местоположении каждого объекта (координаты в пространстве признаков, полученные с помощью *t-SNE*) и

метки кластеров сохраняются в коллекции `GameDataCluster`. В этой коллекции хранятся также временные метки последней кластеризации, что позволяет системе отслеживать актуальность данных и принимать решение о необходимости их пересчета.

Таким образом, в процессе взаимодействия пользователя с системой кластеризация не выполняется заново. Вместо этого, когда пользователь вводит данные новой игры для анализа, система:

- векторизует данные новой игры с применением тех же алгоритмов и весов, что и при исходной кластеризации;
- проецирует вектор новой игры в пространство признаков, используя сохранённую модель *t-SNE*;
- определяет координаты новой игры и находит её наиболее близкий кластер, используя алгоритм *k-Nearest Neighbors (k-NN)*, обученный на сохранённых координатах *t-SNE*.

В результате каждый новый объект (игра) получает не только номер своего кластера, но и координаты в пространстве признаков, что позволяет визуализировать её расположение относительно других объектов и выявить потенциальных конкурентов. Таким образом, кластерный анализ используется не только для априорного группирования данных, но и для поиска наиболее схожих игр, что является основой для последующего этапа ранжирования игр.

2.5.3 Методы ранжирования компьютерных игр

Разработанный алгоритм ранжирования компьютерных игр по оценке доли продаж в сегменте представляет собой подход, учитывающий множество факторов и использующий современные методы анализа данных.

2.5.3.1 Формализация критериев оценки доли продаж игр

Итак, ранжирование игр происходит исключительно внутри одного кластера, поэтому сначала определяются объекты, попадающие под введённые параметры, а затем рассчитывается их оценка доли продаж относительно других игр в кластере. Это позволяет учитывать специфику различных продуктов и обеспечивает более справедливое их сравнение со схожими характеристиками.

При оценке доли продаж компьютерной игры в сегменте похожих игр учитываются следующие ключевые критерии среди рыночных показателей и пользовательских метрик:

- цена игры относительно среднерыночной в своем сегменте;
- количество продаж (медианное время игры);
- соотношение цена/качество.
- рейтинги пользователей;
- количество отзывов;
- активность сообщества.

Каждый рассматриваемый критерий нормализуется в диапазоне $[0, 1]$ для обеспечения сопоставимости различных метрик. В дальнейшем планируется учитывать и технические характеристики продуктов, используя данные из отзывов игроков с большой продолжительностью прохождения игры. Так можно получить информацию о качестве графики, геймплея, звукового сопровождения, а также особенности управления игр.

Для вычисления интегрального показателя доли продаж (sales share – SS) используется простейшая модель – аддитивная свёртка, которая представляет собой взвешенную сумму нормализованных значений критериев:

$$SS = \sum_{i=1}^n w_i \cdot x_i, \quad (2.8)$$

где:

- w_i – вес i -го критерия,
- x_i – нормализованное значение i -го критерия,
- n – общее количество критериев.

Веса критериев определяются методом парного сравнения, который был выбран из-за своей способности формализовать субъективные экспертные оценки и переводить их в количественные значения.

Прежде чем перейти к просчету весов, ознакомимся с основными критериями оценки:

1. *Доля игроков* – отношение количества игроков конкретной игры к максимальному количеству игроков среди всех игр в кластере. Показывает относительную популярность игры.
2. *Соотношение отзывов* – процент положительных отзывов от общего количества отзывов об игре. Характеризует удовлетворенность игроков.

3. *Выручка* – отношение стоимости игры к средней стоимости игр в кластере. Отражает экономическую эффективность.
4. *Активность сообщества* – отношение количества отзывов к числу игроков, купивших игру. Показывает вовлеченность сообщества.
5. *Оценка отзывов* – агрегированная метрика, рассчитываемая на основе текстов пользовательских отзывов. Учитывает долю положительных рецензий, среднюю тональность, полезность, время прохождения игры до написания отзыва и упоминания ключевых аспектов (геймплей, графика и др.). Эта оценка характеризует восприятие качества игры и информативность пользовательского опыта.
6. *Новизна игры* – отношение времени с момента релиза игры к максимальному "возрасту" среди игр кластера. Характеризует актуальность контента.

Результаты ранжирования представляются в виде рейтинговых таблиц с возможностью фильтрации и сортировки, а также графика сравнения ключевых метрик. Такой подход позволяет пользователям системы быстро оценивать положение анализируемой игры относительно конкурентов и принимать обоснованные решения по её развитию и продвижению.

2.5.3.2 Метод парного сравнения для определения весовых коэффициентов

Матрица парных сравнений которая позволяет определить относительную важность каждого критерия. В этой матрице каждый элемент a_{ij} выражает степень предпочтительности критерия i по сравнению с критерием j . Это позволяет перейти от качественных экспертных оценок к количественным весовым коэффициентам. Важным этапом является проверка согласованности полученных результатов с помощью расчета специального индекса, что обеспечивает достоверность выводов. В нашем случае данный метод применяется для определения значимости различных критериев отзывов.

Для просчета весов вышеперечисленных составляющих оценки отзывов игры была сформирована следующая матрица парных сравнений табл.2.7:

Процесс нормализации матрицы и расчета весов включает следующие шаги:

Таблица 2.7

Матрица парных сравнений для критериев оценки отзывов

	Соотн-е отзывов	Настроение	Время игры	Оценка	Ключ. слова
Соотношение отзывов	1	1/2	3/2	3	6
Настроение	2	1	2	4	8
Время игры	2/3	1/2	1	2	4
Оценка	1/3	1/4	1/2	1	2
Ключ. слова	1/6	1/8	1/4	1/2	1

1. Просуммируем значения по каждому столбцу для всех значений. Например, для доли игроков:

$$\begin{aligned}
 S_{\text{Отзывы}} &= 1 + 2 + \frac{2}{3} + \frac{1}{3} + \frac{1}{6} = 4.166 \\
 S_{\text{Настроение}} &= \frac{1}{2} + 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = 2.375 \\
 S_{\text{Время игры}} &= \frac{3}{2} + 2 + 1 + \frac{1}{2} + \frac{1}{4} = 5.25 \\
 S_{\text{Оценка}} &= 3 + 4 + 2 + 1 + \frac{1}{2} = 10.5 \\
 S_{\text{Ключ. слова}} &= 6 + 8 + 4 + 2 + 1 = 21
 \end{aligned} \tag{2.9}$$

2. Разделим каждое значение в столбце на сумму столбца для нормализации. Например, для строки "Соотношение отзывов":

$$\begin{aligned}
 \frac{y_{2,1}}{S_{\text{Отзывы}}} &= \frac{2}{4.166} \approx 0.480 \\
 \frac{y_{2,2}}{S_{\text{Настроение}}} &= \frac{1}{2.375} \approx 0.421 \\
 \frac{y_{2,3}}{S_{\text{Время игры}}} &= \frac{2}{5.25} \approx 0.381 \\
 \frac{y_{2,4}}{S_{\text{Оценка}}} &= \frac{4}{10.5} \approx 0.381 \\
 \frac{y_{2,5}}{S_{\text{Ключ. слова}}} &= \frac{8}{21} \approx 0.381
 \end{aligned} \tag{2.10}$$

3. Расчет весов критериев:

$$\begin{aligned}
 w_{\text{Отзывы}} &= (0.240 + 0.211 + 0.286 + 0.286 + 0.286)/5 \approx 0.262 \\
 w_{\text{Настроение}} &= (0.480 + 0.421 + 0.381 + 0.381 + 0.381)/5 \approx 0.409 \\
 w_{\text{Время игры}} &= (0.360 + 0.211 + 0.190 + 0.190 + 0.190)/5 \approx 0.228 \quad (2.11) \\
 w_{\text{Оценка}} &= (0.720 + 0.421 + 0.381 + 0.095 + 0.095)/5 \approx 0.342 \\
 w_{\text{Ключ. слова}} &= (0.120 + 0.053 + 0.048 + 0.024 + 0.048)/5 \approx 0.059
 \end{aligned}$$

4. Проверка согласованности выполняется в несколько шагов. Сначала считается среднее значение взвешенной суммы, деленное на вес – $\lambda_{\max}$:

$$\begin{aligned}
 \lambda_{\max} &= \frac{1}{5} \left(\frac{0.262 \cdot 4.166}{1} + \frac{0.409 \cdot 2.375}{1} \right. \\
 &\quad \left. + \frac{0.228 \cdot 5.25}{1} + \frac{0.342 \cdot 10.5}{1} + \frac{0.059 \cdot 21}{1} \right) \quad (2.12) \\
 &\approx 5.21
 \end{aligned}$$

затем Индекс согласованности (ИС):

$$\text{ИС} = \frac{5.21 - 5}{4} = 0.052, \quad (2.13)$$

и Отношение согласованности (ОС):

$$\text{ОС} = \frac{0.052}{1.12} \approx 0.046 < 0.1. \quad (2.14)$$

Если ≤ 0.1 , матрица считается согласованной.

2.5.3.3 Применение весовых коэффициентов на практике

Основой для сравнения игр служит линейная комбинация нормализованных показателей:

$$\begin{aligned}
 \text{Доля продаж} = \sum_i w_i S_i = & 0.262 S_{\text{игр}} + 0.409 S_{\text{отз}} + 0.228 S_{\text{выр}} + \\
 & 0.342 S_{\text{акт}} + 0.059 S_{\text{нов}} \quad (2.15)
 \end{aligned}$$

где коэффициенты отражают относительную важность каждого фактора, а $S_{\dots}$ представляют собой нормализованные значения соответствующих критериев (например, косинусное сходство для текстовых описаний или относительная разница в цене).

На основе рассчитанных оценок система выполняет:

- сортировку игр по убыванию оценки доли продаж,
- автоматический учет приоритетов, устанавливая наибольшую важность отзывам (40.9%) и активности сообщества (34.2%), чем доле игроков (26.2%) или числовым показателям (22.8%).

При этом разработанная архитектура позволяет корректировать весовые коэффициенты при изменении бизнес-требований и проводить регулярную проверку согласованности оценок ($OC \leq 0.1$) для исключения субъективных перекосов.

Рассмотрим сравнение игр со следующими характеристиками:

- Доля игроков: 0.75 (нормализованная)
- Положительные отзывы: 0.9
- Уровень выручки: 0.6
- Активность сообщества: 0.4
- Новизна контента: 0.3

При таких показателях итоговая комплексная оценка составит:

$$\begin{aligned} \text{Доли продаж в своем сегменте игр} = & 0.262 \cdot 0.75 + 0.409 \cdot 0.9 + 0.228 \cdot 0.6 + \\ & + 0.342 \cdot 0.4 + 0.059 \cdot 0.3 \approx 0.72 \end{aligned} \quad (2.16)$$

Такой подход позволяет сохранять баланс между различными аспектами сравнения, обеспечивая релевантность построенных рейтингов с сохранением вычислительной эффективности.

2.5.3.4 Парето-оптимальность в ранжировании

Одним из фундаментальных подходов в многокритериальном анализе является метод Парето-ранжирования, основанный на концепции *доминирования*. В отличие от методов, требующих агрегирования показателей в единую свёрнутую оценку, данный метод рассматривает сравнение объектов напрямую по множеству признаков без необходимости задания весов.

Объект (в нашем случае – игра) считается Парето-доминирующим, если он не уступает другому объекту ни по одному из критериев и превосходит хотя бы по одному. Формально, пусть заданы объекты $A = (a_1, a_2, \dots, a_n)$ и $B = (b_1, b_2, \dots, b_n)$, тогда $A \succ B$ по Парето, если:

$$\forall i (a_i \geq b_i) \quad \text{и} \quad \exists j (a_j > b_j) \quad (2.17)$$

На первом этапе формируется *Парето-фронт* первого уровня – множество игр, которые не доминируются ни одной другой. Далее из общего множества игр эти

объекты исключаются, и на оставшемся множестве определяется следующий фронт и т.д. Таким образом, получается иерархия уровней: чем ближе игра к первому фронту, тем она "лучше" с точки зрения сбалансированности характеристик.

Такой подход не требует задания весов и нормализации признаков, обеспечивает устойчивую интерпретацию результатов в задачах с конфликтующими критериями (например, высокая цена vs высокая популярность) и особенно полезен при оценке уникальных или инновационных игр, которые выделяются по нескольким признакам, но не обязательно лидируют по всем.

Однако у него существуют ограничения, которые следует учитывать. Среди них:

- отсутствие возможности упорядочить все объекты линейно – результатом является скорее частичный порядок;
- при большом числе признаков или объектов множество Парето-оптимальных решений может быть чрезмерно широким (размытие фронта);
- не все пользователи готовы воспринимать «равнозначные» рейтинги без однозначного лидера.

Парето-ранжирование может служить хорошим ориентиром (золотым стандартом) в системах, где важна сбалансированность характеристик, а не достижение максимума по одному критерию. В контексте анализа оценки доли продаж компьютерных игр этот подход может быть полезен для оценки новинок или инди-проектов, у которых сильные и слабые стороны распределены неравномерно, однако мы его немного интерпретируем, не удаляя недоминирующие варианты, а снижая вес их оценок, поскольку для нас важно упорядочить объекты именно линейно, понимая, какая игра идёт за какой.

2.5.3.5 Качественная и количественная важность критериев

В системе реализован гибридный подход к управлению важностью критериев [8], сочетающий:

- *качественное ранжирование* — пользователь задаёт порядок обработки критериев через интерфейс. Основываясь на теории важности критериев [7], качественное упорядочение определяется через строгие математические условия:

$$y^{AB} P y \quad \text{при} \quad y_i > y_j \quad \text{для} \quad i \in A, j \in B, \quad (2.18)$$

где y^{AB} – модифицированная векторная оценка, P – отношение строгого предпочтения. Это позволяет ранжировать критерии даже для неаддитивных функций ценности.

- *количественная настройка* — автоматический расчёт весов на основе позиции критерия в ранжировании. Используется модифицированная формула, учитывающая экспоненциальное убывание важности:

$$w_i = \frac{2^{(n-k_i)}}{\sum_{j=1}^n 2^{(n-k_j)}}, \quad (2.19)$$

где k_i – позиция критерия в пользовательском списке, n – общее число критериев.

$$v(y) = \sum_{i=1}^m w_i v_i(y_i), \quad (2.20)$$

а для неаддитивных случаев применяется нормировка через условия взаимной независимости критериев.

Из работы [8] следует, что упорядочение по важности возможно даже при отсутствии аддитивности. Критерии считаются однородными, если их шкалы приведены к общей области. Для проверки согласованности весов с предпочтениями используется условие соответственных замещений:

$$\varepsilon_2 = \varepsilon_3 \quad \text{при} \quad v(y_1 + \delta_1, y_2) = v(y_1, y_2 + \varepsilon), \quad (2.21)$$

где ε – компенсирующее изменение. Его нарушение указывает на неаддитивность, но не препятствует качественному ранжированию.

2.5.4 Метод последовательного уточнения предпочтений

В системе реализован метод последовательного уточнения предпочтений по В.В. Подиновскому [8] с использованием программного обеспечения PyDASS. Метод позволяет корректно ранжировать варианты даже при отсутствии аддитивной функции ценности и включает следующие этапы:

1. Нормализацию критериев, подразумевающую, что все критерии приводятся к общей порядковой шкале Z_0 с единым направлением оптимизации (максимизацией). Для варианта $x \in X$ векторная оценка:

$$y = f(x) = (f_1(x), \dots, f_m(x)), \quad f_i : X \rightarrow Z_0. \quad (2.22)$$

2. Применение принципа Парето, в котором Варианты фильтруются по отношению доминирования:

$$x_i \text{ доминирует } x_j \Leftrightarrow \forall k \in \{1, \dots, m\} : f_k(x_i) \geq f_k(x_j) \wedge \exists k : f_k(x_i) > f_k(x_j). \quad (2.23)$$

Недоминируемые варианты образуют *множество Парето* $X^* \subseteq X$.

3. Качественное ранжирование критериев. Для i -го критерия:

$$f_i \succ f_j \Leftrightarrow \forall x \in X^* : y^{ij} P y, \quad y^{ij} = \text{перестановка } y_i \text{ и } y_j. \quad (2.24)$$

В интерфейсе системы (через окно настроек) пользователь задает строгий порядок $f_1 \succ f_2 \succ \dots \succ f_m$.

4. На основе позиции критерия в ранжировании вычисляется вес w_i по модифицированной формуле:

$$w_i = \frac{2^{(n-k_i)}}{\sum_{j=1}^n 2^{(n-k_j)}}, \quad (2.25)$$

где k_i – позиция i -го критерия, n – общее число критериев. Например, для $n = 4$ и порядка $f_1 \succ f_2 \sim f_3 \succ f_4$ веса будут: $w_1 \approx 0.5$, $w_2 = w_3 \approx 0.25$, $w_4 \approx 0.125$.

5. Для каждого варианта $x \in X^*$ рассчитывается агрегированная оценка:

$$v(x) = \sum_{i=1}^m w_i \cdot f_i(x). \quad (2.26)$$

Варианты сортируются по убыванию $v(x)$. При равенстве оценок применяется лексикографическое сравнение по порядку важности критериев.

При разработке используются методы библиотеки PyDASS [64], модифицированные для работы с текущими критериями и вариантами, а также автоматическое отбрасывание доминируемых вариантов на этапе Парето-фильтрации. Кроме этого в системе реализована поддержка только строгого упорядочения критериев (без равнозначности).

2.6 Формирование рекомендательных решений

2.6.1 Выявление уникальных характеристик

Для выявления уникальных признаков исследуемой игры применяется подход, основанный на многокритериальном анализе характеристик целевой игры и её

сравнении с другими играми на рынке. Учитываются основные признаки объектов, включая жанры, теги и категории с использованием векторного представления и косинусной меры сходства, семантического анализа с использованием векторных эмбеддингов для выявления уникальных особенностей в описании игры и, кроме этого, метрики Чебышева для оценки цены, рейтинга Metacritic и среднего времени прохождения.

В результате анализа выявляются:

- уникальные жанры и теги, отсутствующие у схожих игр;
- общие характеристики с успешными играми того же сегмента;
- отличительные особенности в описании и позиционировании;
- ценовые отклонения от средних показателей в сегменте.

2.6.2 Механизм генерации рекомендаций

После описанного ранее анализа компьютерных игр система выявляет уникальные характеристики пользовательской игры путем определения отличительных жанров и тегов через операции над множествами, поиска общих черт с успешными играми и выделения уникальных концепций в описании.

Далее процесс переходит к формированию и приоритизации рекомендаций. На этом этапе система анализирует ценовое позиционирование относительно схожих игр, оценивает уникальные игровые механики и выявляет возможные конкурентные преимущества. Затем происходит ранжирование сформированных рекомендаций по их важности с учетом рыночного контекста и конкурентной среды, что позволяет сформировать конкретные действия по улучшению позиции игры на рынке.

Система генерирует рекомендации на основе выявленных особенностей:

- при обнаружении *уникальных жанров* или других механик предлагается усилить их в маркетинговых материалах;
- при выявлении *ценовых отклонений* от средней стоимости игр формируются рекомендации по корректировке ценовой политики;
- на основе *анализа описаний* предлагаются улучшения в позиционировании продукта;
- при наличии *общих черт* с успешными играми даются рекомендации по развитию этих аспектов.

В будущем планируется расширить механизм генерации рекомендаций за счёт использования методов машинного обучения, в частности систем рекомендательных правил, основанных на обучении с подкреплением и глубокого обучения для комплексного анализа текстовых описаний и отзывов. Это позволит точнее и персонализированнее формулировать рекомендации, выдаваемые пользователям.

2.6.3 Математическая модель анализа

В этом разделе рассмотрим основные формулы, используемые в системе рекомендаций.

Формирование рекомендаций как объединение компонентов анализа:

$$\text{Recommendations} = \bigcup_{i=1}^6 C_i = C_1 \cup C_2 \cup C_3 \cup C_4 \cup C_5 \cup C_6, \quad (2.27)$$

где

- C_1 – анализ рыночного позиционирования (жанры, теги, категории),
- C_2 – Уникальные особенности игры (эксклюзивные механики),
- C_3 – ценовой анализ относительно конкурентов,
- C_4 – конкурентные преимущества (уникальные selling points),
- C_5 – лингвистический анализ описания игры,
- C_6 – потенциальные улучшения (добавление новых фич).

Анализ рыночного позиционирования выявляет жанры, характерные только для похожих игр (перспективные ниши), и определяет жанры, популярные у других игроков рынка, но не используемые в аналогичных играх – потенциальные направления для расширения. Разность множеств отфильтровывает общераспространенные жанры:

$$\text{NicheGenres} = \text{GenresSimilar} \setminus \text{GenresDissimilar} \quad (2.28)$$

$$\text{PotentialGenres} = \text{GenresDissimilar} \setminus \text{GenresSimilar} \quad (2.29)$$

Функция значимости признаков рассчитывает весовые коэффициенты, которые отражают стратегические приоритеты: максимальный вес у уникальных черт успешных конкурентов.

$$\text{Importance}(f) = \begin{cases} 0.9, & \text{если } f \in \text{SimilarData} \text{ и } f \notin \text{DissimilarData} \\ 0.7, & \text{если } f \in \text{UniqueInTargetData} \\ 0.3, & \text{если } f \in \text{IntersectionData} \\ 0.1, & \text{иначе} \end{cases}, \quad (2.30)$$

где

- SimilarData – признаки, присутствующие в похожих играх,
- DissimilarData – признаки, отсутствующие в похожих играх,
- UniqueInTargetData – уникальные признаки целевой игры,
- IntersectionData – признаки, общие для целевой и похожих игр.

Анализ ценового позиционирования вычисляет рыночный ориентир:

$$\text{AveragePrice} = \frac{1}{N} \sum_{i=1}^N p_i, \quad (2.31)$$

где

- p_i – цена i -й игры,
- N – количество игр в выборке (похожие или разные игры).

Анализ описания (метрика Жаккара) оценивает семантическую близость описаний через отношение общих терминов к общему словарному запасу:

$$\text{SimilarityScore} = \frac{|A \cap B|}{|A \cup B|}, \quad (2.32)$$

где

- A – множество слов в описании целевой игры,
- B – множество слов в описаниях похожих игр.

Формула выявления уникальных характеристик определяет конкурентные преимущества игры – элементы (жанры, теги), отсутствующие у прямых аналогов:

$$\text{UniqueGenreCombination} = \text{GenresTarget} \setminus \text{GenresSimilar} \quad (2.33)$$

$$\text{UniqueTags} = \text{TagsTarget} \setminus \text{TagsSimilar} \quad (2.34)$$

Потенциальные улучшения состоят в выявлении популярных элементов (например, тегов и категорий) у конкурентов, которые можно адаптировать для улучшения своей игры:

$$\text{PotentialTags} = \text{TagsSimilar} \setminus \text{TagsTarget} \quad (2.35)$$

$$\text{PotentialCategories} = \text{CategoriesSimilar} \setminus \text{CategoriesTarget} \quad (2.36)$$

2.7 Пример работы системы

Разберём подробнее работу примера с компьютерной игрой Forza Horizon 5 [55]. План поиска наиболее похожих игр состоит из следующих шагов:

1. Получение и подготовка данных об играх из базы данных MongoDB, включая основные характеристики: название, описание, жанры, теги, категории, цену, рейтинг Metacritic и среднее время прохождения.
2. Предобработка данных, включающая векторизацию текстовых описаний в эмбединги для семантического анализа, бинарное кодирование категориальных признаков (жанров, тегов, категорий) и Нормализацию числовых характеристик (цены, рейтинга, времени игры).
3. Вычисление сходства между анализируемой игрой и другими играми с использованием взвешенной комбинации метрик:
 - косинусное сходство для текстовых описаний (вес 0.5);
 - косинусное сходство для тегов (вес 0.2);
 - косинусное сходство для категорий (вес 0.15);
 - косинусное сходство для жанров (вес 0.1);
 - метрика Чебышева для числовых характеристик (вес 0.15).
4. Анализ уникальных характеристик игры на фоне топ-5 похожих и отличительных продуктов путём:
 - определения уникальных жанров и тегов через операции над множествами;
 - выявления общих характеристик с успешными играми;
 - анализа ценового позиционирования относительно схожих игр;
 - выделения уникальных особенностей в описании игры.

При этом игры, показавшие наибольшее различие, можно увидеть в таблице табл.2.8.

Таблица 2.8

Топ-5 самых разных игр		
Название игры	Схожесть	Жанры игры
NBA 2K20	0.16	Simulation, Sports
The Forest	0.16	Action, Indie, Simulation
Fallout 4	0.16	RPG
Terraria	0.15	Action, Indie, RPG
Rocket League®	0.14	Action, Indie, Racing, Sports

После получения n -похожих и n -разных игр (по умолчанию $n = 50$), мы можем начать определять долю продаж изучаемой игры в её сегменте. Каждый

показатель (например, отношение положительных отзывов к отрицательным, доля игроков в кластере и т.д.) вновь имеет свой вес, который отражает его важность для оценки. Оценка SS введённой игры на момент работы прототипа = 0.80.

Для того, чтобы понять, какие рекомендации может дать система, посмотрим на список самых похожих игр и их оценку доли продаж игры в таблице табл.2.9.

Таблица 2.9

Топ-5 самых похожих игр по показателям схожести и доле продаж

Название игры	Схожесть	Успех	Жанры
Dota 2	0.66	0.93	Action, Strategy, Free To Play
Team Fortress 2	0.32	0.86	Action, Free To Play
Apex Legends™	0.31	0.81	Action, Free To Play
World of Warships	0.30	0.70	Action, Massively Multiplayer, Simulation, Strategy
SMITE®	0.29	0.71	Action, Casual, Massively, Multiplayer, RPG, Strategy

Как можно увидеть, среди ограниченного количества игр из списка топ-100 система отобрала игры, в жанрах которых имеется "Free To Play" чего не наблюдается в списке игр, непохожих на входную. Система может посоветовать уменьшить стоимость игры, либо добавить больше жанров для приближения к продуктам, имеющим лучшую окупаемость – для этого следует обратить внимание на описание наиболее непохожих игр.

Итоговый результат включает:

- количественную оценку доли продаж игры;
- список выявленных уникальных характеристик;
- конкретные рекомендации по улучшению и продвижению;
- сравнительный анализ с конкурентами в сегменте.

2.8 Выводы

В этой главе был описан процесс проектирования системы для оценки и рекомендаций компьютерных игр. В результате проведённого исследования были достигнуты следующие выводы:

1. На основе сравнительного анализа архитектурных решений была выбрана трехуровневая клиент-серверная архитектура, включающая клиентский уровень (веб-интерфейс на React с компонентами Material-UI для визуализации), серверный уровень (FastAPI) и уровень данных (MongoDB),

а также определены основные сценарии её использования различными группами пользователей;

2. Источники данных, включая Steam API и SteamSpy, были выбраны благодаря их актуальности и разнообразию предоставляемой информации. Будут обрабатываться данные как об играх (стоимость, жанры, теги, описание, количество отрицательных и положительных отзывов, объем продаж), так и о взаимодействии пользователей (содержание отзывов и сыгранное время);
3. Важной частью проектирования стала разработка математических моделей и алгоритмов обработки данных. Для кластеризации игр используется метод K-means++.
4. Многокритериальное сравнение игр было принято осуществлять последовательным уточнением предпочтений, используя метод Эджворта Парето, а также оценку качественной и количественной важности критериев.
5. Заключительным этапом проектирования стал механизм генерации рекомендаций, основанный на многоэтапном анализе. Система выявляет уникальные характеристики игр через операции над множествами (пересечение и разность) для жанров и тегов, анализирует ценовое позиционирование и выделяет ключевые особенности из текстовых описаний. На основе этого анализа формируются рекомендации, учитывающие соответствующую конкурентную среду.

Таким образом, данная глава исследования позволила определить основные направления для успешной разработки рекомендательной системы и заложить основу для дальнейших этапов её реализации и внедрения.

3 РЕАЛИЗАЦИЯ СИСТЕМЫ ПОДДЕРЖКИ АНАЛИЗА КОМПЬЮТЕРНЫХ ИГР

В данной главе описывается практическая реализация системы по поддержке анализа и рейтингования компьютерных игр. В разделе 3.1 рассматриваются особенности разработки серверной части системы, включающие описание основного функционала каждого модуля. Реализация клиентского уровня (фронтенда) описываются в разделе 3.2.

3.1 Реализация основных модулей системы

3.1.1 Диаграмма классов

Система построена на принципах композиции и делегирования, где каждый класс имеет четко определенную ответственность и взаимодействует с другими классами через явные зависимости. Архитектура следует принципу "предпочитай композицию наследованию" (Favor Composition Over Inheritance), что делает систему более гибкой и легкой для поддержки.

В текущей архитектуре не используется наследование и полиморфизм по следующим причинам:

1. Каждый анализатор выполняет уникальную функцию и не требует переопределения поведения базового класса. Использование наследования в данном случае могло бы привести к созданию излишне сложной иерархии классов.
2. Композиция позволяет легко заменять компоненты системы без необходимости изменения существующего кода. Это особенно важно при работе с различными алгоритмами анализа, которые могут требовать частых изменений.
3. Сильно упрощена поддержка и тестирование кода, так как в нём отсутствует наследование – зависимости между классами более явные.

Наиболее часто в реализованной системе 3.1 используется такой тип связи, как *композиция*. Так класс `GameAnalysisPipeline` содержит экземпляры всех анализаторов, а каждый анализатор ссылается на методы классов `DataRepository` и `FeatureExtractor`, поскольку они отвечают за работу с базами данных и преобразование признаков. Подробнее о работе каждого модуля системы поговорим далее. На данной диаграмме отсутствуют классы `ReviewAnalyzer`, отвечающий за обработку отзывов, и `CompetitivenessRanker`, в котором реализована система ранжирования игр, – данные классы связаны отношением композиции с `CompetitivenessAnalyzer` и расширяют его функционал.

3.1.2 Класс *DataRepository*

Класс `DataRepository` является одним из ключевых компонентов системы, реализующим паттерн `Repository` для работы с данными. Он обеспечивает абстракцию доступа к данным и инкапсулирует всю логику взаимодействия с базой

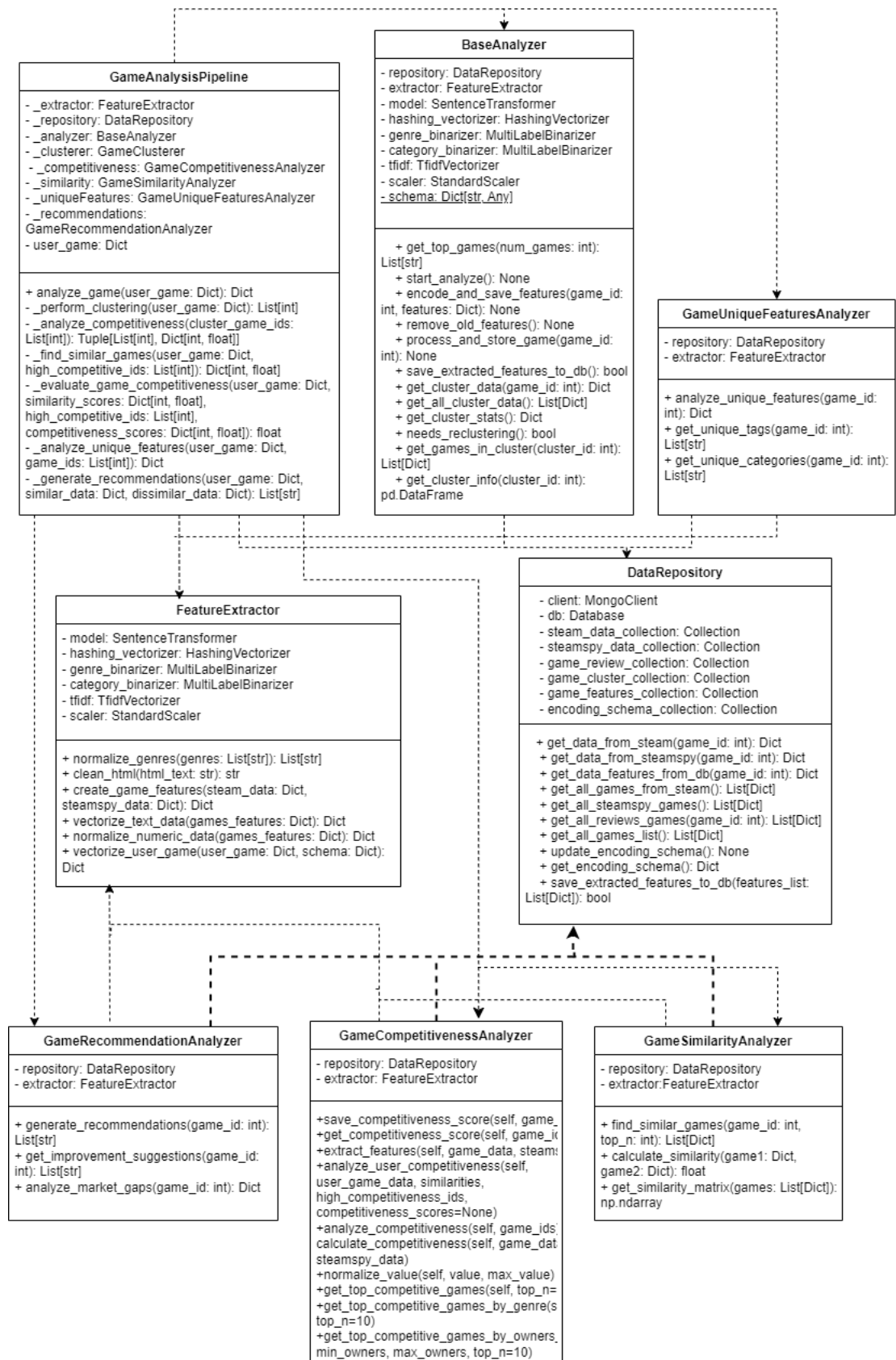


Рис.3.1. Диаграмма классов системы GamesAnalyze

данных MongoDB. Данный класс служит единой точкой доступа к данным для всех остальных компонентов системы.

Класс предоставляет следующие ключевые методы:

- `get_data_from_steam(game_id)` – получение данных игры из Steam;
- `get_data_from_steamspy(game_id)` – получение данных из SteamSpy;
- `get_data_features_from_db(game_id)` – получение признаков игры.

Особое внимание в классе уделено управлению схемой данных через методы:

- `update_encoding_schema()` – обновление схемы кодирования на основе актуальных данных
- `get_encoding_schema()` – получение текущей схемы кодирования

Схема кодирования включает в себя список всех жанров, категорий, тегов, максимальные значения для числовых признаков и статистические показатели (общая стоимость игр и максимальное количество владельцев).

3.1.3 Класс FeatureExtractor

Класс `FeatureExtractor` отвечает за извлечение и преобразование признаков игр в векторное представление. В основном его функции обрабатывают различные типы данных: текстовых описаний, категориальных признаков (жанры, теги, категории) и числовых показателей.

Ключевым методом класса является `create_game_features`, который выполняет предобработку неструктурированных данных игры и возвращает чистую структуру (см. код в 24). Он отвечает за нормализацию жанров, очистку HTML-разметки из описаний, вычисление бинарных признаков (VR, мультиплеер), а также обработку числовых показателей. Особое внимание уделяется обработке отзывов пользователей и расчету соотношения положительных и отрицательных оценок.

Метод `vectorize_text_data` векторизует текстовые данные, используя эмбединги для описаний, бинарное кодирование для жанров и категорий, а также нормализованную матрицу для тегов.

В целом класс `FeatureExtractor` обеспечивает единый интерфейс для работы с признаками, что упрощает интеграцию новых методов анализа и модификацию существующих алгоритмов.

3.1.4 Класс *BaseAnalyzer*

Класс *BaseAnalyzer* выступает в качестве связующего звена между хранилищем данных и процессорами признаков и в основном реализует механизм управления жизненным циклом данных игр.

Например, метод `start_analyze` инициирует процесс обновления данных, проверяя актуальность схемы кодирования. Это включает поиск новых игр и кодирование их признаков при необходимости.

Класс также предоставляет набор методов для извлечения и анализа информации о кластерах игр. Метод `get_cluster_info` формирует структурированное представление данных о играх в кластере.

3.1.5 Класс *GameClusterer*

Класс *GameClusterer* реализует модуль кластеризации компьютерных игр на основе их поведенческих и содержательных признаков (см. приложение 23). Он предназначен для *анализа схожести* между играми и дальнейшей оценки доли продаж новых проектов в своем сегменте игр. Основные функции класса – это кластеризация, визуализация, выбор оптимального числа кластеров и определение кластерной принадлежности по заданным параметрам.

Как уже было сказано, при кластеризации используются два типа признаков: качественные признаки (текстовые описания, теги, жанры, категории) и количественные (цена, оценка `metacritic`, медианное время прохождения). Подготовка признаков осуществляется с помощью класса *BaseAnalyzer*. Примерный сценарий использования данного класса можно увидеть на блок-схеме 3.2.

Для кластеризации по умолчанию используется алгоритм K-Means++. Предусмотрена возможность оценки качества кластеров для подбора оптимального числа кластеров по следующим метрикам:

- Метод локтя (*Elbow method*);
- Коэффициент силуэта (*Silhouette Score*);
- Индекс Дэвиса-Болдина (*Davies-Bouldin Index*).

Для визуализации кластеров в двумерном пространстве используется метод t-SNE. При большом количестве данных берется случайная подвыборка (до 1000 объектов), чтобы сохранить читаемость графика. Кластеризация визуализируется с помощью библиотеки `matplotlib` и `seaborn`, где каждая точка представляет игру, а цвет – кластерную принадлежность. Подписи игр на графике добавляются



Рис.3.2. Блок-схема сценария использования класса GameClusterer.py

выборочно, чтобы избежать перегрузки. Посмотреть примеры графиков можно в разделе 3.2.

3.1.6 Класс *GameCompetitivenessAnalyzer*

Класс *GameCompetitivenessAnalyzer* предназначен для вычисления оценки доли продаж компьютерной игры, что позволяет на ранней стадии оценить её на основе сравнения с наиболее успешными аналогами (см. приложение 27). Это важный этап перед тем, как выполнять поиск наиболее схожих игр – необходимо понять, с кем стоит сравниваться.

Функциональные задачи класса:

- извлечение числовых и текстовых признаков игры;
- расчет итогового рейтинга;
- сохранение и извлечение расчетных значений из базы данных;
- анализ доли продаж пользовательской игры на основе данных о похожих продуктах;
- формирование рейтингов по жанрам и по диапазону количества владельцев.

в текущей реализации класс не хранит заранее рассчитанные взвешенные оценки — все вычисления производятся динамически, а сами веса и критерии определяются в отдельном классе-ранжировщике (*CompetitivenessRanker*). Это позволяет гибко изменять стратегию оценки без необходимости пересчёта всех

данных. Оценка строится на основе набора нормализованных параметров, таких как количество владельцев, соотношение положительных отзывов, выручка, активность сообщества, свежесть игры и анализ отзывов. Дополнительные коэффициенты применяются при наличии VR-режима (+5%) и мультиплеера (+10%).

Прежде чем воспользоваться методами, рассчитывающими долю продаж игры, нам необходимо сформировать список `similar_game_ids`, состоящий из идентификаторов продуктов того же кластера, что и входная игра. Далее именно с найденными играми производится сравнение по схожести признаков, что позволяет избегать сравнений с "неудачными" продуктами и сосредотачиваться на рыночных лидерах.

Класс предоставляет методы для построения различных рейтингов:

- `get_top_competitive_games` – формирует общий рейтинг игр;
- `get_top_competitive_games_by_genre` – позволяет получить топ игр в выбранном жанре;
- `get_top_competitive_games_by_owners_range` – формирует рейтинг по диапазону количества владельцев.

Для анализа доли продаж пользовательской игры реализован метод `analyze_user_competitiveness`, который рассчитывает индивидуальную оценку на основе похожих игр. Алгоритм работы метода следующий:

- 1) Для каждой похожей игры из списка `similarities` определяется её оценка доли продаж.
- 2) Итоговая оценка формируется как взвешенная сумма оценок похожих игр, где вес определяется степенью схожести и дополнительными коэффициентами для лидеров рынка.
- 3) Применяются модификаторы на основе совпадения жанров, ценовой политики, а также наличия VR и мультиплеера.
- 4) Возвращается итоговый балл (от 0.0 до 1.0), отражающий уровень оценки доли продаж пользовательской игры относительно рыночных лидеров.

Таким образом, класс `GameCompetitivenessAnalyzer` не только позволяет вычислять индивидуальные оценки, но и строить рейтинги по различным сегментам рынка, что делает его универсальным инструментом для анализа и сравнения компьютерных игр. Для дальнейшего развития системы планируется внедрение более глубокого анализа отзывов с использованием методов векторизации и определения тональности. Вместо простого усреднения числовых оценок будет применяться модель эмбедингов, позволяющая учитывать смысловое содержание отзывов, что

повысит точность оценки как долю продаж в сегменте, так и близости игр по духу и механикам.

3.1.7 Класс *GameSimilarityAnalyzer*

Класс *GameSimilarityAnalyzer* является самым основным при составлении рейтинга игр и отвечает за определение наиболее схожих по отношению к заданной компьютерных игр с наибольшей долей продаж из соответствующего кластера (см. приложение 30). Он реализует алгоритм сравнения игр, описанный в разделе 2.5.2 и включает следующие этапы:

- 1) Извлечение информации о целевой (пользовательской) игре и наборе сравниваемых игр.
- 2) Приведение всех объектов к единому формату признаков.
- 3) Расчёт индивидуальных метрик сходства между целевой игрой и каждой сравниваемой:
 - текстовые признаки – по формуле (2.6);
 - числовые характеристики – используя (2.7).
- 4) Агрегация метрик в итоговый показатель сходства с использованием взвешенной суммы и метода анализа иерархий (2.8).
- 5) Возврат отсортированного списка кортежей (`game_id`, `similarity_score`) по убыванию значения схожести `similarity`.

Для вычисления сходства реализован метод `calculate_similarity_with_game`. Метод принимает параметры пользовательской игры и возвращает отсортированный список похожих игр. Для объединения различных метрик в единый показатель схожести используется взвешенная сумма, в которой каждому признаку сопоставляется коэффициент значимости. Вес каждого признака отражает его *относительную важность* при сравнении компьютерных игр. Эти веса не задаются произвольно, а определяются с применением АНР [5]. Пример матрицы парных сравнений и полный алгоритм расчёта представлены в разделе 2.5.

3.1.8 Класс *GameUniqueFeaturesAnalyzer*

Класс *GameUniqueFeaturesAnalyzer* отвечает за выявление и анализ уникальных признаков целевой игры по сравнению с группой похожих игр (см. приложение 31). Его основная цель — определить отличительные особенности игры и

сформулировать на основе этого рекомендации по улучшению её доли продаж и привлекательности.

Основные методы класса включают в себя:

1. `create_game_pattern` – формирует векторное представление целевой игры, включающее в себя жанры, теги, категории, описание и числовые признаки.
2. `load_and_analyze_games` – загружает данные о группе игр и рассчитывает общие характеристики (общие жанры, теги, категории, средние значения цены и рейтинга).
3. `compare_with_most_similar_and_dissimilar` – сравнивает целевую игру с группой похожих игр, определяя как общие, так и уникальные признаки по различным характеристикам.
4. `analyze_description_similarity` – рассчитывает степень схожести описания целевой игры и группы игр с использованием косинусного сходства между эмбедингами.
5. `find_unique_description_elements` – выделяет уникальные фразы в описании целевой игры, отсутствующие в группе похожих игр.
6. `find_common_phrases` – определяет общие фразы в описаниях группы игр для анализа типичных паттернов.

Для более детального анализа текстовых данных используются следующие стратегии:

- описание игры разбивается на предложения, которые рассматриваются как потенциальные ключевые фразы;
- сходство между описаниями игр оценивается с помощью косинусного сходства между эмбедингами, что позволяет учитывать контекст и смысл текста;
- анализируются пересечения и уникальные элементы для целевой игры и группы похожих игр.

Текущая реализация класса предоставляет базовый функционал для выявления уникальных признаков игры и генерации простых рекомендаций на их основе. Однако в дальнейшем алгоритм может быть существенно усложнён за счёт применения моделей машинного обучения или нейросетевых подходов для более точного и персонализированного формирования рекомендаций.

В методе `compare_with_most_similar_and_dissimilar`, в котором определяются уникальные черты продукта на основе сравнения со списком похожих

Листинг 3.1

Метод `_analyze_description_similarity`

```

def _analyze_description_similarity(self, target_embedding,
    similar_embeddings):

5  similarities = []
    for emb in similar_embeddings:
        if emb is not None and target_embedding is not None:
            sim = cosine_similarity([target_embedding], [emb])[0][0]
            similarities.append(sim)
10 return np.mean(similarities) if similarities else 0

```

(либо отличительных) игр, схожесть текстовых описаний игр с помощью косинус-ного сходства вычисляется следующим образом (см. листинг 3.1):

3.1.9 Класс *GameRecommendationAnalyzer*

Класс `GameRecommendationAnalyzer` предназначен для генерации рекомендаций по улучшению оценки игры компьютерной игры на основе сравнительного анализа с похожими и разными играми (см. приложение 28). Основной целью является обработка уникальных характеристик игры, анализ её позиции на рынке, а также предложений по улучшению, включая анализ цены и описания.

Рекомендации формируются на основе следующих компонентов:

- анализ позиционирования на рынке (жанры, теги, категории);
- оценка уникальных особенностей игры;
- анализ позиционирования цены;
- выделение конкурентных преимуществ;
- анализ текста описания;
- предложения по улучшению (добавление новых тегов, категорий и т.п.).

Класс реализует описанную в разделе 2.6.3 математическую модель для генерации рекомендаций.

Метод `_analyze_market_positioning` анализирует пересечения жанров, тегов и категорий между целевой игрой, похожими и разными играми, с использованием формул (2.28) и (2.29) (см. листинг 3.2).

Рекомендации формируются на основе принадлежности жанров и тегов целевой игры к указанным группам. Метод `_calculate_feature_importance`

Метод `_analyze_market_positioning`

```

def _analyze_market_positioning(self, user_game,
    similar_data, dissimilar_data):
    recommendations = []
    # Анализ жанрового позиционирования
    similar_genres = set(similar_data['all_genres'])
    dissimilar_genres = set(dissimilar_data['all_genres'])
    user_genres = set(user_game['genres'])
    # Жанры, которые есть в похожих, но нет в разных играх
    niche_genres = similar_genres - dissimilar_genres
    if niche_genres & user_genres:
        recommendations.append(
            f"Игра использует нишевые жанры ({', '.join(
                niche_genres & user_genres)}), "
            f"которые характерны для похожих игр, но отсутствуют в
                разных. "
            f"Это может быть преимуществом для целевой аудитории."
        )
    # Жанры, которые есть в разных, но нет в похожих играх
    potential_genres = dissimilar_genres - similar_genres
    if potential_genres & user_genres:
        recommendations.append(
            f"Игра использует жанры ({', '.join(potential_genres &
                user_genres)}), "
            f"которые характерны для разных игр. "
            f"Это может помочь привлечь новую аудиторию."
        )
    return recommendations

```

определяет значимость признаков (жанры, теги, категории) по формуле (2.30) (см. листинг 3.3).

Метод `_analyze_price_positioning` анализирует ценовое позиционирование игры относительно средних цен похожих и разных игр с расчетом средней цены (2.31), `_analyze_description` анализирует текстовое описание игры. В основе анализа лежит вычисление коэффициента схожести с описаниями похожих игр с применением метрики Жаккара (2.32). `_Analyze_competitive_advantages` выделяет признаки игры, которые отсутствуют в похожих играх, но могут быть востребованы на рынке, через (2.33) и (2.34). Метод `_analyze_potential_improvements` формирует рекомендации по добавлению новых тегов и категорий, которые часто

Метод `_calculate_feature_importance`

```

def _calculate_feature_importance(self, feature,
    similar_data, dissimilar_data):
    # Если признак есть в похожих играх, но отсутствует в разн
    ых - высокая важность
    if feature in similar_data['all_genres'] + similar_data['
        all_tags'] + similar_data['all_categories'] and \
5   feature not in dissimilar_data['all_genres'] +
        dissimilar_data['all_tags'] + dissimilar_data['
        all_categories']:
        return 0.9
    # Если признак уникален для целевой игры - средняя важност
    ь
    elif feature in similar_data['unique_in_target_genres'] +
        similar_data['unique_in_target_tags'] + similar_data['
        unique_in_target_categories']:
        return 0.7
10   # Если признак есть в пересечении - низкая важность
    elif feature in similar_data['intersection_genres'] +
        similar_data['intersection_tags'] + similar_data['
        intersection_categories']:
        return 0.3
    return 0.1

```

встречаются в похожих играх, но отсутствуют в целевой игре, на основе формул (2.35) и (2.36).

Для анализа используются следующие алгоритмы:

- TF-IDF – для оценки текстового описания и тегов.
- Jaccard Similarity – для оценки схожести описаний.
- Среднее арифметическое – для анализа ценового позиционирования.

Таким образом, класс `GameRecommendationAnalyzer` реализует комплексный анализ по математической модели (2.27), объединяя все компоненты рекомендательной системы.

3.1.10 Класс *GameAnalysisPipeline*

Класс `GameAnalysisPipeline` представляет собой централизованный *pipeline* для анализа компьютерных игр с целью определения их рыночного позиционирования, доли продаж и генерации рекомендаций по улучшению. Данный

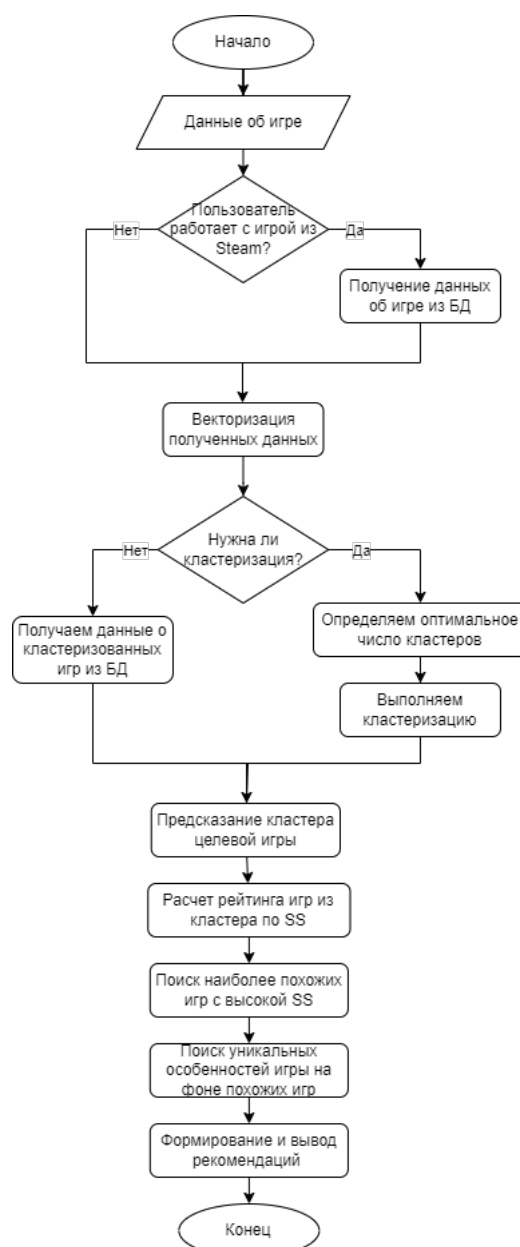


Рис.3.3. Блок-схема возможного сценария работы системы

класс организует последовательность этапов анализа и координирует взаимодействие с отдельными компонентами системы. Рассмотрим возможный сценарий использования системы посредством вызова методов класса `GameAnalysisPipeline` (см.рис.3.3).

Как видно из блок-схемы, кластеризация данных – опциональный процесс, который выполняется только при необходимости. Для входной игры предсказывается её возможный кластер, для игр внутри которого рассчитывается SS (sales share). Для игры пользователя данная оценка рассчитывается уже с расчетом похожих игр. Результатом работы системы становятся рейтинг игр и рекомендации по улучшению пользовательской игры.

3.1.11 Класс *ReviewAnalyzer*

Класс `ReviewAnalyzer` предназначен для комплексного анализа пользовательских отзывов о компьютерных играх. Его основная задача — извлечение и агрегирование различных метрик, отражающих качество и особенности восприятия игры со стороны пользователей. Среди ключевых методов класса:

- `analyze_reviews(game_id: str) → Dict` — основной метод, который извлекает отзывы из базы данных, анализирует их тональность с помощью `TextBlob`, продолжительность игрового времени, оценки пользователей, а также частоту упоминания ключевых особенностей (геймплей, графика, сюжет и др.). На выходе формируется словарь с агрегированными метриками и итоговым интегральным показателем качества отзывов (`review_score`).
- `_analyze_features(reviews: List[Dict]) → Dict[str, float]` — вспомогательный метод, который определяет, насколько часто в отзывах упоминаются различные аспекты игры (например, геймплей или графика), и нормализует эти значения.
- `_calculate_review_score(...)` — рассчитывает итоговый балл на основе нескольких компонент: доли положительных отзывов, средней тональности, среднего времени игры, средней пользовательской оценки и частоты упоминания ключевых особенностей. Для взвешивания этих компонент используется метод анализа иерархий, что позволяет учитывать относительную важность каждого критерия.
- `_get_default_metrics()` — возвращает значения по умолчанию, если для игры отсутствуют отзывы.

Как уже было замечено, для анализа тональности отзывов применяется библиотека `TextBlob`, что позволяет учитывать не только количественные, но и *качественные* аспекты пользовательского опыта. Для расчёта итогового показателя (`review_score`) используется взвешенная сумма нормализованных компонент, где веса определяются с помощью АНР. Это позволяет гибко настраивать вклад каждого критерия в итоговую оценку.

Таким образом, класс `ReviewAnalyzer` обеспечивает многоуровневый анализ отзывов, позволяя учитывать как количественные, так и качественные характеристики именно пользовательского опыта, что существенно повышает инфор-

мативность итоговых метрик для последующего анализа оценки доли продаж игр.

3.1.12 Класс *CompetitivenessRanker*

Класс *CompetitivenessRanker* реализует в системе метод последовательного уточнения предпочтений (см. раздел 2.5.4), позволяющий корректно ранжировать игровые продукты по совокупности критериев оценки доли продаж. В основе лежит подход В.В. Подиновского [8], а также программная реализация на базе библиотеки PyDASS, модифицированная для области анализа компьютерных игр. Ключевые методы класса:

- `rank_games(games_data: List[Dict], max_values: Dict[str, float]) → List[Tuple[str, float]]` – основной метод ранжирования, реализующий все этапы: нормализацию, Парето-фильтрацию, учёт важности критериев и вычисление итоговых оценок.
- `_calculate_importance_coefs() → List[float]` – рассчитывает веса критериев на основе их порядка.
- `_apply_pareto(variants: List[GameVariant])` – реализует Парето-фильтрацию.
- `_apply_quality_importance(variants: List[GameVariant])` – реализует качественное сравнение по важности критериев.
- `_apply_quantitative_importance(variants: List[GameVariant])` – реализует количественное сравнение по важности критериев.

В классе реализованы три ключевые переменные, которые могут быть заданы пользователем через API:

- `criteria` – список критериев, по которым производится оценка доли продаж (например, количество владельцев, выручка, активность, свежесть, анализ отзывов и др.);
- `importance_order` – порядок важности критериев (индексы критериев в порядке убывания значимости);
- `relative_importance` – относительная важность критериев (логический список, где True означает, что следующий критерий менее важен).

Эти параметры позволяют гибко настраивать стратегию ранжирования под конкретные задачи пользователя. Мы также можем рассмотреть, как в нашем модуле реализована Парето-фильтрация (см. листинг 3.4): При этом класс поддерживает

Реализация Парето-фильтрации

```

def apply_pareto(self, variants: List[GameVariant]):
    """Применяет принцип Парето для выявления доминируемых вар
    иантов"""
    for a in variants:
        for b in variants:
            if a == b:
                continue
            # Проверяем доминацию
            dominated = True
            has_better = False
            for s1, s2 in zip(a.scores, b.scores):
                if s1 < s2:
                    dominated = False
                    break
                if s1 > s2:
                    has_better = True
            if dominated and has_better:
                b.nodominated = False
                b.linkedTo.add(a.name)

```

только строгое упорядочение критериев (без равнозначности), все этапы метода реализованы в явном виде, что позволяет гибко модифицировать стратегию ранжирования и все веса критериев автоматически пересчитываются при изменении пользователем порядка важности.

Таким образом, класс *CompetitivenessRanker* обеспечивает корректное и гибкое ранжирование игровых продуктов по совокупности критериев, реализуя современный подход к многокритериальному анализу оценки доли продаж игры в её сегменте.

3.1.13 Реализация API

Для взаимодействия с системой оценки и анализа компьютерных игр был реализован веб-сервис на базе технологии *FastAPI*. В качестве сервера используется *Uvicorn*, который обеспечивает высокую производительность благодаря асинхронной обработке запросов. Структура данных определяется при помощи библиотеки *pydantic*, которая также выполняет автоматическую валидацию входных данных в формате JSON.

В системе реализованы следующие основные эндпоинты:

1. `/test` — тестовый GET-запрос для проверки работоспособности сервера.
2. `/test_game/{game_id}` — GET-запрос для получения базовой информации об игре по её идентификатору.
3. `/settings` — GET-запрос для получения текущих настроек критериев ранжирования (список критериев, порядок важности, относительная важность).
4. `/api/settings` — POST-запрос для обновления настроек критериев ранжирования через API (например, изменение порядка критериев или их включение/отключение).
5. `/run_game_analysis/{game_id}` — GET-запрос для запуска комплексного анализа игры по её идентификатору (включает расчет оценки доли продаж игры в её сегменте *SS*, поиск похожих игр, формирование рекомендаций).
6. `/get_similar_games/{game_id}` — GET-запрос для получения списка схожих игр для существующей в Steam игры.
7. `/rate_game/{game_id}` — GET-запрос для оценки *SS* игры по заданному идентификатору.
8. `/analyze_game` — POST-запрос для запуска анализа новой пользовательской игры (на вход подается полное описание игры).
9. `/get_all_games` — GET-запрос для получения данных о всех играх (названия, теги и другие характеристики).
10. `/get_schema_data` — GET-запрос для получения справочной информации о данных в системе (существующие теги, жанры, категории и др.).
11. `/generate_pdf_report` — POST-запрос для генерации PDF-отчёта с рекомендациями и рейтингом игр на основе результатов анализа.
12. `/get_top_games/{num_games}` — GET-запрос для получения топ-N игр по оценке *SS*.
13. `/games/top-competitive` — GET-запрос для получения топ-N игр с учётом текущих настроек критериев.
14. `/games/top-competitive-by-genre` — GET-запрос для получения топ-N игр в выбранном жанре.
15. `/games/top-competitive-by-owners` — GET-запрос для получения топ-N игр в заданном диапазоне количества владельцев.

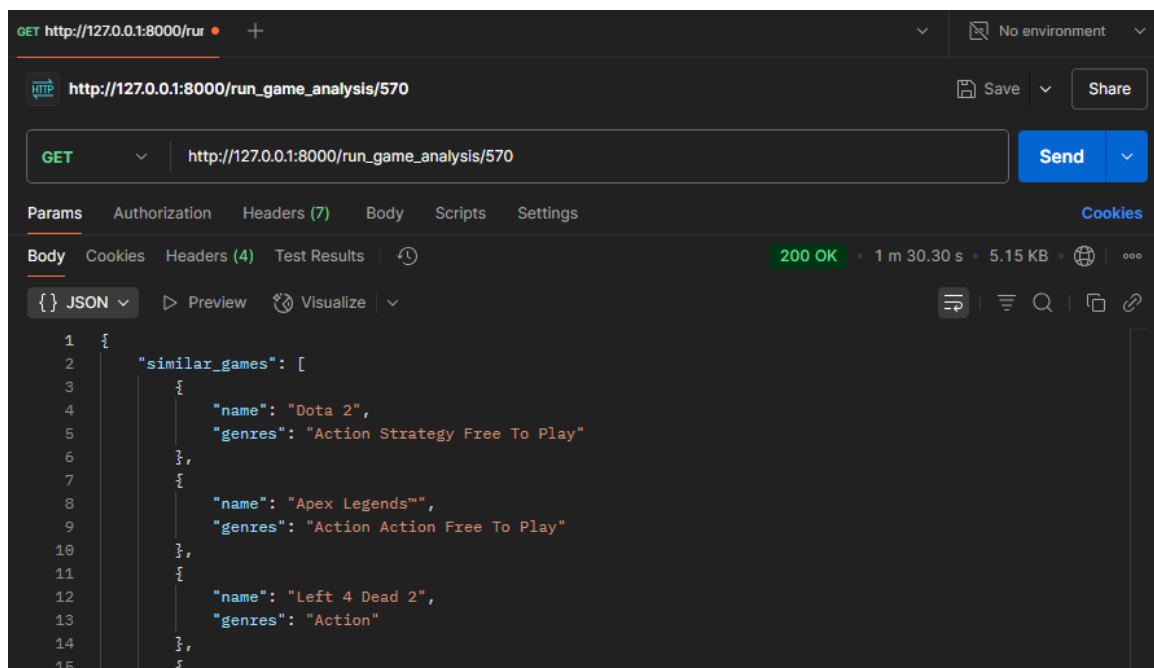


Рис.3.4. Пример POST-запроса к API анализа игры через Postman

Каждый эндпоинт возвращает структурированные данные в формате JSON, что обеспечивает удобную интеграцию с внешними клиентскими приложениями и аналитическими инструментами. Для обеспечения безопасности и удобства работы с фронтендом реализована поддержка CORS и автоматическая обработка ошибок.

Примеры curl-запросов:

```

curl -X POST http://localhost:8000/analyze_game
-H "Content-Type: application/json"
-d @game_data.json

```

Пример взаимодействия через Postman представлен на рис. 3.4.

Архитектура REST (Representational State Transfer) была выбрана за её простоту, масштабируемость и совместимость с веб-технологиями. Она позволяет реализовать логически разделённые ресурсы (анализ, рейтинг, рекомендации), каждому из которых соответствует свой HTTP-метод (GET, POST). Также REST совместим с инструментами тестирования (Postman, curl) и фронтенд-интерфейсами, что удобно при нашей реализации.

3.2 Реализация пользовательского интерфейса

3.2.1 Структура веб-интерфейса

Веб-интерфейс системы реализован как одностраничное приложение на базе *React.js* версии 18. Основной функционал системы сосредоточен на одной странице, которая включает два ключевых компонента: форму для ввода данных и область для отображения результатов анализа (при первоначальном посещении страницы не отображается).

Компоненты приложения организованы по принципу контейнеров и презентационных компонентов. Основная структура интерфейса включает:

1. *GameIdForm.js* – компонент для выбора игры из базы данных Steam. Использует автодополнение и динамическую загрузку данных.
2. *GameAnalysisForm.js* – форма для ввода данных о новой игре. Включает поля для названия игры, описания, жанров, тегов, категорий, цены и продолжительности геймплея.
3. *ResultsDisplay.js* – компонент для отображения результатов анализа, включающий визуализацию оценки доли продаж игры в её сегменте, кластеризацию и рекомендации.
4. *GameRatingDisplay.js* – компонент для отображения рейтинга компьютерных игр с возможностью их сортировки по жанрам, количеству пользователей и настройкой важности критериев.

Как можно понять, пользователь может ввести данные двумя способами. Первый способ – выбор игры из базы данных Steam через компонент *GameIdForm.js*. Данный компонент представляет собой поле с автодополнением, позволяющее найти игру по названию. После выбора игры данные автоматически подгружаются, и запускается анализ. Визуальная индикация загрузки данных реализована с помощью компонента *LoadingIndicator*, который позволяет пользователю отслеживать процесс.

Второй способ – анализ новой игры, для которого предусмотрена форма *GameAnalysisForm.js* (см.рис.3.5). В этой форме пользователю предлагается вручную ввести данные об игре, включая название, описание, жанры, теги, категории, цену и среднюю продолжительность игры. Для удобства ввода используются многоуровневые селекторы для жанров и тегов, что позволяет избежать ручного ввода и минимизировать ошибки. Поля формы валидируются как на клиентской, так и

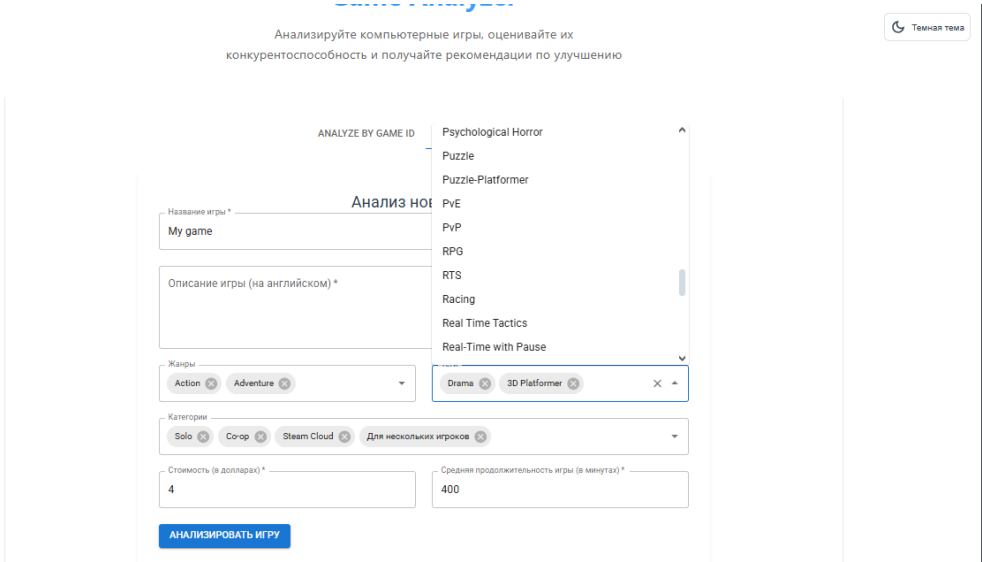


Рис.3.5. Форма загрузки данных пользовательской игры при светлой теме

на серверной стороне, что предотвращает некорректные данные и обеспечивает целостность анализа.

3.2.2 Компонент отображения результатов

Вывод результатов анализа реализован в компоненте ResultsDisplay.js, который структурирован на три основных секции: оценка доли продаж игры, графическое сравнение и рекомендации.

Первая секция (см.рис.3.6) предоставляет оценку SS игры. Визуальная индикация выполнена в виде цветовой шкалы: зелёный цвет обозначает высокую оценку, жёлтый – среднюю, а красный – низкую. Пользователь может быстро оценить позицию своей игры на рынке благодаря данной визуализации.

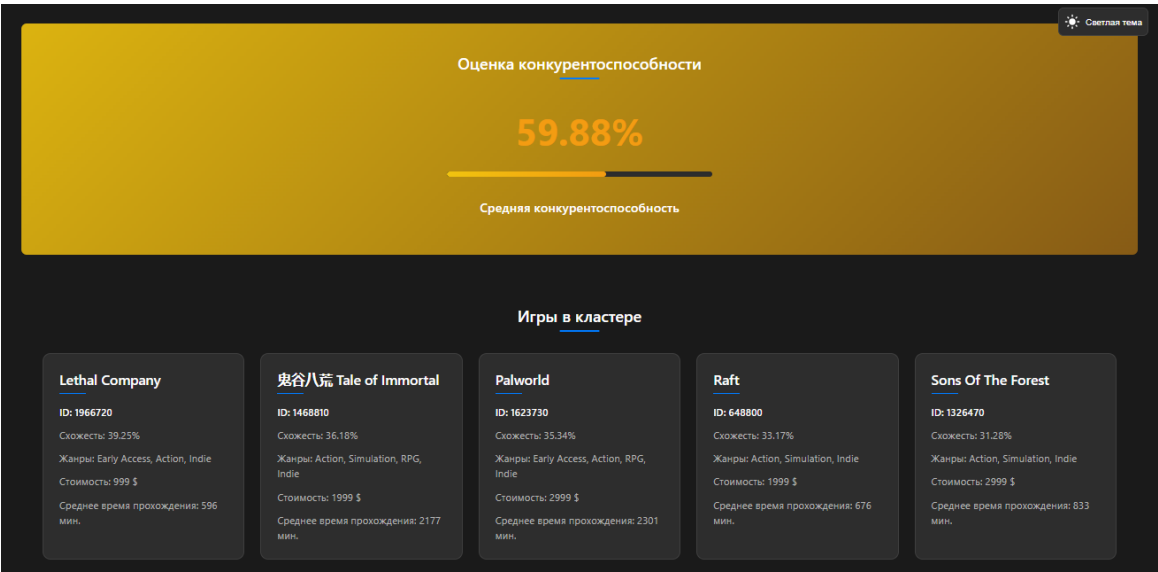


Рис.3.6. Оценка доли продаж игры в её сегменте игр при темной теме

Во второй секции, которую также видно на 3.6, отображается таблица с данными о похожих играх. Таблица поддерживает сортировку по ключевым метрикам, включая цену, количество часов геймплея и оценку доли продаж игры. Сортировка и фильтрация выполняются динамически, что позволяет пользователю быстро находить интересующую информацию. Третья секция 3.7 выводит игроку рейтинг игр, похожих на его. Так пользователь сразу видит, какие игры получают больше всего внимания и насколько они похожи с его продуктом.

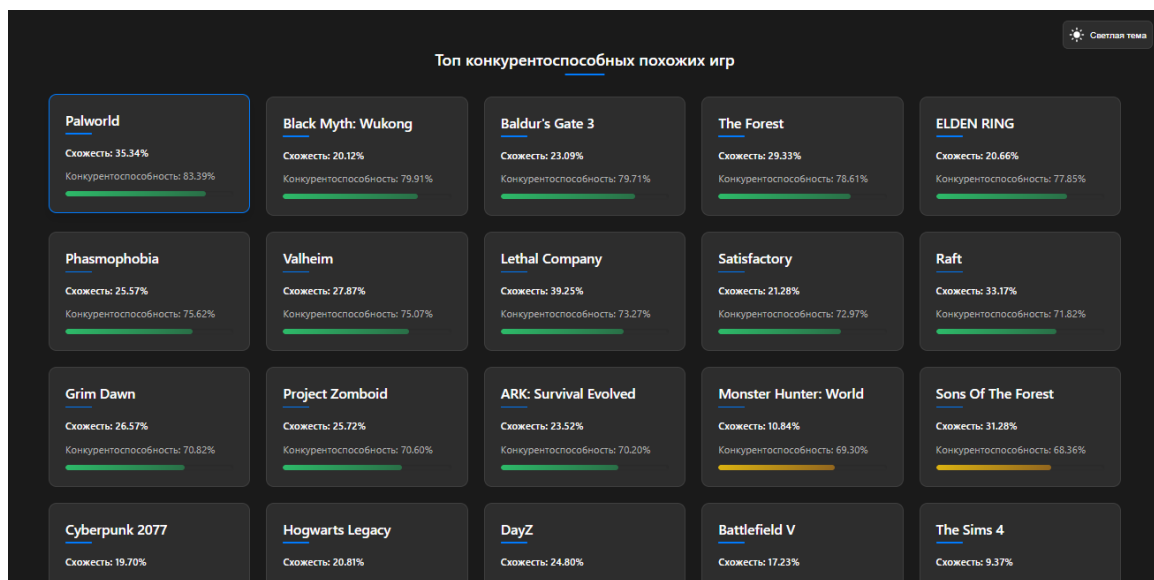


Рис.3.7. Рейтинг похожих игр при темной теме.

Заключительная часть интерфейса результатов – это блок рекомендаций 3.8, который формируется на основе результатов анализа. Здесь пользователь получает конкретные рекомендации по улучшению игры, включая советы по позиционированию, уникальным особенностям и способам повышения оценки доли продаж игры.

3.2.3 Компонент отображения рейтинга игр

Компонент `GameRatingDisplay.js` реализует страницу рейтинга компьютерных игр (см. рис. 3.9), предоставляя пользователю удобный интерфейс для анализа и сравнения оценки доли продаж различных продуктов в их сегменте игр.

Чтобы понять, как работает компонент, можно рассмотреть его ключевые особенности:

- он осуществляет *асинхронные запросы* к серверу для получения списка топовых игр, жанров и другой справочной информации. В зависимости от

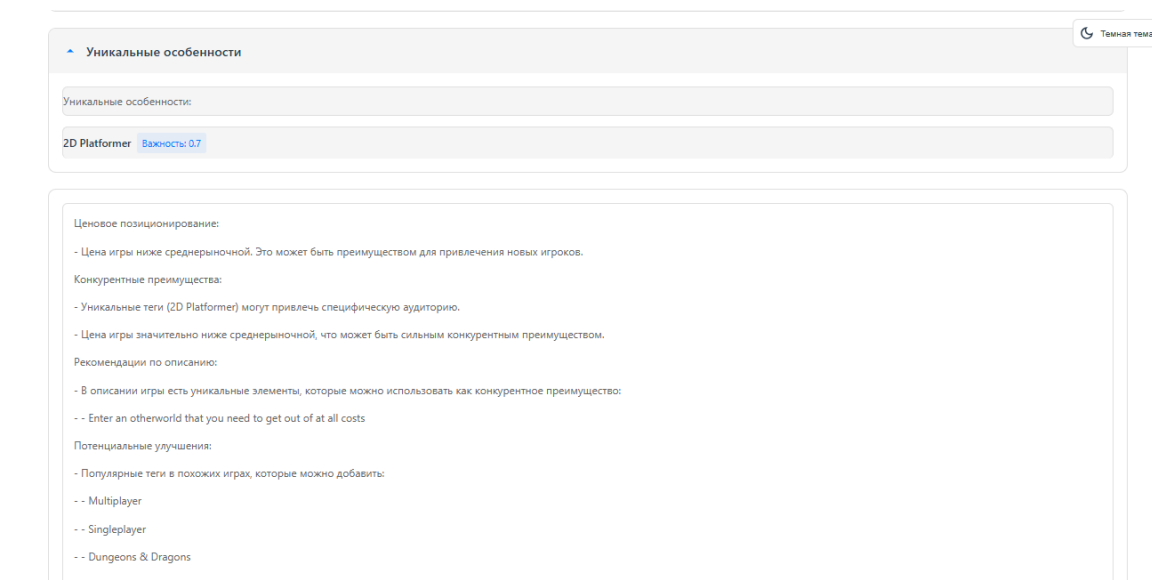


Рис.3.8. Список рекомендаций и анализа составляющей описания игры при светлой теме

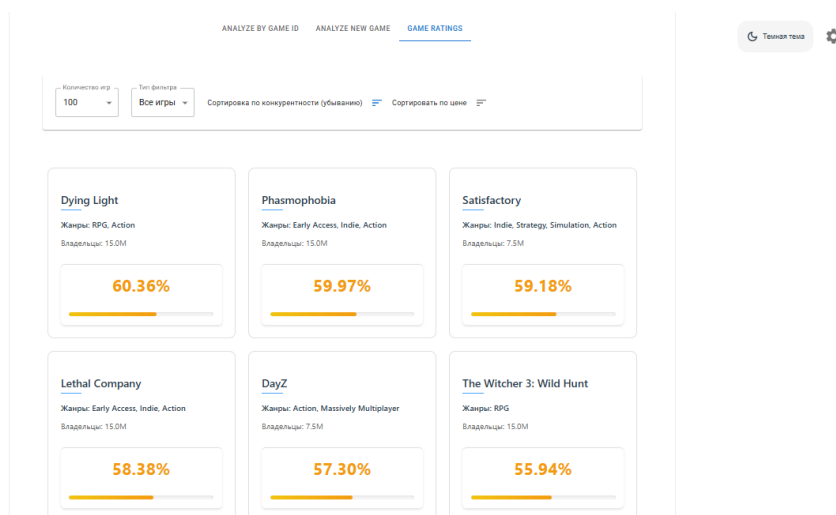


Рис.3.9. Страница рейтинга компьютерных игр

выбранного *фильтра* (по жанру, по диапазону владельцев, либо общий рейтинг) формируется соответствующий запрос к API;

- пользователь может выбрать *количество отображаемых игр*, фильтровать их по *жанру* или *диапазону владельцев*, а также *сортировать* по оценке доли продаж или цене. Все фильтры и сортировки применяются мгновенно, без необходимости перезагрузки страницы.
- для каждой игры отображается название, жанры, цена, количество владельцев и визуальный индикатор оценки доли продаж (цветовая шкала и прогресс-бар). Это позволяет быстро оценить положение игры в общем рейтинге.
- используются современные элементы интерфейса: выпадающие списки, слайдеры для выбора диапазона, кнопки сортировки с иконками и т.д.

В целом компонент позволяет пользователю получить нужные рейтинги игр без привязки к какой-либо игре, как это происходит в компонентах ранее.

3.2.4 Компонент настройки важности критериев

Компонент `SettingsDialog.js` реализует интерфейс для управления критериями оценки доли продаж (см. рис. 3.10). Он позволяет пользователю гибко настраивать стратегию ранжирования игр непосредственно из веб-интерфейса.

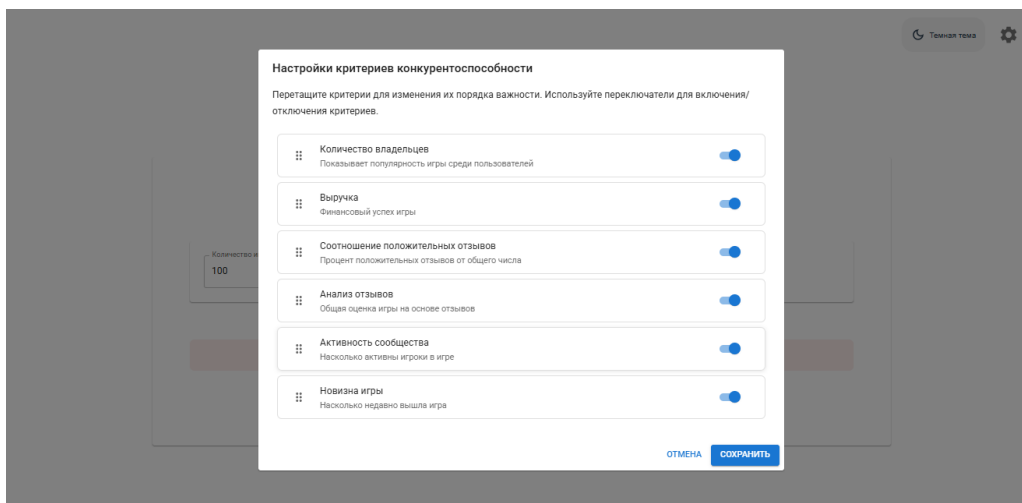


Рис.3.10. Интерфейс управления критериями с drag-and-drop функционалом

Для изменения порядка важности критериев используется библиотека `react-beautiful-dnd`, позволяющая перетаскивать критерии мышью. Порядок критериев напрямую влияет на веса, используемые в ранжировании. Для каждого критерия предусмотрен переключатель (switch), позволяющий временно исключить его из расчёта, а также краткое описание, помогающее пользователю понять его влияние на итоговую оценку.

При открытии диалога компонент запрашивает текущие настройки с сервера, а при сохранении изменений отправляет их обратно через POST-запрос к API (`/api/settings`). Это обеспечивает синхронизацию пользовательских настроек с серверной логикой ранжирования, и в случае ошибок компонент отображает индикаторы загрузки и соответствующие сообщения, что повышает удобство и надёжность работы пользователя.

Таким образом, `SettingsDialog.js` обеспечивает интуитивно понятный и гибкий инструмент для настройки системы ранжирования, позволяя пользователю адаптировать модель под свои задачи без необходимости вмешательства в серверный код.

Можно сказать, что интерфейс системы представляет собой интуитивно понятную, структурированную и функциональную платформу, способную не только проанализировать и визуализировать данные, но и предоставить пользователю конкретные рекомендации по улучшению игрового продукта.

3.2.5 Техническая реализация

Как уже говорилось, веб-интерфейс системы реализован с использованием React.js (версия 18), что обеспечивает одностраничное приложение с динамическим обновлением компонентов без перезагрузки страницы. Стилизация компонентов выполнена с помощью CSS Modules, что позволяет изолировать стили и избежать конфликтов имен классов – это делает разработку множества компонентов более удобной. Взаимодействие клиента с сервером осуществляется через Fetch API с использованием REST API, что обеспечивает стандартизированный обмен данными в формате JSON.

3.2.5.1 Взаимодействие компонентов

Основной поток данных начинается с компонента GameIdForm.js или GameAnalysisForm.js. В первом случае пользователь выбирает существующую игру, и данные загружаются с сервера. Во втором случае данные вводятся вручную и отправляются на сервер для анализа.

После завершения анализа сервер возвращает структурированные данные, которые обрабатываются компонентом ResultsDisplay.js. Компонент организует данные в несколько секций:

- 1) оценка доли продаж – визуализация показателя с динамическим изменением цвета и длины полосы;
- 2) игры в кластере – список игр из соответствующего кластера с сортировкой по схожести;
- 3) топ похожих игр – таблица с конкурентными играми и их оценками доли продаж;
- 4) рекомендации – текстовые советы по позиционированию игры на рынке, выделение уникальных тегов и стратегические рекомендации.

Каждая секция компонента ResultsDisplay.js выполнена с применением React Hooks для управления состоянием и асинхронной загрузки данных. Напри-

Листинг 3.5

Компонент для отображения доли продаж игр renderCompetitivenessScore

```

const renderCompetitivenessScore = () => {
  const score = results.competitiveness_score;
  const scoreClass = score >= 0.7 ? styles.highScore :
5      score >= 0.4 ? styles.mediumScore :
      styles.lowScore;
  return (
    <div className={` ${styles.section} ${scoreClass}`}>
    <h2>Оценка доли продаж игры в её сегменте</h2>
10    <div className={styles.competitivenessScore}>
      <div className={styles.scoreValue}>
        <span>{(score * 100).toFixed(2)}%</span>
      </div>
      <div className={styles.scoreIndicator}>
15        <div
          className={` ${styles.scoreBar} ${scoreClass}`}
          style={{ width: `${score * 100}%` }}
        />
      </div>
      <p className={styles.scoreDescription}>
        {score >= 0.7 ? 'Высокая оценка' :
          score >= 0.4 ? 'Средняя оценка' :
          'Низкая оценка'}
20      </p>
    </div>
  </div>
  );
};

```

мер, в компоненте renderCompetitivenessScore() данные визуализируются с применением динамически изменяемых стилей (см. листинг 3.5):

3.2.5.2 Функциональные возможности

Важной функциональной особенностью системы является возможность скачивания отчета с результатами анализа. Для этого реализована кнопка *DownloadReport*, которая формирует PDF-файл с данными анализа, включая диаграммы, таблицы и текстовые рекомендации. Формирование отчета выполняется на клиентской стороне с использованием библиотеки *jsPDF*, что позволяет пользователю мгновенно получить результаты в удобном формате.

Визуализация данных также включает динамическое изменение цветовой схемы – светлый и тёмный режимы. Смена темы реализована с использованием *React Context API*, позволяющем централизованно управлять состоянием темы и мгновенно обновлять интерфейс.

Интерфейс системы обладает широкими возможностями интерактивной работы с данными:

- Пользователь может *сортировать* и *фильтровать* таблицы с результатами анализа по различным метрикам (например, доля продаж игры, цена, количество владельцев) в обоих направлениях. Сортировка реализована непосредственно на клиенте, что обеспечивает мгновенный отклик и удобство работы.
- В компоненте управления критериями реализована поддержка *drag-and-drop* (перетаскивания) с помощью библиотеки *react-beautiful-dnd*. Это позволяет пользователю интуитивно изменять порядок важности критериев, что напрямую влияет на стратегию ранжирования игр.
- Все элементы интерфейса (таблицы, фильтры, слайдеры, выпадающие списки) формируются *динамически* на основе получаемых с сервера данных. Это позволяет легко масштабировать систему и добавлять новые параметры без необходимости переписывать логику отображения и обновлять страницу.
- Для всех ключевых показателей используются цветовые индикаторы, прогресс-бары и всплывающие подсказки, что облегчает восприятие информации и повышает удобство анализа.
- Интерфейс корректно отображается на различных устройствах и экранах благодаря использованию *CSS Modules*.

При выводе результатов (см. приложение 35) компоненты *renderClusterInfo()* и *renderSimilarGames()* формируют динамическую визуализацию ответа. Логика сортировки, обработки данных и применения стилей сконцентрирована непосредственно в компоненте, что упрощает его поддержку и расширение.

Таким образом, техническая реализация интерфейса обеспечивает удобное и интуитивно понятное взаимодействие с пользователем, а также предоставляет широкие возможности для дальнейшего масштабирования и доработки. Особое внимание уделено адаптивности интерфейса, визуальной обратной связи и гибкости системы за счёт использования *React Hooks*, *drag-and-drop* технологий и *CSS Modules*.

3.3 Выводы

В ходе реализации системы были разработаны и проанализированы пять основных классов: `BaseAnalyzer` (предобработка и извлечение данных), `GameSimilarityAnalyzer` (поиск схожих игр), `GameCompetitivenessAnalyzer` (оценка доли продаж игры в её сегменте), `GameUniqueFeaturesAnalyzer` (выделение уникальных черт) и `GameClusterer` (кластеризация игр). Класс `CompetitivenessRanker` с методом последовательного уточнения предпочтений [7] поддерживает все заявленные этапы, включая нормализацию, Парето-фильтрация, качественное/количественное ранжирование и интеграцию с `PyDASS` для верификации результатов (см. листинг 3.4).

Все компоненты были объединены в единый модуль `GameAnalysisPipeline`, обеспечивающий полный цикл анализа. На основе данных и реализованной логики был создан REST-сервер с использованием фреймворка `FastAPI`.

На стороне клиентской части был разработан веб-интерфейс на основе `React.js`, который позволяет пользователю взаимодействовать с системой и визуализировать результаты анализа. Основные компоненты интерфейса (`GameIdForm`, `GameAnalysisForm`, `ResultsDisplay`, `GameRatingDisplay`) реализованы с применением `CSS Modules` для стилизации и обеспечивают динамическое обновление данных и адаптивное отображение интерфейса. В добавок к этому добавлен `drag-and-drop` интерфейс (рис. 3.10) для изменения порядка важности критериев, учитываемых при ранжировании.

Текущая реализация системы закладывает основу для дальнейшего расширения и интеграции новых аналитических методов, направленных на повышение точности рекомендаций и улучшение пользовательского опыта. Проект можно найти в приложении [9].

4 ТЕСТИРОВАНИЕ И АПРОБАЦИЯ СИСТЕМЫ ПОДДЕРЖКИ АНАЛИЗА КОМПЬЮТЕРНЫХ ИГР

4.1 План тестирования

Тестирование проводится для проверки корректности реализации ключевых компонентов разрабатываемой системы и оценку её качества по следующим критериям:

- 1) *Функциональная пригодность* – степень, в которой система обеспечивают выполнение функции в соответствии с заявленными и подразумеваемыми потребностями при использовании в указанных условиях, в частности проверяется корректность анализа пользовательских данных и генерации рекомендаций.
- 2) *Совместимость системы*, в частности корректность взаимодействия между модулями (функциональная совместимость) и экспорта отчетов.
- 3) *Производительность*, в частности время отклика системы, устойчивость к нагрузке при анализе больших наборов данных.

Для проведения тестирования выбраны следующие инструменты:

- *pytest (версия 7.4.3)* – для модульного и интеграционного тестирования серверной части благодаря простоте написания тестов и возможности генерации подробных отчетов об ошибках и трассировке стека;
- *pytest-cov (версия 4.1.0)* – плагин для оценки покрытия кода тестами. Использовался для генерации отчетов о покрытии кода в формате HTML, выявления непокрытых тестами участков кода и соответствующего анализа эффективности тестового набора;
- *Behave (версия 1.2.6)* – фреймворк для BDD тестирования. Подробнее причина выбора описана в соответствующем разделе, однако основными причинами стали поддержка языка Gherkin для описания сценариев, интеграция с Python и возможности генерации отчетов в различных форматах;
- *@testing-library/react (версия 14.1.2)* – библиотека для тестирования React-компонентов. Приоритетнее других библиотек, поскольку имеет простой API для поиска элементов, интеграцию с Jest, а также доступную и семантическую структуру тестов;
- *Jest (версия 29.7.0)* – фреймворк для оценки производительности и времени отклика React-компонентов.

Кроме того, в рамках тестирования проводится *экспериментальная проверка работоспособности системы на практике*. Особое внимание уделяется сравнению результатов, полученных с использованием разработанной системы, с рекомендациями и рейтингами, формируемыми платформой Steam. Такой подход позволяет оценить эффективность предложенных методов анализа и обоснованность формируемых выводов об оценках игр.

4.2 Тестирование компонентов системы

В данном разделе представлено модульное и интеграционное тестирование компонентов разработанной системы. Целью модульного тестирования является проверка корректности работы отдельных функций и классов backend-приложения в изоляции. Интеграционное тестирование направлено на оценку взаимодействия между модулями, включая работу API, генерацию рекомендаций и экспорт данных.

4.2.1 Модульное тестирование backend

Модульное тестирование было направлено на проверку корректности отдельных компонентов серверной части системы, реализующих основные алгоритмы оценки доли продаж и сравнения игр. Тестирование проводилось с целью выявления и устранения возможных логических ошибок, несоответствий ожидаемым результатам и некорректного взаимодействия между функциями и классами.

Основные модули backend, подлежащие тестированию, включают:

- 1) *Модуль предобработки данных* (FeatureExtractor) – очистка текстовых данных от HTML-тегов, преобразование категориальных данных (жанры, теги) в бинарные векторы и нормализация числовых признаков (цена, оценки Metacritic, продолжительность игры).
- 2) *Модуль кластеризации* (GameClusterer) – группировка игр на основе их характеристик.
- 3) *Модуль анализа оценки доли продаж* (GameCompetitivenessAnalyzer) – расчет итоговой оценки (SS) для каждой игры в рамках одного кластера.

4.2.1.1 Методология тестирования

Модульное тестирование проводилось с использованием *pytest* и *pytest-mock*. Тестовые кейсы включали следующие сценарии:

Листинг 4.1

Пример входных данных для предобработки

```

sample_data = [
{"description": "An epic <b>RPG</b> with dragons!", "price": 59.99, "metacritic": 85, "median_forever": 120},
{"description": "<p>Fast-paced action shooter.</p>", "price": 19.99, "metacritic": 70, "median_forever": 50}
]

```

Листинг 4.2

Ожидаемый результат предобработки

```

expected_output = [
{"description": "an epic rpg with dragons", "price": 0.75, "metacritic": 0.85, "median_forever": 0.6},
{"description": "fast-paced action shooter", "price": 0.25, "metacritic": 0.7, "median_forever": 0.25}
]

```

Листинг 4.3

Сценарий тестирования предобработки

```

def test_data_preprocessing():
    processed_data = preprocess_data(sample_data)
    assert processed_data == expected_output

```

1. *Тестирование обработки некорректных данных (класс `FeatureExtractor`):*
В качестве входных данных используется список игр с HTML-разметкой в описаниях и ненормализованными числовыми значениями (листинг 4.1). После предобработки ожидается результат, приведённый в листинге 4.2. Сам тестовый сценарий выглядит следующим образом (листинг 4.3).
2. *Тестирование кластеризации (класс `GameClusterer`):*
Для оценки работы кластеризации подаются числовые векторы признаков (листинг 4.4).
Ожидается, что алгоритм разобьёт данные на два кластера: первый с двумя, второй с тремя объектами. Тестовая реализация приведена в листинге 4.5.
3. *Тестирование расчёта оценки доли продаж игры:*
Исходные данные и векторы признаков игр представлены в листинге 4.6.

Листинг 4.4

Входные данные для кластеризации

```

5 sample_features = [
    [0.2, 0.8],
    [0.25, 0.75],
    [0.7, 0.3],
    [0.72, 0.28],
    [0.8, 0.2]
]

```

Листинг 4.5

Сценарий тестирования кластеризации

```

5 def test_clustering_algorithm():
    clusters = perform_clustering(sample_features, n_clusters
        =2)
    assert len(clusters[0]) == 2
    assert len(clusters[1]) == 3

```

Листинг 4.6

Пример входных данных для расчёта оценки доли продаж игры

```

5 sample_games = [
    {
        "name": "Game1",
        "price": 0.5,
        "metacritic": 0.8,
        "median_forever": 0.7,
        "features": [0.5, 0.8, 0.7]
    },
10 {
        "name": "Game2",
        "price": 0.6,
        "metacritic": 0.9,
        "median_forever": 0.75,
15 "features": [0.6, 0.9, 0.75]
    }
]

```

Сценарий тестирования оценки доли продаж игры

```

def test_competitiveness_index():
    weights = [0.4, 0.3, 0.3]
    cp_values = calculate_competitiveness_index(sample_games,
        weights)
5  assert round(cp_values[0], 2) == 0.65
    assert round(cp_values[1], 2) == 0.73

```

Расчёт выполняется по формуле:

$$CP = w_1 \cdot p + w_2 \cdot m + w_3 \cdot t,$$

где $w_1 = 0.4$, $w_2 = 0.3$, $w_3 = 0.3$. Ожидаемые значения: Game1: $CP = 0.65$, Game2: $CP = 0.73$.

См. тест в листинге 4.7.

4. Тестирование уникальных особенностей игр (см. приложение 43) нацелено на проверку корректности анализа уникальных характеристик игры, включая сравнение с похожими и непохожими играми, проверку наличия всех необходимых полей в результатах сравнения и проверку структуры данных о схожести (описание, жанры, категории).
5. Тесты конвейера анализа игр (см. приложение 44) проверяет корректность работы полного конвейера анализа, включая:
 - кластерный анализ;
 - анализ оценки доли продаж;
 - анализ схожести;
 - выявление уникальных особенностей.
6. Тест обработки пустых данных (см. приложение 42) проверяет устойчивость системы к некорректным входным данным (пустые строки в текстовых полях, нулевые значения в числовых полях, пустые массивы в категориальных данных).
7. Тестирование анализатора рекомендаций игр (см. приложение 45) проверяет генерацию комплексных рекомендаций, включая:
 - позиционирование на рынке;
 - уникальные особенности;
 - ценовое позиционирование;
 - конкурентные преимущества;

```

tests/test_feature_extractor_html.py::test_html_attributes PASSED [ 20%]
tests/test_feature_extractor_html.py::test_multiple_spaces_after_html_removal PASSED [ 22%]
tests/test_game_analysis_pipeline.py::test_pipeline_initialization PASSED [ 23%]
tests/test_game_analysis_pipeline.py::test_rate_top_games PASSED [ 25%]
tests/test_game_analysis_pipeline.py::test_analyze_game PASSED [ 27%]
tests/test_game_analysis_pipeline.py::test_vectorize_game PASSED [ 28%]
tests/test_game_analysis_pipeline.py::test_empty_game_analysis PASSED [ 30%]
tests/test_game_analysis_pipeline.py::test_pipeline_consistency PASSED [ 32%]
tests/test_game_clusterer.py::test_kmeans_clustering PASSED [ 33%]
tests/test_game_clusterer.py::test_normalize_features PASSED [ 35%]
tests/test_game_clusterer.py::test_calculate_similarity PASSED [ 37%]
tests/test_game_clusterer.py::test_evaluate_clusters PASSED [ 38%]
tests/test_game_clusterer.py::test_get_features_matrix PASSED [ 40%]
tests/test_game_clusterer.py::test_error_on_empty_db PASSED [ 42%]
tests/test_game_clusterer_stability.py::test_get_features_matrix_shape PASSED [ 44%]
tests/test_game_clusterer_stability.py::test_evaluate_clusters_returns_optimal_k PASSED [ 45%]
tests/test_game_clusterer_stability.py::test_perform_clustering_and_balance PASSED [ 47%]
tests/test_game_clusterer_stability.py::test_predict_cluster PASSED [ 49%]
tests/test_game_clusterer_stability.py::test_visualize_clusters_tsne PASSED [ 50%]
tests/test_game_clusterer_stability.py::test_get_similar_games_for_user_game PASSED [ 52%]
tests/test_game_clusterer_stability.py::test_error_on_empty_db PASSED [ 54%]
tests/test_game_competitiveness_analyzer.py::test_calculate_competitiveness PASSED [ 55%]
tests/test_game_competitiveness_analyzer.py::test_analyze_competitiveness PASSED [ 57%]

```

Рис.4.1. Результаты выполнения тестовых функций

- рекомендации по описанию;
- потенциальные улучшения.

Ожидаемым результатом является структурированный список рекомендаций, охватывающий все аспекты анализа.

8. Тестирование анализа ценового позиционирования (см. приложение 45) нацелено на проверку корректности анализа ценовой политики: оценку ценового позиционирования и формирование рекомендаций по ценообразованию. Проверяются различные ценовые сценарии (низкая и высокая цена).
9. Тест анализа уникальных особенностей (см. приложение 43) проверяет выявление уникальных характеристик игры (анализ комбинаций жанров, выявление уникальных тегов, оценка конкурентных преимуществ). Ожидаемый результат: список уникальных особенностей и конкурентных преимуществ.
10. Тест анализа описания (см. приложение 45) выполняет проверку оценки схожести с другими играми, выявления общих и уникальных фраз и корректность формирования рекомендаций по улучшению описания. В основном проверяются различные уровни схожести (высокая и низкая).

Результаты выполнения модульных тестов: 60 тестовых функций успешно пройдены (на рис. 4.1 можно увидеть часть функций), несколько функций имело результат *failed*, что позволило заметить ошибки в коде и успешно исправить их. Тесты охватывают все ключевые аспекты работы системы, включая анализ рекомендаций, ценовое позиционирование, уникальные особенности и обработку граничных случаев.

Как уже было сказано, по результатам модульного тестирования были выявлены следующие замечания и пути их исправления:

- 1) В модуле предобработки данных обнаружена ошибка при очистке текста от HTML-тегов, приводящая к удалению значимых слов (например, `<b>RPG</b>` становилось пустым). Ошибка устранена путем использования регулярных выражений с учетом специфики HTML-разметки.
- 2) В модуле кластеризации выявлена нестабильность алгоритма при малых значениях `n_clusters` (менее 3). Внедрен контроль корректности определения числа кластеров на основе `silhouette score`.
- 3) В модуле расчета оценки доли продаж при недостатке данных для одного из критериев (например, отсутствует метрика `metacritic`) возникала ошибка деления на ноль. Внедрена проверка данных и использование среднего значения по выборке в случае отсутствующих данных.

Таким образом, модульное тестирование помогло выявить и устранить критические ошибки, связанные с некорректной обработкой данных и вычислением метрик оценки доли продаж. В результате была налажена корректная передача данных между компонентами *FeatureExtractor*, *GameClusterer* и *GameCompetitivenessAnalyzer*, внедрена проверка формата данных и их согласованности на каждом этапе анализа, а также оптимизирована обработка некорректных данных и исключений. Реализованные тесты обеспечивают *стабильность* основных алгоритмов системы и минимизируют вероятность возникновения сбоев при интеграции компонентов.

4.2.2 Интеграционное тестирование

Интеграционное тестирование – это вид программного тестирования, при котором отдельные модули программного обеспечения объединяются и тестируются как единое целое. Его основная цель – проверить корректность взаимодействия между компонентами системы, выявить ошибки обмена данными и убедиться в согласованности логики работы модулей.

Алгоритм интеграционного тестирования включает следующие этапы:

- 1) Подготовка стратегии тестирования и тестового плана.
- 2) Разработка интеграционных сценариев и подготовка тестовых данных.
- 3) Проведение тестирования, фиксация и анализ дефектов.
- 4) Повторное тестирование после устранения ошибок.

Учитывая структуру приложения и особенности взаимодействия модулей, при разработке тестов был выбран поведенческий подход – *Behavior-Driven Development* (далее – BDD), позволяющий формализовать тестовые сценарии в форме бизнес-логики и упростить коммуникацию между заинтересованными сторонами.

4.2.2.1 Методология тестирования

Behavior-Driven Development — это методология разработки, направленная на формулирование требований в виде сценариев, понятных как техническим, так и нетехническим участникам проекта [26]. Основной единицей спецификации является *сценарий* (*Scenario*), описывающий поведение системы через шаги Given (предусловия), When (действие) и Then (ожидаемый результат). Дополнительно используются ключевые слова And, But.

Пример сценария на человеко-читаемом языке для описания поведения системы – Gherkin [50], – описывающем, что именно должна делать система, а не как она это реализует:

```
Scenario: Расчет оценки доли продаж новой игры
Given Пользователь находится на странице анализа игр
When Вводит данные о новой игре
And Нажимает кнопку "Анализировать игру"
Then Пользователь получает список похожих игр
And Каждая игра имеет рассчитанный рейтинг оценки доли продаж
```

В таблице 4.1 представлены основные тестовые сценарии и ожидаемые результаты.

4.2.2.2 Фреймворк Behave

Для автоматизации выполнения Gherkin-сценариев в рамках проекта был выбран фреймворк *Behave* [49] — разновидность инструмента *Cucumber* [50], ориентированная на язык программирования Python. Behave позволяет писать тесты в соответствии с методологией BDD, используя ключевые слова языка Gherkin.

Для настройки среды тестирования были выполнены следующие шаги:

- 1) Установлена версия Python 3.11.

Таблица 4.1

План тестирования

№	Тестовый сценарий	Описание	Ожидаемый результат	Статус
1	Анализ существующей игры	Выбрать игру из выпадающего списка и нажать "Анализировать"	Отображаются результаты анализа с рекомендациями	Пройден
2	Анализ новой игры	Заполнить форму с данными новой игры и отправить	Система добавляет игру в базу и показывает результаты анализа	Пройден
3	Валидация формы анализа	Отправить форму с пустыми обязательными полями	Отображаются сообщения об ошибках валидации	Пройден
4	Проверка кластеризации игр	Запустить анализ группы похожих игр	Система корректно группирует игры по схожести характеристик	Пройден
5	Генерация PDF-отчета	Нажать кнопку "Скачать PDF"	Создается PDF-файл с результатами анализа	Пройден
6	Анализ уникальных особенностей	Запустить анализ для игры с уникальными характеристиками	Система выявляет и подчеркивает уникальные особенности	Пройден

- 2) Установлен пакетный менеджер `pip`.
- 3) Установлена среда разработки `PyCharm`.
- 4) С помощью `pip` установлен фреймворк `Behave` командой: `pip install behave`.

`Behave` предоставляет удобный способ описания поведения системы с использованием понятного полужформального языка, что облегчает взаимодействие между разработчиками, тестировщиками и заказчиком. Сценарии являются при этом своего рода документацией, описывающей логику работы системы. Дополнительным плюсом является возможность *повторного использования шагов* (steps) в разных сценариях, что упрощает поддержку и расширение тестов. `Behave` также хорошо *масштабируется* благодаря модульной структуре.

Однако у этого фреймворка есть и недостатки, которые, впрочем, не мешают работе нашей системы:

- оптимально подходит только для тестирования методом «чёрного ящика» [2], где внутреннее устройство тестируемого компонента не учитывается.
- высокая степень многословности в описании шагов может привести к усложнению поддержки сценариев.

В таблице 4.2 представлено сравнение популярных IDE для работы с `Gherkin` и BDD-фреймворками, на основании которого был сделан выбор в пользу *Visual*

Studio Code [33], соответствующего языку Python и предоставляющего полный набор необходимых возможностей.

Таблица 4.2

Сравнение IDE для написания Gherkin-сценариев

<i>IDE / Инструмент</i>	<i>Язык</i>	<i>Поддержка фреймворков</i>	<i>Функции (подсветка, отладка, отчёты)</i>	<i>Визуализация сценариев</i>
Cucumber Studio [50]	Java, JS, Ruby	JUnit, Selenium, Appium и др.	Да	Да
Gauge [22]	Java, C#, Ruby	Cucumber, JUnit, TestNG	Да	Нет
Behave Pro [16]	Python	Selenium, PyTest и др.	Да	Нет
VS Code + Cucumber Extension [33]	JS, TS, C#, Java	JUnit, Gradle, TestNG и др.	Да	Нет
SpecFlow+Runner [69]	.NET (C#, VB.NET)	NUnit, MSTest, Selenium	Да	Да

4.2.2.3 Тестовые сценарии

Итак, BDD-тесты написаны с использованием фреймворка Behave и языка Gherkin, которые обеспечивают четкую структуру и читаемость тестовых сценариев, каждый тестовый сценарий состоит из следующих компонентов:

- 1) *Функционал (Feature)* – общее описание тестируемой функциональности
- 2) *Сценарий (Scenario)* – конкретный тестовый случай
- 3) *Шаги (Steps)* – последовательность действий, описываемая ключевыми словами:
 - *Дано (Given)* – начальное состояние системы
 - *Когда (When)* – действия пользователя
 - *Тогда (Then)* – ожидаемый результат

Ниже представлены ключевые тестовые сценарии, демонстрирующие основные возможности системы (полный список предоставлен в приложении П49.1):

Листинг 4.8

Описание шага "Дано я нахожусь на странице анализа игр"

```

5  @given('я нахожусь на странице анализа игр')
    def step_impl(context):
        try:
            response = requests.get(f"{API_URL}/get_all_games")
            assert response.status_code == 200
            context.games = response.json()
        except requests.exceptions.RequestException as e:
            logger.error(f"Ошибка при получении списка игр: {e}")
10  raise

```

Функционал: Анализ игр

Как пользователь системы анализа игр

Я хочу иметь возможность анализировать существующие и новые игры

Чтобы получать рекомендации по улучшению и позиционированию

Сценарий: Анализ существующей игры

Дано я нахожусь на странице анализа игр

Когда я выбираю игру "Dota 2" из выпадающего списка

И нажимаю кнопку "Анализировать"

Тогда система должна отобразить результаты анализа

Тестовые сценарии реализованы с использованием Python и библиотеки Behave. Каждый шаг сценария имеет соответствующую реализацию в файле `game_analysis_steps.py`. Например, шаг «Дано я нахожусь на странице анализа игр» реализован следующим образом (см. листинг 4.8):

Тесты интегрированы с API системы через HTTP-запросы. Основные аспекты интеграции:

1) *Взаимодействие с API:*

- все тесты используют единый базовый URL API;
- реализована обработка HTTP-ответов и ошибок;
- добавлено логирование всех запросов и ответов.

2) *Управление состоянием:*

- контекст теста (context) используется для хранения данных между шагами;
- реализована валидация наличия необходимых данных;

- обеспечена изоляция тестовых сценариев.

3) Проверка результатов:

- валидация HTTP-статусов ответов;
- проверка структуры и содержимого JSON-ответов;
- верификация бизнес-логики в результатах анализа.

Сценарий: Анализ новой игры

Дано я нахожусь на странице добавления новой игры

Когда я заполняю форму следующими данными:

Поле	Значение
name	New Fantasy RPG
description	Epic fantasy adventure game
price	59.99
genres	RPG, Action, Adventure
tags	Space, Sandbox, Mystery
categories	Single-player, Multi-player

И нажимаю кнопку "Добавить и проанализировать"

Тогда система должна добавить игру в базу данных

Этот тест демонстрирует полный цикл интеграции:

- 1) Начинается всё с получения схемы данных формы.
- 2) Происходит отправка данных новой игры с соответствующими заполненными полями.
- 3) Валидация ответа API.
- 4) Проверка результатов анализа.

4.2.2.4 Результаты тестирования с использованием BDD-подхода

Как можно увидеть на практике, тестовые сценарии, оформленные с использованием языка Gherkin, позволяют использовать сценарии в качестве живой документации, описывающей поведение системы при различных условиях, что полезно на ранних стадиях развития проекта.

В ходе тестирования была обнаружена серия дефектов, включая несоответствие формата API-ответов, отсутствие проверки входных данных на отдельных маршрутах, а также ошибки, возникавшие при генерации отчетов. После их анализа были внесены важные изменения в архитектуру системы: *стандартизирован формат ответов*, улучшена система логирования и оптимизирована производительность при работе с крупными выборками.

Рисунок 4.2 демонстрирует успешное выполнение всех автоматизированных тестовых сценариев: большинство тестовых функций завершились со статусом `passed`, что подтверждает корректную работу ключевых функций системы.

```
0 features passed, 1 failed, 0 skipped
3 scenarios passed, 3 failed, 0 skipped
16 steps passed, 1 failed, 3 skipped, 2 undefined
Took 26m22.536s
```

Рис.4.2. Результаты выполнения интеграционного тестирования

Тестирование улучшило *читаемость* и упростило *сопровождение проекта*. Автоматизация сценариев позволила *быстрее проводить регрессионное тестирование*, снизить время на ручную проверку и обеспечить стабильную работу системы при её доработке.

Таким образом, методология BDD, реализованная с помощью фреймворка *Behave*, позволила эффективно организовать интеграционное тестирование системы оценки и продвижения игр, задокументировать поведение приложения в удобочитаемой форме и обеспечить прозрачность результатов тестирования для всех участников проекта. В дальнейшем планируется расширение набора сценариев и развитие системы тестирования для охвата новых функций и обеспечения устойчивости системы к изменениям.

4.2.3 Анализ покрытия кода

Для оценки полноты модульного тестирования использовался инструмент *pytest-cov*, который позволяет определить процент покрытых строк кода. В таблице 4.3 представлены данные о покрытии кода тестами для каждого модуля системы.

Основные аналитические методы системы демонстрируют высокий уровень покрытия тестами:

- модуль кластеризации игр (*GameClusterer*) - 80% покрытия;
- анализ оценки доли продаж (*GameCompetitivenessAnalyzer*) - 83% покрытия;
- поиск похожих игр (*GameSimilarityAnalyzer*) - 74% покрытия.

Особенно высокое покрытие тестами (91%) демонстрирует модуль извлечения признаков (*FeatureExtractor*), что обеспечивает надежность базового функционала системы.

Общее покрытие кода тестами составляет 77%, что является хорошим показателем для системы такого масштаба. Особенно важно отметить высокое

Таблица 4.3

Покрывание кода тестами по модулям

Модуль	Строк кода	Пропущено	Покрывание
BaseAnalyzer	159	95	40%
DataRepository	70	25	64%
FeatureExtractor	160	15	91%
GameAnalysisPipeline	174	45	74%
GameClusterer	229	45	80%
GameCompetitivenessAnalyzer	121	20	83%
GameRecommendationAnalyzer	141	35	75%
GameSimilarityAnalyzer	98	25	74%
Decorators	18	1	94%
<i>Итого</i>	<i>1310</i>	<i>306</i>	<i>77%</i>

покрытие тестами ключевых аналитических модулей, отвечающих за расчет оценки доли продаж игр, их кластеризацию и поиск похожих продуктов, что обеспечивает надежность и стабильность работы системы.

4.2.4 Профилирование времени выполнения и использования памяти

Для оценки эффективности работы системы было проведено профилирование времени выполнения и использования оперативной памяти для ключевых методов каждого модуля. В качестве тестовых данных использовались реальные игровые объекты из базы данных (100 случайных игр). Измерения проводились с помощью инструментов *memory_profiler* и *time* на машине с процессором Intel(R) Core(TM) i3-6006U CPU @ 2.00GHz, 8 ГБ оперативной памяти, что также могло повлиять на задержки при выполнении ресурсоёмких операций. Результаты сведены в таблицу 4.4. В таблице приведены средние значения времени выполнения и максимального потребления памяти для каждого метода.

Результаты показывают, что наиболее ресурсоёмкими являются методы, связанные с кластеризацией и комплексным анализом игры, что объясняется необходимостью обработки больших объёмов данных и вычисления эмбедингов.

В то же время, методы получения справочной информации из базы данных работают практически мгновенно и не требуют значительных ресурсов. Методы ранжирования оценки доли продаж (*analyze_competitiveness*,

Таблица 4.4

Производительность ключевых методов системы

Класс	Метод	Время (с)	Память (MiB)	Результат
GameAnalysisPipeline	analyze_game	59.16	124.5	cluster_info: [...]
GameClusterer	get_features_matrix	93.23	-146.9	array([...])
GameClusterer	clusters	78.60	48.8	-
GameRecommendation...	generate_recs	0.20	0	Рекомендации [...]
GameSimilarity...	calc_similarity	7.26	44.9	730: 0.98 [...]
GameUniqueFeatures...	compare_games	6.09	-6.1	size: 5 [...]
GameCompetitiveness...	analyze_comp	2.13	2.5	[(730, 0.95)]
GameCompetitiveness...	user_analysis	0.34	0.1	0.87
ReviewAnalyzer	analyze_reviews	0.41	0.2	score: 0.82
DataRepository	get_all_games	79.32	804.2	[game_id...]
DataRepository	get_games_list	1.65	24.0	[game_id...]
DataRepository	get_schema	0.25	-7.0	genres: [...]

analyze_user_competitiveness) и анализа отзывов (analyze_reviews) показали высокую производительность: время выполнения не превышает нескольких секунд даже на относительно слабом оборудовании. Проведённый анализ подтверждает, что система обладает хорошей производительностью и эффективным использованием памяти, что позволяет масштабировать её для работы с большими объёмами данных.

4.3 Тестирование пользовательского интерфейса

Тестирование пользовательского интерфейса включает проверку основных компонентов React приложения с целью выявления возможных проблем, связанных с удобством использования (юзабилити) и отзывчивостью интерфейса. Для реализации тестирования использовалась библиотека *@testing-library/react*, которая позволяет симулировать взаимодействие с компонентами и проверять их поведение.

4.3.1 Функциональное тестирование интерфейса

Целью функционального тестирования было оценить удобство взаимодействия с основными формами и элементами интерфейса системы. Тесты охватывали следующие компоненты:

- форма выбора игры (GameIdForm);
- форма анализа новой игры (GameAnalysisForm);

Тест для компонента GameIdForm

```

test('handles game selection and submission', async () => {
  render(<GameIdForm onSubmit={mockOnSubmit} loading={false}
    />);
  await waitFor(() => {
5     expect(screen.getByLabelText(/выберите игру/i)).
      toBeInTheDocument();
  });

  const input = screen.getByLabelText(/выберите игру/i);
  fireEvent.mouseDown(input);
10  const option = await screen.findByText('Game 1');
  fireEvent.click(option);

  const submitButton = screen.getByText(/анализировать/i);
  fireEvent.click(submitButton);
15

  expect(mockOnSubmit).toHaveBeenCalled();
});

```

- отображение результатов (ResultsDisplay);
- главная страница (HomePage).

4.3.1.1 Форма выбора игры GameIdForm

Основные сценарии тестирования включали:

- 1) Проверку корректной загрузки списка игр при монтировании компонента.
- 2) Валидацию выбора игры перед отправкой формы.
- 3) Обработку состояния загрузки данных.
- 4) Корректную обработку ошибок при загрузке данных.

Пример теста для компонента GameIdForm (см. листинг 4.9 и полный код в приложении 46):

Все основные сценарии взаимодействия с формой были успешно выполнены. Компонент корректно загружает список игр и отображает его при монтировании. Обработка состояния загрузки данных работает корректно: отображается индикатор загрузки, пока данные не получены. Валидация выбора игры перед отправкой формы выполнена корректно, поскольку при попытке отправить форму без выбора игры появляется соответствующее уведомление об ошибке. Однако в

Тест для валидации обязательных полей

```

test('validates required fields', async () => {
  render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
    false} />);
  const submitButton = screen.getByText(/анализировать игру/
5    i);
  fireEvent.click(submitButton);

  await waitFor(() => {
    expect(screen.getAllByText(/обязательно/i)).toHaveLength
      (4);
  });
10 });

```

одном из тестов был выявлен дефект: при попытке повторной отправки формы без изменения выбора игра оставалась невыбранной. Проблема была устранена путём сброса состояния формы после успешной отправки.

4.3.1.2 Форма анализа новой игры *GameAnalysisForm*

В рамках тестирования формы *GameAnalysisForm* проводилась проверка следующих сценариев:

- 1) Отображение всех полей формы.
- 2) Валидация обязательных полей (например, название игры, описание, стоимость, средняя продолжительность).
- 3) Проверка множественного выбора в полях с автодополнением.
- 4) Валидация числовых значений (например, стоимость и средняя продолжительность игры).

Пример теста для валидации обязательных полей (см. листинг 4.10 и полный код в приложении 47):

Отображение всех полей формы прошло успешно, элементы корректно отображаются и доступны для взаимодействия:

- при отсутствии данных в обязательных полях отображаются соответствующие сообщения об ошибке;
- при вводе некорректных значений (например, отрицательной стоимости) отображается уведомление об ошибке, что означает корректную работу валидации числовых значений.

Листинг 4.11

Тест для проверки раскрывающихся секций

```

5 test('expands and collapses recommendation sections', () =>
  {
    render(<ResultsDisplay results={mockResults} />);
    const marketPositioning = screen.getByText(/позиционирован
      ие на рынке/i);

    fireEvent.click(marketPositioning);
    expect(screen.getByText(/рассмотрите нишевые жанры/i)).
      toBeInTheDocument();

    fireEvent.click(marketPositioning);
10 expect(screen.queryByText(/рассмотрите нишевые жанры/i)).
      not.toBeVisible();
  });

```

Была выявлена проблема: при многократном выборе жанров и категорий обнаружено дублирование выбранных элементов в поле автодополнения. Внесены изменения в логику обработки множественного выбора.

4.3.1.3 Форма анализа новой игры ResultsDisplay

Компонент ResultsDisplay тестировался по следующим аспектам:

- 1) Корректное отображение секций результатов, включая блоки рекомендаций и информации о кластерах.
- 2) Сортировка похожих игр по оценке доли продаж в их сегменте.
- 3) Работа интерактивных элементов (например, раскрывающиеся секции рекомендаций).
- 4) Отображение индикаторов оценок.

Пример теста для проверки раскрывающихся секций (см. листинг 4.11 и приложение 48):

По результатам тестирования можно сказать, что компонент корректно отображает основные секции результатов, включая блоки рекомендаций, оценку доли продаж и позиционирования на рынке:

- сортировка похожих игр по оценке доли продаж в сегменте работает корректно и отображается в порядке убывания значений;

Тест для переключения между вкладками

```

test('switches between tabs', () => {
  render(<HomePage />);

5   expect(screen.getByText(/analyze by game id/i)).
      toHaveAttribute('aria-selected', 'true');

  fireEvent.click(screen.getByText(/analyze new game/i));
  expect(screen.getByText(/analyze new game/i)).
      toHaveAttribute('aria-selected', 'true');
});

```

- интерактивные элементы (раскрывающиеся секции) успешно обрабатывают события нажатия (click и focuse).

Однако был обнаружен дефект: при переключении между рекомендациями и секциями информации о кластерах некоторые элементы оставались в свернутом состоянии. Проблема устранена путем обновления логики управления состояниями.

4.3.1.4 Главная страница HomePage

Тестирование компонента HomePage включало:

- 1) Проверку переключения между вкладками (Analyze by Game ID и Analyze New Game).
- 2) Сохранение состояния при переключении вкладок.
- 3) Обработку темной и светлой тем.

Пример теста для переключения между вкладками (см. листинг 4.12 приложение 49):

Тесты на переключение между вкладками (Analyze by Game ID и Analyze New Game) выполнены успешно. Активная вкладка корректно изменяется при нажатии. Сохранение состояния при переключении вкладок реализовано корректно: данные, введенные в формах, не теряются при переключении. Темная и светлая темы также переключаются корректно, основные элементы интерфейса адаптируются под выбранную тему. Возможные проблемы в данном компоненте устраняются путем оптимизации логики рендеринга компонентов.

4.3.1.5 Результаты функционального тестирования интерфейса

Тестирование пользовательского интерфейса выявило несколько незначительных дефектов, связанных с состояниями компонентов и обработкой форм. Все обнаруженные проблемы были исправлены, и компоненты прошли повторное тестирование. В текущей версии приложения интерфейс функционирует стабильно, корректно обрабатывает все сценарии взаимодействия и соответствует требованиям по юзабилити и отзывчивости.

4.3.2 Проверка отзывчивости интерфейса

Тестирование отзывчивости интерфейса было направлено на анализ быстродействия и корректной обработки различных состояний компонентов, что включает в себя *время отклика компонентов при взаимодействии*, а также *обработку состояний загрузки и ошибок при взаимодействии с сервером*.

Основные тестовые сценарии:

1. Время отклика интерфейса должно соответствовать следующим требованиям:
 - загрузка списка игр — менее 20 секунд;
 - переключение вкладок — менее 100 мс;
 - отображение результатов — менее 1 минуты.

Пример теста для времени отклика (см. листинг 4.13 и полный код в приложении 48) :

2. Обработка ошибок:
 - отображение сообщений об ошибках при сбое загрузки данных;
 - возможность повторной попытки отправки формы;
 - сохранение данных формы при ошибке.

Пример теста для обработки ошибки API (см. листинг 4.14):

3. Состояния загрузки:
 - отображение индикаторов загрузки (например, Spinner);
 - блокировка кнопок и форм во время загрузки данных;
 - плавные переходы между состояниями компонентов.

Пример теста для отображения индикатора загрузки (см. листинг 4.15):

Таким образом, тестирование интерфейса позволило выявить и устранить ряд критических ошибок, а также оптимизировать работу компонентов приложения.

Листинг 4.13

Тест для времени отклика

```
test('renders loading state and handles data fetch', async
    () => {
      global.fetch = jest.fn(() =>
        Promise.resolve({
          ok: true,
          json: () => Promise.resolve(mockResults)
        })
      );

      render(<HomePage />);

      const submitButton = screen.getByText(/анализировать/i);
      fireEvent.click(submitButton);

      await waitFor(() => {
        expect(screen.getByText(/оценка доли продаж/i)).
          toBeInTheDocument();
      });
    });
```

Листинг 4.14

Тест обработки ошибок API в компоненте HomePage

```
test('handles API errors gracefully', async () => {
  global.fetch = jest.fn(() => Promise.reject('API Error'))
    );

  render(<HomePage />);
  const submitButton = screen.getByText(/анализировать/i);
  fireEvent.click(submitButton);

  await waitFor(() => {
    expect(screen.getByText(/произошла ошибка/i)).
      toBeInTheDocument();
  });
});
```

Тест для отображения индикатора загрузки

```

5 | test('shows loading state', () => {
    |   render(<GameAnalysisForm onSubmit={mockOnSubmit} loading
    |     = {true} />);
    |   expect(screen.getByRole('progressbar')).
    |     toBeInTheDocument();
    | });

```

На основе проведенных тестов были внесены изменения в код для повышения отзывчивости и удобства использования системы.

4.4 Экспериментальное сравнение с существующими решениями

В данном разделе проводится анализ существующих решений в области рекомендационных систем для компьютерных игр. Основное внимание уделено системе платформы Steam, являющейся цифровой дистрибуцией игр.

4.4.1 Система рекомендаций Steam

Steam [68] предоставляет несколько публичных чартов («Top Sellers», «Most Played», «New Releases», «Upcoming Releases» и др.). Детали алгоритмов ранжирования Valve не публикует полностью, но в документации Steamworks [4] и выступлениях разработчиков содержатся ключевые сведения. Так, Steamworks Doc ясно указывает, что «списки Top Sellers генерируются автоматически на основе всех источников выручки для игры, включая DLC и внутриигровые покупки». Ключевыми параметрами при построении рейтингов являются выручка (продажи) и игровая активность (число игроков и игровое время), а не «популярность» через просмотры или отзывы, которые учитываются в нашей системе.

В целом в литературе указывается, что конкретные детали «ранжирующего весового коэффициента» закрыты. Можно сказать точно только то, что Steam ранжирует игры прежде всего по признаку реальной «ценности» — объёма денег и времени, которые игроки в них вкладывают.

При этом основной механизм рекомендаций — *Discovery Queue*, который основывается на ранее сыгранных играх, списке желаемого и активности друзей пользователя. Несмотря на регулярное обновление алгоритмов, около 68%

пользователей оценивают рекомендации как не соответствующие их ожиданиям [44].

Метакритик предлагает принципиально иной подход:

- он основан на *экспертных оценках* (70-80%) и пользовательских рейтингах (20-30%);
- оценивает *качество контента*, а не популярность;
- не учитывает коммерческие показатели (продажи, активность игроков);
- имеет проблему *накрутки оценок* и субъективности.

Поэтому наша система имеет ряд преимуществ перед Metacritic, в основном за счет объективных количественных метрик вместо субъективных оценок, учет реальных показателей успешности игры (выручка, активность) и возможности прогнозирования оценки для новых игр.

4.4.2 Сравнительный анализ рейтинговой системы

В рамках разрабатываемой системы была реализована функциональность отображения рейтингов компьютерных игр с использованием различных критериев и визуальных индикаторов. Вкладка «Game Ratings» позволяет пользователям просматривать список игр с информацией о названии, жанрах, цене и уровне оценки доли продаж. Система предоставляет возможность *фильтрации* по количеству отображаемых игр (5, 10, 15, 20, 50, 100) и жанрам, а также *сортировки* по цене и оценке доли продаж. Для удобства восприятия данных реализована цветовая индикация уровня оценки (зелёный – высокий уровень, жёлтый – средний, красный – низкий) и адаптивная сетка карточек с поддержкой анимаций и темной/светлой темы интерфейса.

В отличие от рейтинговой системы платформы Steam, которая основывается преимущественно на данных о *еженедельной посещаемости игр* и формирует топ-100 игр без возможности детальной фильтрации и сортировки, предлагаемая система предоставляет пользователю более широкий спектр возможностей для анализа. В частности, пользователи могут фильтровать игры по жанрам, регулировать количество отображаемых игр и сортировать их по ключевым показателям.

Тем не менее, подробный алгоритм формирования рейтингов в Steam остаётся *закрытым*. Неизвестны конкретные параметры, используемые для определения наиболее популярных игр, а также способы подбора похожих игр. В то же время,

предлагаемая система явно определяет и визуализирует оценку доли продаж каждой игры, что обеспечивает прозрачность и интерпретируемость результатов.

Основным недостатком предлагаемой системы является *менее оперативное обновление данных* — информация о статистиках игр обновляется раз в месяц, что может снижать её актуальность по сравнению с Steam, где данные о количестве игроков обновляются еженедельно. Это ограничение обусловлено необходимостью снижения нагрузки на сервер и уменьшения времени отклика системы.

Таким образом, предлагаемая рейтинговая система предоставляет пользователям более гибкий инструментарий для анализа игр по различным параметрам, но уступает Steam по оперативности обновления данных и учёту временных трендов. Давайте сравним полученные рейтинги 15 самых популярных игр для более детального анализа и выявления отличий в подходах к формированию рейтингов в следующем подразделе.

4.4.3 Анализ полученных рейтингов

Для оценки корректности алгоритма расчета рейтингов было проведено сравнение полученных автором рейтингов с реальными рейтингами из системы Steam. В таблице 4.5 представлены результаты для нескольких игр (на выборке из 100 продуктов).

В указанной таблице вводятся следующие обозначения:

- R_S – рейтинг Steam «top 100 in 2 weeks», подробности составления рейтинга не раскрываются – скорее всего построен на среднем количестве активных игроков;
- R_M – рейтинг Metacritic, построенный на взвешенной оценке экспертов и пользователей;
- R_A – рейтинг реализованной системы по критериям (в порядке важности): количество владельцев, активность сообщества;
- R_B – рейтинг реализованной системы по критериям (в порядке важности): Соотношение положительных отзывов, выручка, активность сообщества;
- R_C – рейтинг реализованной системы по критериям (в порядке важности): новизна игры и анализ отзывов.

Приведём пример расчета изменения порядка рейтингов между Steam и нашей системой с параметрами R_A :

1. Всего игр в таблице $n = 18$.

Таблица 4.5

Различия в рейтингах Steam и разработанной системы

Игра	R_S	R_M	R_A	R_B	R_C	Разница в квадрате				
						SA	MB	MC	SM	AB
Counter-Strike: Global Offensive	1	7	1	11	14	0	100	49	36	9
PUBG: BATTLEGROUNDS	2	8	2	14	10	0	36	4	36	16
Apex Legends	3	6	5	16	16	4	100	100	196	0
Palworld	4	18	9	18	18	25	0	0	1	0
Team Fortress 2	5	4	6	13	13	1	4	81	81	0
Call of Duty: Modern Warfare II	6	15	8	10	5	4	25	100	49	25
New World: Aeternum	7	14	10	9	8	9	25	36	49	1
Black Myth: Wukong	8	11	4	6	1	16	25	100	9	25
Grand Theft Auto V	9	1	3	2	11	36	1	100	25	81
Left 4 Dead 2	10	5	7	5	9	9	16	16	1	16
Lost Ark	11	12	11	17	6	0	25	36	1	121
War Thunder	12	13	12	15	15	0	4	4	9	0
HELLDIVERS 2	13	10	13	8	4	0	4	36	4	16
Warframe	14	16	15	12	17	1	16	1	144	25
ELDEN RING	15	3	14	3	3	1	0	0	144	0
Terraria	16	11	16	1	12	0	100	1	16	121
Path of Exile 2	17	8	18	7	7	1	1	1	81	0
Baldur's Gate 3	18	2	17	4	2	1	4	0	256	4
Сумма отклонений						108	476	780	1138	460
Изменение порядка (%)						11.15	49.13	78.49	117.4	48.48

2. Различие рейтингов без учета порядка = $12/18 * 100\% = 66\%$.

3. Изменение порядка рассчитаем с помощью коэффициента Спирмена:

– Сумма квадратов разниц:

$$\sum d_i^2 = 0+0+4+25+1+4+9+16+36+9+0+0+0+1+1+0+1+1 = 108$$

– Расчёт коэффициента Спирмена:

$$\begin{aligned}
 r_s &= 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \\
 &= 1 - \frac{648}{5814} \Bigg|_{\substack{n=18 \\ \sum d_i^2=108}} \\
 &\approx \boxed{0.8885}
 \end{aligned}$$

– Изменение порядка = $(1 - r_s) \times 100\% = (1 - 0.8885) \times 100\% \approx 11.15\%$

4. Изменение шкалы = 0% .

Исходя из таблицы, можно сделать следующие выводы:

1. Столбец SA – статистический анализ показал значимую корреляцию между рангами Steam и оценками системы ($r = 0.89$), что при уровне расхождений 11% подтверждает конвергентную валидность разработанного алгоритма. Особенно важно, что такая согласованность достигнута без использования оперативных метрик посещаемости, составляющих основу ранжирования в Steam.
2. Столбец MB – разница между рейтингом Metacritic, основанным на взвешенных средних оценках экспертов, и рейтингом нашей системы, в которой самым важным критерием было соотношение положительных отзывов, затем выручка и после – активность сообщества. Повторить в точности рейтинг с оценками, предоставляемыми критиками, не всегда возможно, так как оценка часто является эмпирической, однако изменение порядка рейтинга составляет около 49%, связь средняя – критерии нужно задавать количественной важностью.
3. Столбец MC – между Metacritic и R_C наблюдается слабая корреляция ($r \approx 0.22$), из чего можно сделать предположение, что критерий «новизна игры» не учитывается в экспертных оценках, а анализ отзывов в нашей системе может конфликтовать с пользовательскими рейтингами Metacritic из-за разницы в методах сбора данных.
4. Столбец SM – отсутствие корреляции между Steam и Metacritic (117%) подтверждает, что популярность игры (Steam) и её критическая оценка (Metacritic) – независимые метрики. Успешная игра может быть либо массовой, либо высокооценённой, но редко сочетает оба фактора.
5. AB – разница между R_A и R_B , у которых нет общих показателей, однако корреляционная связь между рейтингами имеется и равна 51% - то есть рейтинги отчасти сходятся.

Несмотря на *неизвестность алгоритма* формирования рейтингов в Steam, нам удалось выделить ключевые факторы, которые, вероятно, также учитываются их системой. Возможно, помимо активности игроков, данная платформа использует и другие параметры, такие как отзывы пользователей, среднее время игры или популярность среди конкретных целевых аудиторий. Благодаря включению подобных признаков, а также проектированию нашей системы сверху-вниз, мы смогли добиться *выборочно хорошей аппроксимации рейтинга Steam* и приблизиться к его логике ранжирования, сохранив при этом прозрачность алгоритма.

С другой стороны, рейтинг Metacritic, основанный на экспертных оценках и пользовательских отзывах, слабо коррелирует с нашими алгоритмами (изменение порядка 49–78%). Это ожидаемо, так как Metacritic оценивает качество игры, а наша система фокусируется на популярности и коммерческих показателях. Для более точного прогнозирования успеха игры стоит комбинировать оба подхода.

Кроме того, наша система может предсказать оценку доли продаж разрабатываемой игры, отсутствующей в Steam, и таким образом предположить её место в реальных рейтингах в будущем и спрогнозировать уровень коммерческого успеха.

Таким образом, разработанная система рейтингов демонстрирует высокую согласованность с популярностью игр в Steam (11% расхождений), но средне предсказывает оценки Metacritic. Это подтверждает гипотезу её применимости для оценки игр, но не художественной ценности.

4.4.4 Анализ полученных схожих игр и формирование рекомендаций

В Steam подбор списка похожих игр осуществляется не только на основе схожести игровых характеристик, но и с учётом предпочтений конкретного пользователя. Основной подход, используемый для формирования рекомендаций, — это *коллаборативная фильтрация*, которая анализирует поведение пользователей и выявляет закономерности в их взаимодействии с играми. В отличие от этого, разработанная система использует *контентно-ориентированный подход*, анализирующий объективные параметры игр для формирования рекомендаций и оценки доли продаж в определённом сегменте.

4.4.4.1 Пример работы системы рекомендаций

Для демонстрации функциональности рассмотрим анализ игры *Arma 3*. Система генерирует развернутые рекомендации в трех аспектах (см. листинг 4.16):

Таким образом, разработанная система рекомендаций не только выявляет объективные характеристики игры, но и предоставляет разработчикам:

- аналитически обоснованные направления оптимизации;
- измеримые параметры для принятия решений;
- инструменты для баланса между нишевой специализацией и массовой привлекательностью.

Пример вывода системы для Arma 3

```

5  === ПОЗИЦИОНИРОВАНИЕ НА РЫНКЕ ===
   - Игра использует нишевые жанры (Strategy), которые характер
     ны для похожих игр, но отсутствуют в разных. Это может бы
     ть преимуществом для целевой аудитории.
   - Игра использует жанры (Simulation), которые характерны для
     разных игр. Это может помочь привлечь новую аудиторию.

10  === УНИКАЛЬНЫЕ ОСОБЕННОСТИ ===
   Жанры:
     Military (важность: 0.7)
     Tactical (важность: 0.7)
15   Realistic (важность: 0.6)
   Технические особенности:
     Steam Workshop (важность: 0.8)
     Moddable (важность: 0.9)

   === РЕКОМЕНДАЦИИ ПО УЛУЧШЕНИЮ ===
15  Уникальные теги (Tactical, Military, Moddable, Simulation,
     Shooter, First-Person, Realistic, War, Team-Based, FPS) м
     огут привлечь специфическую аудиторию.
   Описание игры значительно отличается от описаний похожих игр
     . Рекомендуется добавить больше общих элементов, характер
     ных для жанра.
   Добавить популярные теги:
     Story Rich
     Team-Based
20   Dark Fantasy
   Расширить поддержку (добавить категории):
     Remote Play Together
     Steam Cloud
   Акцентировать в описании:
25  "With a massive arsenal at your disposal, Arma 3 moves you
     into a world of tactical opportunities"
   "Run through the competitive firing drills to hone your
     shooting and movement skills, and complete your training
     by signing up to Arma 3 Bootcamp, which features SP and
     MP tutorials, and a dedicated Virtual Reality practice
     environment"

```

Данный пример подтверждает, что система успешно решает поставленную задачу и предоставляет практические рекомендации для повышения оценки доли продаж игровых продуктов на основе комплексного анализа их характеристик.

4.4.4.2 Анализ полученных схожих игр

Список похожих игр для *Arma 3* в Steam может включать такие игры, как *Team Fortress 2*, *The Finals*, *Garry's Mod*, *CS:GO* и *PAYDAY 2*, не только потому, что они обладают схожими жанровыми характеристиками, но и потому, что их предпочитали пользователи с похожими игровыми паттернами.

В отличие от этого, в разработанной системе для игры *Arma 3* формирование списка похожих игр происходит исключительно на основе *контентной схожести*. Алгоритм оценивает схожесть по таким признакам, как жанры, цена, метакритик и медианное время игры, что позволяет выявить кластер игр с аналогичными характеристиками. Например, для *Arma 3* такими играми стали:

- *Warframe* (RPG, Free To Play, Action) – схожесть: 50%;
- *Halo Infinite* (Free To Play, Action) – схожесть: 44%;
- *PAYDAY 2* (RPG, Action) – схожесть 37%.

Разработанная система не учитывает персональные предпочтения пользователя, а формирует список только на основе схожести контента, однако этого и не требуется – функция поиска похожих игр введена для более точной оценки доли продаж в определенном сегменте и составления рекомендаций по улучшению входной игры, и как можно заметить, система подбирает игры подобно платформе Steam, что вполне удовлетворяет поставленные задачи в нашем проекте.

4.4.5 Выводы на основе экспериментального сравнения

Как уже говорилось, платформа Steam формирует рейтинг на основе актуальной вовлечённости пользователей (например, количества запусков, времени онлайн, поведения игроков в последние дни или недели), что делает её модель динамичной и чувствительной к краткосрочным колебаниям популярности. В противоположность этому, разработанная система ориентирована на оценку *долгосрочной доли продаж* игр на основе структурных, стабильных признаков: жанров, отзывов, выручки, времени игры и ключевых аспектов контента. Это принципиально иной подход, не преследующий цель имитировать текущий трендовый рейтинг, а направленный на *обоснованную и воспроизводимую оценку качества продукта*.

Оценка качества работы разработанной системы ранжирования проводилась путём сравнения позиций игр, полученных в нашей системе, с официальными рейтингами платформы Steam. Коэффициент ранговой корреляции Спирмена составил $r_s = 0.8885$, что указывает на сильную корреляционную связь двух рейтингов.

Таким образом, *разработанная система работает корректно в контексте поставленной цели* – она даёт интерпретируемую, обоснованную и стабильную оценку доли продаж, пригодную для анализа как выпущенных, так и разрабатываемых игр. Сравнение с Metacritic же показывает не ошибки модели, а различие в логике и задачах ранжирования.

Есть и другие ключевые особенности разработанной системы по сравнению с платформой Steam:

1. Система может анализировать игры, ещё не выпущенные на платформе, и прогнозировать их потенциальное место в рейтинге.
2. Система не просто подбирает похожие игры, но и формулирует рекомендации по улучшению коммерческой привлекательности продукта.
3. Пользователи могут фильтровать и сортировать игры по различным критериям, включая жанры, цену и оценку доли продаж игр.
4. Система использует единый подход для анализа как выпущенных, так и разрабатываемых игр, что упрощает процесс оценки и прогнозирования их доли продаж.

Таким образом, несмотря на упрощённую модель ранжирования и отсутствие учёта пользовательских предпочтений, разработанная система обеспечивает стабильное качество ранжирования, высокую интерпретируемость результатов и уникальные аналитические возможности по сравнению с Steam.

4.5 Апробация на конференции

В апреле 2025 года на конференции студентов, аспирантов и молодых ученых «Современные технологии в теории и практике программирования» был представлен доклад на тему «Разработка системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению». Основная цель доклада заключалась в демонстрации возможностей предлагаемой системы по анализу текстовых и числовых данных о компьютерных играх для прогнозирования их доли продаж в сегменте и формирования рекомендаций для разработчиков.

В ходе выступления было отмечено, что система основана на комбинированном подходе, включающем кластеризацию данных и метод взвешенной суммы для оценки доли продаж игр. В процессе обсуждения были заданы вопросы, которые позволили уточнить отдельные аспекты работы системы.

Наиболее значимым был вопрос, касающийся механизма расчета оценки доли продаж:

«Как строится оценка игры, что больше влияет – отзывы или рейтинг игры, ведь отзывы могут быть негативными, а рейтинг высоким?»

В ответе было отмечено, что оценка доли продажи игры определяется как взвешенная оценка, учитывающая как текстовые отзывы, так и числовые рейтинги. Однако в модели используется повышенный вес для текстовых отзывов, так как они содержат более детализированную информацию о пользовательских предпочтениях и проблемах.

В дальнейшем можно ввести дополнительную проверку и, в случае противоречивых данных (например, высокий рейтинг и негативные отзывы), использовать анализ частотности упоминания ключевых слов и общую тональность текста отзывов, чтобы лучше определить оценку игры. На данный момент отзывы обладают большей значимостью в контексте определения оценки доли продажи игры.

Также был поднят вопрос о применении системы обычными пользователями и разработчиками:

«Как именно использовать систему обычным пользователям и как это может помочь разработчикам?»

Для обычных пользователей система предоставляет возможность ввести данные о разрабатываемой игре (жанр, описание, основные механики и ценовую категорию) и получить список аналогичных игр с рекомендациями по улучшению. Например, если система выявит, что игры аналогичного жанра с определенными механиками имеют более высокий рейтинг при наличии многопользовательского режима, она предложит включить аналогичную механику в проектируемую игру.

Для разработчиков данная система позволяет:

- выявить конкурентные продукты и проанализировать их сильные и слабые стороны;
- оценить текущее положение разрабатываемой игры на рынке, сравнив её с уже выпущенными проектами;

- получить рекомендации по улучшению игры на основе анализа отзывов и метрик наиболее успешных аналогов;
- спрогнозировать оценку доли продаж игры в сегменте похожих игр до её релиза, что может быть полезно для маркетинговой стратегии и привлечения инвестиций.

Таким образом, представленная система способна не только оценивать долю продаж игр в нужном сегменте, но и предоставлять стратегические рекомендации по улучшению проекта, повышая его шансы на коммерческий успех.

4.6 Выводы

В данной главе были протестированы ключевые модули системы, включая алгоритмы кластеризации, анализа текстовых данных и расчета оценки доли продаж игр в сегменте похожих игр. Для тестирования использовались заранее подготовленные игровые данные, включающие текстовые описания, жанры, рейтинги и ценовые категории. В результате тестирования были выявлены и устранены ошибки в логике обработки данных и вычислении метрик оценки доли продаж, а также выполнены следующие задачи:

- 1) Проведено модульное тестирование (60+ тестов, 77% покрытие кода), интеграционное тестирование (BDD-подход с Behave), проверяющее корректность обмена данными с сервером, и профилирование производительности, показавшее хорошую производительность системы и эффективное использование памяти.
- 2) Выполнено сравнение с существующими решениями: система показала высокую корреляцию с рейтингами Steam (11% расхождений), среднюю с Metacritic.
- 3) В апреле 2025 года результаты проведенных исследований и тестирования системы были представлены на конференции студентов, аспирантов и молодых ученых «Современные технологии в теории и практике программирования».

Таким образом, был осуществлён комплекс мероприятий по тестированию и апробации системы поддержки анализа компьютерных игр. Полученные результаты подтвердили корректность работы системы, а также выявили направления для её дальнейшего развития – улучшение пользовательского интерфейса, рас-

ширение набора анализируемых данных и внедрение дополнительных методов прогнозирования.

ЗАКЛЮЧЕНИЕ

Несмотря на стремительное развитие игровой индустрии и рост числа отечественных разработчиков, особенно в сегменте инди-игр, проблема прогнозирования успеха компьютерных игр на начальном этапе проектирования остается недостаточно изученной. Решение этой проблемы может значительно повлиять на эффективность разработки, продвижения и дальнейшей поддержки игр на рынке.

В результате работы была разработана система поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению.

Настоящее исследование *подтвердило гипотезу* о возможности построения системы ранжирования, сочетающей современные методы анализа данных и теорию принятия решений. Основные научные и практические результаты работы заключаются в следующем:

- 1) Проведён анализ существующих методов и средств ранжирования компьютерных игр, основной минус которых заключается в фокусе на поведенческих данных (количество пользователей, социальные связи) и, как следствие, непригодность для оценки новых игр.
- 2) Платформа Steam определена как оптимальный источник данных, предоставляющий метаинформацию об играх (жанры, теги, описания, среднее количество часов на прохождение, категории и т.д.), объективные метрики (активность игроков и количество пользователей) и текстовые отзывы.
- 3) Разработана математическая модель поддержки анализа компьютерных игр, сочетающая кластерный анализ (K-means++ с метриками силуэта и Дэвиса-Болдина), метод последовательного уточнения предпочтений Подиновского, а также алгоритмы формирования рекомендаций по улучшению игр, включающие анализ уникальных способностей, выявление конкурентных преимуществ и рекомендации по ценовому позиционированию.
- 4) Разработано веб-приложение на библиотеке React, предоставляющее интерактивный интерфейс для настройки критериев ранжирования, возможность просмотра и сортировки рейтинга игр по различным показателям, а также гибкую систему фильтрации.
- 5) Проведена всесторонняя проверка системы, позволившая оценить функциональную пригодность системы, совместимость (взаимодействие между модулями, интеграция с API и корректность экспорта отчетов) и произ-

водительность (время отклика меньше 3 сек для 100 игр при получении рейтингов) системы.

- 6) Проведено сравнение с существующими решениями, выявившее высокую корреляцию (89%) с рейтингом Steam Top и среднюю корреляцию с Metacritic. При этом разработанная система работает по открытым правилам рейтингования и позволяет настраивать качественную важность критериев.
- 7) Полученные результаты были представлены и обсуждены на профильной конференции, что подтвердило актуальность и корректность выбранной методологии.

Проведенная работа демонстрирует эффективность предложенного подхода для решения актуальной задачи прогнозирования успеха компьютерных игр. Разработанное приложение может помочь как независимым разработчикам, так и крупным студиям *сократить финансовые расходы*, связанным с разработкой нерентабельных игр. Кроме того, оно позволит *оптимизировать продукты* на стадии разработки и корректировать жанровую принадлежность на основе выявленных нишевых комбинаций. Создание системы рекомендаций будет *способствовать разработке уникальных и востребованных игр*, тем самым минимизируя риски, связанные с копированием идей или невостребованностью проекта.

Перспективы развития системы включают:

- 1) Внедрение BERT-моделей для анализа отзывов.
- 2) Добавление модуля прогнозирования жизненного цикла игр.
- 3) Интеграцию с Unity/Unreal Engine для предиктивного анализа на этапе разработки.

Выражаю благодарность научному руководителю Пархоменко Владимиру Андреевичу за экспертные рекомендации в ходе выполнения работы.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

TLG	Top-Level Genres.
NLP	Natural Language Processing.
API	Application programming interface.
RS	Recommender System.
TF	Term Frequency.
IDF	Inverse Document Frequency.
CF	Collaborative Filtering.
ЦА	Целевая Аудитория.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Lyadova L. N., Malkova K. M., Timofeev M. V.* Архитектура рекомендательной системы, настраиваемой на предметные области // Название сборника конференции. — Город проведения, 2017.
2. *Бейзер Б.* Тестирование чёрного ящика. Технологии функционального тестирования программного обеспечения и систем. — СПб.: Питер, 2004. — 320 с.
3. Документация Django. — URL: <https://django.fun/docs/django/5.2/> (дата обращения: 10.05.2025).
4. Документация Steamworks. — URL: <https://partner.steamgames.com/doc/home> (дата обращения: 08.01.2025).
5. *Микони С. В.* Теория принятия управленческих решений: учебное пособие для вузов. — 2-е изд. — Город: Издательство, 2022. — 384 с.
6. *Платонов Е. Н., Мартынова И. Р.* Семантический анализ отзывов об организациях методами машинного обучения // Моделирование и анализ данных. — 2024. — Т. 14, № 1. — С. 7—26. — DOI 10.17759/mda.2024140101.
7. *Подиновский В. В.* Идеи и методы теории важности критериев в многокритериальных задачах принятия решений. — 1-е изд. — Москва, 2019. — 102 с. — (Дата обращения: 15.01.2025).
8. *Подиновский В. В.* Количественная важность критериев и аддитивные функции ценности // Журнал вычислительной математики и математической физики. — 2013. — Т. 53, № 1. — С. 133—142.
9. Система по поддержке анализа и рейтингования компьютерных игр, а также составления рекомендаций по их улучшению. — URL: <https://github.com/Gonerr/SystemGA> (дата обращения: 11.06.2025).
10. Список пользовательских обзоров. — URL: <https://partner.steamgames.com/doc/store/getreviews> (дата обращения: 08.01.2025).
11. *Ahn S., Kang J., Park S.* What makes the difference between popular games and unpopular games? analysis of online game reviews from steam platform using word2vec and bass model // ICIC Express Letters. — 2017. — Т. 11, № 12. — С. 1729—1737.
12. *Alves M., Oliveira B., Guimarães F.* Ensemble ranking: An aggregation of multiple multicriteria methods and scenarios and its application to power generation planning // Decision Analytics Journal. — 2024. — Т. 10. — С. 100435. — DOI 10.1016/j.dajour.2024.100435.

13. An Empirical Study of Game Reviews on the Steam Platform / D. Lin [и др.] // Empirical Software Engineering. — 2019. — Т. 24. — DOI 10.1007/s10664-018-9627-4.
14. Andreev G., Saxena D., Verma J. K. Impact of review sentiment and magnitude on customers' recommendations for video games // 2021 international conference on computational performance evaluation (ComPE). — IEEE. 2021. — С. 992—995.
15. Becker K. 4PEG: A structured rating system for games for learning //. — 2015. — С. 1—6. — DOI 10.1109/GEM.2015.7377212.
16. Behave Pro Team. Behave Pro: BDD for Jira / Behave Pro. — URL: <https://behavepro.app> (дата обращения: 20.05.2025); (Официальный сайт Behave Pro).
17. Bunga R., Batista F., Ribeiro R. From Implicit Preferences to Ratings: Video Games Recommendation based on Collaborative Filtering //. — 2021. — С. 209—216. — DOI 10.5220/0010655900003064.
18. Condensing Steam: Distilling the Diversity of Gamer Behavior / M. O'Neill [и др.] // Proceedings of the 2016 Internet Measurement Conference. — 2016. — URL: <https://api.semanticscholar.org/CorpusID:15134651>.
19. DRGame: Diversified Recommendation for Multi-category Video Games with Balanced Implicit Preferences / K. Liu [и др.]. — 2023. — DOI 10.48550/arXiv.2308.15823.
20. Ferreira F., Borges H. S., Valente M. T. Refactoring react-based Web apps // Journal of Systems and Software. — 2024. — Т. 215. — С. 112105. — DOI <https://doi.org/10.1016/j.jss.2024.112105>. — URL: <https://www.sciencedirect.com/science/article/pii/S016412122400150X>.
21. Game Experience Evaluation. A Study of Game Reviews on the Steam Platform / B. I. [и др.] // Digital Transformation and Global Society. — 2020. — С. 10. — URL: <https://publications.hse.ru/en/chapters/424293771>.
22. Gauge: Lightweight Cross-Platform Test Automation / ThoughtWorks. — URL: <https://gauge.org> (дата обращения: 20.05.2025); (Официальный сайт фреймворка Gauge).
23. Gong J., Ye Y., Stefanidis K. A Hybrid Recommender System for Steam Games //. — 2020. — С. 133—144. — DOI 10.1007/978-3-030-44900-1_9.
24. Guzsvinecz T., Szűcs J. Length and sentiment analysis of reviews about top-level video game genres on the steam platform // Computers in Human Behavior. — 2023. — Т. 149. — С. 107955. — DOI <https://doi.org/10.1016/j.chb.2023.107955>. — URL: <https://www.sciencedirect.com/science/article/pii/S0747563223003060>.

25. *Ikram F., Farooq H.* Multimedia Recommendation System for Video Game Based on High-Level Visual Semantic Features // *Scientific Programming*. — 2022. — Т. 2022. — DOI 10.1155/2022/6084363.
26. *Irshad M., Britto R., Petersen K.* Adapting Behavior Driven Development (BDD) for large-scale software systems // *Journal of Systems and Software*. — 2021. — Т. 177. — С. 110944. — DOI <https://doi.org/10.1016/j.jss.2021.110944>. — URL: <https://www.sciencedirect.com/science/article/pii/S0164121221000418>.
27. *Kang H.-N., Yong H.-R., Hwang H.-S.* A Study of analyzing on online game reviews using a data mining approach: STEAM community data // *International Journal of Innovation, Management and Technology*. — 2017. — Т. 8, № 2. — С. 90. — DOI <https://doi.org/10.18178/ijimt.2017.8.2.709>.
28. Learning to Hash for Recommendation: A Survey / F. Luo [и др.]. — 2024. — DOI 10.48550/arXiv.2412.03875.
29. *Li X., Zhang B.* A preliminary network analysis on steam game tags: another way of understanding game genres // — Tampere, Finland: Association for Computing Machinery, 2020. — С. 65—73. — (Сер.: AcademicMindtrek '20). — DOI 10.1145/3377290.3377300. — URL: <https://doi.org/10.1145/3377290.3377300>.
30. *Li X., Zhang Z., Stefanidis K.* A Data-Driven Approach for Video Game Playability Analysis Based on Players' Reviews // *Information*. — 2021. — Т. 12, № 3. — DOI 10.3390/info12030129. — URL: <https://www.mdpi.com/2078-2489/12/3/129>.
31. *Manhes S.* Steam Games Dataset 2025. — 2025. — URL: <https://www.kaggle.com/datasets/srgiomanhes/steam-games-dataset-2025>; (Версия 3.1 (2025-04-15)).
32. *Marchand A., Hennig-Thurau T.* Value Creation in the Video Game Industry: Industry Economics, Consumer Benefits, and Research Opportunities // *Journal of Interactive Marketing*. — 2013. — Т. 27. — С. 141—157. — URL: <https://doi.org/10.1016/j.intmar.2013.05.001> (дата обращения: 19.11.2024).
33. *Microsoft Corporation.* Visual Studio Code: Code Editing. Redefined / Microsoft. — URL: <https://code.visualstudio.com> (дата обращения: 20.05.2025); (Официальный сайт VS Code).
34. *Mukherjee T.* Videogame Distribution and Steam's Imperialist Practices: Platform Coloniality in Game Distribution // *Journal of Games Criticism*. — 2023. — Т. 5. — С. 15. — URL: <https://gamescriticism.org/2023/08/23/mukherjee-5-a/>.
35. *Philp M., Nepomuceno M. V.* How reviews influence product usage post-purchase: An examination of video game playtime // *Journal of Business Research*. —

2024. — T. 172. — C. 114456. — DOI <https://doi.org/10.1016/j.jbusres.2023.114456>. — URL: <https://www.sciencedirect.com/science/article/pii/S0148296323008159>.

36. Rating Generation of Video Games using Sentiment Analysis and Contextual Polarity from Microblog / S. Chakraborty [и др.] // — 2018. — C. 157—161. — DOI 10.1109/CTEMS.2018.8769149.

37. *Reimers N., Gurevych I.* Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. — 2019. — DOI 10.48550/arXiv.1908.10084.

38. *Rojas H., Arias K. A., Renteria R.* Service-oriented architecture design for small and medium enterprises with infrastructure and cost optimization. // *Procedia Computer Science*. — 2021. — T. 179. — C. 488—497. — DOI <https://doi.org/10.1016/j.procs.2021.01.032>. — URL: <https://www.sciencedirect.com/science/article/pii/S1877050921000375>; (5th International Conference on Computer Science and Computational Intelligence 2020).

39. Sentiment analysis of product review / K. Wase [и др.] // *International Journal of Innovations in Engineering and Science*. — 2018. — T. 3, № 5. — C. 8—13.

40. *Signorini A.* A Survey of Ranking Algorithms. — 2005.

41. Solving multicriteria problems of rating alternatives based on pairwise comparisons / Krivulin [и др.]. — 2024. — DOI 10.32603/2071-2340-2024-2-5-29. — URL: <http://cte.eltech.ru/ojs/index.php/kio/article/view/1833>.

42. *Sukardi Teja A., Hanafi M., Qomariyah N. N.* Predicting Steam Games Rating with Regression // *E3S Web of Conferences*. — 2023. — T. 388. — DOI 10.1051/e3sconf/202338802001.

43. The influencing factors of game brand loyalty / X. Zhao [и др.] // *Heliyon*. — 2024. — T. 10. — e31324. — DOI 10.1016/j.heliyon.2024.e31324.

44. *Wang D., Moh M., Moh T.-S.* Using Deep Learning and Steam User Data for Better Video Game Recommendations // — Tampa, FL, USA: Association for Computing Machinery, 2020. — C. 154—159. — (Сер.: ACMSE '20). — DOI 10.1145/3374135.3385283. — URL: <https://doi.org/10.1145/3374135.3385283>.

45. *Zhang W., Wu Z.* E-commerce recommender system based on improved K-means commodity information management model // *Heliyon*. — 2024. — T. 10, № 9. — e29045. — DOI <https://doi.org/10.1016/j.heliyon.2024.e29045>. — URL: <https://www.sciencedirect.com/science/article/pii/S240584402405076X>.

46. *Zhi Wang Victor Chang G. H.* Explaining and Predicting Helpfulness and Funniness of Online Reviews on the Steam Platform // *Journal of Global Information*

Management (JGIM). — 2021. — Т. 29, вып. 6. — С. 23. — URL: <https://www.igi-global.com/gateway/article/273880>.

47. *Павлович К. С.* Влияние оценок игроков и критиков на продажи видеоигр // StudNet. — 2020. — Т. 8, вып. 6. — С. 23. — URL: <https://cyberleninka.ru/article/n/vliyanie-otsenok-igrokov-i-kritikov-na-prodazhi-videoigr>.

48. Angular. — URL: <https://angular.dev/> (visited on 11.10.2024).

49. Behave documentation. — URL: <https://behave.readthedocs.io/en/latest/> (visited on 20.05.2025).

50. Cucumber documentation. — URL: <https://cucumber.io/docs/gherkin/reference/> (visited on 20.05.2025).

51. Dependency injection in Angular. — URL: <https://angular.dev/guide/di> (visited on 11.01.2024).

52. Entertainment Software Association. — URL: <https://www.theesa.com/> (visited on 08.01.2025).

53. Epic Games Store. — URL: <https://store.epicgames.com/ru/?ref=website-popularity> (visited on 08.01.2025).

54. FastAPI documentation. — URL: <https://fastapi.tiangolo.com/> (visited on 10.05.2025).

55. Forza Horizon 5. — URL: https://store.steampowered.com/app/1551360/Forza_Horizon_5/ (visited on 15.01.2024).

56. *Foundation P. S.* Python Documentation. — URL: <https://docs.python.org/> (visited on 20.10.2024).

57. Game Jolt - Share your creations. — URL: <https://gamejolt.com/games> (visited on 11.01.2025).

58. GamersGate Store. — URL: <https://www.gamersgate.com/> (visited on 11.01.2025).

59. GOG.com. — URL: <https://www.gog.com/ru/> (visited on 08.01.2025).

60. *Kim K., Ahn H.* A recommender system using GA K-means clustering in an online shopping market // Expert Systems with Applications. — 2008. — Vol. 34, no. 2. — P. 1200–1209. — DOI 10.1016/j.eswa.2006.12.025. — URL: <https://doi.org/10.1016/j.eswa.2006.12.025>.

61. Machine Learning Market Size, Share, and Trends 2024 to 2034. — URL: <https://www.precedenceresearch.com/machine-learning-market> (visited on 24.12.2024).

62. Node.js — Run JavaScript Everywhere. — URL: <https://nodejs.org/en> (visited on 11.10.2024).

63. Origin. — URL: <https://www.ea.com/ru-ru/games/library/pc-download#!> (visited on 11.01.2025).
64. PyDASS. — URL: <https://github.com/maksimio/pydass> (visited on 10.05.2025).
65. React is the library for web and native user interfaces. — URL: <https://react.dev/> (visited on 11.10.2024).
66. Spring Boot. — URL: <https://spring.io/projects/spring-boot/> (visited on 10.05.2025).
67. Steam Recommender. — URL: <https://store.steampowered.com/recommender> (visited on 11.01.2024).
68. Steam Store. — URL: <https://store.steampowered.com/> (visited on 12.09.2024).
69. steam-video-games. — URL: <https://www.kaggle.com/tamber/steam-video-games/> (visited on 15.01.2024).
70. SteamDB / Steam Database. — URL: <https://steamdb.info/> (visited on 22.10.2024).
71. SteamSpy - All the data and stats about Steam games. — URL: <https://steamspy.com/> (visited on 16.02.2025).
72. The Developer Data Platform | MongoDB. — URL: <https://www.mongodb.com/?ref=streamlit> (visited on 11.01.2025).
73. The Progressive JavaScript Framework | Vue.js. — URL: <https://vueframework.com/> (visited on 11.10.2024).
74. The world's most advanced open source... — PostgreSQL. — URL: <https://www.postgresql.org/> (visited on 11.01.2025).
75. Top selling and top played games across Steam. — URL: <https://store.steampowered.com/charts/topselling/NL> (visited on 02.06.2025).
76. Video Game Reviews, Articles, Trailers and more - Metacritic. — URL: <https://www.metacritic.com/game/> (visited on 12.09.2024).
77. VK Play. — URL: <https://vkplay.ru/> (visited on 16.02.2025).

Приложение 1

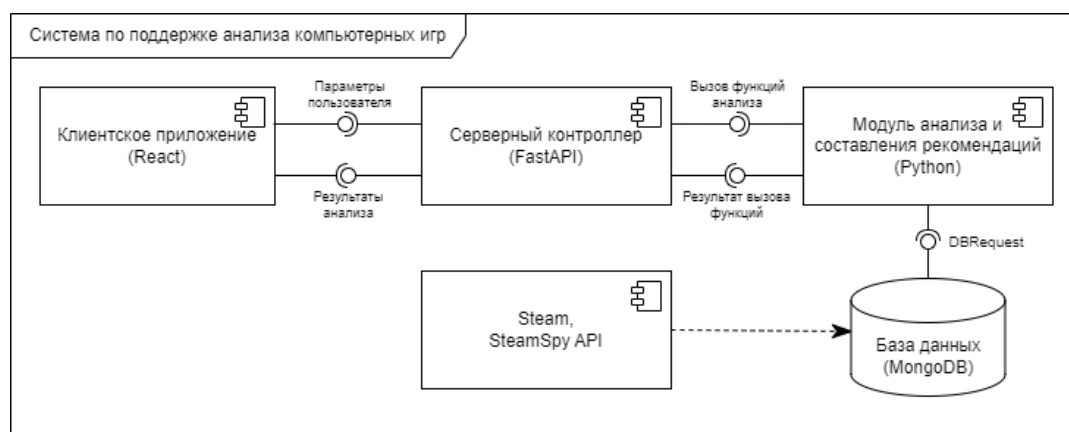
**Диаграмма компонентов системы поддержки анализа компьютерных игр
для их ранжирования и составления рекомендаций по улучшению**

Рис.П1.1. Диаграмма компонентов системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

Приложение 2

**Диаграмма случаев использования системы поддержки анализа
компьютерных игр для их ранжирования и составления рекомендаций по
улучшению**

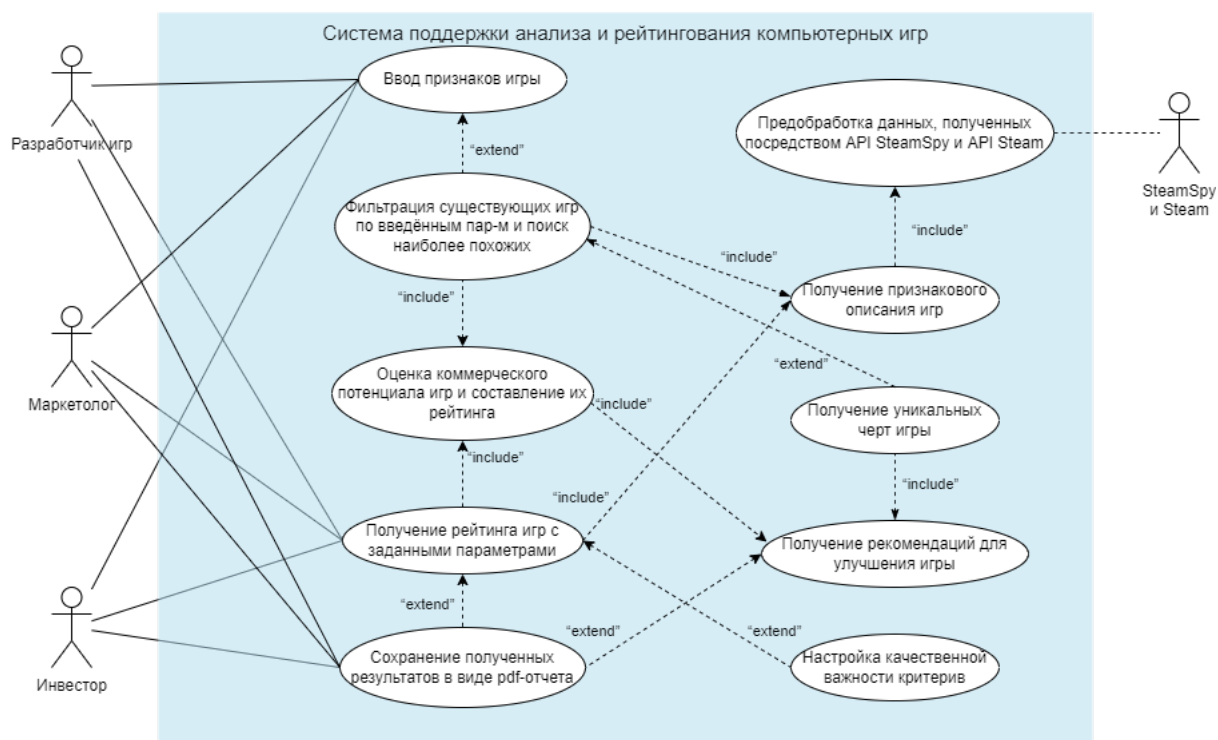


Рис.П2.1. Диаграмма случаев использования системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

Приложение 3

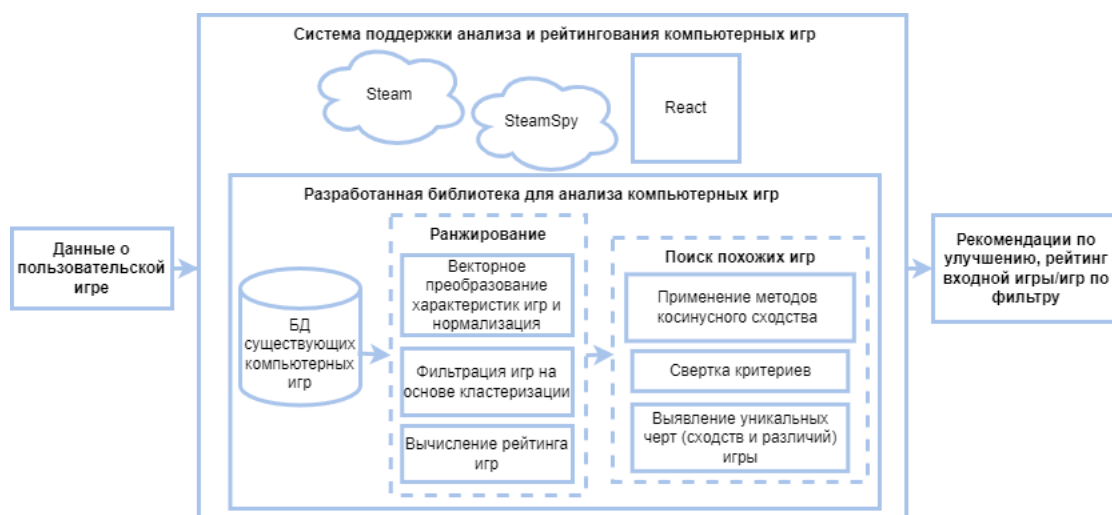
Архитектура системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

Рис.ПЗ.1. Архитектура системы поддержки анализа компьютерных игр для их ранжирования и составления рекомендаций по улучшению

Приложение 4

Логическая модель документоориентированной БД GamesAnalyzer с цветовой маркировкой источников данных

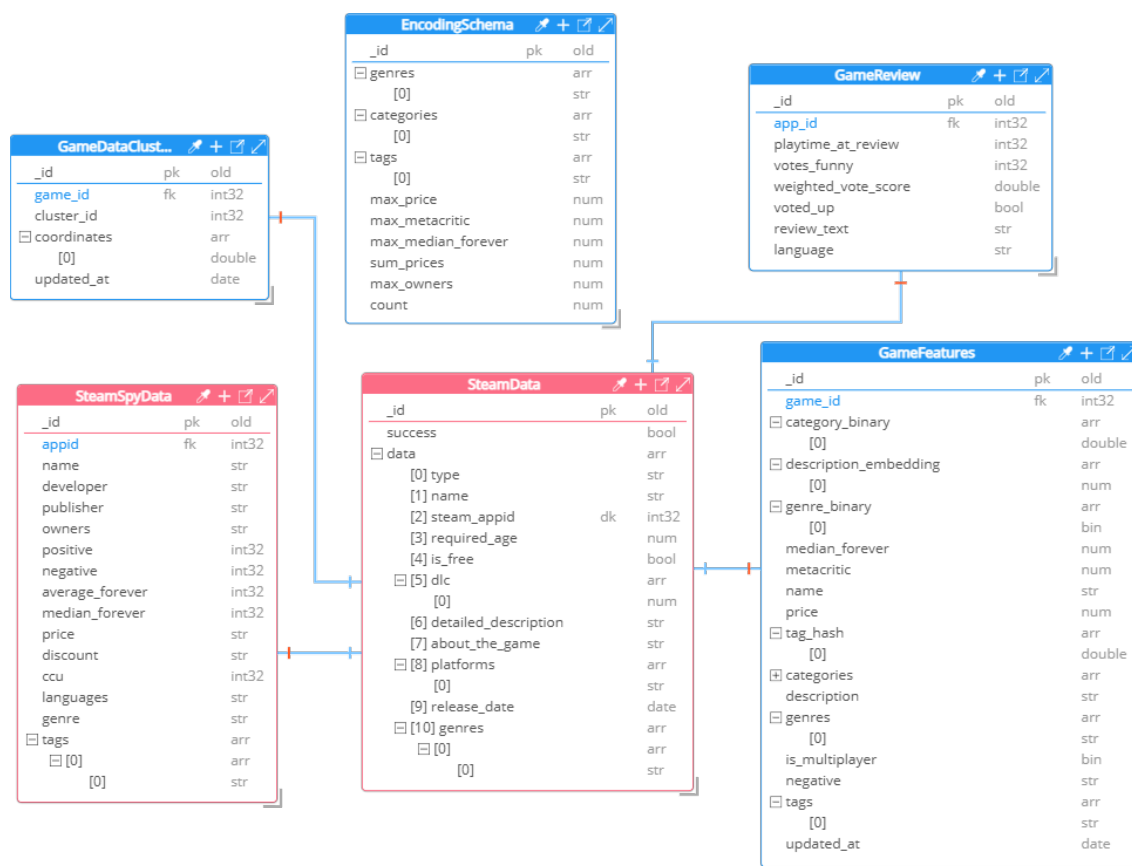


Рис.П4.1. Логическая модель документоориентированной БД GamesAnalyzer с цветовой маркировкой источников данных

Диаграмма классов системы GamesAnalyze

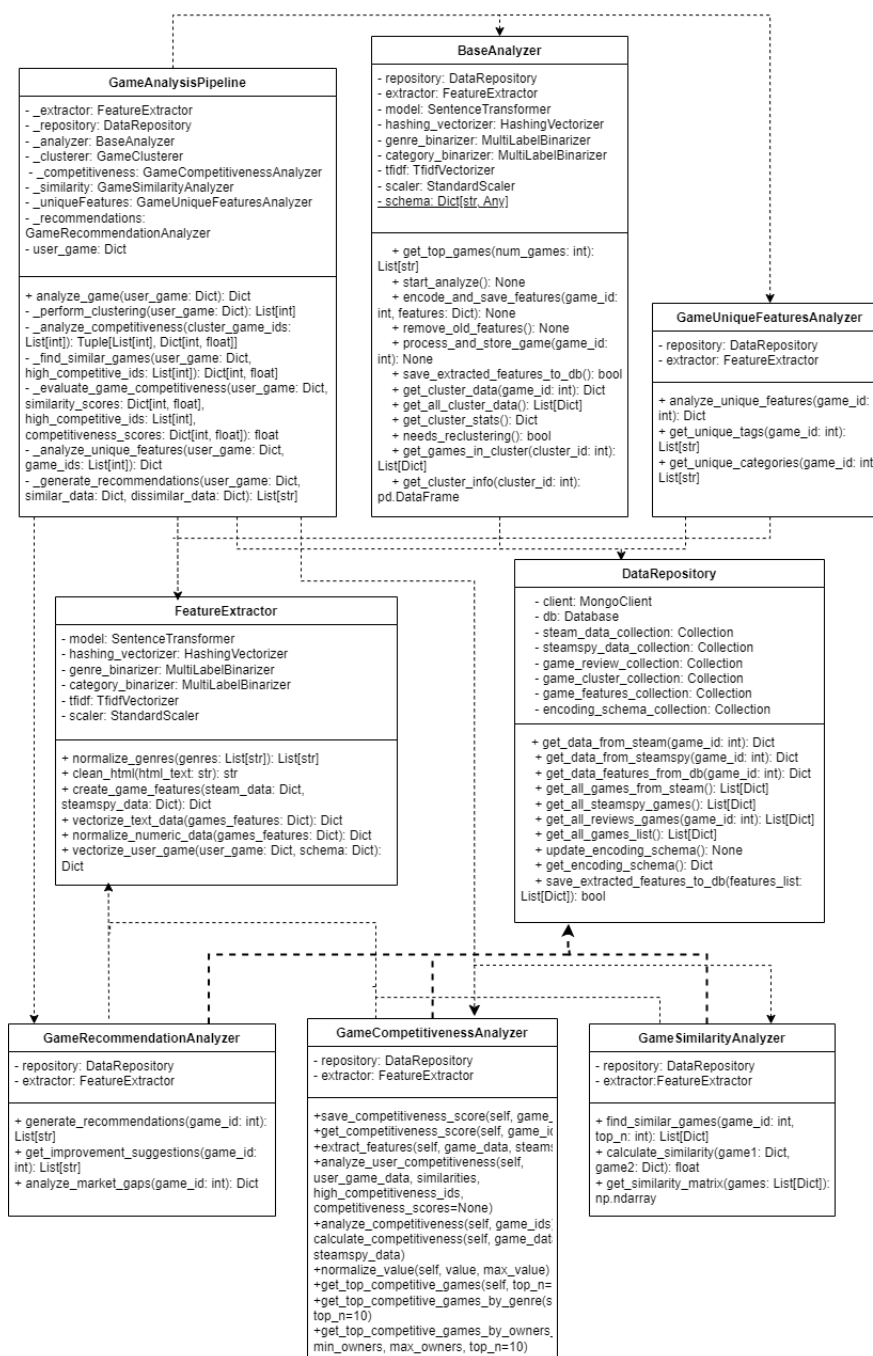


Рис.П5.1. Диаграмма классов системы GamesAnalyze

Приложение 6

Основные параметры игр из Steam и SteamSpy

Таблица П6.1

Основные параметры игр из Steam и SteamSpy

Данные из Steam	Описание	Тип данных
<i>detailed_description</i>	Описание выбранной игры	string
<i>demos</i>	Наличие демо-версии игры, ID и описание демо	list или string
<i>metacritic</i>	Оценка metacritic (от 0 до 100)	integer или string
<i>categories</i>	Категории предоставленной игры	list
<i>release_date</i>	Дата выпуска игры (формат: "DD MMM, YYYY")	string
Данные из SteamSpy	Описание	Тип данных
<i>positive</i>	Количество позитивных отзывов	integer
<i>negative</i>	Количество негативных отзывов	integer
<i>owners</i>	Количество владельцев игры (например, "1000000..2000000")	string
<i>price</i>	Текущая стоимость игры (\$)	integer или string
<i>tags</i>	Пользовательские теги игры	list (строки)
<i>median_forever</i>	Среднее время игры за всё время	integer (в минутах)

Приложение 7

Основные параметры отзывов пользователей

Таблица П7.1

Основные параметры отзывов пользователей

Параметр отзыва	Описание	Тип данных
<i>voted_up</i>	Отметка "Рекомендую"	bool
<i>playtime_at_review</i>	Время игры на момент написания отзыва (минуты)	integer
<i>votes_funny</i>	Количество пользователей, сочтяших отзыв забавным	integer
<i>weighted_vote_score</i>	Оценка полезности отзыва (от 0 до 1)	float
<i>review_text</i>	Текст отзыва	string
<i>language</i>	Язык отзыва	string

Приложение 8

Сравнение количества игр в Steam и на других игровых платформах для ПК

Таблица П8.1

Сравнение количества игр в Steam и на других игровых платформах для ПК (по состоянию на 11 января 2025 года)

Название платформы	Количество компьютерных игр
Game Jolt [57]	1,644,492
Steam	73,000
Good Old Games (GOG) [59]	9,685
GamersGate [58]	9,090
VK Play [77]	7,072
Epic Games Store [53]	406
Origin [63]	144

Приложение 9

Сравнительный анализ методов векторизации текстовых данных

Таблица П9.1

Сравнительный анализ методов векторизации текстовых данных

Метод	Преимущества	Недостатки	Применимость
TF-IDF	– простота реализации; – низкие вычислительные затраты.	– игнорирует контекст; – плохо работает с синонимами.	Устарел для семантического анализа [6]
Bag-of-Words	– быстрая обработка; – подходит для простых задач.	– высокая размерность; – потеря семантики.	Не учитывает смысл описаний
Sentence-BERT	– учитывает контекст; – компактные векторы; – многоязычность.	– требует GPU; – сложная настройка.	Основной метод для векторизации текстовых описаний [6]

Приложение 10

Определение оптимального числа кластеров методом локтя

Таблица П10.1

Сравнение методов свёртки критериев

Критерий	Аддитивная	Мультипликативная
Интерпретируемость	Высокая	Низкая
Учёт сбалансированных решений	Подходит при Парето-фильтрации	Уязвим к дисбалансу
Чувствительность к нулям	Не критична	Критична
Интеграция разнородных данных	Гибкая	Сложная
Научное обоснование [7]	Да	Нет

Приложение 11

Объектно-признаковая таблица моделей ранжирования

Таблица П11.1

Объектно-признаковая таблица моделей ранжирования

Признак	Оценка доли продаж	Дивер- сификация	Мульти- модальность	Кластери- зация	Рекомендации по улучшению
Steam	-	-	-	-	-
STEAMer [44]	-	+	+	-	-
DRGame [19]	-	+	-	+	-
DeepFM [17]	-	-	-	-	-
D-VSMR [25]	-	-	+	-	-
Разрабатываемая система	+	+	+	+	+

Приложение 12

Сравнительный анализ подходов к ранжированию игр

Таблица П12.1

Сравнительный анализ подходов к ранжированию игр

Система / метод	Тип данных	Метод свёртки критериев	Контентный анализ	Оценка доли продаж
Steam (официальные чарты)	Коммерческие и поведенческие метрики	Не раскрыт	–	–
STEAMer	Поведенческие, социальные и контентные признаки	Взвешенная сумма (агрегация)	Частично (жанры, разработчик)	–
DRGame	Графы взаимодействий и категории	Неявная агрегация в GNN	Категории (кластеры)	–
DeepFM	Поведенческие признаки (время игры)	FM + DNN (суммирование)	–	–
D-VSMR	Визуальные и текстовые признаки	Векторная агрегация	+	–
Разрабатываемая система	Контентные и рыночные признаки	Аддитивная свёртка + метод АНР	+	+

Приложение 13

Сравнение инструментальных средств для разработки веб-приложения

Таблица П13.1

Сравнение инструментальных средств для разработки веб-приложения

Название инструментального средства	React[65]	Angular[48]	Vue.js[73]
Производительность	+	+-	+-
Сообщество и поддержка	+	+	+
Интеграция с сервером	+	+	+-
Гибкость	+	-	+
Модульность и повторное использование компонентов	+	-	+
Наличие библиотек и инструментов	+	+-	+
Поддержка мобильной разработки	+	+	-

Приложение 14

Производительность ключевых методов системы

Таблица П14.1

Производительность ключевых методов системы

Класс	Метод	Время (с)	Память (MiB)	Результат
GameAnalysisPipeline	analyze_game	59.16	124.5	cluster_info: [...]
GameClusterer	get_features_matrix	93.23	-146.9	array([...])
GameClusterer	clusters	78.60	48.8	-
GameRecommendation...	generate_recs	0.20	0	Рекомендации [...]
GameSimilarity...	calc_similarity	7.26	44.9	730: 0.98 [...]
GameUniqueFeatures...	compare_games	6.09	-6.1	size: 5 [...]
GameCompetitiveness...	analyze_comp	2.13	2.5	[(730, 0.95)]
GameCompetitiveness...	user_analysis	0.34	0.1	0.87
ReviewAnalyzer	analyze_reviews	0.41	0.2	score: 0.82
DataRepository	get_all_games	79.32	804.2	[game_id...]
DataRepository	get_games_list	1.65	24.0	[game_id...]
DataRepository	get_schema	0.25	-7.0	genres: [...]

Приложение 15

Определение оптимального числа кластеров методом локтя

Таблица П15.1

Различия в рейтингах Steam и разработанной системы

Игра	R_S	R_M	R_1	R_2	R_3	Разница				
						$dS1_i^2$	$dM2_i^2$	$dM3_i^2$	dSM_i^2	$d12_i^2$
Counter-Strike: Global Offensive	1	7	1	11	14	0	100	49	36	9
PUBG: BATTLEGROUNDS	2	8	2	14	10	0	36	4	36	16
Apex Legends	3	6	5	16	16	4	100	100	196	0
Palworld	4	18	9	18	18	25	0	0	1	0
Team Fortress 2	5	4	6	13	13	1	4	81	81	0
Call of Duty: Modern Warfare II	6	15	8	10	5	4	25	100	49	25
New World: Aeternum	7	14	10	9	8	9	25	36	49	1
Black Myth: Wukong	8	11	4	6	1	16	25	100	9	25
Grand Theft Auto V	9	1	3	2	11	36	1	100	25	81
Left 4 Dead 2	10	5	7	5	9	9	16	16	1	16
Lost Ark	11	12	11	17	6	0	25	36	1	121
War Thunder	12	13	12	15	15	0	4	4	9	0
HELLDIVERS 2	13	10	13	8	4	0	4	36	4	16
Warframe	14	16	15	12	17	1	16	1	144	25
ELDEN RING	15	3	14	3	3	1	0	0	144	0
Terraria	16	11	16	1	12	0	100	1	16	121
Path of Exile 2	17	8	18	7	7	1	1	1	81	0
Baldur's Gate 3	18	2	17	4	2	1	4	0	256	4
Сумма отклонений						108	476	780	1138	460
Изменение порядка (%)						11.15	49.13	78.49	117.4	48.48

Определение оптимального числа кластеров методом локтя

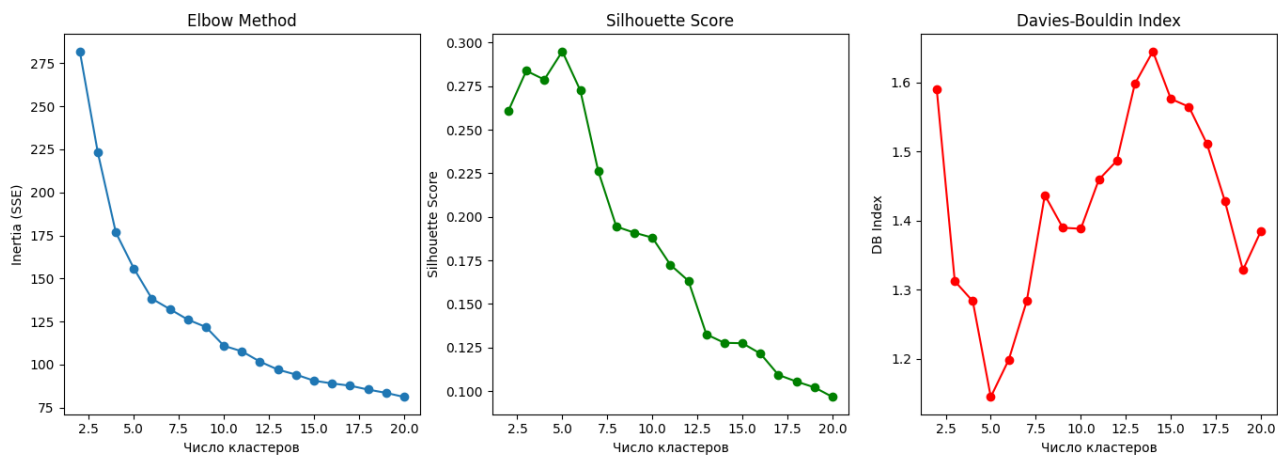


Рис.П16.1. Определение оптимального числа кластеров методом локтя

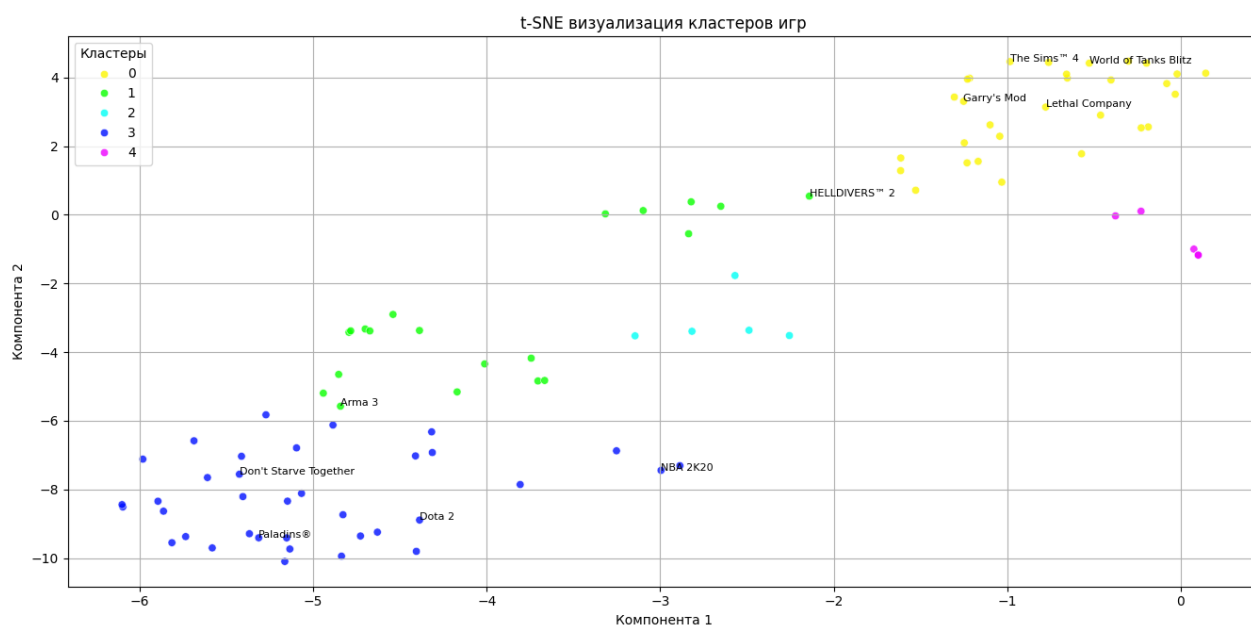
t-SNE визуализация кластеров компьютерных игр

Рис.П17.1. t-SNE визуализация кластеров компьютерных игр

Приложение 18

Блок-схема выполнения одного из возможных сценариев работы системы

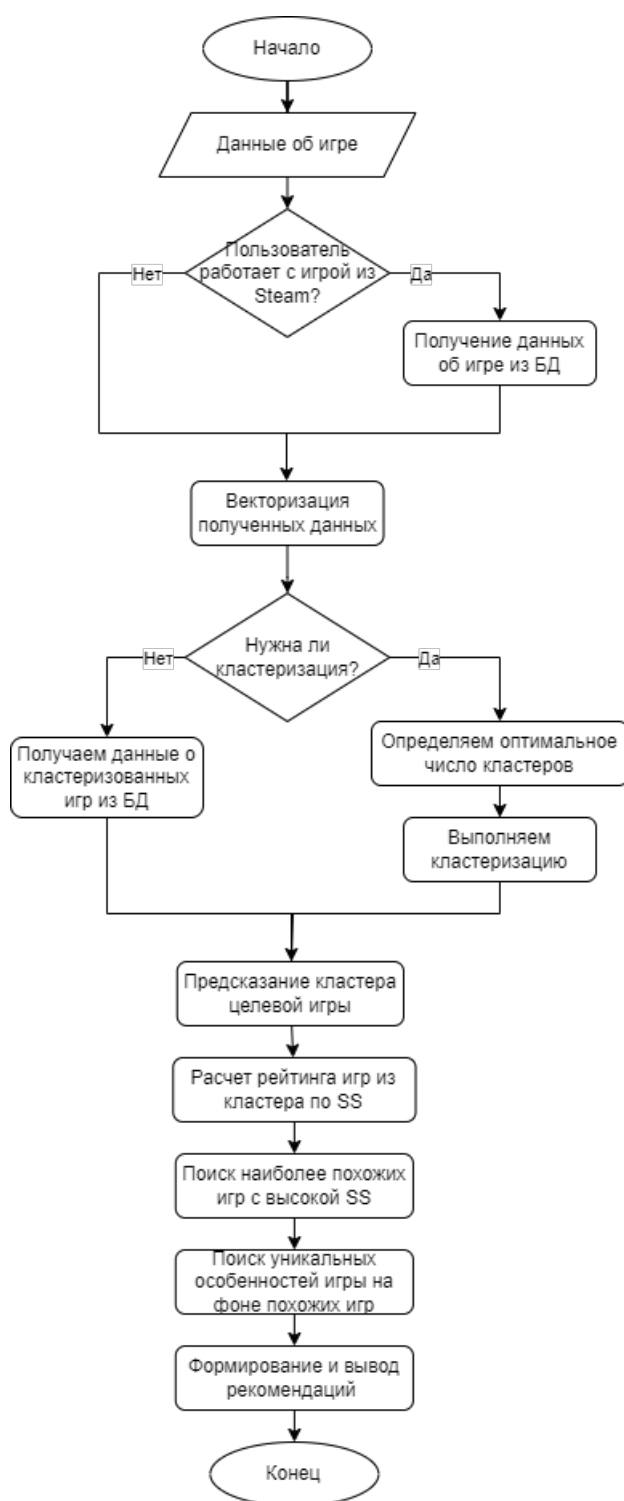


Рис.П18.1. Блок-схема обработки данных в классе BazeAnalyzer

Приложение 19

Пример POST-запроса к API анализа игры через Postman

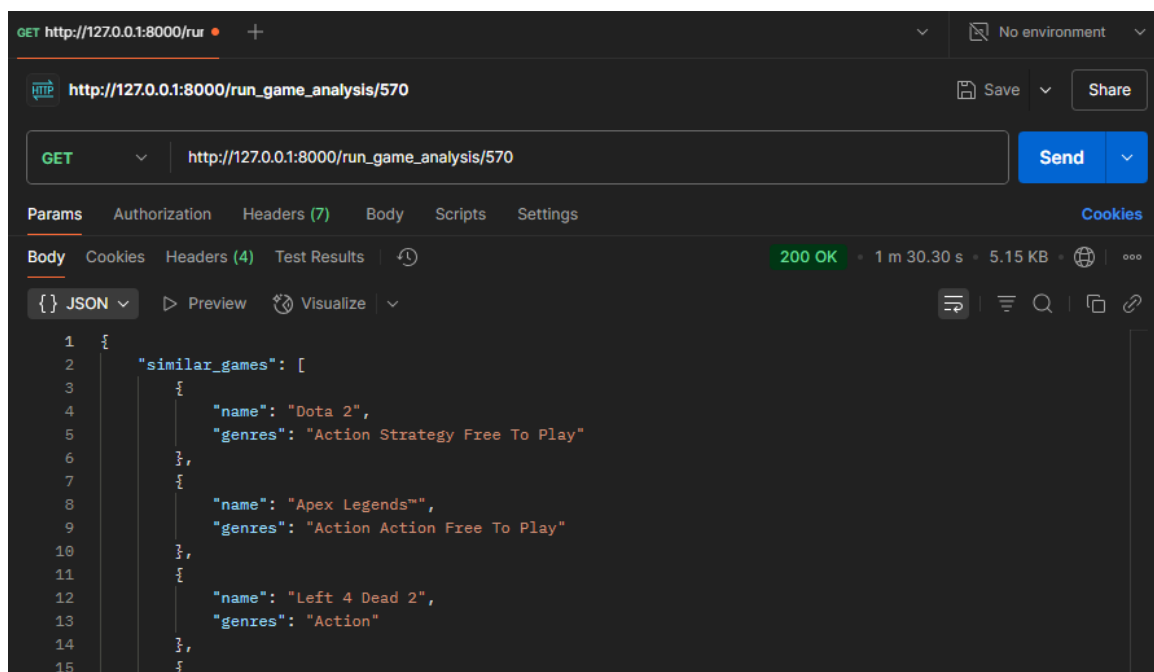


Рис.П19.1. Пример POST-запроса к API анализа игры через Postman

Приложение 20

Форма загрузки данных пользовательской игры при светлой теме

Анализируйте компьютерные игры, оценивайте их конкурентоспособность и получайте рекомендации по улучшению

Темная тема

ANALYZE BY GAME ID

Psychological Horror
Puzzle
Puzzle-Platformer
PvE
PvP
RPG
RTS
Racing
Real Time Tactics
Real-Time with Pause

Анализ новой игры

Название игры *

My game

Описание игры (на английском) *

Жанры

Action Adventure

Категории

Solo Co-op Steam Cloud Для нескольких игроков

Стоимость (в долларах) *

4

Средняя продолжительность игры (в минутах) *

400

АНАЛИЗИРОВАТЬ ИГРУ

Рис.П20.1. Форма загрузки данных пользовательской игры при светлой теме

Программный код модуля BaseAnalyzer

```

from datetime import datetime, timedelta

import pandas as pd
5 import requests
from sklearn.feature_extraction.text import TfidfVectorizer,
    HashingVectorizer
from sklearn.preprocessing import StandardScaler,
    MultiLabelBinarizer, normalize
from sentence_transformers import SentenceTransformer
import numpy as np
10 from typing import Dict, List, Optional, Tuple
from pymongo import UpdateOne

from modules.DataRepository import DataRepository
from modules.FeatureExtractor import FeatureExtractor
15 from modules.decorators import log_execution_time

class BaseAnalyzer:
    def __init__(self):
        self.repository = DataRepository()
        20 self.extractor = FeatureExtractor()
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.hashing_vectorizer = HashingVectorizer(n_features
            =256)
        self.genre_binarizer = MultiLabelBinarizer()
        self.category_binarizer = MultiLabelBinarizer()
        25 self.tfidf = TfidfVectorizer(max_features=5000,
            stop_words='english', ngram_range=(1, 2))
        self.scaler = StandardScaler()
        self.schema = {"genres": [], "categories": [], "tags":
            [], "count": 0}
        self._clusterer = None
        self._similarity = None
        30 self._uniqueFeatures = None
        self._recommendations = None

    @property
    def clusterer(self):
        35 if self._clusterer is None:
            from modules.GameClusterer import GameClusterer

```

```

        self._clusterer = GameClusterer()
        return self._clusterer

40 @property
    def similarity(self):
        if self._similarity is None:
            from modules.GameSimilarityAnalyzer import
                GameSimilarityAnalyzer
            self._similarity = GameSimilarityAnalyzer()
45         return self._similarity

    @property
    def uniqueFeatures(self):
        if self._uniqueFeatures is None:
50             from modules.GameUniqueFeaturesAnalyzer import
                GameUniqueFeaturesAnalyzer
            self._uniqueFeatures = GameUniqueFeaturesAnalyzer
                ()
        return self._uniqueFeatures

    @property
55     def recommendations(self):
        if self._recommendations is None:
            from modules.GameRecommendationAnalyzer import
                GameRecommendationAnalyzer
            self._recommendations = GameRecommendationAnalyzer
                ()
        return self._recommendations

60     def get_top_games(self, num_games=100):
        url = f"https://steamspy.com/api.php?request=
            top100in2weeks"
        response = requests.get(url)
        if response.status_code == 200:
65             return list(response.json().keys())[:num_games]
        else:
            return None

    def start_analyze(self):
70         self.schema = self.repository.get_encoding_schema()
        all_games = self.repository.get_all_games()
        num_all_games = len(all_games)

        if self.schema["count"] < num_all_games:

```

```

75         print(f'Найдено новых игр для загрузки: {
            num_all_games - self.schema["count"]}')
        self.repository.update_encoding_schema()
        self.schema = self.repository.get_encoding_schema
            ()
    else:
        print(f'Игр в схеме = {self.schema['count']}')
80         print(f'Игр в базе данных = {num_all_games}')

        game_ids = [game["game_id"] for game in all_games]
        descriptions = [game.get("description", "") for game
            in all_games]
        genres = [game.get("genres", []) for game in all_games
            ]
        categories = [game.get("categories", []) for game in
            all_games]
        tags = [game.get("tags", []) for game in all_games]
        names = [game.get("name", "") for game in all_games]
90         prices = [game.get("price", 0) for game in all_games]
        metacritics = [game.get("metacritic", 0) for game in
            all_games]
        median_forevers = [game.get("median_forever", 0) for
            game in all_games]

        print(f"Начинаем батчевое кодирование признаков для {
            num_all_games} игр...")
95         game_features_list = self.batch_encode_features(
            game_ids, names, descriptions, genres, categories,
            tags, prices, metacritics, median_forevers,
            self.schema
        )
        print("Батчевое кодирование завершено. Начинаем батчев
            ое сохранение в БД...")
        requests = []
100         for features in game_features_list:
            game_id = features["game_id"]
            requests.append(
                UpdateOne(
                    {"game_id": game_id},
                    {"$set": features},
                    upsert=True
                )
            )
105         )
    )

```

```

110     # Execute bulk write
    if requests:
        try:
            result = self.repository.
                game_features_collection.bulk_write(
                    requests)
            print(f"Батчевое сохранение завершено. Вставле
                но: {result.upserted_count}, Совпадений: {
                    result.matched_count}, Изменено: {result.
                        modified_count}")
115        except Exception as e:
            print(f"Ошибка при выполнении bulk_write: {e}"
                )
        else:
            print("Нет признаков для сохранения.")

120 def batch_encode_features(self, game_ids, names,
    descriptions, genres_list, categories_list, tags_list,
    prices, metacritics, median_forevers, schema):
    """
    Encodes features for a list of games in batches.
    """
    num_games = len(game_ids)
125    if num_games == 0:
        return []

    # Batch encode descriptions
    description_embeddings = self.model.encode(
        descriptions).tolist()

130

    # Batch binarize genres
    genre_binarizer = MultiLabelBinarizer(classes=schema["
        genres"])

    genre_binaries = genre_binarizer.fit_transform(
        genres_list).tolist()

135

    category_texts = [" ".join(categories) for categories
        in categories_list]
    tag_texts = [" ".join(tags) for tags in tags_list]

    category_matrices = self.hashing_vectorizer.transform(
        category_texts).toarray().tolist()

```



```

140 tag_matrices = self.hashing_vectorizer.transform(
    tag_texts).toarray().tolist()

    max_price = schema.get("max_price", 1) if schema.get("
        max_price", 1) > 0 else 1
    max_metacritic = schema.get("max_metacritic", 1) if
        schema.get("max_metacritic", 1) > 0 else 1
    max_median_forever = schema.get("max_median_forever",
        1) if schema.get("max_median_forever", 1) > 0 else
        1
145
    normalized_prices = [price / max_price for price in
        prices]
    normalized_metacritics = [metacritic / max_metacritic
        for metacritic in metacritics]
    normalized_median_forevers = [median /
        max_median_forever for median in median_forevers]

150 game_features_list = []
    for i in range(num_games):
        game_features = {
            "game_id": game_ids[i],
            "name": names[i],
155            "description_embedding":
                description_embeddings[i],
            "genre_binary": genre_binaries[i],
            "category_binary": category_matrices[i],
            "tag_hash": tag_matrices[i],
            "price": normalized_prices[i],
160            "metacritic": normalized_metacritics[i],
            "median_forever":
                normalized_median_forevers[i]
        }
        game_features_list.append(game_features)

165 return game_features_list

def encode_and_save_features(self, game_id, features):
    schema = self.repository.get_encoding_schema()
    genre_binarizer = MultiLabelBinarizer(classes=
        schema["genres"])
170    description_embedding = self.extractor.model.
        encode(features["description"]).tolist()
    genre_binary = genre_binarizer.fit_transform([
        features["genres"]])[0].tolist()

```

```

category_matrix = self.extractor.
    hashing_vectorizer.transform([" ".join(features
["categories"])]).toarray()[0].tolist()
tag_matrix = self.extractor.hashing_vectorizer.
    transform([" ".join(features["tags"])]).toarray
()[0].tolist()
game_features = {
175     "game_id": game_id,
        "name": features["name"],
        "description_embedding": description_embedding
        ,
        "genre_binary": genre_binary,
        "category_binary": category_matrix,
180     "tag_hash": tag_matrix,
        "price": features["price"] / schema["max_price
        "],
        "metacritic": features["metacritic"] / schema[
        "max_metacritic"],
        "median_forever": features["median_forever"] /
        schema["max_median_forever"]
    }
185 self.repository.game_features_collection.
    update_one({"game_id": game_id}, {"$set":
    game_features}, upsert=True)

def save_extracted_features_to_db(self):
    try:
        steam_games = self.repository.
            get_all_games_from_steam()
190     total_games = len(steam_games)
        processed = 0
        print(f"Начинаем обработку {total_games} игр...")
        for game in steam_games:
            try:
195                 game_id = game["data"]["steam_appid"]
                    steamspy_data = self.repository.
                        get_data_from_steamspy(game_id)
                    if not steamspy_data:
                        continue
                    features = self.extractor.
                        create_game_features(game,
                            steamspy_data)
200                 update_data = {
                    "name": features["name"],

```

```

        "description": features["description"],
        "genres": features["genres"],
        "tags": list(features["tags"]),
205     "price": features["price"],
        "categories": features["categories"],
        "metacritic": features["metacritic"],
        "median_forever": features["
            median_forever"],
        "is_vr": features["is_vr"],
210     "is_multiplayer": features["
            is_multiplayer"],
        "owners": features["owners"],
        "positive": features["positive"],
        "negative": features["negative"],
        "positive_ratio": features["
            positive_ratio"],
215     "updated_at": datetime.now()
    }

    self.repository.game_features_collection.
        update_one(
            {"game_id": game_id},
            {"$set": update_data},
220     upsert=True
        )

        processed += 1
        if processed % 100 == 0:
            print(f"Обработано {processed}/{
                total_games} игр...")
225     except Exception as e:
        print(f"Ошибка при обработке игры: {str(e)}")
        continue
    print(f"Обработка завершена. Успешно обработано {
        processed} игр.")
except Exception as e:
230     print(f"Ошибка при сохранении признаков в базу дан
        ных: {str(e)}")
    return False
return True

def get_cluster_data(self, game_id):
235     """
    Получает информацию о кластере игры из базы данных.
    Args:

```

```

        game_id (int): ID игры
Returns:
240         dict: Данные о кластере или None, если данные не н
            айдены
        """
        return self.repository.game_cluster_collection.
            find_one({"game_id": int(game_id)})

def get_all_cluster_data(self):
245     """Получает все данные о кластерах из базы данных"""
    return list(self.repository.game_cluster_collection.
        find({}))

def get_cluster_stats(self):
    """
250     Получает статистику по кластерам.
    Returns:
        dict: Статистика по кластерам
    """
    stats = {
255         "total_games": self.repository.
            game_features_collection.count_documents({}),
        "clustered_games": self.repository.
            game_cluster_collection.count_documents({}),
        "clusters": {}
    }
    # Получаем количество игр в каждом кластере
260     pipeline = [
        {"$group": {
            "_id": "$cluster_id",
            "count": {"$sum": 1}
        }}
    ]
265     cluster_counts = list(self.repository.
        game_cluster_collection.aggregate(pipeline))
    for cluster in cluster_counts:
        stats["clusters"][str(cluster["_id"])] = cluster["
            count"]
    return stats

270 def needs_reclustering(self):
    """Проверяет, нужно ли проводить повторную кластеризац
        ию"""
    stats = self.get_cluster_stats()
    # Проверяем, есть ли некластеризованные игры

```

```

275     if stats["total_games"] > stats["clustered_games"]:
        return True
        # Проверяем, не устарели ли данные кластеризации
    latest_cluster = self.repository.
        game_cluster_collection.find_one(
            sort=[("updated_at", -1)]
280 )
    if latest_cluster:
        # Если данные старше 7 дней, обновляем кластеризац
        ию
        update_threshold = datetime.now() - timedelta(days
            =7)
        if latest_cluster["updated_at"] < update_threshold
            :
285         return True

    return False

def get_games_in_cluster(self, cluster_id):
290     """Получает все игры в указанном кластере"""
    return list(self.repository.game_cluster_collection.
        find({"cluster_id": int(cluster_id)}))

def get_cluster_info(self, cluster_id):
    """
295     Получает информацию об играх в указанном кластере.

    Args:
        cluster_id (int): ID кластера

    Returns:
        pd.DataFrame: Таблица с информацией о играх в клас
        тере
    """
    try:
        games_in_cluster = self.get_games_in_cluster(
            cluster_id)
305         if not games_in_cluster:
            print(f"Кластер {cluster_id} не содержит игр."
                )
            return pd.DataFrame()

        game_data = []
310         for game in games_in_cluster:

```

```

game_id = game['game_id']
coordinates = game.get('coordinates', [])

315     steam_data = self.repository.
        get_data_from_steam(game_id)
    steamspy_data = self.repository.
        get_data_from_steamspy(game_id)
    if not steam_data:
        continue
    features = self.extractor.create_game_features
        (steam_data, steamspy_data)

320
    game_data.append({
        "Game ID": game_id,
        "Name": features["name"],
        "Cluster ID": cluster_id,
325     "Coordinates": coordinates,
        "Genres": features["genres"],
        "Price": features["price"],
        "Metacritic": features["metacritic"],
        "Median Forever": features["median_forever
            "]
330     })

    df = pd.DataFrame(game_data)
    return df

335 except Exception as e:
    print(f"Ошибка при получении информации о кластере
        {cluster_id}: {str(e)}")
    return pd.DataFrame()

def get_all_clusters_info(self):
340     """
        Получает информацию о всех кластерах.

        Returns:
            dict: Словарь, где ключ - ID кластера, значение -
                DataFrame с данными об играх в кластере
345     """
    cluster_stats = self.get_cluster_stats()
    cluster_info = {}

    for cluster_id in cluster_stats["clusters"]:
350         cluster_id = int(cluster_id)

```

```

        cluster_info[cluster_id] = self.get_cluster_info(
            cluster_id)

    return cluster_info

355 def print_games_clusters(self):
    all_clusters_info = self.get_all_clusters_info()

    for cluster_id, cluster_data in all_clusters_info.
        items():
            print(f"--- Кластер {cluster_id} ---")
360         if not cluster_data.empty:
            print(cluster_data.head())
        else:
            print("Нет данных в кластере.")

365 def get_similar_games_info(self, similarity_scores, top_n
    =50):
    """
    Получает информацию об играх из словаря
        similarity_scores (ID игры: степень схожести).

    Args:
370     similarity_scores (dict): Словарь {game_id:
        similarity_score}
        top_n (int): Количество топовых игр для вывода (по
            умолчанию 10)

    Returns:
        pd.DataFrame: Таблица с информацией об играх, отсо
            ртированная по схожести
375 """
    try:
        if not similarity_scores:
            print("Словарь similarity_scores пуст.")
            return pd.DataFrame()

380         sorted_scores = sorted(similarity_scores.items(),
            key=lambda x: x[1], reverse=True)[:top_n]

        game_data = []

385         for game_id, similarity in sorted_scores:

```

```

steam_data = self.repository.
    get_data_from_steam(game_id)
steamspy_data = self.repository.
    get_data_from_steamspy(game_id)
390 if not steam_data:
        continue

features = self.extractor.create_game_features
    (steam_data, steamspy_data)

395 game_data.append({
        "Game_ID": game_id,
        "Name": features["name"],
        "Similarity_score": similarity,
        "Genres": features["genres"],
        "Price": features["price"],
        "Metacritic": features["metacritic"],
        "Median Forever": features["median_forever
            "]
    })

400 df = pd.DataFrame(game_data)
405 return df

except Exception as e:
    print(f"Ошибка при получении информации об играх:
        {str(e)}")
    return pd.DataFrame()

```


Приложение 22

Программный код модуля CompetitivenessRanker

```

import numpy as np
from typing import List, Dict, Tuple
from dataclasses import dataclass
5 import logging

logger = logging.getLogger(__name__)

@dataclass
10 class GameVariant:
    """Класс для представления игры как варианта в PYDASS"""
    name: str
    scores: List[float]
    nodedominated: bool = True
15 linkedTo: set = None

    def __init__(self, name: str, scores: List[float]):
        self.name = name
        self.scores = scores
        self.nodedominated = True
20 self.linkedTo = set()

class CompetitivenessRanker:
    def __init__(self):
25         # Критерии оценки конкурентоспособности
        self.criteria = [
            "owners",          # Количество владельцев
            "positive_ratio",  # Соотношение положительных от
                               # зывов
            "revenue",         # Выручка
30         "activity",        # Активность сообщества
            "freshness",       # Новизна игры
            "review_score"     # Анализ отзывов
        ]

35         # Порядок важности критериев (от более важного к менее
            # важному)
        self.importance_order = [0, 2, 1, 5, 3, 4] # owners,
            revenue, positive_ratio, review_score, activity,
            freshness

```

```

# Относительная важность критериев (True если следующий
# критерий менее важен)
self.relative_importance = [True, True, True, True,
                             True]

# Включенные/отключенные критерии
self.enabled_criteria = {criterion: True for criterion
                         in self.criteria}

# Коэффициенты важности (рассчитываются автоматически)
self.importance_coefs = self.
    _calculate_importance_coefs()
logger.debug(f"Initialized CompetitivenessRanker with
              criteria: {self.criteria}")

# def _calculate_importance_coefs(self) -> List[float]:
#     """Рассчитывает коэффициенты важности на основе поряд
#     ка и относительной важности"""
#     # Фильтруем только включенные критерии
#     enabled_order = [i for i in self.importance_order if
#                      self.enabled_criteria[self.criteria[i]]]
#     n = len(enabled_order)
#     coefs = [1 - 0.5 * (i / (n - 1)) if n > 1 else 1.0
#              for i in range(n)]
#     # Нормализуем, чтобы сумма была 1
#     total = sum(coefs)
#     coefs = [c / total for c in coefs]
#     return coefs

def _calculate_importance_coefs(self) -> List[float]:
    """Рассчитывает коэффициенты важности по экспоненциаль
        ной формуле:

$$w_i = 2^{\{(n-k_i)\}} / \sum_j 2^{\{(n-k_j)\}}$$

        где  $k_i$  позиция критерия (0 самый важный)
    """
    enabled_order = [i for i in self.importance_order if
                     self.enabled_criteria[self.criteria[i]]]
    n = len(enabled_order)
    #  $k_i$  позиция критерия в порядке (0 самый важн
        ый)
    weights = [2 ** (n - k - 1) for k in range(n)]
    total = sum(weights)
    coefs = [w / total for w in weights]
    print(coefs)
    return coefs

```

```

def _normalize_scores(self, scores: List[float],
max_values: List[float]) -> List[float]:
    """Нормализует оценки к шкале [0, 1]"""
    return [score / max_val if max_val > 0 else 0 for
            score, max_val in zip(scores, max_values)]

def rank_games(self, games_data: List[Dict], max_values:
Dict[str, float]) -> List[Tuple[str, float]]:
    """
    Ранжирует игры с использованием теории важности критер
иев

    Args:
        games_data: Список словарей с данными игр
        max_values: Словарь максимальных значений для кажд
ого критерия

    Returns:
        Список кортежей (game_id, score), отсортированный
        по убыванию
    """
    logger.debug(f"Ranking {len(games_data)} games with
max_values: {max_values}")

    # Создаем варианты (игры) для PYDASS
    variants = []
    for game in games_data:
        try:
            # Собираем оценки только по включенным критери
ям
            scores = []
            max_vals = []
            for criterion in self.criteria:
                if self.enabled_criteria[criterion]:
                    value = float(game.get(criterion, 0))
                    max_val = max_values.get(criterion, 1)
                    # Приводим к [0, 1]
                    norm_value = value / max_val if
                        max_val > 0 else 0
                    # Ограничиваем диапазон [0, 1]
                    norm_value = min(max(norm_value, 0),
                                      1)
                    scores.append(norm_value)

```

```

105         max_vals.append(1.0)  # теперь все при
                                # знаки в [0, 1]

        logger.debug(f"Game {game.get('game_id', '
unknown')} normalized scores: {scores}")

        # Создаем вариант с учетом порядка важности кр
        # итериев
110        ordered_scores = [scores[i] for i in range(len
            (scores))]
        variant = GameVariant(game['game_id'],
            ordered_scores)
        variants.append(variant)

    except Exception as e:
115        logger.error(f"Error processing game {game.get
            ('game_id', 'unknown')}: {str(e)}",
            exc_info=True)
        continue

    self._apply_pareto(variants)
    self._apply_quality_importance(variants)
120    self._apply_quantitative_importance(variants)

    print(variants)
    print(self.importance_coefs)
    # Рассчитываем финальные оценки
125    final_scores = []
    for variant in variants:
        if variant.nodominated:
            # Для недоминируемых вариантов используем взве
            # шенную сумму
            weighted_sum = sum(s * w for s, w in zip(
                variant.scores, self.importance_coefs))
130            # Нормализуем к [0, 100]
            max_possible_sum = sum(self.importance_coefs)
            # Максимально возможная сумма

            score = (weighted_sum / max_possible_sum)

135        else:
            # Для доминируемых вариантов используем средне
            # взвешенное значение
            weighted_sum = sum(s * w for s, w in zip(
                variant.scores, self.importance_coefs))

```

```

max_possible_sum = sum(self.importance_coefs)
score = (weighted_sum / max_possible_sum)

# Уменьшаем оценку на 20% для доминируемых вар
иантов
score *= 0.8

final_scores.append((variant.name, score*100))
logger.debug(f"Game {variant.name} final score: {
score}")

return sorted(final_scores, key=lambda x: x[1],
reverse=True)

def _apply_pareto(self, variants: List[GameVariant]):
    """Применяет принцип Парето для выявления доминируемых
    вариантов"""
    for a in variants:
        for b in variants:
            if a == b:
                continue

            # Проверяем доминацию
            dominated = True
            has_better = False
            for s1, s2 in zip(a.scores, b.scores):
                if s1 < s2:
                    dominated = False
                    break
                if s1 > s2:
                    has_better = True

            if dominated and has_better:
                b.nodominated = False
                b.linkedTo.add(a.name)

def _apply_quality_importance(self, variants: List[
GameVariant]):
    """Применяет качественную важность критериев"""
    for variant in variants:
        if not variant.nodominated:
            continue

            # Создаем матрицу качественной важности
matrix = []

```

```

for k in range(1, len(self.criteria)):
    values = [v if i <= k else 0 for i, v in
               enumerate(variant.scores)]
    values.sort()
    matrix.append(values)

# Проверяем доминанцию с учетом качественной важнос
ти
for other in variants:
    if other == variant or not other.nodominated:
        continue

    other_matrix = []
    for k in range(1, len(self.criteria)):
        values = [v if i <= k else 0 for i, v in
                   enumerate(other.scores)]
        values.sort()
        other_matrix.append(values)

# Сравниваем матрицы
dominated = True
has_better = False
for m1, m2 in zip(matrix, other_matrix):
    for v1, v2 in zip(m1, m2):
        if v1 < v2:
            dominated = False
            break
        if v1 > v2:
            has_better = True

    if dominated and has_better:
        other.nodominated = False
        other.linkedTo.add(variant.name)

def _apply_quantitative_importance(self, variants: List[
    GameVariant]):
    """Применяет количественную важность критериев"""
    try:
        # Создаем N-модель
        n_model = [1]
        for coef in self.importance_coefs:
            n_model.append(coef * n_model[-1])

        # Нормализуем N-модель

```

```

n_model = [float(x) for x in n_model]  # Преобразу
ем в float
max_val = max(n_model)
220 n_model = [x / max_val for x in n_model]  # Нормал
изуем к [0,1]

# Применяем N-модель к вариантам
for variant in variants:
    if not variant.nodominated:
225         continue

    # Создаем удлиненные оценки
    long_scores = []
    for score, n in zip(variant.scores, n_model):
230         # Умножаем на 1000 для получения целых чис
ел
        n_int = int(n * 1000)
        long_scores.extend([score] * n_int)
    long_scores.sort(reverse=True)

235 # Проверяем доминацию с учетом количественной
важности
for other in variants:
    if other == variant or not other.
nodominated:
        continue

240 other_long_scores = []
for score, n in zip(other.scores, n_model)
:
    n_int = int(n * 1000)
    other_long_scores.extend([score] *
n_int)
other_long_scores.sort(reverse=True)

245 # Сравниваем удлиненные оценки
dominated = True
has_better = False
for s1, s2 in zip(long_scores,
other_long_scores):
250     if s1 < s2:
        dominated = False
        break
    if s1 > s2:
        has_better = True

```

```

255         if dominated and has_better:
                other.nodominated = False
                other.linkedTo.add(variant.name)
except Exception as e:
260     logger.error(f"Error in
        _apply_quantitative_importance: {str(e)}",
        exc_info=True)
    pass

def _calculate_importance_coefs_pairwise_logchebyshev(self
, quant_weights: dict) -> List[float]:
    """
265     Рассчитывает коэффициенты важности на основе количеств
        енных весов критериев (от 0 до 10)
        методом парных сравнений с лог-чебышевской аппроксимац
        ией.
        quant_weights: dict {criterion: weight (0..10)}
        Возвращает нормированные веса в порядке self.criteria
        (или включённых критериев).
    """
270     # Используем только включённые критерии
    enabled_criteria = [c for c in self.criteria if self.
        enabled_criteria[c]]
    n = len(enabled_criteria)
    # Если не заданы веса по 1 для всех
    weights = [max(quant_weights.get(c, 1), 1e-6) for c in
        enabled_criteria]
275     # Строим матрицу парных сравнений
    A = np.ones((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
280                 # Отношение важностей
                A[i, j] = weights[i] / weights[j] if
                    weights[j] > 0 else 1e6

    # Логарифмируем
    logA = np.log(A)
    # Среднее по строкам
285     log_means = np.mean(logA, axis=1)
    # Экспоненцируем и нормируем
    raw_coefs = np.exp(log_means)
    coefs = raw_coefs / np.sum(raw_coefs)
    return coefs.tolist()

```


Программный код модуля GameClusterer

```

from joblib import dump, load
import numpy as np
import requests
5 import seaborn as sns
from matplotlib import pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
10 from sklearn.metrics import silhouette_score,
    davies_bouldin_score
from sklearn.preprocessing import StandardScaler
from typing import List, Dict, Optional
import pandas as pd
from datetime import datetime
15
from modules.DataRepository import DataRepository
from modules.FeatureExtractor import FeatureExtractor

20 class GameClusterer:
    def __init__(self):
        self.repository = DataRepository()
        self.extractor = FeatureExtractor()
        self.scaler = StandardScaler()
25 self.kmeans = KMeans(n_clusters=10, random_state=42)
        self.cluster_centers = None
        self.cluster_labels = None
        self.feature_matrix = None
        self.game_ids = None
30
    def get_features_matrix(self):
        all_games = self.get_top_games() #self.repository.
            get_all_games_list()
        game_ids = [game for game in all_games]
        games = []
35 for game_id in game_ids:
            game = self.repository.get_data_features_from_db(
                game_id)
            if game:
                games.append(game)

```

```

40     if not games:
        raise ValueError("В базе данных отсутствуют закоди-
            рованные данные для кластеризации.")

    features_matrix = []
    game_ids = []
45     game_names = []

    category_features = []
    description_features = []
    genre_features = []
50     tag_features = []
    numerical_features = []

    for game in games:
        game_ids.append(game['game_id'])
60         game_names.append(game['name'])
        print(game)
        if game.get('category_binary') is None:
            continue

        category_features.append(np.array(game['
            category_binary']).flatten())
        description_features.append(np.array(game['
            description_embedding']).flatten())
        genre_features.append(np.array(game['genre_binary',
            ]).flatten())
        tag_features.append(np.array(game['tag_hash']).
            flatten())
65         numerical_features.append(np.array([
            game['price'],
            game['metacritic'],
            game['median_forever']
        ]))

70     category_features = np.array(category_features)
    description_features = np.array(description_features)
    genre_features = np.array(genre_features)
    tag_features = np.array(tag_features)
    numerical_features = np.array(numerical_features)

75     numerical_scaler = StandardScaler()
    numerical_features = numerical_scaler.fit_transform(
        numerical_features)

```

```

n_samples, n_features = description_features.shape
n_components = min(50, n_samples, n_features)
pca = PCA(n_components=n_components)
description_features = pca.fit_transform(
    description_features)

dump(pca, 'pca_description.joblib')

category_scaler = StandardScaler()
genre_scaler = StandardScaler()
tag_scaler = StandardScaler()

category_features = category_scaler.fit_transform(
    category_features)
genre_features = genre_scaler.fit_transform(
    genre_features)
tag_features = tag_scaler.fit_transform(tag_features)

weights = {
    'category': 4,
    'description': 8,
    'genre': 4,
    'tag': 3,
    'numerical': 3
}
weights = self._calculate_ahp_weights(weights)
features_matrix = np.hstack([
    category_features * weights['category'],
    description_features * weights['description'],
    genre_features * weights['genre'],
    tag_features * weights['tag'],
    numerical_features * weights['numerical']
])

dump(numerical_scaler, 'numerical_scaler.joblib')
dump(category_scaler, 'category_scaler.joblib')
dump(genre_scaler, 'genre_scaler.joblib')
dump(tag_scaler, 'tag_scaler.joblib')

return features_matrix, game_ids, game_names

def get_top_games(self, num_games=100):

```

```

url = f"https://steampy.com/api.php?request=
    top100in2weeks"
120 response = requests.get(url)
    if response.status_code == 200:
        return list(response.json().keys())[:num_games]
    else:
        return None
125
def clusters(self):
    features_matrix, game_ids, game_names = self.
        get_features_matrix()

    optimal_k = self.evaluate_clusters(features_matrix,
        game_ids, game_names)
130 print(f"Оптимальное количество кластеров: {optimal_k}"
        )

    # Кластеризация с оптимальным количеством кластеров
    kmeans = KMeans(n_clusters=optimal_k, random_state=42)
    labels = kmeans.fit_predict(features_matrix)
135

    # Визуализация
    #self.visualize_clusters_tsne(features_matrix, labels,
        game_ids, game_names, label_frequency=5)

def visualize_clusters_tsne(self, feature_matrix, labels,
    game_ids, names=None, sample_size=1000, label_frequency
    =10):
140     """Функция для визуализации кластеров с помощью t-SNE
        с подписями точек"""
    if len(feature_matrix) > sample_size:
        rng = np.random.RandomState(42)
        indices = rng.choice(len(feature_matrix),
            sample_size, replace=False)
        feature_matrix = feature_matrix[indices]
145         labels = labels[indices]
        if names is not None:
            names = [names[i] for i in indices]

    # t-SNE с адаптивным perplexity
150     perplexity = min(30, len(feature_matrix) - 1)
    tsne = TSNE(
        n_components=2,
        perplexity=perplexity,
        learning_rate='auto',

```

```

155         n_iter=1000,
            init='pca',  # Используем PCA для инициализации
            random_state=42
        )
        reduced = tsne.fit_transform(feature_matrix)
160        dump(tsne, 'tsne_model.joblib')
        dump(feature_matrix, 'original_features.joblib')

        # Сохраняем данные о кластерах и координатах в БД
        for game_id, label, coord in zip(game_ids, labels,
165            reduced):
            self.repository.save_cluster_data(game_id, label,
                coord.tolist())

        # Визуализация
        plt.figure(figsize=(12, 8))
        palette = sns.color_palette("husl", len(set(labels)))
            # Используем husl для лучшего разделения цветов
170        scatter = sns.scatterplot(
            x=reduced[:, 0],
            y=reduced[:, 1],
            hue=labels,
            palette=palette,
175            legend='full',
            alpha=0.8,
            s=100
        )

180        # Добавляем подписи для некоторых точек
        if names is not None:
            for i, name in enumerate(names):
                if i % label_frequency == 0:
185                    plt.text(
                        reduced[i, 0],
                        reduced[i, 1],
                        name,
                        fontsize=8,
                        alpha=0.75,
190                        bbox=dict(facecolor='white', alpha
                            =0.7, edgecolor='none', pad=1)
                    )

        plt.title("t-SNE визуализация кластеров игр", fontsize
            =14)
        plt.xlabel("Компонента 1", fontsize=12)

```

```

195 plt.ylabel("Компонента 2", fontsize=12)
plt.legend(title='Кластеры', bbox_to_anchor=(1.05, 1),
           loc='upper left')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

200

def evaluate_clusters(self, features_matrix, ids, names,
                      max_clusters=20):
    try:
        k_range = range(2, max_clusters + 1)
        inertia = []
        silhouette_scores = []
        db_scores = []

        n_samples, n_features = features_matrix.shape
        n_components = min(5, n_samples, n_features)
        pca = PCA(n_components=n_components, random_state
                  =42)
        pca_features = pca.fit_transform(features_matrix)

        for k in k_range:
            kmeans = KMeans(n_clusters=k, init='k-means++',
                           , random_state=42)

            labels = kmeans.fit_predict(pca_features)
            inertia.append(kmeans.inertia_)
            silhouette_scores.append(silhouette_score(
                pca_features, labels))
            db_scores.append(davies_bouldin_score(
                pca_features, labels))

        # Визуализация
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 3, 1)
        plt.plot(k_range, inertia, marker='o')
        plt.title('Elbow Method')
        plt.xlabel('Число кластеров')
        plt.ylabel('Inertia (SSE)')

        plt.subplot(1, 3, 2)
        plt.plot(k_range, silhouette_scores, marker='o',
                 color='green')

```

```

plt.title('Silhouette Score')
plt.xlabel('Число кластеров')
plt.ylabel('Silhouette Score')

plt.subplot(1, 3, 3)
plt.plot(k_range, db_scores, marker='o', color='
    red')
plt.title('Davies-Bouldin Index')
plt.xlabel('Число кластеров')
plt.ylabel('DB Index')

plt.tight_layout()
plt.show()

optimal_k = k_range[np.argmax(silhouette_scores)]
print(f"Оптимальное число кластеров по Silhouette:
    {optimal_k}")
return optimal_k

except Exception as e:
    print(f"Ошибка в процессе оценки кластеров: {str(e
        )}")
    raise

def predict_cluster(self, vectorized_game_data):
    """
    Предсказывает кластер для пользовательской игры по её
    векторным данным.
    Использует сохраненные t-SNE координаты и KNN для пред
    сказания.

    Args:
        vectorized_game_data (dict): Векторные данные поль
        зовательской игры.

    Returns:
        int: Предсказанный кластер.
    """
    try:
        numerical_scaler = load('numerical_scaler.joblib')
        category_scaler = load('category_scaler.joblib')
        genre_scaler = load('genre_scaler.joblib')
        tag_scaler = load('tag_scaler.joblib')
        pca = load('pca_description.joblib')
    except FileNotFoundError as e:

```

```

print(f"Не удалось загрузить модель: {str(e)}. Сна
      чала выполните кластеризацию.")
return None

275     try:
        desc_embedding = np.array(vectorized_game_data.get
            ("description_embedding", []))
        if len(desc_embedding) == 0:
            print("Отсутствует description_embedding")
            return None

280     if desc_embedding.shape[0] != pca.n_features_in_:
        desc_embedding = pca.transform([desc_embedding
            ])[0]
    else:
        desc_embedding = pca.transform([desc_embedding
            ])[0]

285     numerical_features = numerical_scaler.transform([[
        float(vectorized_game_data.get("price", 0) or
            0),
        float(vectorized_game_data.get("median_forever
            ", 0) or 0),
        float(vectorized_game_data.get("metacritic",
            0) or 0)

290     ]])[0]

    category_vector = category_scaler.transform([
        vectorized_game_data.get("category_binary", [])
    ])[0]
    genre_vector = genre_scaler.transform([
        vectorized_game_data.get("genre_binary", [])
    ])[0]
    tag_vector = tag_scaler.transform([
        vectorized_game_data.get("tag_hash", [])
    ])[0]

295     weights = {
        'category': 4,
        'description': 8,
        'genre': 4,
        'tag': 3,
        'numerical': 3
    }

300     weights = self._calculate_ahp_weights(weights)

```



```

305 user_vector = np.hstack([
        category_vector * weights['category'],
        desc_embedding * weights['description'],
        genre_vector * weights['genre'],
        tag_vector * weights['tag'],
310 numerical_features * weights['numerical']
    ]).reshape(1, -1)

    try:
        tsne = load('tsne_model.joblib')
315 except FileNotFoundError:
        print("t-SNE модель не найдена. Сначала выполните кластеризацию.")
        return None

    try:
320 original_features = load('original_features.joblib')
    except FileNotFoundError:
        print("Исходные признаки не найдены. Сначала выполните кластеризацию.")
        return None

325 all_cluster_data = list(self.repository.
        game_cluster_collection.find({}))
    if not all_cluster_data:
        print("Нет данных о кластерах в БД.")
        return None

330 # Собираем координаты и метки кластеров
    X_tsne = []
    cluster_labels = []

    for game_data in all_cluster_data:
335 coords = game_data.get("coordinates", None)
        if coords and len(coords) == 2:
            X_tsne.append(coords)
            cluster_labels.append(game_data["cluster_id"])

340 if not X_tsne:
        print("Нет координат для определения кластера.")
        return None

```

```

X_tsne = np.array(X_tsne)
cluster_labels = np.array(cluster_labels)

# Используем KNN для предсказания
from sklearn.neighbors import KNeighborsClassifier
    , NearestNeighbors
nbrs = NearestNeighbors(n_neighbors=5).fit(
    original_features)
_, indices = nbrs.kneighbors(user_vector)
# Берем t-SNE координаты этих соседей и усредняем
их
predicted_coords = np.mean(X_tsne[indices[0]],
    axis=0)
# Предсказываем кластер
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tsne, cluster_labels)
predicted_cluster = int(knn.predict([
    predicted_coords])[0])

return predicted_cluster

except Exception as e:
    print(f"Ошибка при предсказании кластера: {str(e)}")
    return None

def get_similar_games_for_user_game(self, user_game):
    """
    Получает игры из того же кластера, что и пользовательс
        кая игра.
    Args:
        user_game (dict): Векторные данные пользовательско
            й игры.
    Returns:
        int: Номер кластера
    """
    try:
        vectorized_data = self.extractor.
            vectorize_user_game(user_game, None)
        if vectorized_data is None:
            print("Не удалось векторизовать пользовательск
                ую игру.")
            return None
        # Предсказание кластера
        cluster_id = self.predict_cluster(vectorized_data)

```

```

380         if cluster_id is not None:
            return cluster_id
        else:
            print("Не удалось определить кластер для пользо-
                  вательской игры.")
            return None
    except Exception as e:
385         print(f"Ошибка при определении кластера: {str(e)}"
              )
        return None

def _normalize_features(self, features):
    features = np.array(features)
390    min_vals = features.min(axis=0)
    max_vals = features.max(axis=0)
    denom = np.where(max_vals - min_vals == 0, 1, max_vals
                     - min_vals)
    return (features - min_vals) / denom

395 def _calculate_similarity(self, f1, f2):
    f1, f2 = np.array(f1), np.array(f2)
    if np.linalg.norm(f1) == 0 or np.linalg.norm(f2) == 0:
        return 0.0
    return float(np.dot(f1, f2) / (np.linalg.norm(f1) * np
                                   .linalg.norm(f2)))

400 def _calculate_ahp_weights(self, quant_weights: dict) ->
    dict:
        keys = list(quant_weights.keys())
        n = len(keys)
        weights = [max(quant_weights[k], 1e-6) for k in keys]
405        A = np.ones((n, n))
        for i in range(n):
            for j in range(n):
                if i != j:
                    A[i, j] = weights[i] / weights[j] if
                        weights[j] > 0 else 1e6

410        logA = np.log(A)
        log_means = np.mean(logA, axis=1)
        raw_coefs = np.exp(log_means)
        norm_coefs = raw_coefs / np.sum(raw_coefs)
        return {k: w for k, w in zip(keys, norm_coefs)}

```

Программный код модуля FeatureExtractor

```

from collections import Counter
import html
import re
5 from bs4 import BeautifulSoup
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer,
    HashingVectorizer
from sklearn.preprocessing import StandardScaler,
    MultiLabelBinarizer, normalize
from sentence_transformers import SentenceTransformer
10
from .DataRepository import DataRepository

class FeatureExtractor:
15     def __init__(self):
        self.schema = DataRepository().get_encoding_schema()
        # Try to load the model from a local path first
        try:
            self.model = SentenceTransformer('modules/models/
                all-MiniLM-L6-v2')
20        except Exception:
            # Fallback to downloading if local load fails
            self.model = SentenceTransformer('all-MiniLM-L6-v2
                ')
        self.hashing_vectorizer = HashingVectorizer(n_features
            =256)
        self.genre_binarizer = MultiLabelBinarizer()
25        self.category_binarizer = MultiLabelBinarizer()
        self.tfidf = TfidfVectorizer(max_features=5000,
            stop_words='english', ngram_range=(1, 2))
        self.scaler = StandardScaler()

    def normalize_genres(self, genres):
30        synonyms_for_genres = {
            "Racing": ["Driving", "Car", "Automobile"],
            "Action": ["Adventure", "Shooter"]
        }
        normalized_genres = set()
35        for genre in genres:

```

```

        for key, synonyms in synonyms_for_genres.items():
            if genre in synonyms:
                normalized_genres.add(key)
                break
40     else:
            normalized_genres.add(genre)
        return list(normalized_genres)

def clean_html(self, html_text):
45     soup = BeautifulSoup(html_text, "html.parser")
    return soup.get_text()

def preprocess_text(self, text):
    text = self.clean_html(text)
50     text = html.unescape(text)
    text = text.lower()
    # Удаляем лишние пробелы
    text = re.sub(r'\\s+', ' ', text)
    # Удаляем неалфавитные символы, кроме дефиса
55     text = re.sub(r'[^a-za-я0-9\\- ]', '', text)
    text = re.sub(r'\\s+', ' ', text)
    return text

def create_game_features(self, steam_data, steamspy_data=
None):
60     """Создает словарь признаков игры из данных Steam и
        SteamSpy."""
    try:
        if steamspy_data is None:
            steamspy_data = {}

65         genres = []
        if steamspy_data.get("genre"):
            genres = [genre for genre in steamspy_data["
                genre"].split(', ') if genre]
            normalized_genres = self.normalize_genres(genres)

70         # Безопасное получение описания
        description = self.preprocess_text(steam_data.get(
            "data", {}).get("detailed_description", ""))

        # Безопасное получение категорий
        categories = steam_data.get("data", {}).get("
            categories", [])
75         if isinstance(categories, list):

```

```

        categories_list = [cat.get("description", "")
                            for cat in categories if isinstance(cat,
                                                                    dict)]
    else:
        categories_list = []

    is_vr = int("VR Supported" in categories_list)
    is_multiplayer = int("Multi-player" in
                         categories_list or "Online PvP" in
                         categories_list)

    tags_counter = Counter()
    tags = steamspy_data.get('tags', {})
    if isinstance(tags, dict):
        tags_counter.update(tags.keys())

    owners_raw = steamspy_data.get("owners", "0-0")
    owners_split = [o for o in owners_raw.split('.')
                    if o.strip()]
    if owners_split:
        try:
            owners_midpoint = np.mean([int(o.replace('-', '').replace(',', '')) for o in
                                       owners_split])
        except (ValueError, TypeError):
            owners_midpoint = 0
    else:
        owners_midpoint = 0

    pos = steamspy_data.get("positive", 0)
    neg = steamspy_data.get("negative", 0)
    pos = 0 if pos is None else int(pos)
    neg = 0 if neg is None else int(neg)
    positive_ratio = pos / (pos + neg) if (pos + neg)
        > 0 else 0

    release_date = steam_data.get('data', {}).get('
        release_date', {}).get('date', '')

    try:
        price = float(steamspy_data.get("price", 0) or
                        0)
    except (ValueError, TypeError):
        price = 0.0

```

```

try:
    metacritic_score = float(steam_data.get("data"
        , {}).get("metacritic", {}).get("score", 0)
        or 0)
except (ValueError, TypeError):
    metacritic_score = 0.0

try:
    median_forever = float(steamspy_data.get("
        median_forever", 0) or 0)
except (ValueError, TypeError):
    median_forever = 0.0

features = {
    "name": steamspy_data.get('name', ''),
    "description": description,
    "genres": normalized_genres,
    "tags": list(tags_counter.keys()),
    "price": price,
    "categories": categories_list,
    "metacritic": metacritic_score,
    "median_forever": median_forever,
    "is_vr": is_vr,
    "is_multiplayer": is_multiplayer,
    "owners": owners_midpoint,
    "positive": pos,
    "negative": neg,
    "positive_ratio": positive_ratio,
    "release_date": release_date
}
return features
except Exception as e:
    print(f"Ошибка при создании признаков игры: {str(e)}")
    return None

def vectorize_text_data(self, games_features):
    """
    Векторизация текстовых данных (описание, жанры, теги).
    Каждый элемент в списках жанров, тегов и категорий век-
    торизуется отдельно.

    :param games_features: Словарь, где ключи ID игр,
        а значения словари с данными об играх.
    :return: Матрицы для описания, жанров и тегов.

```

```

150 """
    descriptions = [features.get("description", "") for
        features in games_features.values()]
    model = SentenceTransformer('all-MiniLM-L6-v2')
    description_embeddings = model.encode(descriptions)

155 genres = [features.get("genres", []) for features in
    games_features.values()]
    genre_vectorizer = MultiLabelBinarizer()
    genre_matrix = genre_vectorizer.fit_transform(genres)

    categories = [features.get("categories", []) for
        features in games_features.values()]
160 category_vectorizer = MultiLabelBinarizer()
    category_matrix = category_vectorizer.fit_transform(
        categories)

    all_tags = set()
    for features in games_features.values():
165         tags = features.get("tags", [])
        if isinstance(tags, (list, set)):
            all_tags.update(tags)

    tag_matrix = np.zeros((len(games_features), len(
        all_tags)))
170 tag_names = sorted(list(all_tags)) # Сортируем для со
    хранения порядка
    for i, features in enumerate(games_features.values()):
        tags = features.get("tags", [])
        if isinstance(tags, (list, set)):
            for j, tag in enumerate(tag_names):
175                 tag_matrix[i, j] = 1 if tag in tags else 0
    tag_matrix = normalize(tag_matrix, norm='l2', axis=1)

    return {
180         'description_embeddings': description_embeddings,
        'genre_matrix': genre_matrix,
        'category_matrix': category_matrix,
        'tag_matrix': tag_matrix,
        'tag_names': tag_names,
        'genre_names': genre_vectorizer.classes_,
185         'category_names': category_vectorizer.classes_
    }

```



```

def normalize_numeric_data(self, games_features):
    prices = np.array([features["price"] for features in
        games_features.values()])
    metacritic_scores = np.array([features["metacritic"]
        for features in games_features.values()])
    median_forever_times = np.array([features["
        median_forever"] for features in games_features.
        values()])

    max_price = np.max(prices) if np.max(prices) != 0 else
        1
    max_metacritic = np.max(metacritic_scores) if np.max(
        metacritic_scores) != 0 else 1
    max_median_forever = np.max(median_forever_times) if
        np.max(median_forever_times) != 0 else 1

    normalized_prices = prices / max_price
    normalized_metacritic = metacritic_scores /
        max_metacritic
    normalized_median_forever = median_forever_times /
        max_median_forever

    return {
        "prices": normalized_prices,
        "metacritic": normalized_metacritic,
        "median_forever": normalized_median_forever,
    }

def vectorize_user_game(self, user_game, schema):
    """Векторизует данные пользовательской игры."""
    try:
        if not user_game:
            print("Отсутствуют данные пользовательской игр
                ы")
            return None

        if not schema:
            schema = self.schema

        required_fields = ['name', 'description', 'genres',
            , 'categories', 'tags']
        if not all(field in user_game for field in
            required_fields):

```

```

        print(f"Отсутствуют обязательные поля в данных
              игры: {required_fields}")
        return None

name = user_game.get('name', '')
description = user_game.get('description', '')
genres = user_game.get('genres', [])
categories = user_game.get('categories', [])
tags = user_game.get('tags', [])
price = float(user_game.get('price', 0) or 0)
median_forever = float(user_game.get('
median_forever', 0) or 0)
metacritic = float(user_game.get('metacritic', 0)
or 0)

try:
    genre_binarizer = MultiLabelBinarizer(classes=
        schema["genres"])
    description_embedding = self.model.encode(
        description).tolist()
    genre_binary = genre_binarizer.fit_transform([
        genres])[0].tolist()
    category_matrix = self.hashing_vectorizer.
        transform([" ".join(categories)]).toarray()
        [0].tolist()
    tag_matrix = self.hashing_vectorizer.transform
        ([" ".join(tags)]).toarray()[0].tolist()

    # Нормализация числовых значений
    max_price = schema.get("max_price", 1)
    max_median_forever = schema.get("
max_median_forever", 1)

    normalized_price = price / max_price if
        max_price > 0 else 0
    normalized_median_forever = median_forever /
        max_median_forever if max_median_forever >
        0 else 0

    vectorized_data = {
        "game_id": 0,
        "name": name,
        "description_embedding":
            description_embedding,
        "genre_binary": genre_binary,

```

```

255         "category_binary": category_matrix,
        "tag_hash": tag_matrix,
        "price": normalized_price,
        "median_forever":
            normalized_median_forever,
        "metacritic": metacritic / 100.0 # Нормал
            изуем метакритик к диапазону [0, 1]
    }
    return vectorized_data

260 except Exception as e:
    print(f"Ошибка при векторизации данных: {str(e)}")
    return None

265 except Exception as e:
    print(f"Ошибка при векторизации игры: {str(e)}")
    return None

```

Программный код модуля GameAnalysisPipeline

```

import pandas as pd
from typing import Dict, List, Optional, Tuple, Any, Union
import numpy as np
5
from modules.BaseAnalyzer import BaseAnalyzer
from modules.GameCompetitivenessAnalyzer import
    GameCompetitivenessAnalyzer
from modules.ReviewAnalyzer import ReviewAnalyzer
from modules.DataRepository import DataRepository
10 from modules.FeatureExtractor import FeatureExtractor
from modules.GameClusterer import GameClusterer
from modules.GameSimilarityAnalyzer import
    GameSimilarityAnalyzer
from modules.GameUniqueFeaturesAnalyzer import
    GameUniqueFeaturesAnalyzer
from modules.GameRecommendationAnalyzer import
    GameRecommendationAnalyzer
15 from .decorators import log_execution_time

pd.set_option('display.max_rows', None) # Показывать все стро
ки
pd.set_option('display.max_columns', None) # Показывать все с
толбцы
pd.set_option('display.width', None) # Автоматически подбира
ть ширину
20 pd.set_option('display.max_colwidth', None)

def convert_numpy_types(obj: Any) -> Any:
    """
    Рекурсивно преобразует все numpy типы в Python native типы
    .
25
    Args:
        obj: Любой объект, который может содержать numpy типы

    Returns:
        Объект с преобразованными типами
30
    """
    if isinstance(obj, (np.int64, np.int32, np.int16, np.int8)
        ):

```

```

        return int(obj)
    elif isinstance(obj, (np.float64, np.float32, np.float16)):
        :
35     return float(obj)
    elif isinstance(obj, dict):
        return {key: convert_numpy_types(value) for key, value
                in obj.items()}
    elif isinstance(obj, list):
        return [convert_numpy_types(item) for item in obj]
40     elif isinstance(obj, tuple):
        return tuple(convert_numpy_types(item) for item in obj
                    )
    elif isinstance(obj, set):
        return {convert_numpy_types(item) for item in obj}
    return obj
45

class GameAnalysisPipeline:
    def __init__(self, repository: DataRepository):
        self.repository = repository
        self.competitiveness_analyzer =
            GameCompetitivenessAnalyzer(repository)
50     self.review_analyzer = ReviewAnalyzer(repository)
        self._extractor = None
        self._clusterer = None
        self._similarity = None
        self._uniqueFeatures = None
55     self._recommendations = None
        self._analyzer = None

    @property
    def analyzer(self):
60         if self._analyzer is None:
            self._analyzer = BaseAnalyzer()
        return self._analyzer

    @property
65     def extractor(self):
        if self._extractor is None:
            self._extractor = FeatureExtractor()
        return self._extractor

    @property
70     def clusterer(self):
        if self._clusterer is None:
            self._clusterer = GameClusterer()

```

```

75         return self._clusterer

@property
def similarity(self):
    if self._similarity is None:
        self._similarity = GameSimilarityAnalyzer()
80     return self._similarity

@property
def uniqueFeatures(self):
    if self._uniqueFeatures is None:
85         self._uniqueFeatures = GameUniqueFeaturesAnalyzer()
    return self._uniqueFeatures

@property
def recommendations(self):
90     if self._recommendations is None:
        self._recommendations = GameRecommendationAnalyzer()
    return self._recommendations

def _convert_numpy_types(self, obj: Any) -> Any:
95     """
    Рекурсивно преобразует все numpy типы в Python native
    типы.

    Args:
        obj: Любой объект, который может содержать numpy т
            ипы

    Returns:
        Объект с преобразованными типами
    """
    if isinstance(obj, (np.int64, np.int32, np.int16, np.
105         int8)):
        return int(obj)
    elif isinstance(obj, (np.float64, np.float32, np.
        float16)):
        return float(obj)
    elif isinstance(obj, np.ndarray):
        return obj.tolist()
110    elif isinstance(obj, dict):
        return {key: self._convert_numpy_types(value) for
            key, value in obj.items()}

```

```

elif isinstance(obj, list):
    return [self._convert_numpy_types(item) for item
            in obj]
elif isinstance(obj, tuple):
115     return tuple(self._convert_numpy_types(item) for
                  item in obj)
elif isinstance(obj, set):
    return {self._convert_numpy_types(item) for item
           in obj}
return obj

120 @log_execution_time()
def rate_top_games(self, user_game, top_n=5):
    """Оценивает топ-N игр для пользователя."""
    try:
        # Векторизация пользовательской игры
125     vectorized_user_game = self.extractor.
        vectorize_user_game(user_game, None)
        if vectorized_user_game is None:
            return []

        # Получение похожих игр
130     similar_games = self.similarity.find_similar_games
        (vectorized_user_game, top_n=top_n)
        if not similar_games:
            return []

        # Анализ уникальных особенностей
135     unique_features = self.uniqueFeatures.
        compare_with_most_similar_and_dissimilar(
            user_game, similar_games)
        if not unique_features:
            return []

        # Анализ конкурентоспособности
140     competitiveness = self competitiveness_analyzer.
        analyze_competitiveness(similar_games)
        if not competitiveness:
            return []

        # Формирование результата
145     rated_games = []
        for game_id, game_data in similar_games.items():
            game_id_str = str(game_id) # Convert to
            string for dictionary key

```

```

150         game_info = {
            'game_id': game_id_str,
            'name': game_data.get('name', ''),
            'similarity_score': game_data.get('
                similarity_score', 0.0),
            'unique_features': unique_features.get(
                game_id_str, {}),
            'competitiveness_score': competitiveness.
                get(game_id_str, 0.0)
155         }
        rated_games.append(game_info)

        return sorted(rated_games, key=lambda x: x['
            similarity_score'], reverse=True)

160     except Exception as e:
        print(f"Ошибка при оценке топ игр: {str(e)}")
        return []

    @log_execution_time()
165     def analyze_game(self, user_game):
        """Анализирует игру и возвращает результаты анализа."""
        try:
            print("Starting analyze_game")
            print(f"User game data: {user_game.keys() if
                user_game else 'None'}")

170
            # 1. Кластеризация
            cluster_game_ids = self._perform_clustering(
                user_game)
            if not cluster_game_ids:
                cluster_game_ids = []
175            print(f'cluster_game_ids = {cluster_game_ids}')

            # 2. Анализ конкурентоспособности
            high_competitive_ids, competitiveness_scores =
                self._analyze_competitiveness(cluster_game_ids)
            print(f'high_competitive_ids = {
                high_competitive_ids}')
            print(f'competitiveness_scores = {
                competitiveness_scores}')

180
            # 3. Поиск похожих игр

```



```

similarity_scores = self._find_similar_games(
    user_game, high_competitive_ids)
print(f'similarity_scores = {similarity_scores}')

# 4. Оценка конкурентоспособности целевой игры
competitiveness_score = self.
    _evaluate_game_competitiveness(
user_game,
similarity_scores,
high_competitive_ids,
competitiveness_scores
)
print(f'competitiveness_score = {
    competitiveness_score}')

# 5. Анализ уникальных особенностей
print("Starting unique features analysis")
print(f"Similar games IDs: {list(similarity_scores
    .keys())[0:3]}")
similar_data = self._analyze_unique_features(
user_game,
list(similarity_scores.keys())[0:3]
)
print(f"Similar data: {similar_data}")

dissimilar_data = self._analyze_unique_features(
user_game,
list(similarity_scores.keys())[-3:]
)
print(f"Dissimilar data: {dissimilar_data}")

# 6. Генерация рекомендаций
game_recommendations = self.
    _generate_recommendations(
user_game,
similar_data,
dissimilar_data
)
print(f"Game recommendations: {
    game_recommendations}")

# Получаем результаты анализа
cluster_info = self.analyzer.get_cluster_info(self
    .clusterer.get_similar_games_for_user_game(
user_game))

```

```

220     similar_games_info = self.analyzer.
        get_similar_games_info(similarity_scores)

    if hasattr(cluster_info, 'to_dict'):
        cluster_info = cluster_info.to_dict(orient='
            records')
    if hasattr(similar_games_info, 'to_dict'):
225         similar_games_info = similar_games_info.
            to_dict(orient='records')

    # Преобразуем все numpy массивы в Python native массивы
    result = {
        'cluster_info': self._convert_numpy_types(
            cluster_info),
230         'competitiveness_scores': self.
            _convert_numpy_types(competitiveness_scores
            ),
        'competitiveness_score': self.
            _convert_numpy_types(competitiveness_score)
        ,
        'similar_games': self._convert_numpy_types(
            similar_games_info),
        'recommendations': self._convert_numpy_types(
            game_recommendations)
    }

235     print(f"Final result keys: {result.keys()}")
    return result

    except Exception as e:
240         print(f"Ошибка при анализе игры: {str(e)}")
        import traceback
        print(f"Traceback: {traceback.format_exc()}")
        return {
            'similarity_analysis': {},
245             'unique_features': {},
            'competitiveness_analysis': {},
            'cluster_analysis': {'cluster_id': -1}
        }

250 def rate_game(self, user_game):
    cluster_game_ids = self._perform_clustering(user_game)
    if not cluster_game_ids:
        cluster_game_ids = []
    print(f'cluster_game_ids = {cluster_game_ids}')

```

```

255     # 2. Анализ конкурентоспособности
        high_competitive_ids, competitiveness_scores = self.
            _analyze_competitiveness(cluster_game_ids)

        # 3. Поиск похожих игр
260     similarity_scores = self._find_similar_games(user_game
        , high_competitive_ids)

        competitiveness_score = self.competitiveness_analyzer.
            analyze_user_competitiveness(
                user_game,
                similarity_scores,
265                high_competitive_ids,
                competitiveness_scores
            )
        return competitiveness_score

270 def similarities_search(self, user_game):
    # 1. Кластеризация
    all_games_ids = [game['game_id'] for game in self.
        repository.get_all_games_list()]

275     # 2. Анализ конкурентоспособности
        high_competitive_ids, competitiveness_scores = self.
            _analyze_competitiveness(all_games_ids)
        print(f'high_competitive_ids = {high_competitive_ids}'
            )
        print(f'competitiveness_scores = {
            competitiveness_scores}')

280     # 3. Поиск похожих игр
        similarity_scores = self._find_similar_games(user_game
        , high_competitive_ids)
        return similarity_scores

285 def _perform_clustering(self, games_data):
    """Выполняет кластеризацию игр."""
    try:
        # if self.analyzer.needs_reclustering():
290         # self.clusterer.clusters()

```

```

cluster_id = self.clusterer.
    get_similar_games_for_user_game(games_data)
cluster_docs = self.analyzer.get_games_in_cluster(
    cluster_id)

295     # Извлекаем game_id из документов MongoDB
cluster_game_ids = [doc['game_id'] for doc in
    cluster_docs]
return cluster_game_ids

except Exception as e:
300     print(f"Ошибка в процессе кластеризации: {str(e)}"
        )
    return []

#def save_features_to_db(self) -> bool:
#     """Сохраняет извлеченные признаки в базу данных"""
305 #     return self.repository.save_extracted_features_to_db
        ()

def vectorize_game(self, game_data: Dict) -> Dict:
    """Векторизует данные игры"""
    return self.extractor.vectorize_user_game(game_data,
        None)

310

def _analyze_competitiveness(self, cluster_game_ids: List[
int]) -> Tuple[List[int], Dict[int, float]]:
    """Анализирует конкурентоспособность игр в кластере"""
    try:
        # Получаем данные всех игр в кластере
315 games_data = []
        for game_id in cluster_game_ids:
            game_data = self.repository.
                get_data_from_steam(game_id)
            steamspy_data = self.repository.
                get_data_from_steamspy(game_id)
            if game_data and steamspy_data:
320                 features = self.competitiveness_analyzer.
                    extract_features(game_data,
                        steamspy_data)
                review_analysis = self.review_analyzer.
                    analyze_reviews(str(game_id))

                game_features = self.
                    competitiveness_analyzer.

```

```

        _get_game_features(features, game_id,
                           review_analysis)
    games_data.append({
325         'game_id': game_id,
        **game_features
    })

    # Рассчитываем максимальные значения для кластера
330 segment_max = {
        'owners': max(float(data.get('owners', 0)) for
                        data in games_data),
        'price': max(float(data.get('price', 0)) for
                      data in games_data),
        'revenue': max(float(data.get('price', 0)) *
                        float(data.get('owners', 0)) for data in
                        games_data),
        'activity': max((float(data.get('positive', 0))
                          + float(data.get('negative', 0))) /
335                          max(1, float(data.get('owners', 0))) for data
                              in games_data),
        'positive_ratio': 1.0, # Максимальное значени
                               e для positive_ratio всегда 1.0 (100%)
        'freshness': 1.0, # Максимальное значение для
                           freshness всегда 1.0
        'review_score': 1.0 # Максимальное значение д
                               ля review_score всегда 1.0
    }

340 ranked_games = self.competitiveness_analyzer.
    ranker.rank_games(games_data, segment_max)

    high_competitive_ids = [int(gid) for gid, score in
                             ranked_games if score > 20] # score > 50% от
                                                           максимального
    competitiveness_scores = {int(gid): score/100 for
                                gid, score in ranked_games} # нормализуем к
                                                                [0,1]
345 return high_competitive_ids,
    competitiveness_scores

except Exception as e:
    print(f"Ошибка при анализе конкурентоспособности:
          {str(e)}")

```

```

350         import traceback
        print(f"Traceback: {traceback.format_exc()}")
        return [], {}

    def _find_similar_games(self, user_game: Dict,
        high_competitive_ids: List[int]) -> Dict[int, float]:
355         """Находит похожие игры"""
        return self.similarity.calculate_similarity_with_game(
            user_game, high_competitive_ids)

    def _evaluate_game_competitiveness(self, user_game: Dict,
        similarity_scores: Dict[int, float],
        high_competitive_ids:
            List[int],
            competitiveness_scores
                : Dict[int, float]) ->
            float:
360         """Оценивает конкурентоспособность целевой игры"""
        return self.competitiveness_analyzer.
            analyze_user_competitiveness(
                user_game,
                similarity_scores,
                high_competitive_ids,
365                competitiveness_scores
            )

    def _analyze_unique_features(self, user_game: Dict,
        game_ids: List[int]) -> Dict:
370         """Анализирует уникальные особенности игры"""
        print(f"_analyze_unique_features called with game_ids:
            {game_ids}")
        try:
            result = self.uniqueFeatures.
                compare_with_most_similar_and_dissimilar(
                    user_game, game_ids)
            print(f"_analyze_unique_features result: {result}"
                )
            return result
375        except Exception as e:
            print(f"Error in _analyze_unique_features: {str(e)}")
            import traceback
            print(f"Traceback: {traceback.format_exc()}")
            return {'size': 0}
380

```

385

```
def _generate_recommendations(self, user_game: Dict,
    similar_data: Dict, dissimilar_data: Dict) -> List[str]:
    """Генерирует рекомендации по улучшению игры"""
    return self.recommendations.generate_recommendations(
        user_game, similar_data, dissimilar_data)

if __name__ == "__main__":

    pipeline = GameAnalysisPipeline(user_game_id=570)
```

Программный код модуля GameAnalyzer

```

import React, { useState } from 'react';
import { TextField, Button, CircularProgress, Alert,
  IconButton } from '@mui/material';
import SettingsIcon from '@mui/icons-material/Settings';
5 import ResultsDisplay from '../results/ResultsDisplay';
import SettingsDialog from '../settings/SettingsDialog';
import styles from '../GameAnalyzer.module.css';

const GameAnalyzer = () => {
10   const [gameId, setGameId] = useState('');
   const [results, setResults] = useState(null);
   const [loading, setLoading] = useState(false);
   const [error, setError] = useState(null);
   const [criteriaSettings, setCriteriaSettings] = useState(
     null);
15   const [settingsOpen, setSettingsOpen] = useState(false);

   const handleAnalyze = async () => {
     if (!gameId) {
       setError('Пожалуйста, введите ID игры');
20       return;
     }

     setLoading(true);
     setError(null);

25     try {
       const response = await fetch('http://localhost:5000/api/
         analyze', {
         method: 'POST',
         headers: {
30           'Content-Type': 'application/json',
         },
         body: JSON.stringify({
           game_id: gameId,
           criteria_settings: criteriaSettings
35         }),
       });

       const data = await response.json();

```



```

40     if (data.error) {
        setError(data.error);
    } else {
        setResults(data);
    }
45 } catch (err) {
    setError('Ошибка при анализе игры: ' + err.message);
} finally {
    setLoading(false);
}
50 };

const handleSettingsChange = (newSettings) => {
    setCriteriaSettings(newSettings);
    // Перезапускаем анализ с новыми настройками
55 if (gameId) {
    handleAnalyze();
}
};

60 return (
    <div className={styles.container}>
        <div className={styles.header}>
            <h1>Анализатор игр</h1>
            <IconButton
65         onClick={() => setSettingsOpen(true)}
            className={styles.settingsButton}
            title="Настройки критериев"
            size="large"
            >
70         <SettingsIcon fontSize="large" />
        </IconButton>
    </div>

    <div className={styles.inputSection}>
75     <TextField
        label="ID игры в Steam"
        value={gameId}
        onChange={(e) => setGameId(e.target.value)}
        variant="outlined"
80     fullWidth
        margin="normal"
        />
        <Button

```

```

85         variant="contained"
           color="primary"
           onClick={handleAnalyze}
           disabled={loading}
           className={styles.analyzeButton}
90     >
       {loading ? <CircularProgress size={24} /> : 'Анализи
           ровать'}
       </Button>
     </div>

     {error && (
95       <Alert severity="error" className={styles.errorAlert}>
         {error}
       </Alert>
     )}

100   {results && (
     <ResultsDisplay
       results={results}
       onSettingsChange={handleSettingsChange}
     />
105   )}

     <SettingsDialog
       open={settingsOpen}
       onClose={() => setSettingsOpen(false)}
110       onSave={handleSettingsChange}
       initialSettings={{
         criteria: Object.keys(results?.criteria || {}),
         enabledCriteria: Object.fromEntries(
           Object.keys(results?.criteria || {}).map(key => [
115             key, true])
         )
       }}
     />
     </div>
   );
120 };

export default GameAnalyzer;

```

Программный код модуля GameCompetitivenessAnalyzer

```

import numpy as np
import requests
from scipy.special import expit
5 from datetime import datetime
from typing import Dict, List, Optional, Tuple

from modules.DataRepository import DataRepository
from modules.FeatureExtractor import FeatureExtractor
10 from modules.ReviewAnalyzer import ReviewAnalyzer
from modules.CompetitivenessRanker import
    CompetitivenessRanker

class GameCompetitivenessAnalyzer:
15     def __init__(self, repository: DataRepository, ranker:
        Optional[CompetitivenessRanker] = None):
        self.repository = repository
        self.extractor = FeatureExtractor()
        self.review_analyzer = ReviewAnalyzer(repository)
        self.ranker = ranker if ranker is not None else
            CompetitivenessRanker()
20     self.schema = self.repository.get_encoding_schema()

    def save_competitiveness_score(self, game_id, score):
        """Сохраняет оценку конкурентоспособности в базу данны
            x"""
        try:
25             self.repository.game_features_collection.
                update_one(
                    {"game_id": game_id},
                    {
                        "$set": {
                            "competitiveness_score": score,
30                             "competitiveness_updated_at": datetime
                                .now()
                        }
                    }
                )
        except Exception as e:

```

```

35         print(f"Ошибка при сохранении оценки конкурентоспо
           способности для игры {game_id}: {str(e)}")

def get_competitiveness_score(self, game_id):
    """Получает сохраненную оценку конкурентоспособности и
       з базы данных"""
    try:
40         game_data = self.repository.
           game_features_collection.find_one(
               {"game_id": game_id},
               {"competitiveness_score": 1, "
                 competitiveness_updated_at": 1}
           )
        if game_data and "competitiveness_score" in
           game_data:
45             return game_data["competitiveness_score"]
        return None
    except Exception as e:
        print(f"Ошибка при получении оценки конкурентоспос
           обности для игры {game_id}: {str(e)}")
        return None

50 def extract_features(self, game_data, steamspy_data):
    """ Извлекает и рассчитывает признаки для анализа конк
       урентоспособности """
    features = self.extractor.create_game_features(
        game_data, steamspy_data)

55     # Дополнительные признаки
    features['revenue'] = float(features['price']) * float
        (features['owners'])
    features['total_reviews'] = features['positive'] +
        features['negative']
    features['review_activity'] = features['total_reviews'
        ] / max(1, features['owners'])

60     # Рассчитываем возраст игры
    release_date = game_data.get('data', {}).get('
        release_date', {}).get('date', '')
    if release_date:
        try:
            release = datetime.strptime(release_date, '%d
                %b, %Y')
65             age_days = (datetime.now() - release).days
            features['age_days'] = age_days

```

```

        except:
            features['age_days'] = 365*40
    else:
        features['age_days'] = 365*40

    return features

def analyze_user_competitiveness(self, user_game_data,
    similarities, high_competitiveness_ids,
    competitiveness_scores=None):
    """
    Анализирует конкурентоспособность пользовательской игры
    на основе похожих игр
    Args:
        user_game_data: Данные пользовательской игры
        similarities: Словарь {game_id: similarity_score}
        похожих игр
        high_competitiveness_ids: Список ID игр с высокой
        конкурентоспособностью
        competitiveness_scores: Словарь {game_id:
        competitiveness_score} предварительно рассчитан
        ных оценок
    Returns:
        Оценка конкурентоспособности пользовательской игры
        (0-1)
    """
    try:
        if not similarities:
            return 0.0

        user_features = user_game_data
        weighted_scores = []
        total_weight = 0

        for game_id, similarity in similarities.items():
            if competitiveness_scores and game_id in
                competitiveness_scores:
                competitiveness = competitiveness_scores[
                    game_id]
            else:
                game_data = self.repository.
                    get_data_from_steam(game_id)
                steamspy_data = self.repository.
                    get_data_from_steamspy(game_id)
                if not game_data or not steamspy_data:

```

```

100         continue
        competitiveness = self.
            calculate_competitiveness(game_data,
            steamspy_data)

        # Увеличиваем вес для игр с высокой конкуренто
        # способностью
        weight = similarity * (1.2 if game_id in
            high_competitiveness_ids else 1.0)
105         weighted_scores.append(competitiveness *
            weight)
        total_weight += weight

    if not weighted_scores:
        return 0.0

110     base_score = sum(weighted_scores) / total_weight

    modifiers = []
    # 1. Модификатор на основе жанров
115     genre_overlap = sum(1 for game_id in similarities.
        keys())

        if any(genre in user_features['
            genres'])
            for genre in self.
                repository.
                    get_data_from_steamspy(
                        game_id).get('genre', '')
                        ).split(', ')))
        genre_modifier = 1.0 + (genre_overlap / len(
            similarities)) * 0.1
        modifiers.append(genre_modifier)

120     # 2. Модификатор на основе цены
    avg_price = sum(float(self.repository.
        get_data_from_steamspy(game_id).get('price', 0)
        ))

        for game_id in similarities.keys()
            / len(similarities)
    price_diff = abs(float(user_features['price']) -
        avg_price) / max(avg_price, 1)
125     price_modifier = 1.0 - (price_diff * 0.2) # Штраф
        за значительное отклонение от средней цены
    modifiers.append(price_modifier)

```

```

# 3. Модификатор на основе VR и мультиплеера
vr_factor = 1.05 if 'VR' in user_features['
categories'] else 1.0
130 multiplayer_factor = 1.1 if 'Multi-player' in
user_features['categories'] else 1.0
modifiers.extend([vr_factor, multiplayer_factor])

# Применяем все модификаторы
135 final_score = base_score
for modifier in modifiers:
    final_score *= modifier

return max(0.0, min(1.0, final_score))

140 except Exception as e:
    print(f"Ошибка анализа конкурентоспособности: {str
(e)}")
    return 0.0

def analyze_competitiveness(self, game_ids):
145 """
    Анализирует конкурентоспособность списка игр.
    Args:
        game_ids: Список ID игр для анализа
    Returns:
150 Список кортежей (game_id, competitiveness_score),
    отсортированный по убыванию
    """
    scores = []

    for game_id in game_ids:
155 try:
        saved_score = self.get_competitiveness_score(
            game_id)
        if saved_score is not None:
            scores.append((game_id, saved_score))
            continue

        # Если сохраненной оценки нет, рассчитываем но
        # вую
        game_data = self.repository.
            get_data_from_steam(game_id)
        steamspy_data = self.repository.
            get_data_from_steamspy(game_id)

```



```

195         case 'review_score':
            review_analysis = self.review_analyzer
                .analyze_reviews(str(game_data.get(
                    'game_id', '')))
            value = review_analysis['review_score']
        case _:
            continue
200         normalized_metrics.append(value)
    competitiveness = np.sum(np.array(
        normalized_metrics) * np.array(self.ranker.
        importance_coefs))

    vr_factor = 1.05 if features['is_vr'] else 1.0
    multiplayer_factor = 1.1 if features['
        is_multiplayer'] else 1.0
205

    final_score = competitiveness * vr_factor *
        multiplayer_factor

    return max(0.0, min(1.0, final_score))

210 except Exception as e:
    print(f"Ошибка расчета конкурентоспособности: {str
        (e)}")
    return 0.0

def normalize_value(self, value, max_value):
215     """
        Нормализует значение к диапазону [0, 1] с использовани
            ем сигмоидной функции.
        Добавлен параметр steepness для контроля крутизны сигм
            оиды.
        """
    if max_value <= 0:
220         return 0.0

    steepness = 5.0
    return expit((value / max_value) * steepness)

225 def _get_game_features(self, features, game_id,
    review_analysis):
    """Вспомогательный метод для получения признаков игры
        """
    game_features = {}

```

```

for criterion in self.ranker.criteria:
    match criterion:
230         case 'owners':
            game_features[criterion] = features['
                owners']
        case 'positive_ratio':
            game_features[criterion] = features['
                positive_ratio']
        case 'revenue':
235         game_features[criterion] = features['
            revenue']
        case 'activity':
            game_features[criterion] = features['
                review_activity']
        case 'freshness':
            game_features[criterion] = 1 - self.
                normalize_value(features['age_days'],
                356 * 20)
240         case 'review_score':
            game_features[criterion] = review_analysis
                ['review_score']
        case _:
            continue
    return game_features
245

def get_top_games(self, num_games=100):
    url = f"https://steamspy.com/api.php?request=
        top100in2weeks"
    response = requests.get(url)
250    if response.status_code == 200:
        return list(response.json().keys())[:num_games]
    else:
        return None

255 def get_max_values(self, segment_data):
    """ Получает максимальные значения для всех признаков
        """

    segment_max = {
        'owners': max(float(data.get('owners', 0)) for
            data in segment_data),
260        'price': max(float(data.get('price', 0)) for data
            in segment_data),

```

```

    'revenue': max(float(data.get('price', 0)) * float
        (data.get('owners', 0)) for data in
        segment_data),
    'activity': max((float(data.get('positive', 0)) +
        float(data.get('negative', 0))) /
        max(1, float(data.get('owners',
            , 0)))) for data in
            segment_data),
    'positive_ratio': 1.0,  # Максимальное значение дл
        я positive_ratio всегда 1.0 (100%)
    'freshness': 1.0,  # Максимальное значение для
        freshness всегда 1.0
    'review_score': 1.0  # Максимальное значение для
        review_score всегда 1.0
}

return segment_max

def get_top_competitive_games(self, top_n=10):
    """ Получает топ-N самых конкурентоспособных игр среди
        всех игр """
    try:
        # Получаем все игры
        all_games = self.get_top_games()#self.repository.
            get_all_games_list()
        game_ids = [game for game in all_games]

        # Получаем данные всех игр в сегменте один раз
        segment_data = []
        for game_id in game_ids:
            game = self.repository.
                get_data_features_from_db(game_id)
            if game:
                segment_data.append(game)

        segment_max = self.get_max_values(segment_data)

        games_data = []
        for game_id in game_ids:
            game_data = self.repository.
                get_data_from_steam(game_id)
            steamspy_data = self.repository.
                get_data_from_steamspy(game_id)
            if game_data and steamspy_data:

```

```

        features = self.extract_features(game_data
                                         , steamspy_data)
        review_analysis = self.review_analyzer.
            analyze_reviews(str(game_id))
295

        game_features = self._get_game_features(
            features, game_id, review_analysis)
        games_data.append({
            'game_id': game_id,
            **game_features
300        })

        ranked_games = self.ranker.rank_games(games_data,
            segment_max)

        return ranked_games[:top_n]
305

    except Exception as e:
        print(f"Ошибка при получении топ игр: {str(e)}")
        return []

310 def get_top_competitive_games_by_genre(self, genre, top_n
    =10):
    """Получает топ-N самых конкурентоспособных игр в опре
        деленном жанре"""
    try:
        all_games = self.get_top_games()
        games = list(self.repository.
            game_features_collection.find(
315         {"genres": genre,
            "game_id": {"$in": all_games}
            },
            {"_id": 0}
        ))
        game_ids = [game['game_id'] for game in games]

        # Получаем данные всех игр в сегменте один раз
        segment_data = []
        for game_id in game_ids:
325         game = self.repository.
            get_data_features_from_db(game_id)
            if game:
                segment_data.append(game)

        segment_max = self.get_max_values(segment_data)

```

```

330     # Подготавливаем данные для ранжирования
    games_data = []
    for game_id in game_ids:
        game_data = self.repository.
            get_data_from_steam(game_id)
335         steamspy_data = self.repository.
            get_data_from_steamspy(game_id)
        if game_data and steamspy_data:
            features = self.extract_features(game_data
                , steamspy_data)
            review_analysis = self.review_analyzer.
                analyze_reviews(str(game_id))

340         game_features = self._get_game_features(
            features, game_id, review_analysis)
        games_data.append({
            'game_id': game_id,
            **game_features
        })

345     # Ранжируем игры с использованием настроек из
        ranker
    ranked_games = self.ranker.rank_games(games_data,
        segment_max)

    # Возвращаем топ-N игр
350    return ranked_games[:top_n]

    except Exception as e:
        print(f"Ошибка при получении топ игр по жанру: {
            str(e)}")
        return []

355
def get_top_competitive_games_by_owners_range(self,
    min_owners, max_owners, top_n=10):
    """
    Получает топ-N самых конкурентоспособных игр в определ
        енном диапазоне владельцев
    """
360    try:
        # Получаем все игры в указанном диапазоне владельц
            ев
        all_games = self.get_top_games()

```

```

games = list(self.repository.
    game_features_collection.find(
        {
365             "owners": {
                "$gte": min_owners,
                "$lte": max_owners
            },
            "game_id": {"$in": all_games}
370         },
        {"_id": 0}
    ))
game_ids = [game['game_id'] for game in games]

375 # Получаем данные всех игр в сегменте один раз
segment_data = []
for game_id in game_ids:
    game = self.repository.
        get_data_features_from_db(game_id)
    if game:
380         segment_data.append(game)
if len(segment_data) == 0:
    return None
# Рассчитываем максимальные значения для сегмента
один раз
segment_max = self.get_max_values(segment_data)

385 # Подготавливаем данные для ранжирования
games_data = []
for game_id in game_ids:
    game_data = self.repository.
        get_data_from_steam(game_id)
390    steamspy_data = self.repository.
        get_data_from_steamspy(game_id)
    if game_data and steamspy_data:
        features = self.extract_features(game_data
            , steamspy_data)
        review_analysis = self.review_analyzer.
            analyze_reviews(str(game_id))

395        game_features = self._get_game_features(
            features, game_id, review_analysis)
        games_data.append({
            'game_id': game_id,
            **game_features
        })

```

400

```
# Ранжируем игры с использованием настроек из  
ranker
```

```
ranked_games = self.ranker.rank_games(games_data,  
                                       segment_max)
```

405

```
# Возвращаем топ-N игр
```

```
return ranked_games[:top_n]
```

```
except Exception as e:
```

```
    print(f"Ошибка при получении топ игр по количеству  
          владельцев: {str(e)}")
```

```
    return []
```

Приложение 28

Программный код модуля GameRecommendationAnalyzer

```

from sklearn.feature_extraction.text import TfidfVectorizer

class GameRecommendationAnalyzer:
5   def __init__(self):
        self.vectorizer = TfidfVectorizer()

    def _calculate_feature_importance(self, feature,
        similar_data, dissimilar_data):
        """Рассчитывает важность признака на основе его присут-
            ствия в похожих и разных играх"""
10        # Если признак есть в похожих играх, но отсутствует в
            разных - высокая важность
        if feature in similar_data['all_genres'] +
            similar_data['all_tags'] + similar_data['
            all_categories'] and \
            feature not in dissimilar_data['all_genres'] +
                dissimilar_data['all_tags'] + dissimilar_data['
                all_categories']:
            return 0.9
        # Если признак уникален для целевой игры - средняя важ-
            ность
15        elif feature in similar_data['unique_in_target_genres'] +
            similar_data['unique_in_target_tags'] +
            similar_data['unique_in_target_categories']:
            return 0.7
        # Если признак есть в пересечении - низкая важность
        elif feature in similar_data['intersection_genres'] +
            similar_data['intersection_tags'] + similar_data['
            intersection_categories']:
            return 0.3
20        return 0.1

    def _analyze_price_positioning(self, user_game,
        similar_data, dissimilar_data):
        """Анализирует позиционирование цены"""
        price = user_game['price']
25        similar_avg_price = similar_data['avg_price_diff'] +
            price
        dissimilar_avg_price = dissimilar_data['avg_price_diff
            '] + price

```



```

if price < similar_avg_price and price <
    dissimilar_avg_price:
        return "Цена игры ниже среднерыночной. Это может б
            ыть преимуществом для привлечения новых игроков
            ."
30 elif price > similar_avg_price and price >
    dissimilar_avg_price:
        return "Цена игры выше среднерыночной. Рекомендует
            ся обосновать более высокую цену уникальными ос
            обенностями."
    elif similar_avg_price < price < dissimilar_avg_price:
        return "Цена игры находится в оптимальном диапазон
            е между похожими и разными играми."
    return None
35
def _analyze_feature_combinations(self, user_game,
    similar_data):
    """Анализирует уникальные комбинации признаков"""
    combinations = []

40
    # Анализ комбинаций жанров
    if len(user_game['genres']) > 1:
        genre_comb = ' + '.join(user_game['genres'])
        if genre_comb not in ' '.join(similar_data['
            all_genres']):
            combinations.append(f"Уникальная комбинация жа
                нров: {genre_comb}")
45
    # Анализ комбинаций тегов
    if len(user_game['tags']) > 1:
        tag_comb = ' + '.join(user_game['tags'])
        if tag_comb not in ' '.join(similar_data['all_tags
            ']):
50
            combinations.append(f"Уникальная комбинация те
                гов: {tag_comb}")

    return combinations

def _analyze_description(self, user_game, similar_data,
    dissimilar_data):
55
    """Анализирует описание игры и генерирует рекомендации
        """
    recommendations = []

```

```

if 'description_analysis' in similar_data:
    desc_analysis = similar_data['description_analysis']

# Анализ схожести описаний
if 'similarity_score' in desc_analysis:
    similarity = desc_analysis['similarity_score']
    if similarity < 0.3:
        recommendations.append(
            "Описание игры значительно отличается
              от описаний похожих игр. "
            "Рекомендуется добавить больше общих э
              лементов, характерных для жанра."
        )
    elif similarity > 0.8:
        recommendations.append(
            "Описание игры очень похоже на описани
              я других игр. "
            "Рекомендуется добавить больше уникаль
              ных элементов для выделения игры."
        )

# Анализ общих фраз
if 'common_phrases' in desc_analysis and
    desc_analysis['common_phrases']:
    recommendations.append(
        "В описании похожих игр часто встречаются
          следующие фразы, "
        "которые можно использовать для улучшения
          описания:"
    )
    for phrase in desc_analysis['common_phrases'][:3]:
        # Берем топ-3 фразы
        recommendations.append(f"- {phrase}")

# Анализ уникальных фраз
if 'unique_phrases' in desc_analysis and
    desc_analysis['unique_phrases']:
    recommendations.append(
        "В описании игры есть уникальные элементы,
          которые можно использовать "
        "как конкурентное преимущество:"
    )
    for phrase in desc_analysis['unique_phrases'][:3]:
        # Берем топ-3 фразы

```

```

        recommendations.append(f"- {phrase}")

    return recommendations

95 def _analyze_market_positioning(self, user_game,
    similar_data, dissimilar_data):
    """Анализирует позиционирование игры на рынке"""
    recommendations = []

    # Анализ жанрового позиционирования
100 similar_genres = set(similar_data['all_genres'])
    dissimilar_genres = set(dissimilar_data['all_genres'])
    user_genres = set(user_game['genres'])

    # Жанры, которые есть в похожих, но нет в разных играх
105 niche_genres = similar_genres - dissimilar_genres
    if niche_genres & user_genres:
        recommendations.append(
            f"Игра использует нишевые жанры ({', '.join(
                niche_genres & user_genres)}), "
            f"которые характерны для похожих игр, но отсут-
            ствуют в разных. "
110 f"Это может быть преимуществом для целевой ауд-
            итории."
        )

    # Жанры, которые есть в разных, но нет в похожих играх
    potential_genres = dissimilar_genres - similar_genres
115 if potential_genres & user_genres:
        recommendations.append(
            f"Игра использует жанры ({', '.join(
                potential_genres & user_genres)}), "
            f"которые характерны для разных игр. "
            f"Это может помочь привлечь новую аудиторию."
120 )

    return recommendations

def _analyze_competitive_advantages(self, user_game,
    similar_data, dissimilar_data):
125 """Анализирует конкурентные преимущества игры"""
    advantages = []

    # Анализ уникальных комбинаций жанров
    user_genre_comb = set(user_game['genres'])

```

```

130 similar_genres = set(similar_data['all_genres'])

# Проверяем, есть ли уникальные комбинации жанров
if len(user_genre_comb) > 1 and not any(genre in
    similar_genres for genre in user_genre_comb):
    advantages.append(
135         f"Уникальная комбинация жанров ({', '.join(
            user_genre_comb)}) "
        f"может привлечь игроков, ищущих новый игровой
            опыт."
    )

# Анализ уникальных тегов
140 unique_tags = set(user_game['tags']) - set(
    similar_data['all_tags'])
if unique_tags:
    advantages.append(
        f"Уникальные теги ({', '.join(unique_tags)}) "
        f"могут привлечь специфическую аудиторию."
145     )

# Анализ ценового преимущества
price = user_game['price']
similar_avg_price = similar_data['avg_price_diff'] +
    price
150 if price < similar_avg_price * 0.7: # Если цена на
    30% ниже средней
        advantages.append(
            f"Цена игры значительно ниже среднерыночной, "
            f"что может быть сильным конкурентным преимуще
                ством."
155         )

    return advantages

def _analyze_potential_improvements(self, user_game,
    similar_data, dissimilar_data):
    """Анализирует потенциальные улучшения игры"""
160     improvements = []

# Анализ популярных тегов в похожих играх
similar_tags = set(similar_data['all_tags'])
user_tags = set(user_game['tags'])
165 potential_tags = similar_tags - user_tags
if potential_tags:

```

```

        improvements.append(
            "Популярные теги в похожих играх, которые можн
              о добавить:"
        )
170     for tag in list(potential_tags)[:5]:  # Топ-5 мего
        в
            improvements.append(f"- {tag}")

    # Анализ категорий
    similar_categories = set(similar_data['all_categories'
175     ])
    user_categories = set(user_game['categories'])
    potential_categories = similar_categories -
        user_categories
    if potential_categories:
        improvements.append(
            "Категории, которые можно добавить для расшире
              ния функционала:"
180     )
        for category in list(potential_categories)[:3]:  #
            Топ-3 категории
                improvements.append(f"- {category}")

    return improvements

185
def generate_recommendations(self, user_game, similar_data
, dissimilar_data):
    """Генерирует рекомендации на основе анализа данных"""
    recommendations = []

190     # Анализ позиционирования на рынке
    market_recommendations = self.
        _analyze_market_positioning(user_game, similar_data
        , dissimilar_data)
    if market_recommendations:
        recommendations.append("\nПозиционирование на рынк
            е:")
        recommendations.extend(market_recommendations)

195     # Анализ уникальных особенностей
    unique_features = []

    # Анализ жанров
200     for genre in user_game['genres']:

```

```

        importance = self._calculate_feature_importance(
            genre, similar_data, dissimilar_data)
        if genre in similar_data['unique_in_target_genres']:
            unique_features.append(("Уникальный жанр",
                                   genre, importance))

205     # Анализ тегов
    for tag in user_game['tags']:
        importance = self._calculate_feature_importance(
            tag, similar_data, dissimilar_data)
        if tag in similar_data['unique_in_target_tags']:
            unique_features.append(("Уникальный тег", tag,
                                   importance))

210     # Анализ категорий
    for category in user_game['categories']:
        importance = self._calculate_feature_importance(
            category, similar_data, dissimilar_data)
        if category in similar_data['
            unique_in_target_categories']:
215             unique_features.append(("Уникальная категория"
                                     , category, importance))

    # Формирование рекомендаций на основе уникальных особе
    нностей
    if unique_features:
        recommendations.append("\nУникальные особенности:"
                               )
220     for feature_type, feature, importance in
        unique_features:
            if importance > 0.7:
                recommendations.append(
                    f"- Игра имеет уникальный {
                        feature_type.lower()} '{feature}' (
                            важность: {importance:.1f}). "
                    f"Это может быть значительным конкурен
                        тным преимуществом."
                )
225     elif importance > 0.3:
        recommendations.append(
            f"- Игра имеет {feature_type.lower()}
                '{feature}' (важность: {importance
                    :.1f}). "

```

```

230         f"Это может привлечь определенную нише
           вую аудиторию."
    )

    # Анализ позиционирования цены
    price_recommendation = self._analyze_price_positioning(
        user_game, similar_data, dissimilar_data)
    if price_recommendation:
235         recommendations.append("\nЦеновое позиционирование
           :")
        recommendations.append(f"- {price_recommendation}"
                                )

    # Анализ конкурентных преимуществ
    advantages = self._analyze_competitive_advantages(
        user_game, similar_data, dissimilar_data)
240    if advantages:
        recommendations.append("\nКонкурентные преимущесств
           а:")
        recommendations.extend([f"- {adv}" for adv in
                                advantages])

    # Анализ описания
245    description_recommendations = self._
        _analyze_description(user_game, similar_data,
                               dissimilar_data)
    if description_recommendations:
        recommendations.append("\nРекомендации по описанию
           :")
        recommendations.extend([f"- {rec}" for rec in
                                description_recommendations])

    # Анализ потенциальных улучшений
250    improvements = self._analyze_potential_improvements(
        user_game, similar_data, dissimilar_data)
    if improvements:
        recommendations.append("\nПотенциальные улучшения:
           ")
        recommendations.extend([f"- {imp}" for imp in
                                improvements])

255    # Если нет рекомендаций, добавляем общую рекомендацию
    if not recommendations:
        recommendations.append(

```

260

"Игра имеет схожие характеристики с другими играми в жанре. "

"Рекомендуется обратить внимание на развитие уникальных особенностей. "

)

return recommendations

Программный код модуля GameRecommendationAnalyzer

```

import numpy as np
from textblob import TextBlob
from collections import Counter
5 import re
from typing import List, Dict, Tuple

class ReviewAnalyzer:
    def __init__(self, repository):
10         self.repository = repository
        self.key_features = {
            'gameplay': ['gameplay', 'mechanics', 'controls',
                        'combat', 'movement'],
            'graphics': ['graphics', 'visuals', 'art', 'design',
                        'animation'],
            'story': ['story', 'narrative', 'plot', 'characters', 'dialogue'],
15         'performance': ['performance', 'fps', 'lag', 'optimization', 'stuttering'],
            'content': ['content', 'content', 'levels', 'missions', 'quests'],
            'multiplayer': ['multiplayer', 'coop', 'pvp', 'online', 'matchmaking'],
            'value': ['value', 'price', 'worth', 'content', 'hours'],
            'support': ['support', 'updates', 'patches', 'devs',
                        'community']
20         }
        # Веса для разных компонентов
        self.quant_weights = {
            'positive_ratio': 7,
            'sentiment': 6,
25         'playtime': 4,
            'vote_score': 4,
            'features': 8
        }
        self.weights = self._calculate_ahp_weights(self.quant_weights)
30
    def analyze_reviews(self, game_id: str) -> Dict:
        """

```

```

Анализирует отзывы игры и возвращает различные метрики
"""
35 try:
    # Получаем отзывы из базы данных
    reviews = list(self.repository.
        game_review_collection.find({"app_id": game_id
        }))
    if not reviews:
        return self._get_default_metrics()

40     # Базовые метрики
    total_reviews = len(reviews)
    if total_reviews == 0:
        print(f"Пустой список отзывов для игры {
            game_id}")
45     return self._get_default_metrics()

    positive_reviews = sum(1 for r in reviews if r.get
        ('voted_up', False))
    negative_reviews = total_reviews -
        positive_reviews

50     # Анализ тональности
    sentiments = []
    for review in reviews:
        if 'review_text' in review:
            try:
155         blob = TextBlob(str(review['
            review_text']))
            sentiments.append(float(blob.sentiment
                .polarity))
            except Exception as e:
                print(f"Ошибка при анализе тональности
                    отзыва: {str(e)}")
                continue

60     avg_sentiment = float(np.mean(sentiments)) if
        sentiments else 0.0

    # Анализ времени игры
    playtimes = []
65     for r in reviews:
        try:
            playtime = float(r.get('playtime_at_review
                ', 0))

```

```

        if playtime > 0: # Учитываем только положи-
            тельное время
            playtimes.append(playtime)
    except Exception as e:
        print(f"Ошибка при обработке времени игры:
              {str(e)}")
        continue
    avg_playtime = float(np.mean(playtimes)) if
    playtimes else 0.0

# Анализ качества отзывов
vote_scores = []
for r in reviews:
    try:
        score = float(r.get('weighted_vote_score',
                             0))
        if score > 0: # Учитываем только положи-
            тельные оценки
            vote_scores.append(score)
    except Exception as e:
        print(f"Ошибка при обработке оценки отзыва
              : {str(e)}")
        continue
    avg_vote_score = float(np.mean(vote_scores)) if
    vote_scores else 0.0

# Анализ ключевых особенностей
feature_mentions = self._analyze_features(reviews)

# Рассчитываем финальный score
review_score = self._calculate_review_score(
    positive_reviews=positive_reviews,
    total_reviews=total_reviews,
    avg_sentiment=avg_sentiment,
    avg_playtime=avg_playtime,
    avg_vote_score=avg_vote_score,
    feature_mentions=feature_mentions
)

return {
    'review_score': float(review_score),
    'metrics': {
        'positive_ratio': float(positive_reviews /
                                total_reviews if total_reviews > 0
                                else 0),

```

```

105         'avg_sentiment': float(avg_sentiment),
        'avg_playtime': float(avg_playtime),
        'avg_vote_score': float(avg_vote_score),
        'feature_mentions': {k: float(v) for k, v
                               in feature_mentions.items()}
    }
    }

110 except Exception as e:
    print(f"Ошибка при анализе отзывов для игры {
          game_id}: {str(e)}")
    return self._get_default_metrics()

115 def _analyze_features(self, reviews: List[Dict]) -> Dict[
    str, float]:
    """
    Анализирует упоминания ключевых особенностей в отзывах
    """
    feature_scores = {feature: 0.0 for feature in self.
                      key_features.keys()}
    processed_reviews = 0

120     for review in reviews:
        if 'review_text' not in review:
            continue

125         try:
            text = str(review['review_text']).lower()
            voted_up = bool(review.get('voted_up', False))

            for feature, keywords in self.key_features.
              items():
                for keyword in keywords:
                    if keyword in text:
                        # Увеличиваем счетчик особенности
                        # Положительные отзывы имеют больш
                          ий вес
                        weight = 1.2 if voted_up else 0.8
                        feature_scores[feature] += float(
                          weight)
                    processed_reviews += 1
        except Exception as e:
            print(f"Ошибка при анализе особенностей в отзы
                  ве: {str(e)}")
            continue
140

```

```

# Нормализуем scores
if processed_reviews == 0:
    return {k: 0.0 for k in feature_scores.keys()}

max_score = max(feature_scores.values()) if
    feature_scores.values() else 1.0
if max_score == 0:
    return {k: 0.0 for k in feature_scores.keys()}

return {k: float(v/max_score) for k, v in
        feature_scores.items()}

def _calculate_review_score(
    self,
    positive_reviews: int,
    total_reviews: int,
    avg_sentiment: float,
    avg_playtime: float,
    avg_vote_score: float,
    feature_mentions: Dict[str, float]
) -> float:
    """
    Рассчитывает финальный score на основе всех метрик
    """
    try:
        if total_reviews == 0:
            return 0.0

        # Нормализуем playtime (предполагаем, что 100 часо
        # в - это много)
        normalized_playtime = float(min(avg_playtime /
            100, 1.0))

        # Рассчитываем score для особенностей
        features_score = float(sum(feature_mentions.values
            ()) / len(feature_mentions)) if
            feature_mentions else 0.0

        # Собираем все компоненты
        components = {
            'positive_ratio': float(positive_reviews /
                total_reviews if total_reviews > 0 else 0),

```

```

        'sentiment': float((avg_sentiment + 1) / 2),
        # Нормализуем от [-1,1] до [0,1]
        'playtime': float(normalized_playtime),
        'vote_score': float(avg_vote_score),
        'features': float(features_score)
    }

    # Рассчитываем взвешенную сумму
    final_score = float(sum(components[k] * self.
        weights[k] for k in self.weights.keys()))
    return float(max(0.0, min(1.0, final_score)))

except Exception as e:
    print(f"Ошибка при расчете review score: {str(e)}"
        )
    return 0.0

def _calculate_ahp_weights(self, quant_weights: dict) ->
dict:
    """
    195     Рассчитывает веса методом анализа иерархий (АИР) с лог
        -чебышевской аппроксимацией.
    quant_weights: dict {ключ: вес}
    Возвращает dict {ключ: нормированный вес}
    """
    keys = list(quant_weights.keys())
    200     n = len(keys)
    weights = [max(quant_weights[k], 1e-6) for k in keys]
    A = np.ones((n, n))
    for i in range(n):
        for j in range(n):
            205             if i != j:
                A[i, j] = weights[i] / weights[j] if
                    weights[j] > 0 else 1e6
    logA = np.log(A)
    log_means = np.mean(logA, axis=1)
    raw_coefs = np.exp(log_means)
    210     norm_coefs = raw_coefs / np.sum(raw_coefs)
    print(f'Веса для отзывов: {norm_coefs}')
    return {k: w for k, w in zip(keys, norm_coefs)}

def _get_default_metrics(self) -> Dict:
    215     """
    Возвращает метрики по умолчанию, если отзывов нет
    """

```

```
220     return {
225         'review_score': 0.2,
        'metrics': {
            'positive_ratio': 0.0,
            'avg_sentiment': 0.0,
            'avg_playtime': 0.0,
            'avg_vote_score': 0.0,
            'feature_mentions': {feature: 0.0 for feature
                                in self.key_features.keys()}
        }
    }
```

Программный код модуля GameSimilarityAnalyzer

```

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from typing import Dict, List, Optional
5
from modules.DataRepository import DataRepository
from modules.FeatureExtractor import FeatureExtractor

10 class GameSimilarityAnalyzer:
    def __init__(self):
        self.repository = DataRepository()
        self.extractor = FeatureExtractor()

15     # Веса для различных характеристик
    self.quant_weights = {
        "description": 8,
        "genres": 6,
        "tags": 6,
20     "categories": 4,
        "numeric": 4
    }
    self.WEIGHTS = self._calculate_ahp_weights(self.
        quant_weights)

25     def get_games_features(self, user_game_data, games_ids):
        '''Получение векторизованных и нормализованных данных
            об играх'''
        games_features = {}

        target_game_id = user_game_data["game_id"]
30     games_features[target_game_id] = self.extractor.
        vectorize_user_game(user_game_data, None)

        # Добавляем данные о других играх
        for game_id in games_ids:
            games_features[game_id] = self.repository.
                get_data_features_from_db(game_id)

35     if not games_features:

```



```

        print("Не удалось получить данные ни об одной игре
              .")
        return None

40     return games_features

def numeric_similarity(self, value1, value2):
    """
    Вычисляет сходство между двумя числовыми значениями на
    45     основе
    Чебышевского расстояния.
    Сходство нормализовано и находится в диапазоне [0, 1].
    """
    chebyshev_distance = np.abs(value1 - value2)

    50     similarity = 1 - chebyshev_distance / 1.0
    return max(0, min(similarity, 1))

def calculate_similarity_with_game(self, user_game_data,
    games_ids):
    """
    55     Вычисляет схожесть пользовательской игры с другими игр
    ами.

    Использует косинусное сходство для текстовых данных, т
    ак как:
    1. Оно инвариантно к длине текста
    2. Учитывает семантическую близость
    60     3. Хорошо работает с разреженными данными
    """
    games_features = self.get_games_features(
        user_game_data, games_ids)
    if not games_features:
        return None

    65     target_game_id = user_game_data["game_id"]
    if target_game_id not in games_features:
        return None

    70     # Фильтруем игры, у которых есть все необходимые данны
    е
    valid_games = []
    for game_id in games_ids:
        if game_id in games_features and all(key in
            games_features[game_id] for key in [

```

```

        "description_embedding", "tag_hash", "
            genre_binary",
75         "category_binary", "price", "metacritic", "
            median_forever"
    ]):
        valid_games.append(game_id)

    if not valid_games:
80         return None

    similarities = []
    user_game_features = games_features[target_game_id]

85     # Числовые данные
    numeric_sims = []
    for game_id in valid_games:
        game_features = games_features[game_id]
        numeric_sim = np.mean([
90             self.numeric_similarity(user_game_features["
                price"], game_features["price"]),
            self.numeric_similarity(user_game_features["
                metacritic"], game_features["metacritic"]),
            self.numeric_similarity(user_game_features["
                median_forever"], game_features["
                median_forever"]),
        ])
        numeric_sims.append(numeric_sim)

95     numeric_sims = np.array(numeric_sims)

    # Получаем эмбединги целевой игры
    target_description = np.array(user_game_features["
        description_embedding"]).reshape(1, -1)
100    target_tags = np.array(user_game_features["tag_hash"])
        .reshape(1, -1)
    target_genres = np.array(user_game_features["
        genre_binary"]).reshape(1, -1)
    target_categories = np.array(user_game_features["
        category_binary"]).reshape(1, -1)

    # Инициализируем массивы для хранения сходств
105    description_sims = []
    tag_sims = []
    genre_sims = []
    category_sims = []

```

```

110     # Вычисляем сходство с каждой игрой по отдельности
    for game_id in valid_games:
        game_features = games_features[game_id]

        # Вычисляем сходство для каждого признака
115        desc_sim = cosine_similarity(
            target_description,
            np.array(game_features["description_embedding"]
                ).reshape(1, -1)
        )[0][0]

120        tag_sim = cosine_similarity(
            target_tags,
            np.array(game_features["tag_hash"]).reshape(1,
                -1)
        )[0][0]

125        genre_sim = cosine_similarity(
            target_genres,
            np.array(game_features["genre_binary"]).
                reshape(1, -1)
        )[0][0]

130        category_sim = cosine_similarity(
            target_categories,
            np.array(game_features["category_binary"]).
                reshape(1, -1)
        )[0][0]

135        description_sims.append(desc_sim)
        tag_sims.append(tag_sim)
        genre_sims.append(genre_sim)
        category_sims.append(category_sim)

140    # Преобразуем списки в numpy массивы
    description_sims = np.array(description_sims)
    tag_sims = np.array(tag_sims)
    genre_sims = np.array(genre_sims)
    category_sims = np.array(category_sims)

145    # Взвешенная сумма сходств
    similarities = (
        self.WEIGHTS["description"] * description_sims
        +

```

```

150         self.WEIGHTS["tags"] * tag_sims +
            self.WEIGHTS["genres"] * genre_sims +
            self.WEIGHTS["categories"] * category_sims +
            self.WEIGHTS["numeric"] * numeric_sims
        )

155     # Возвращаем словарь с результатами
    similarity_scores = {valid_games[i]: similarities[i]
        for i in range(len(valid_games))}
    similarity_scores.pop(target_game_id, None) # Удаляем
        собственное сходство (с самим собой)

    return similarity_scores

160 def find_similar_games(self, user_game_data, top_n=5):
    """
    Находит похожие игры для пользовательской игры.

165     Args:
        user_game_data (dict): Данные пользовательской игр
                               ы
        top_n (int): Количество похожих игр для возврата

    Returns:
        dict: Словарь с похожими играми и их оценками схож
              ести
    """
    try:
        if not user_game_data:
            print("Отсутствуют данные пользовательской игр
                  ы")
175         return {}

        # Проверяем наличие необходимых полей
        required_fields = ['name', 'description', 'genres',
            , 'categories', 'tags']
        if not all(field in user_game_data for field in
            required_fields):
            print(f"Отсутствуют обязательные поля в данных
                  игры: {required_fields}")
            return {}

        # Получаем все игры из базы данных
        all_games = list(self.repository.
            game_features_collection.find({}))
    
```

```

185         if not all_games:
            print("В базе данных нет игр для сравнения")
            return {}

        # Получаем ID всех игр
190     game_ids = [game['game_id'] for game in all_games
        if 'game_id' in game]
    if not game_ids:
        print("Не удалось получить ID игр из базы данн
            ых")
        return {}

    # Вычисляем схожесть
195     similarity_scores = self.
        calculate_similarity_with_game(user_game_data,
            game_ids)
    if not similarity_scores:
        print("Не удалось вычислить схожесть игр")
        return {}

    # Сортируем игры по схожести и берем топ-N
    sorted_games = sorted(similarity_scores.items(),
        key=lambda x: x[1], reverse=True)[:top_n]

    # Формируем результат
205     similar_games = {}
    for game_id, score in sorted_games:
        game_data = self.repository.
            get_data_features_from_db(game_id)
        if game_data:
            # Добавляем только необходимые поля
210             similar_games[str(game_id)] = {
                'name': game_data.get('name', ''),
                'description': game_data.get('
                    description', ''),
                'genres': game_data.get('genres', []),
                'categories': game_data.get('
                    categories', []),
                'tags': game_data.get('tags', []),
                'price': game_data.get('price', 0),
                'metacritic': game_data.get('
                    metacritic', 0),
                'median_forever': game_data.get('
                    median_forever', 0),
215                 'similarity_score': score

```

```

220         }

        return similar_games

    except Exception as e:
225         print(f"Ошибка при поиске похожих игр: {str(e)}")
        return {}

def _calculate_ahp_weights(self, quant_weights: dict) ->
dict:
    """
230     Рассчитывает веса методом анализа иерархий (АИР) с лог
        -чебышевской аппроксимацией.
        quant_weights: dict {ключ: вес}
        Возвращает dict {ключ: нормированный вес}
    """

    import numpy as np
235     keys = list(quant_weights.keys())
    n = len(keys)
    weights = [max(quant_weights[k], 1e-6) for k in keys]
    A = np.ones((n, n))
    for i in range(n):
240         for j in range(n):
            if i != j:
                A[i, j] = weights[i] / weights[j] if
                    weights[j] > 0 else 1e6

    logA = np.log(A)
    log_means = np.mean(logA, axis=1)
245     raw_coefs = np.exp(log_means)
    norm_coefs = raw_coefs / np.sum(raw_coefs)
    print(f'веса для схожести: {dict(zip(keys, norm_coefs))}')
    return {k: w for k, w in zip(keys, norm_coefs)}

```

Приложение 31

Программный код модуля GameUniqueFeaturesAnalyzer

```

from .BaseAnalyzer import BaseAnalyzer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
5 from collections import Counter
import re

from .DataRepository import DataRepository
from .FeatureExtractor import FeatureExtractor
10

class GameUniqueFeaturesAnalyzer:
    def __init__(self):
        self.analyzer = BaseAnalyzer()
        self.repository = DataRepository()
        self.extractor = FeatureExtractor()
15

    def _extract_key_phrases(self, text):
        """Извлекает ключевые фразы из текста"""
        # Разбиваем текст на предложения
        sentences = re.split(r'[!?!]+', text)
        # Удаляем пустые предложения и лишние пробелы
        sentences = [s.strip() for s in sentences if s.strip()]
        ]
        return sentences
20

    def _find_common_phrases(self, descriptions):
        """Находит общие фразы в описаниях игр"""
        all_phrases = []
        for desc in descriptions:
            phrases = self._extract_key_phrases(desc)
            all_phrases.extend(phrases)
30

        # Подсчитываем частоту фраз
        phrase_counter = Counter(all_phrases)
        # Возвращаем фразы, которые встречаются более чем в од
        ной игре
        return [phrase for phrase, count in phrase_counter.
            items() if count > 1]
35

```

```

def _analyze_description_similarity(self, target_embedding
, similar_embeddings):
    """Анализирует схожесть описаний с помощью эмбедингов
    """
40     similarities = []
    for emb in similar_embeddings:
        if emb is not None and target_embedding is not
            None:
                sim = cosine_similarity([target_embedding], [
                    emb])[0][0]
                similarities.append(sim)
45     return np.mean(similarities) if similarities else 0

def _find_unique_description_elements(self, target_desc,
similar_descs):
    """Находит уникальные элементы в описании"""
    target_phrases = set(self._extract_key_phrases(
        target_desc))
50     all_similar_phrases = set()

    for desc in similar_descs:
        phrases = self._extract_key_phrases(desc)
        all_similar_phrases.update(phrases)
55

    # Находим фразы, которые есть только в целевом описани
    и
    unique_phrases = target_phrases - all_similar_phrases
    return list(unique_phrases)

60 def create_game_pattern(self, game_data):
    """
    :param game_data: данные входной игры
    :return:
        "game_id": 0,
65         "name": user_game["name"],
        "description_embedding": description_embedding
        ,
        "genre_vector": genre_binary,
        "category_vector": category_matrix,
        "tag_vector": tag_matrix,
70         "price": price,
        "median_forever": median_forever,
        "metacritic": 0.5
    """

```



```

    return self.extractor.vectorize_user_game(game_data,
        None)
75
def load_and_analyze_games(self, game_ids):
    """
    Загружает и анализирует данные игр из
    game_features_collection
    Args:
    80     game_ids: Список ID игр для анализа
    Returns:
        dict: Словарь с общими характеристиками игр
    """
    jd = {
    85         'common_genres': set(),
        'common_tags': set(),
        'common_categories': set(),
        'avg_price': 0,
        'avg_metacritic': 0,
    90         'avg_median_forever': 0,
        'avg_positive_ratio': 0,
        'all_genres': [],
        'all_tags': [],
        'all_categories': [],
    95         'descriptions': [],
        'description_embeddings': []
    }
    try:
        # Получаем данные всех игр из
        game_features_collection
    100     games_data = []
    for game_id in game_ids:
        game_data = self.repository.
            game_features_collection.find_one({"game_id":
                game_id})
        if game_data:
            games_data.append(game_data)
    105
    if not games_data:
        return jd

    # Собираем все жанры, теги и категории
    110     all_genres = set()
    all_tags = set()
    all_categories = set()

```

```

115 for game in games_data:
    all_genres.update(game.get('genres', []))
    all_tags.update(game.get('tags', []))
    all_categories.update(game.get('categories',
    []))
    if 'description' in game:
        jd['descriptions'].append(game['
        description'])
120 if 'description_embedding' in game:
        jd['description_embeddings'].append(game['
        description_embedding'])

# Для поиска общих элементов используем списки мно
# жеств
genres_sets = [set(game.get('genres', [])) for
    game in games_data]
125 tags_sets = [set(game.get('tags', [])) for game in
    games_data]
categories_sets = [set(game.get('categories', []))
    for game in games_data]

# Находим общие элементы
common_genres = set.intersection(*genres_sets) if
    genres_sets else set()
130 common_tags = set.intersection(*tags_sets) if
    tags_sets else set()
common_categories = set.intersection(*
    categories_sets) if categories_sets else set()

# Рассчитываем средние значения
prices = [game.get('price', 0) for game in
    games_data]
135 metacritics = [game.get('metacritic', 0) for game
    in games_data]
median_forevers = [game.get('median_forever', 0)
    for game in games_data]
positive_ratios = [game.get('positive_ratio', 0)
    for game in games_data]

avg_price = sum(prices) / len(prices) if prices
    else 0
140 avg_metacritic = sum(metacritics) / len(
    metacritics) if metacritics else 0
avg_median_forever = sum(median_forevers) / len(
    median_forevers) if median_forevers else 0

```

```

    avg_positive_ratio = sum(positive_ratios) / len(
        positive_ratios) if positive_ratios else 0

    return {
145         'common_genres': list(common_genres),
        'common_tags': list(common_tags),
        'common_categories': list(common_categories),
        'avg_price': avg_price,
        'avg_metacritic': avg_metacritic,
150         'avg_median_forever': avg_median_forever,
        'avg_positive_ratio': avg_positive_ratio,
        'all_genres': list(all_genres),
        'all_tags': list(all_tags),
        'all_categories': list(all_categories),
155         'descriptions': jd['descriptions'],
        'description_embeddings': jd['
            description_embeddings']
    }

    except Exception as e:
160         print(f"Ошибка при анализе игр: {str(e)}")
        return jd

def compare_with_most_similar_and_dissimilar(self,
    user_game, similar_games):
    """Сравнивает целевую игру с группами похожих и отличи
        тельных игр"""
165     # Анализируем похожие игры
    similar_combined = self.load_and_analyze_games(
        similar_games)

    # Получаем уникальные жанры и теги
    target_genres = set(user_game.get('genres', []))
170     target_tags = set(user_game.get('tags', []))
    target_categories = set(user_game.get('categories',
        []))

    # Анализ описания
    description_analysis = {}
175     if 'description' in user_game and similar_combined['
        descriptions']:
        # Находим общие фразы в описаниях
        common_phrases = self._find_common_phrases(
            similar_combined['descriptions'])

```

```

description_analysis['common_phrases'] =
    common_phrases

180     # Находим уникальные элементы в описании
unique_phrases = self.
    _find_unique_description_elements(
        user_game['description'],
        similar_combined['descriptions']
    )
185 description_analysis['unique_phrases'] =
    unique_phrases

    # Анализируем схожесть описаний с помощью эмбедин
    гов
    if 'description_embedding' in user_game and
        similar_combined['description_embeddings']:
        similarity_score = self.
            _analyze_description_similarity(
190                 user_game['description_embedding'],
                    similar_combined['description_embeddings']
            )
        description_analysis['similarity_score'] =
            similarity_score

195 similar_unique_genres = target_genres - set(
    similar_combined['all_genres'])
similar_unique_tags = target_tags - set(
    similar_combined['all_tags'])
similar_unique_categories = target_categories - set(
    similar_combined['all_categories'])

similar_intersection_genres = target_genres & set(
    similar_combined['all_genres'])
200 similar_intersection_tags = target_tags & set(
    similar_combined['all_tags'])
similar_intersection_categories = target_categories &
    set(similar_combined['all_categories'])

return {
205     'size': len(similar_games),

    'common_genres_for_all_games': similar_combined['
        common_genres'],
    'common_categories_for_all_games':
        similar_combined['common_categories'],

```

```

210     'common_tags_for_all_games': similar_combined['
        common_tags'],

    'unique_in_target_genres': list(
        similar_unique_genres),
    'unique_in_target_categories': list(
        similar_unique_categories),
    'unique_in_target_tags': list(similar_unique_tags)
        ,

    'intersection_genres': list(
        similar_intersection_genres),
215    'intersection_categories': list(
        similar_intersection_categories),
    'intersection_tags': list(
        similar_intersection_tags),

    'all_genres': similar_combined['all_genres'],
    'all_tags': similar_combined['all_tags'],
220    'all_categories': similar_combined['all_categories
        '],

    'description_analysis': description_analysis,

    'avg_price_diff': round(abs(user_game.get('price',
        0) - similar_combined['avg_price']), 2)
225 }

```

Приложение 32

Программный код компонента GameAnalysisForm

```

import React, { useEffect } from 'react';
import { Box, TextField, Button, Grid, CircularProgress,
  Typography, Autocomplete } from '@mui/material';
import styles from './GameAnalysisForm.module.css';
5
const GameAnalysisForm = ({ onSubmit, loading }) => {
  const [gameData, setGameData] = React.useState({
    name: '',
    description: '',
10    genres: [],
    tags: [],
    categories: [],
    median_forever: 0,
    price: 0
15  });

  const [schemaData, setSchemaData] = React.useState({
    genres: [],
    tags: [],
20    categories: []
  });

  const [schemaLoading, setSchemaLoading] = React.useState(
    true);

25  useEffect(() => {
    const fetchSchemaData = async () => {
      try {
        const response = await fetch('http://localhost:8000/
          get_schema_data');
        if (!response.ok) {
30          throw new Error('Failed to fetch schema data');
        }
        const data = await response.json();
        setSchemaData(data);
      } catch (error) {
35        console.error('Error fetching schema data:', error);
      } finally {
        setSchemaLoading(false);
      }
    }
  });

```

```

    };

    fetchSchemaData();
  }, []));

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit(gameData);
  };

  const handleGameDataChange = (field, value) => {
    setGameData(prev => ({
      ...prev,
      [field]: value
    }));
  };

  if (schemaLoading) {
    return (
      <Box className={styles.form}>
        <CircularProgress />
      </Box>
    );
  }

  return (
    <Box component="form" onSubmit={handleSubmit} className={
      styles.form}>
      <Typography variant="h5" className={styles.formTitle}>
        Анализ новой игры
      </Typography>

      <Grid container spacing={3}>
        <Grid item xs={12}>
          <div className={styles.inputGroup}>
            <TextField
              fullWidth
              label="Название игры"
              value={gameData.name}
              onChange={(e) => handleGameDataChange('name', e.
                target.value)}
              required
              variant="outlined"
            />
          </div>

```

```

</Grid>

<Grid item xs={12}>
85   <div className={styles.inputGroup}>
      <TextField
        fullWidth
        multiline
        rows={4}
90        label="Описание игры (на английском)"
        value={gameData.description}
        onChange={(e) => handleGameDataChange('
          description', e.target.value)}
        required
        variant="outlined"
95      />
    </div>
  </Grid>

  <Grid item xs={12} md={6}>
100   <div className={styles.inputGroup}>
      <Autocomplete
        multiple
        options={schemaData.genres}
        value={gameData.genres}
105        onChange={(_, newValue) => handleGameDataChange
          ('genres', newValue)}
        renderInput={(params) => (
          <TextField
            {...params}
            label="Жанры"
            variant="outlined"
110          />
        )}
      />
    </div>
115  </Grid>

  <Grid item xs={12} md={6}>
    <div className={styles.inputGroup}>
      <Autocomplete
120        multiple
        options={schemaData.tags}
        value={gameData.tags}
        onChange={(_, newValue) => handleGameDataChange
          ('tags', newValue)}

```



```

125         renderInput={ (params) => (
            <TextField
                {...params}
                label="Терм"
                variant="outlined"

130            />
        )}
    />
</div>
</Grid>

135 <Grid item xs={12}>
    <div className={styles.inputGroup}>
        <Autocomplete
            multiple
            options={schemaData.categories}
            value={gameData.categories}
            onChange={(_, newValue) => handleGameDataChange
140                ('categories', newValue)}
            renderInput={ (params) => (
                <TextField
                    {...params}
                    label="Категории"
                    variant="outlined"

145                />
            )}
        />
    </div>
</Grid>

150 <Grid item xs={12} md={6}>
    <div className={styles.inputGroup}>
        <TextField
            fullWidth
            type="number"
            label="Стоимость (в долларах)"
            value={gameData.price}
            onChange={(e) => handleGameDataChange('price',
160                parseFloat(e.target.value))}
            required
            variant="outlined"
            InputProps={{
                inputProps: { min: 0, step: 0.01 }

165            }}
        </TextField>
    </div>
</Grid>

```

```

        />
      </div>
    </Grid>

    <Grid item xs={12} md={6}>
      <div className={styles.inputGroup}>
        <TextField
          fullWidth
          type="number"
          label="Средняя продолжительность игры (в минутах)"
          value={gameData.median_forever}
          onChange={(e) => handleGameDataChange('median_forever', parseFloat(e.target.value))}
          required
          variant="outlined"
          InputProps={{
            inputProps: { min: 0, step: 1 }
          }}
        />
      </div>
    </Grid>

    <Grid item xs={12}>
      <div className={styles.buttonGroup}>
        <Button
          type="submit"
          variant="contained"
          color="primary"
          className={styles.submitButton}
          disabled={loading}
        >
          {loading ? <CircularProgress size={24} /> : 'Анализировать игру'}
        </Button>
      </div>
    </Grid>
  </Grid>
</Box>
);
};

export default GameAnalysisForm;

```

Программный код компонента GameIdForm

```

import React, { useState, useEffect } from 'react';
import { TextField, Button, CircularProgress, Autocomplete }
  from '@mui/material';
import styles from './GameIdForm.module.css';
5
const GameIdForm = ({ onSubmit, loading }) => {
  const [gameId, setGameId] = useState('');
  const [games, setGames] = useState([]);
  const [selectedGame, setSelectedGame] = useState(null);
10  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchGames = async () => {
      try {
15        const response = await fetch('http://localhost:8000/
          get_all_games');
        if (!response.ok) {
          throw new Error('Failed to fetch games');
        }
        const data = await response.json();
20        setGames(data);
      } catch (err) {
        setError(err.message);
      }
    };

25    fetchGames();
  }, []);

  const handleSubmit = (e) => {
30    e.preventDefault();
    if (selectedGame) {
      onSubmit(selectedGame.game_id);
    }
  };
35

  return (
    <form onSubmit={handleSubmit} className={styles.form}>
      <Autocomplete
        options={games}

```

```

40      getOptionLabel={option => option.name || ''}
      value={selectedGame}
      onChange={(event, newValue) => {
        setSelectedGame(newValue);
      }}
45      renderInput={params => (
        <TextField
          {...params}
          label="Выберите игру"
          variant="outlined"
50          fullWidth
          required
          error={!!error}
          helperText={error}
        />
55      )}
      isOptionEqualToValue={option, value) => option.
        game_id === value.game_id}
    />
    <Button
      type="submit"
60      variant="contained"
      color="primary"
      disabled={loading || !selectedGame}
      className={styles.submitButton}
    >
65      {loading ? <CircularProgress size={24} /> : 'Анализиро
        вать'}
    </Button>
  </form>
);
};
70 export default GameIdForm;

```

Приложение 34

Программный код компонента GameRatingDisplay

```

import React, { useState, useEffect } from 'react';
import {
  5   Select,
      MenuItem,
      FormControl,
      InputLabel,
      Slider,
      Box,
  10  Typography,
      IconButton,
      Paper,
      Button
} from '@mui/material';
  15 import { Sort as SortIcon } from '@mui/icons-material';
import styles from './ResultsDisplay.module.css';

const GameRatingDisplay = () => {
  const [games, setGames] = useState([]);
  20  const [genres, setGenres] = useState(['all']);
      const [selectedGenre, setSelectedGenre] = useState('all');
      const [numGames, setNumGames] = useState(100);
      const [sortField, setSortField] = useState('
        competitiveness_score');
      const [sortDirection, setSortDirection] = useState('desc');
  25  const [loading, setLoading] = useState(false);
      const [error, setError] = useState(null);
      const [ownersRange, setOwnersRange] = useState([0,
        1000000000]);
      const [ownersRangeDraft, setOwnersRangeDraft] = useState([0,
        1000000000]);
      const [filterType, setFilterType] = useState('all'); // 'all
        ', 'genre', 'owners'
  30

  const fetchGenres = async () => {
    try {
      const response = await fetch('http://localhost:8000/
        get_schema_data');
      if (!response.ok) throw new Error('Failed to fetch
        schema data');
  35  const data = await response.json();

```

```

        setGenres(['all', ...data.genres.sort()]);
    } catch (err) {
        setError(err.message);
    }
40 };

const fetchGames = async () => {
    try {
        setLoading(true);
45        setError(null);
        let url;

        switch (filterType) {
            case 'genre':
50            url = 'http://localhost:8000/games/top-competitive-
                by-genre?genre=${selectedGenre}&top_n=${numGames
                }';
            break;
            case 'owners':
                url = 'http://localhost:8000/games/top-competitive-
                    by-owners?min_owners=${ownersRange[0]}&max_owners
                    =${ownersRange[1]}&top_n=${numGames}';
                break;
55            default:
                url = 'http://localhost:8000/games/top-competitive?
                    top_n=${numGames}';
        }

        const response = await fetch(url);
60        if (!response.ok) throw new Error('Failed to fetch games
            ');
        const data = await response.json();
        setGames(data.data || []);
    } catch (err) {
        setError(err.message);
65    } finally {
        setLoading(false);
    }
};

70 useEffect(() => {
    fetchGenres();
}, []);

useEffect(() => {

```

```

75     fetchGames();
    }, [numGames, selectedGenre, filterType, ownersRange]);

    const handleSort = (field) => {
      if (field === sortField) {
80         setSortDirection(sortDirection === 'asc' ? 'desc' : 'asc'
            ');
      } else {
        setSortField(field);
        setSortDirection('desc');
      }
85    };

    const handleOwnersRangeChange = (event, newValue) => {
      setOwnersRangeDraft(newValue);
    };

90    const applyOwnersRange = () => {
      setOwnersRange(ownersRangeDraft);
      // fetchGames вызовется через useEffect на ownersRange
    };

95    const filteredAndSortedGames = games
      .sort((a, b) => {
        const aValue = a[sortField];
        const bValue = b[sortField];
100        const direction = sortDirection === 'asc' ? 1 : -1;
        return direction * (aValue - bValue);
      });

    const getScoreClass = (score) => {
105      return score >= 70 ? styles.highScore :
        score >= 40 ? styles.mediumScore :
        styles.lowScore;
    };

110    const formatOwners = (value) => {
      if (value >= 1000000) {
        return `${(value / 1000000).toFixed(1)}M`;
      } else if (value >= 1000) {
        return `${(value / 1000).toFixed(1)}K`;
115      }
      return value.toString();
    };
  };

```

```

return (
120   <div className={styles.container}>
      <Paper className={styles.filtersContainer}>
        <div className={styles.filters}>
          <FormControl sx={{ minWidth: 120 }}>
            <InputLabel>Количество игр</InputLabel>
125     <Select
          value={numGames}
          label="Количество игр"
          onChange={(e) => setNumGames(e.target.value)}
        >
130     <MenuItem value={5}>5</MenuItem>
      <MenuItem value={10}>10</MenuItem>
      <MenuItem value={15}>15</MenuItem>
      <MenuItem value={20}>20</MenuItem>
      <MenuItem value={50}>50</MenuItem>
135     <MenuItem value={100}>100</MenuItem>
      </Select>
    </FormControl>

    <FormControl sx={{ minWidth: 120 }}>
140     <InputLabel>Тип фильтра</InputLabel>
      <Select
        value={filterType}
        label="Тип фильтра"
        onChange={(e) => setFilterType(e.target.value)}
145     >
      <MenuItem value="all">Все игры</MenuItem>
      <MenuItem value="genre">По жанру</MenuItem>
      <MenuItem value="owners">По владельцам</MenuItem>
      </Select>
150    </FormControl>

    {filterType === 'genre' && (
      <FormControl sx={{ minWidth: 120 }}>
        <InputLabel>Жанр</InputLabel>
155     <Select
        value={selectedGenre}
        label="Жанр"
        onChange={(e) => setSelectedGenre(e.target.
          value)}
      >
160     {genres.map(genre => (
      <MenuItem key={genre} value={genre}>

```



```

        {genre === 'Action' ? 'Action' : genre}
      </MenuItem>
    )))
  </Select>
</FormControl>
})

{filterType === 'owners' && (
  <Box sx={{ width: 300 }}>
    <Typography gutterBottom>
      Диапазон владельцев: {formatOwners(
        ownersRangeDraft[0])} - {formatOwners(
        ownersRangeDraft[1])}
    </Typography>
    <Slider
      value={ownersRangeDraft}
      onChange={handleOwnersRangeChange}
      valueLabelDisplay="auto"
      min={0}
      max={1000000000}
      step={100000}
      valueLabelFormat={formatOwners}
    />
    <Button onClick={applyOwnersRange} variant="
      contained" sx={{ mt: 1 }}>
      Применить
    </Button>
  </Box>
})

<Box sx={{ display: 'flex', gap: 1, alignItems: '
  center' }}>
  <Box sx={{ display: 'flex', alignItems: 'center'
    }}>
    <Typography variant="body2" sx={{ mr: 1 }}>
      {sortField === 'competitiveness_score'
        ? 'Сортировка по ком.потенциалу (${
          sortDirection === 'asc' ? 'возрастанию' :
          'убыванию'})'
        : 'Сортировать по ком.потенциалу'}
    </Typography>
    <IconButton
      onClick={() => handleSort('
        competitiveness_score')}

```

```

200         color={sortField === 'competitiveness_score' ?
            'primary' : 'default'}
        aria-label="Сортировать по ком.потенциалу"
    >
        <SortIcon style={{
            transform: sortField === '
                competitiveness_score' && sortDirection
                === 'asc' ? 'rotate(180deg)' : 'none'
        }}/>
        </IconButton>
205 </Box>

    <Box sx={{ display: 'flex', alignItems: 'center'
        }}>
        <Typography variant="body2" sx={{ mr: 1 }}>
            {sortField === 'price'
210             ? 'Сортировка по цене (${sortDirection === '
                asc' ? 'возрастанию' : 'убыванию'})'
                : 'Сортировать по цене'}
        </Typography>
        <IconButton
            onClick={() => handleSort('price')}
215             color={sortField === 'price' ? 'primary' : '
                default'}
            aria-label="Сортировать по цене"
        >
            <SortIcon style={{
                transform: sortField === 'price' &&
                sortDirection === 'asc' ? 'rotate(180deg)'
                : 'none'
220             }}/>
            </IconButton>
        </Box>
    </Box>
</div>
225 </Paper>

{error && (
    <div className={styles.error}>
        {error}
230    </div>
)}

{loading ? (
    <div className={styles.loading}>Загрузка...</div>

```

```

235     ) : (
        <div className={styles.gamesGrid}>
          {filteredAndSortedGames.map((game, index) => (
            <div key={game.game_id} className={` ${styles.
              gameCard} ${getScoreClass(game.
                competitiveness_score)} `}>
              <div className={styles.rankBadge}>{index + 1}</div>
240              <h3>{game.name}</h3>
              <div className={styles.gameDetails}>
                <p>Жанры: {game.genres.join(', ')}</p>
                <p>Владельцы: {formatOwners(game.owners)}</p>
                <div className={styles.competitivenessScore}>
245                  <div className={styles.scoreValue}>
                    <span>{(game.competitiveness_score).
                      toFixed(2)}%</span>
                  </div>
                  <div className={styles.scoreIndicator}>
                    <div
250                      className={` ${styles.scoreBar} ${
                        getScoreClass(game.
                          competitiveness_score)} `
                      style={{ width: `${game.
                        competitiveness_score}%` }}
                    />
                  </div>
                </div>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
255  );
  };
260  </div>
  );
};

export default GameRatingDisplay;

```

Программный код компонента ResultsDisplay

```

import React, { useState } from 'react';
import { Paper } from '@mui/material';
import styles from './ResultsDisplay.module.css';
5
const ResultsDisplay = ({ results }) => {
  const [expandedSections, setExpandedSections] = useState({})
  ;

  function groupRecommendations(recommendations) {
10
    const sections = [];
    let currentSection = null;

    recommendations.forEach(line => {
      if (line.includes(':')) {
15
        // Новая секция
        if (currentSection) sections.push(currentSection);
        currentSection = {
          title: line.replace(/^[- \s]+/, '').trim(),
          items: []
20
        };
      } else if (currentSection) {
        currentSection.items.push(line.replace(/^[- \s]+/,
          '').trim());
      }
    });
25
    if (currentSection) sections.push(currentSection);
    return sections;
  }

  if (!results) return null;
30

  const toggleSection = (sectionId) => {
    setExpandedSections(prev => ({
      ...prev,
      [sectionId]: !prev[sectionId]
35
    }));
  };

  const formatOwners = (value) => {
    if (value >= 1000000) {

```

```

40     return `${(value / 1000000).toFixed(1)}M`;
    } else if (value >= 1000) {
        return `${(value / 1000).toFixed(1)}K`;
    }
    return value.toString();
45 };

const renderCompetitivenessScore = () => {
    if (results.competitiveness_score === undefined) return
        null;
    const score = results.competitiveness_score;
50     const scoreClass = score >= 0.7 ? styles.highScore :
        score >= 0.4 ? styles.mediumScore :
        styles.lowScore;

    return (
55     <div className={` ${styles.section} ${scoreClass}`}>
        <h2>Оценка конкурентоспособности</h2>
        <div className={styles.competitivenessScore}>
            <div className={styles.scoreValue}>
                <span>{(score * 100).toFixed(2)}%</span>
60            </div>
            <div className={styles.scoreIndicator}>
                <div
                    className={` ${styles.scoreBar} ${scoreClass}`}
                    style={{ width: `${score * 100}%` }}
65                />
            </div>
            <p className={styles.scoreDescription}>
                {score >= 0.7 ? 'Высокий коммерческий потенциал' :
                score >= 0.4 ? 'Средний коммерческий потенциал' :
70                'Низкий коммерческий потенциал'}
            </p>
        </div>
    </div>
    );
75 };

const renderClusterInfo = () => {
    if (!results.cluster_info || !results.similar_games)
        return null;

80     // Создаем мапу для быстрого доступа к схожести по game_id
    const similarityMap = {};
    results.similar_games.forEach(game => {

```

```

    similarityMap[game.Game_ID] = game.Similarity_score;
  });

85
  // Сортируем игры по схожести (от большей к меньшей)
  const sortedClusterGames = [...results.cluster_info].sort
    ((a, b) => {
      const similarityA = similarityMap[a["Game ID"]] || 0;
      const similarityB = similarityMap[b["Game ID"]] || 0;
90      return similarityB - similarityA;
    });

  return (
    <div className={styles.section}>
95      <h2>Игры в кластере</h2>
      <div className={styles.clusterGames}>
        {sortedClusterGames.map((game) => (
          <div key={game["Game ID"]} className={styles.
            clusterGameCard}>
100          <h3>{game.Name || 'Без названия'}</h3>
          <div className={styles.gameDetails}>
            <p>ID: {game["Game ID"]}</p>
            <p>Схожесть: {((similarityMap[game["Game ID"]]
              || 0) * 100).toFixed(2)}%</p>
            <p>Жанры: {(game.Genres || []).join(', ') || '
              Нет данных'}</p>
            <p>Стоимость: ${game.Price || 0}</p>
105          <p>Метакритик: {game["Metacritic"] || 0}</p>
          <p>Среднее время: {game["Median Forever"] ||
            0} ч</p>
          </div>
        </div>
      </div>
    )})
110    </div>
  </div>
);
};

115 const renderSimilarGames = () => {
  if (!results.similar_games || !results.
    competitiveness_scores) return null;

  // Сортируем игры по схожести (от большей к меньшей)
  const sortedSimilarGames = [...results.similar_games].sort
    ((a, b) => {
120      return b.Similarity_score - a.Similarity_score;
    });

```

```

});

return (
  <div className={styles.section}>
125   <h2>Топ похожих игр</h2>
    <div className={styles.similarGames}>
      {sortedSimilarGames.map((game) => {
        const competitivenessScore = results.
          competitiveness_scores[game.Game_ID] || 0;
        return (
130   <div key={game.Game_ID} className={styles.
            gameCard}>
            <h3>{game.Name || 'Без названия'}</h3>
            <div className={styles.gameDetails}>
              <p>Коммерческий потенциал: {(
                competitivenessScore * 100).toFixed(2)}%</p>
              <p>Схожесть: {(game.Similarity_score * 100).
                toFixed(2)}%</p>
135   <p>Жанры: {(game.Genres || []).join(', ') ||
              'Нет данных'}</p>
              <p>Стоимость: ${game.Price || 0}</p>
              <p>Метакритик: {game.Metacritic || 0}</p>
              <p>Среднее время: {game["Median Forever"] ||
                0} ч</p>
              <div className={styles.scoreIndicator}>
140   <div
                className={` ${styles.scoreBar} ${
                  competitivenessScore >= 0.7 ? styles.
                    highScore :
                  competitivenessScore >= 0.4 ? styles.
                    mediumScore :
                    styles.lowScore
                }`
                style={{ width: `${competitivenessScore
                  * 100}%` }}
145   />
              </div>
            </div>
          </div>
150   </div>
        );
      })}
    </div>
155  </div>
);

```

```

};

const renderRecommendations = () => {
  if (!results.recommendations) return null;

  const grouped = groupRecommendations(results.
    recommendations);

  return (
    <div className={styles.section}>
      <h2>Рекомендации</h2>
      <div className={styles.recommendations}>
        {grouped.map((section, idx) => (
          <div key={idx} className={styles.
            recommendationSection}>
            <div className={styles.recommendationTitle}>{
              section.title}</div>
            <ul className={styles.recommendationList}>
              {section.items.map((item, i) => (
                <li key={i} className={styles.
                  recommendationParagraph}>
                  {item}
                </li>
              )))}
            </ul>
          </div>
        )))}
      </div>
    </div>
  );
};

return (
  <div className={styles.container}>
    <div className={styles.header}>
      <h1>Результаты анализа</h1>
    </div>
    {renderCompetitivenessScore()}
    {/* {renderClusterInfo()} */}
    {renderSimilarGames()}
    {renderRecommendations()}
  </div>
);
};

```



```
| export default ResultsDisplay;
```

Программный код компонента SettingsDialog

```

import React, { useState, useCallback, useEffect } from 'react'
';
import {
  Dialog,
  DialogTitle,
  DialogContent,
  DialogActions,
  Button,
  Typography,
  Switch,
  FormControlLabel,
  CircularProgress,
  Alert
} from '@mui/material';
import { DragDropContext, Droppable, Draggable } from 'react-
  beautiful-dnd';
import DragIndicatorIcon from '@mui/icons-material/
  DragIndicator';
import styles from './SettingsDialog.module.css';
import QuantitativeWeights from './QuantitativeWeights';

const criteriaInfo = {
  owners: {
    name: "Количество владельцев",
    description: "Показывает популярность игры среди пользоват
      елей"
  },
  positive_ratio: {
    name: "Соотношение положительных отзывов",
    description: "Процент положительных отзывов от общего числ
      а"
  },
  revenue: {
    name: "Выручка",
    description: "Финансовый успех игры"
  },
  activity: {
    name: "Активность сообщества",
    description: "Насколько активны игроки в игре"
  },

```

```

freshness: {
  name: "Новизна игры",
  description: "Насколько недавно вышла игра"
},
review_score: {
  name: "Анализ отзывов",
  description: "Общая оценка игры на основе отзывов"
}
};

const SettingsDialog = ({ open, onClose, onSave,
  initialSettings }) => {
  const [criteria, setCriteria] = useState([]);
  const [enabledCriteria, setEnabledCriteria] = useState({});
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [showQuantitative, setShowQuantitative] = useState(
    false);
  const [quantWeights, setQuantWeights] = useState({});

  // Загружаем начальные настройки при открытии диалога
  useEffect(() => {
    if (open) {
      fetchSettings();
    }
  }, [open]);

  const fetchSettings = async () => {
    try {
      setLoading(true);
      setError(null);

      const response = await fetch('http://localhost:8000/
        settings');
      if (!response.ok) {
        throw new Error('Failed to fetch settings');
      }

      const data = await response.json();

      // Преобразуем порядок важности в список критериев
      const orderedCriteria = data.importance_order.map(index
        => data.criteria[index]);

      setCriteria(orderedCriteria);

```

```

// Инициализируем все критерии как включенные
setEnabledCriteria(
80   Object.fromEntries(data.criteria.map(key => [key, true
      ]))
    );
} catch (err) {
  setError(err.message);
  // Используем начальные настройки в случае ошибки
85   setCriteria(initialSettings.criteria || Object.keys(
      criteriaInfo));
  setEnabledCriteria(initialSettings.enabledCriteria ||
    Object.fromEntries(Object.keys(criteriaInfo).map(key
      => [key, true]))
    );
} finally {
90   setLoading(false);
}
};

const handleDragEnd = useCallback((result) => {
95   if (!result.destination) return;

   const sourceIndex = result.source.index;
   const destinationIndex = result.destination.index;

100   if (sourceIndex === destinationIndex) return;

   setCriteria(prevCriteria => {
     const newCriteria = Array.from(prevCriteria);
     const [removed] = newCriteria.splice(sourceIndex, 1);
105     newCriteria.splice(destinationIndex, 0, removed);
     return newCriteria;
   });
}, []);

110 const handleToggleCriterion = useCallback((criterion) => {
   setEnabledCriteria(prev => ({
     ...prev,
     [criterion]: !prev[criterion]
   }));
115 }, []);

const handleSave = useCallback(async () => {
  try {
    setLoading(true);

```

```

120      setError(null);

      const response = await fetch('http://localhost:8000/api/
        settings', {
          method: 'POST',
          headers: {
125            'Content-Type': 'application/json',
          },
          body: JSON.stringify({
            criteria,
            enabledCriteria
130          }),
        });

      if (!response.ok) {
        throw new Error('Failed to save settings');
135      }

      onSave({
        criteria,
        enabledCriteria
140      });
      onClose();
    } catch (err) {
      setError(err.message);
    } finally {
145      setLoading(false);
    }
  }, [criteria, enabledCriteria, onSave, onClose]);

  return (
150    <Dialog open={open} onClose={onClose} maxWidth="md"
      fullWidth>
      <DialogTitle>Настройки критериев оценки</DialogTitle>
      <DialogContent>
        {error && (
          <Alert severity="error" className={styles.errorAlert
155            }>
            {error}
          </Alert>
        )}

        <Typography variant="subtitle1" gutterBottom>

```



```

200 {...provided.draggableProps}
style={{
    ...provided.draggableProps.
        style,
    marginBottom: '8px',
    height: 'auto',
    opacity: enabledCriteria[
        criterion] ? 1 : 0.5
205 }}
className={
    enabledCriteria[criterion]
        ? styles.criterionWrapper
        : styles.disabledCriterion
210 }
>
<div
    className={` ${styles.
        criterionItem} ${snapshot.
            isDragging ? styles.
                dragging : ''} `}
    style={{ height: '100%' }}
215 >
    <div {...provided.
        dragHandleProps} className
            = {styles.dragHandle}>
        <DragIndicatorIcon />
    </div>
    <div className={styles.
        criterionContent}>
220 <div className={styles.
        criterionText}>
        <Typography variant="
            subtitle1" noWrap>
            {criteriaInfo[criterion
                ].name}
        </Typography>
        <Typography variant="body2
            " color="textSecondary"
            noWrap>
225 {criteriaInfo[criterion
                ].description}
        </Typography>
    </div>
    <FormControlLabel
        control={

```

```

230         <Switch
            edge="end"
            checked={
                enabledCriteria[
                    criterion]}
            onChange={() =>
                handleToggleCriterion
                    (criterion)}
            />
235     }
        className={styles.switch}
    />
    </div>
    </div>
240 </div>
    )}
    </Draggable>
    )})
    {provided.placeholder}
245 </div>
    )}
    </Droppable>
    </DragDropContext>
    )}
250 </>
    )}
</DialogContent>
<DialogActions>
    <Button onClick={onClose} disabled={loading}>Отмена</
        Button>
255 <Button
    onClick={handleSave}
    variant="contained"
    color="primary"
    disabled={loading}
260 >
    {loading ? <CircularProgress size={24} /> : 'Сохрани
        ть'}
    </Button>
    </DialogActions>
</Dialog>
265 );
};

export default SettingsDialog;

```


Программный код тестов модуля FeatureExtractor

```

import numpy as np
import pytest
from modules.FeatureExtractor import FeatureExtractor
5
@pytest.fixture
def sample_data():
    return [
        {
10
            "data": {
                "detailed_description": "An epic <b>RPG</b>
                    with dragons!",
                "metacritic": {"score": 85},
                "categories": [{"description": "Single-player"
                    }, {"description": "Multi-player"}]
            },
15
            "name": "Game1",
            "genre": "RPG, Action, Adventure",
            "tags": {"RPG": 1, "Action": 1, "Fantasy": 1},
            "price": 59.99,
            "median_forever": 120,
20
            "positive": 100,
            "negative": 20,
            "owners": "1000 .. 2000"
        },
        {
25
            "data": {
                "detailed_description": "<p>Fast-paced action
                    shooter.</p>",
                "metacritic": {"score": 70},
                "categories": [{"description": "Single-player"
                    }, {"description": "Multi-player"}, {""
                    description": "Co-op"}]
            },
30
            "name": "Game2",
            "genre": "Action, FPS",
            "tags": {"FPS": 1, "Action": 1, "Shooter": 1},
            "price": 19.99,
            "median_forever": 50,
35
            "positive": 50,
            "negative": 10,

```

```

        "owners": "500 .. 1000"
    }
]
40
@pytest.fixture
def feature_extractor():
    return FeatureExtractor()

45 def test_text_preprocessing(feature_extractor, sample_data):
    processed_texts = [feature_extractor.preprocess_text(game[
        "data"]["detailed_description"]) for game in
        sample_data]
    expected_texts = ["an epic rpg with dragons", "fast-paced
        action shooter"]
    assert processed_texts == expected_texts

50 def test_create_game_features(feature_extractor, sample_data):
    features = [feature_extractor.create_game_features(game)
        for game in sample_data]
    assert len(features) == len(sample_data)
    for f in features:
        assert "name" in f
        assert "description" in f
55         assert "genres" in f
        assert "tags" in f
        assert "price" in f
        assert "categories" in f
        assert "metacritic" in f
60         assert "median_forever" in f
        assert "is_vr" in f
        assert "is_multiplayer" in f
        assert "owners" in f
65         assert "positive" in f
        assert "negative" in f
        assert "positive_ratio" in f

def test_empty_data_handling(feature_extractor):
70     features = []
    assert features == []

def test_missing_values_handling(feature_extractor):
    data_with_missing = {
75         "data": {
            "detailed_description": "Test game",
            "metacritic": {},

```

```

        "categories": [],
    },
    "name": "Game3",
    "genre": "",
    "tags": {},
    "price": None,
    "median_forever": None,
    "positive": None,
    "negative": None,
    "owners": ""
}
features = feature_extractor.create_game_features(
    data_with_missing)
assert "name" in features
assert "description" in features
assert "genres" in features
assert "tags" in features
assert "price" in features
assert "categories" in features
assert "metacritic" in features
assert "median_forever" in features
assert "is_vr" in features
assert "is_multiplayer" in features
assert "owners" in features
assert "positive" in features
assert "negative" in features
assert "positive_ratio" in features

def test_vectorize_text_data(feature_extractor, sample_data):
    features_dict = {i: feature_extractor.create_game_features
                     (game) for i, game in enumerate(sample_data)}
    result = feature_extractor.vectorize_text_data(
        features_dict)
    assert "description_embeddings" in result
    assert "genre_matrix" in result
    assert "category_matrix" in result
    assert "tag_matrix" in result
    assert "tag_names" in result
    assert "genre_names" in result
    assert "category_names" in result
    # Проверяем размерности
    n = len(features_dict)
    assert result["description_embeddings"].shape[0] == n
    assert result["genre_matrix"].shape[0] == n

```

```

120     assert result["category_matrix"].shape[0] == n
       assert result["tag_matrix"].shape[0] == n

def test_normalize_numeric_data(feature_extractor, sample_data
):
    features_dict = {i: feature_extractor.create_game_features
                     (game) for i, game in enumerate(sample_data)}
125     result = feature_extractor.normalize_numeric_data(
         features_dict)
    assert "prices" in result
    assert "metacritic" in result
    assert "median_forever" in result
    n = len(features_dict)
130     assert result["prices"].shape[0] == n
    assert result["metacritic"].shape[0] == n
    assert result["median_forever"].shape[0] == n
    # Проверяем, что значения нормализованы (от 0 до 1)
    assert np.all((0 <= result["prices"]) & (result["prices"]
        <= 1))
135     assert np.all((0 <= result["metacritic"]) & (result["
        metacritic"] <= 1))
    assert np.all((0 <= result["median_forever"]) & (result["
        median_forever"] <= 1))

def test_vectorize_user_game(feature_extractor):
    # Пример пользовательской игры
140     user_game = {
        "name": "Test Game",
        "description": "A unique adventure with puzzles.",
        "genres": ["Adventure", "Puzzle"],
        "categories": ["Single-player"],
145         "tags": ["Puzzle", "Logic"],
        "price": 25.0,
        "median_forever": 40
    }
    # Пример схемы (можно взять из feature_extractor.schema, е
        сли она корректная)
150     schema = {
        "genres": ["Adventure", "Puzzle", "Action"],
        "max_price": 100.0,
        "max_median_forever": 100.0
    }
155     result = feature_extractor.vectorize_user_game(user_game,
        schema)
    # Проверяем, что результат это словарь с нужными ключами

```

```
assert isinstance(result, dict)
for key in [
    "game_id", "name", "description_embedding", "
160     genre_vector",
    "category_vector", "tag_vector", "price", "
        median_forever", "metacritic"
]:
    assert key in result
    # Проверяем размерности векторов
    assert isinstance(result["description_embedding"], list)
165     assert isinstance(result["genre_vector"], list)
    assert isinstance(result["category_vector"], list)
    assert isinstance(result["tag_vector"], list)
    # Проверяем нормализацию числовых признаков
    assert 0 <= result["price"] <= 1
170     assert 0 <= result["median_forever"] <= 1
```

Приложение 38

Программный код тестов модуля FeatureExtractor для HTML

```

import pytest
from modules.FeatureExtractor import FeatureExtractor

5 @pytest.fixture
def html_sample_data():
    return [
        {
            "description": "An epic <b>RPG</b> with <i>dragons
10             </i>!",
            "price": 59.99,
            "metacritic": 85,
            "median_forever": 120
        },
        {
15         "description": "<p>Fast-paced <strong>action</strong> shooter.</p>",
            "price": 19.99,
            "metacritic": 70,
            "median_forever": 50
        },
20         {
            "description": "<div>Open world <span>adventure</span> game</div>",
            "price": 39.99,
            "metacritic": 75,
            "median_forever": 80
25         }
    ]

@pytest.fixture
def feature_extractor():
30     return FeatureExtractor()

def test_html_tag_removal(feature_extractor, html_sample_data)
:
    processed_texts = [feature_extractor.preprocess_text(game[
        "description"]) for game in html_sample_data]
    expected_texts = [
35         "an epic rpg with dragons",
        "fast-paced action shooter",
    ]

```

```

        "open world adventure game"
    ]
    assert processed_texts == expected_texts
40
def test_nested_html_tags(feature_extractor):
    nested_html = "<div>Game with <b>nested <i>tags</i></b></div>"
    processed = feature_extractor.preprocess_text(nested_html)
    assert processed == "game with nested tags"
45
def test_special_html_characters(feature_extractor):
    special_chars = "Game with &lt;special&gt; characters &
        ; symbols"
    processed = feature_extractor.preprocess_text(
        special_chars)
    assert processed == "game with special characters symbols"
50
def test_empty_html_tags(feature_extractor):
    empty_tags = "<div></div><p></p>Empty content"
    processed = feature_extractor.preprocess_text(empty_tags)
    assert processed == "empty content"
55
def test_html_attributes(feature_extractor):
    with_attributes = '<div class="game" id="123">Game with
        attributes</div>'
    processed = feature_extractor.preprocess_text(
        with_attributes)
    assert processed == "game with attributes"
60
def test_multiple_spaces_after_html_removal(feature_extractor):
    :
    multiple_spaces = "<div>Game</div> <p>with</p> <span>
        spaces</span>"
    processed = feature_extractor.preprocess_text(
        multiple_spaces)
    assert processed == "game with spaces"

```

Программный код тестов модуля GameClusterer

```

import pytest
import numpy as np
from modules.GameClusterer import GameClusterer
5
@pytest.fixture
def sample_features():
    return np.array([
        [0.2, 0.8, 0.5],
10        [0.25, 0.75, 0.45],
        [0.7, 0.3, 0.8],
        [0.72, 0.28, 0.75],
        [0.8, 0.2, 0.9]
    ])
15
@pytest.fixture
def clusterer():
    return GameClusterer(n_clusters=2)

20 def test_kmeans_clustering(clusterer, sample_features):
    kmeans = clusterer.kmeans
    labels = kmeans.fit_predict(sample_features)
    assert len(labels) == len(sample_features)
    assert set(labels) <= {0, 1}
25
def test_normalize_features(clusterer, sample_features):
    normalized = clusterer._normalize_features(sample_features)
    assert np.all((0 <= normalized) & (normalized <= 1))

30 def test_calculate_similarity(clusterer, sample_features):
    sim = clusterer._calculate_similarity(sample_features[0],
        sample_features[1])
    assert 0 <= sim <= 1

def test_evaluate_clusters(clusterer, sample_features):
35     ids = list(range(len(sample_features)))
    names = [f"Game{i}" for i in ids]
    optimal_k = clusterer.evaluate_clusters(sample_features,
        ids, names, max_clusters=3)
    assert 2 <= optimal_k <= 3

```



```

40 def test_get_features_matrix(monkeypatch, clusterer):
    # Мокаем пеностопу
    fake_games = [{
        'game_id': i,
        'name': f'Game{i}',
45     'category_binary': [0, 1],
        'description_embedding': np.random.rand(384).tolist(),
        'genre_binary': [1, 0],
        'tag_hash': [0.5, 0.5],
        'price': 10.0 * i,
50     'metacritic': 50 + i,
        'median_forever': 20 + i
    } for i in range(5)]
    monkeypatch.setattr(clusterer.repository.
        game_features_collection, "find", lambda *a, **k:
            fake_games)
    features_matrix, game_ids, game_names = clusterer.
        get_features_matrix()
55     assert features_matrix.shape[0] == len(fake_games)

def test_error_on_empty_db(monkeypatch, clusterer):
    monkeypatch.setattr(clusterer.repository.
        game_features_collection, "find", lambda *a, **k: [])
    with pytest.raises(ValueError):
60         clusterer.get_features_matrix()

```

Программный код тестов стабильности GameClusterer

```

import pytest
import numpy as np
from modules.GameClusterer import GameClusterer
5
@pytest.fixture
def clusterer():
    # n_clusters не важен для большинства тестов, но зададим д
    # ля явности
    return GameClusterer(n_clusters=2)
10
def test_get_features_matrix_shape(monkeypatch, clusterer):
    # Мокаем метод репозитория, чтобы не обращаться к MongoDB
    fake_games = [{
        'game_id': i,
        'name': f'Game{i}',
        'category_binary': [0, 1],
        'description_embedding': np.random.rand(384).tolist(),
        'genre_binary': [1, 0],
        'tag_hash': [0.5, 0.5],
        'price': 10.0 * i,
        'metacritic': 50 + i,
        'median_forever': 20 + i
    } for i in range(10)]
    monkeypatch.setattr(clusterer.repository.
        game_features_collection, "find", lambda *a, **k:
        fake_games)
25
    features_matrix, game_ids, game_names = clusterer.
        get_features_matrix()
    assert features_matrix.shape[0] == len(fake_games)
    assert len(game_ids) == len(fake_games)
    assert len(game_names) == len(fake_games)

30 def test_evaluate_clusters_returns_optimal_k(clusterer):
    # Простая матрица с двумя кластерами
    X = np.vstack([np.random.normal(0, 1, (10, 10)), np.random
        .normal(5, 1, (10, 10))])
    ids = list(range(20))
    names = [f"Game{i}" for i in ids]
35
    optimal_k = clusterer.evaluate_clusters(X, ids, names,
        max_clusters=4)

```

```

    assert 2 <= optimal_k <= 4

def test_perform_clustering_and_balance(clusterer):
    # Два явных кластера
40 X = np.vstack([np.random.normal(0, 1, (10, 5)), np.random.
        normal(5, 1, (10, 5))])
    kmeans = clusterer.kmeans
    kmeans.n_clusters = 2
    labels = kmeans.fit_predict(X)
    # Проверяем, что кластеры примерно равны по размеру
45 sizes = np.bincount(labels)
    assert abs(sizes[0] - sizes[1]) <= 2

def test_predict_cluster(monkeypatch, clusterer):
    # Мокаем все загрузки моделей и коллекций
50 monkeypatch.setattr("joblib.load", lambda path: np.eye(2)
        if "scaler" in path else np.eye(50))
    monkeypatch.setattr(clusterer.repository, "
        game_cluster_collection", type("MockCol", (), {
            "find": lambda self: [{"coordinates": [0, 0], "
                cluster_id": 0}, {"coordinates": [1, 1], "
                cluster_id": 1}]
        })())
    monkeypatch.setattr("joblib.load", lambda path: np.random.
        rand(2, 50) if "original_features" in path else np.eye
        (2))

55 vectorized_game_data = {
    "description_embedding": np.random.rand(50),
    "price": 0.5,
    "median_forever": 0.5,
60 "metacritic": 0.5,
    "category_vector": [0, 1],
    "genre_vector": [1, 0],
    "tag_vector": [0.5, 0.5]
}

65 result = clusterer.predict_cluster(vectorized_game_data)
    assert result is None or isinstance(result, int)

def test_visualize_clusters_tsne(clusterer):
70 # Проверяем, что функция не падает на малых данных
    X = np.random.rand(10, 10)
    labels = np.random.randint(0, 2, 10)
    game_ids = list(range(10))

```

```

75     names = [f"Game{i}" for i in game_ids]
    clusterer.visualize_clusters_tsne(X, labels, game_ids,
        names, sample_size=10, label_frequency=2)

def test_get_similar_games_for_user_game(monkeypatch,
    clusterer):

    clusterer.extractor.vectorize_user_game = lambda user_game
        , _: {
80         "description_embedding": np.random.rand(50),
         "price": 0.5,
         "median_forever": 0.5,
         "metacritic": 0.5,
85         "category_vector": [0, 1],
         "genre_vector": [1, 0],
         "tag_vector": [0.5, 0.5]
    }
    clusterer.predict_cluster = lambda v: 1
    user_game = {
90         "name": "Test",
         "description": "desc",
         "genres": ["Action"],
         "categories": ["Single-player"],
         "tags": ["Action"],
95         "price": 10,
         "median_forever": 5
    }
    cluster_id = clusterer.get_similar_games_for_user_game(
        user_game)
    assert cluster_id == 1

100 def test_error_on_empty_db(monkeypatch, clusterer):
    # Проверяем, что выбрасывается ValueError при пустой базе
    monkeypatch.setattr(clusterer.repository.
        game_features_collection, "find", lambda *a, **k: [])
    with pytest.raises(ValueError):
105         clusterer.get_features_matrix()

```

Программный код тестов модуля GameCompetitivenessAnalyzer

```

import pytest
import numpy as np
from modules.GameCompetitivenessAnalyzer import
    GameCompetitivenessAnalyzer
5 from modules.FeatureExtractor import FeatureExtractor
from datetime import datetime

@pytest.fixture
def sample_game_data():
10     return {
        "data": {
            "categories": [
                {"description": "Single-player"},
                {"description": "Multi-player"},
15                {"description": "VR Support"}
            ],
            "genres": ["Action", "Adventure"],
            "name": "Test Game",
            "release_date": {
20                "date": "01 Jan, 2023"
            },
            "metacritic": {"score": 80},
            "median_forever": 120,
            "detailed_description": "Test game description"
25        }
    }

@pytest.fixture
def sample_steamspy_data():
30     return {
        "genre": "Action, Adventure",
        "name": "Test Game",
        "negative": 2000,
        "owners": "1000000",
35        "positive": 8000,
        "price": "29.99",
        "tags": {
            "Action": 100,
            "Adventure": 80,
40            "RPG": 60
        }
    }

```

```

        }
    }

@pytest.fixture
45 def analyzer():
    return GameCompetitivenessAnalyzer()

@pytest.fixture
50 def extractor():
    return FeatureExtractor()

def test_calculate_competitiveness(analyzer, extractor,
    sample_game_data, sample_steamspy_data):
    """Расчет конкурентоспособности теста"""
    score = analyzer.calculate_competitiveness(
        sample_game_data, sample_steamspy_data)
55 assert isinstance(score, float)
    assert 0 <= score <= 1
    assert not np.isnan(score)

def test_analyze_competitiveness(analyzer):
60 """Тест, анализирующий конкурентоспособность в нескольких
    играх"""
    game_ids = [570, 730] # Dota 2, CS:GO
    results = analyzer.analyze_competitiveness(game_ids)
    assert isinstance(results, list)
    assert len(results) > 0
65 for game_id, score in results:
    assert isinstance(game_id, int)
    assert isinstance(score, float)
    assert 0 <= score <= 1
    assert not np.isnan(score)

70
def test_analyze_user_competitiveness(analyzer):
    """Тест, анализирующий конкурентоспособность пользовательс
    кой игры"""
    user_game = {
        "data": {
75         "categories": [
            {"description": "Single-player"},
            {"description": "Multi-player"}
        ],
        "genres": ["Action", "Adventure"],
80         "name": "User Game",
        "release_date": {"date": "01 Jan, 2023"},
    }

```

```

        "metacritic": {"score": 75},
        "median_forever": 100,
        "detailed_description": "User game description"
85     }
    }
    similarities = [570, 730]
    high_competitiveness_ids = [570]
    competitiveness_scores = {"570": 0.9, "730": 0.7}
90    score = analyzer.analyze_user_competitiveness(user_game,
        similarities, high_competitiveness_ids,
        competitiveness_scores)
    assert isinstance(score, float)
    assert 0 <= score <= 1
    assert not np.isnan(score)

95 def test_normalize_value(analyzer):
    """Нормализация тестового значения"""

    test_cases = [
100         (100, 0, 100, 1.0),      # Maximum value
        (0, 0, 100, 0.0),         # Minimum value
        (50, 0, 100, 0.5),        # Middle value
        (1000, 0, 1000, 1.0),      # Maximum value
        (500, 0, 1000, 0.5)       # Middle value
    ]

105    for value, min_val, max_val, expected in test_cases:
        normalized = analyzer.normalize_value(value, max_val)
        assert isinstance(normalized, float)
        assert 0 <= normalized <= 1
        assert abs(normalized - expected) < 0.1

110 def test_empty_game_analysis(analyzer, extractor):
    """Тестовая обработка пустых игровых данных"""
    empty_game = {
        "data": {
115         "categories": [],
        "genres": [],
        "name": "",
        "release_date": {"date": None},
        "metacritic": {"score": 0},
120         "median_forever": 0,
        "detailed_description": ""
        }
    }

    empty_steamspy = {

```

```

125         "genre": "",
            "name": "",
            "negative": 0,
            "owners": "0",
            "positive": 0,
130         "price": "0",
            "tags": {}
        }
        score = analyzer.calculate_competitiveness(empty_game,
            empty_steamspy)
        assert isinstance(score, float)
135         assert 0 <= score <= 1
        assert not np.isnan(score)

def test_invalid_game_data(analyzer, extractor):
    """Тестовая обработка неверных игровых данных"""
140     invalid_game = {
        "data": {
            "categories": None,
            "genres": None,
            "name": None,
145         "release_date": None,
            "metacritic": None,
            "median_forever": None,
            "detailed_description": None
        }
150     }
    invalid_steamspy = {
        "genre": None,
        "name": None,
        "negative": None,
155         "owners": None,
        "positive": None,
        "price": None,
        "tags": None
    }
160     score = analyzer.calculate_competitiveness(invalid_game,
        invalid_steamspy)
    assert isinstance(score, float)
    assert 0 <= score <= 1
    assert not np.isnan(score)

165 def test_competitiveness_score_persistence(analyzer):
    """Сохранение результатов тестирования и получение результ
        атов оценки конкурентоспособности"""

```


170

```
game_id = 570
test_score = 0.8
analyzer.save_competitiveness_score(game_id, test_score)
retrieved_score = analyzer.get_competitiveness_score(
    game_id)
assert isinstance(retrieved_score, float)
assert 0 <= retrieved_score <= 1
assert abs(retrieved_score - test_score) < 0.001
```

Программный код тестов обработки пропущенных данных в GameCompetitivenessAnalyzer

```

import pytest
import numpy as np
from modules.FeatureExtractor import FeatureExtractor
5 from modules.GameCompetitivenessAnalyzer import
    GameCompetitivenessAnalyzer

@pytest.fixture
def sample_game_data_missing():
    return {
10         "data": {
            "name": "Test Game",
            "release_date": {
                "date": None
            },
15         "categories": [],
            "genres": []
        }
    }

20 @pytest.fixture
def sample_steamspy_data_missing():
    return {
        "name": "Test Game",
        "price": None,
25         "owners": None,
        "positive": None,
        "negative": None,
        "genre": "",
        "tags": {}
30     }

@pytest.fixture
def analyzer():
    return GameCompetitivenessAnalyzer()
35

def extractor():
    return FeatureExtractor()

def test_analyzer_initialization(analyzer):

```

```

40     """Проверьте правильность инициализации анализатора необхо-
        димыми компонентами"""
        assert analyzer.analyzer is not None
        assert analyzer.repository is not None
        assert analyzer.extractor is not None

45 def test_extract_features_with_missing_data(analyzer,
        extractor, sample_game_data_missing,
        sample_steamspy_data_missing):
        """Извлечение тестового признака с недостающими данными"""
        features = extractor.create_game_features(
            sample_game_data_missing, sample_steamspy_data_missing)

        assert isinstance(features, dict)
50     assert 'revenue' in features
        assert 'total_reviews' in features
        assert 'review_activity' in features
        assert 'age_days' in features

55     assert features['age_days'] == 365*40
        assert features['revenue'] == 0.0
        assert features['total_reviews'] == 0
        assert features['review_activity'] == 0.0

60 def test_calculate_competitiveness_with_missing_data(analyzer,
        sample_game_data_missing, sample_steamspy_data_missing):
        """Test competitiveness calculation with missing data"""
        score = analyzer.calculate_competitiveness(
            sample_game_data_missing, sample_steamspy_data_missing)

        assert isinstance(score, float)
65     assert 0 <= score <= 1
        assert not np.isnan(score) # not return NaN

def test_analyze_competitiveness_with_missing_data(analyzer):
    """Test analyzing competitiveness for games with missing
        data"""
70     game_ids = [570, 999999, 730] # Dota 2, invalid ID, CS:GO
        results = analyzer.analyze_competitiveness(game_ids)

        assert isinstance(results, list)
        assert len(results) > 0
75     for game_id, score in results:
            assert isinstance(game_id, int)
            assert isinstance(score, float)

```

```

        assert 0 <= score <= 1
        assert not np.isnan(score)

80 def test_analyze_user_competitiveness_with_missing_data(
    analyzer):
    """Тест, анализирующий конкурентоспособность пользователс
        кой игры с отсутствующими данными"""
    user_game = {
        "name": "User Game",
85         "price": None,
        "genres": [],
        "categories": []
    }

    similarities = {570: 0.8, 730: 0.6}
    high_competitiveness_ids = [570]
    competitiveness_scores = {570: 0.9, 730: 0.7}

    score = analyzer.analyze_user_competitiveness(
95         user_game,
        similarities,
        high_competitiveness_ids,
        competitiveness_scores
    )

100    assert isinstance(score, float)
    assert 0 <= score <= 1
    assert not np.isnan(score)

105 def test_normalize_value_with_missing_data(analyzer):
    """Нормализация тестового значения при отсутствии или неве
        рных данных"""
    test_cases = [
        (None, 1000, 0.0),
        (100, None, 0.0),
110         (None, None, 0.0),
        (-100, 1000, 0.0),
        (100, -1000, 0.0),
        (100, 0, 0.0)
    ]

115    for value, max_value, expected in test_cases:
        normalized = analyzer.normalize_value(value, max_value
        )
        assert isinstance(normalized, float)

```

```

    assert 0 <= normalized <= 1
    assert not np.isnan(normalized)

def test_competitiveness_score_persistence_with_missing_data(
    analyzer):
    """Тестовое сохранение и извлечение оценок конкурентоспо
        бности с отсутствующими данными"""
    game_id = 570
    test_score = None

    analyzer.save_competitiveness_score(game_id, test_score)

    retrieved_score = analyzer.get_competitiveness_score(
        game_id)
    assert retrieved_score is None or (isinstance(
        retrieved_score, float) and 0 <= retrieved_score <= 1)

def test_partial_data_handling(analyzer):
    """Test handling of games with partial data"""
    partial_game_data = {
        "data": {
            "name": "Partial Game",
            "release_date": {
                "date": "01 Jan, 2023" # Только дата предоста
                влена
            },
            "categories": [],
            "genres": []
        }
    }

    partial_steamspy_data = {
        "name": "Partial Game",
        "price": "29.99", # предоставлена только стоимос
        ть
        "owners": None,
        "positive": None,
        "negative": None,
        "genre": "",
        "tags": {}
    }

    score = analyzer.calculate_competitiveness(
        partial_game_data, partial_steamspy_data)
    assert isinstance(score, float)

```

```
assert 0 <= score <= 1
assert not np.isnan(score)
```

Программный код тестов модуля GameUniqueFeaturesAnalyzer

```

import pytest
import numpy as np
from modules.GameUniqueFeaturesAnalyzer import
    GameUniqueFeaturesAnalyzer
5
@pytest.fixture
def analyzer():
    return GameUniqueFeaturesAnalyzer()

10
@pytest.fixture
def sample_user_game():
    return {
        "data": {
            "categories": [
15
                {"description": "Single-player"},
                {"description": "Multi-player"}
            ],
            "genres": ["RPG", "Action", "Adventure"],
            "name": "Test Game",
20
            "detailed_description": "An epic RPG with dragons
                and unique combat system!",
            "metacritic": {"score": 85},
            "median_forever": 120
        }
    }
25
@pytest.fixture
def sample_similar_games():
    return {
        "570": { # Dota 2
30
            "data": {
                "categories": [
                    {"description": "Multi-player"},
                    {"description": "Online PvP"}
                ],
35
                "genres": ["Action", "Strategy"],
                "name": "Dota 2",
                "detailed_description": "A competitive MOBA
                    game with unique heroes and abilities",
                "metacritic": {"score": 90},

```

```

        "median_forever": 1000
    },
    "730": { # CS:GO
        "data": {
            "categories": [
                {"description": "Multi-player"},
                {"description": "Online PvP"}
            ],
            "genres": ["Action", "FPS"],
            "name": "Counter-Strike: Global Offensive",
            "detailed_description": "A competitive FPS
            game with tactical gameplay",
            "metacritic": {"score": 83},
            "median_forever": 800
        }
    }
}

def test_analyzer_initialization(analyzer):
    """Test that analyzer is properly initialized"""
    assert analyzer.analyzer is not None
    assert analyzer.repository is not None
    assert analyzer.extractor is not None

def test_compare_with_most_similar_and_dissimilar(analyzer,
    sample_user_game, sample_similar_games):
    """Test comparison of user game with similar games"""
    comparison = analyzer.
        compare_with_most_similar_and_dissimilar(
            sample_user_game, sample_similar_games)

    assert isinstance(comparison, dict)
    assert 'size' in comparison
    assert 'features' in comparison
    assert comparison['size'] > 0

    # Проверяем структуру features
    features = comparison['features']
    assert isinstance(features, dict)
    for game_id, feature_data in features.items():
        assert isinstance(feature_data, dict)
        assert 'description_similarity' in feature_data
        assert 'genre_similarity' in feature_data
        assert 'category_similarity' in feature_data

```



```

80     assert 'tag_similarity' in feature_data
       assert 'overall_similarity' in feature_data

       # Проверяем, что все значения находятся в диапазоне
       [0, 1]
       for value in feature_data.values():
85         assert isinstance(value, float)
           assert 0 <= value <= 1

def test_empty_data_handling(analyzer):
    """Test handling of empty data"""
90     empty_game = {
        "data": {
            "categories": [],
            "genres": [],
            "name": "Empty Game",
95            "detailed_description": "",
            "metacritic": {"score": 0},
            "median_forever": 0
        }
    }

100     comparison = analyzer.
        compare_with_most_similar_and_dissimilar(empty_game,
        {})

    assert isinstance(comparison, dict)
    assert 'size' in comparison
    assert comparison['size'] == 0

105 def test_missing_data_handling(analyzer):
    """Test handling of missing data"""
    game_with_missing = {
        "data": {
110            "categories": None,
            "genres": None,
            "name": None,
            "detailed_description": None,
            "metacritic": None,
115            "median_forever": None
        }
    }

    comparison = analyzer.
        compare_with_most_similar_and_dissimilar(
        game_with_missing, {})

```

```

120     assert isinstance(comparison, dict)
        assert 'size' in comparison
        assert comparison['size'] == 0

def test_text_similarity_calculation(analyzer):
125     """Test text similarity calculation"""
    # Тест с нормальными текстами
    text1 = "This is a test description"
    text2 = "This is another test description"
    similarity = analyzer._calculate_text_similarity(text1,
130         text2)
    assert isinstance(similarity, float)
    assert 0 <= similarity <= 1

    # Тест с пустыми текстами
    assert analyzer._calculate_text_similarity("", "") == 0.0
135     assert analyzer._calculate_text_similarity(None, None) ==
        0.0
    assert analyzer._calculate_text_similarity("", None) ==
        0.0

    # Тест с очень разными текстами
    text3 = "Completely different text"
140     similarity = analyzer._calculate_text_similarity(text1,
        text3)
    assert similarity < 0.5 # Должно быть меньше 0.5 для очень
        разных текстов

def test_set_similarity_calculation(analyzer):
145     """Test set similarity calculation"""
    # Тест с пересекающимися множествами
    set1 = {"Action", "RPG", "Adventure"}
    set2 = {"Action", "RPG", "Strategy"}
    similarity = analyzer._calculate_set_similarity(set1, set2
150         )
    assert isinstance(similarity, float)
    assert 0 <= similarity <= 1
    assert similarity > 0.5 # Должно быть больше 0.5 для множеств
        с общими элементами

    # Тест с пустыми множествами
    assert analyzer._calculate_set_similarity(set(), set()) ==
155         0.0
    assert analyzer._calculate_set_similarity(None, None) ==
        0.0

```

```

assert analyzer._calculate_set_similarity(set(), None) ==
    0.0

# Тест с непересекающимися множествами
set3 = {"Strategy", "Simulation", "Sports"}
160 similarity = analyzer._calculate_set_similarity(set1, set3
    )
assert similarity == 0.0 # Должно быть 0 для непересекающ
    ихся множеств

def test_unique_elements_finding(analyzer):
    """Test finding unique elements"""
165 # Тест с нормальными текстами
    text1 = "This is a test description"
    text2 = "This is another description"
    unique = analyzer.find_unique_elements(text1, text2)
    assert isinstance(unique, list)
170 assert "test" in unique

    # Тест с пустыми текстами
    assert analyzer.find_unique_elements("", "") == []
    assert analyzer.find_unique_elements(None, None) == []
175 assert analyzer.find_unique_elements("", None) == []

    # Тест с полностью разными текстами
    text3 = "Completely different text"
    unique = analyzer.find_unique_elements(text1, text3)
180 assert len(unique) > 0
    assert all(word in text1.lower().split() for word in
        unique)

def test_load_and_analyze_games(analyzer):
    """Test loading and analyzing games"""
185 # Тест с существующими играми
    game_ids = [570, 730] # Dota 2 and CS:GO
    analysis = analyzer.load_and_analyze_games(game_ids)

    assert isinstance(analysis, dict)
190 assert 'common_genres' in analysis
    assert 'common_tags' in analysis
    assert 'common_categories' in analysis
    assert 'avg_price' in analysis
    assert 'avg_metacritic' in analysis
195 assert 'avg_median_forever' in analysis
    assert 'avg_positive_ratio' in analysis

```

```

assert 'all_genres' in analysis
assert 'all_tags' in analysis
assert 'all_categories' in analysis
200 assert 'descriptions' in analysis
assert 'description_embeddings' in analysis

# Тест с пустым списком игр
empty_analysis = analyzer.load_and_analyze_games([])
205 assert isinstance(empty_analysis, dict)
assert empty_analysis['common_genres'] == []
assert empty_analysis['common_tags'] == []
assert empty_analysis['common_categories'] == []
assert empty_analysis['avg_price'] == 0
210 assert empty_analysis['avg_metacritic'] == 0
assert empty_analysis['avg_median_forever'] == 0
assert empty_analysis['avg_positive_ratio'] == 0

# Тест с несуществующими играми
215 non_existent_analysis = analyzer.load_and_analyze_games
    ([999999, 888888])
assert isinstance(non_existent_analysis, dict)
assert non_existent_analysis['common_genres'] == []
assert non_existent_analysis['common_tags'] == []
assert non_existent_analysis['common_categories'] == []
220

def test_extract_key_phrases(analyzer):
    """Test key phrase extraction"""
    # Тест с нормальным текстом
    text = "This is a test. This is another test. This is
           unique!"
225 phrases = analyzer._extract_key_phrases(text)
assert isinstance(phrases, list)
assert len(phrases) == 3
assert all(isinstance(phrase, str) for phrase in phrases)

# Тест с пустым текстом
230 assert analyzer._extract_key_phrases("") == []
assert analyzer._extract_key_phrases(None) == []

# Тест с текстом без точек
235 text_no_periods = "This is a test This is another test"
phrases = analyzer._extract_key_phrases(text_no_periods)
assert len(phrases) == 1

def test_find_common_phrases(analyzer):

```

```

240 """Test finding common phrases"""
    # Тест с нормальными описаниями
    descriptions = [
        "This is a test. This is common.",
        "This is another test. This is common.",
245     "This is unique. This is common."
    ]
    common_phrases = analyzer._find_common_phrases(
        descriptions)
    assert isinstance(common_phrases, list)
    assert len(common_phrases) > 0
250     assert "This is common" in common_phrases

    # Тест с пустыми описаниями
    assert analyzer._find_common_phrases([]) == []
    assert analyzer._find_common_phrases(["", "", ""]) == []
255

    # Тест с уникальными описаниями
    unique_descriptions = [
        "This is unique one.",
        "This is unique two.",
260     "This is unique three."
    ]
    common_phrases = analyzer._find_common_phrases(
        unique_descriptions)
    assert len(common_phrases) == 0

265 def test_analyze_description_similarity(analyzer):
    """Test description similarity analysis"""
    # Создаем тестовые эмбединги
    target_embedding = np.random.rand(384)
    similar_embeddings = [np.random.rand(384) for _ in range
270         (2)]

    # Тест с нормальными эмбедингами
    similarity = analyzer._analyze_description_similarity(
        target_embedding, similar_embeddings)
    assert isinstance(similarity, float)
    assert 0 <= similarity <= 1
275

    # Тест с пустыми эмбедингами
    assert analyzer._analyze_description_similarity(None, [])
        == 0
    assert analyzer._analyze_description_similarity(
        target_embedding, []) == 0

```

```

    assert analyzer._analyze_description_similarity(None,
        similar_embeddings) == 0
280
def test_find_unique_description_elements(analyzer):
    """Test finding unique description elements"""
    # Тест с нормальными описаниями
    target_desc = "This is a unique description with special
        features"
285
    similar_descs = [
        "This is a common description",
        "This is another common description"
    ]
    unique_elements = analyzer.
        _find_unique_description_elements(target_desc,
            similar_descs)
290
    assert isinstance(unique_elements, list)
    assert len(unique_elements) > 0
    assert "unique" in " ".join(unique_elements).lower()
    assert "special" in " ".join(unique_elements).lower()

295
    # Тест с пустыми описаниями
    assert analyzer._find_unique_description_elements("", [])
        == []
    assert analyzer._find_unique_description_elements(None,
        []) == []
    assert analyzer._find_unique_description_elements("", [" "
        ]) == []

300
    # Тест с идентичными описаниями
    same_desc = "This is the same description"
    assert analyzer._find_unique_description_elements(
        same_desc, [same_desc]) == []

```

Программный код тестов конвейера анализа игр

```

import pytest
from modules.GameAnalysisPipeline import GameAnalysisPipeline

5 @pytest.fixture
def sample_game():
    return {
        "name": "Test Game",
        "description": "An epic RPG with dragons!",
10        "price": 59.99,
        "metacritic": 85,
        "median_forever": 120,
        "genres": ["RPG", "Action", "Adventure"],
        "categories": ["Single-player", "Multi-player"],
15        "tags": ["RPG", "Action", "Fantasy"]
    }

@pytest.fixture
def pipeline():
20    user_game = {
        "name": "User Game",
        "description": "A test game for analysis",
        "price": 29.99,
        "metacritic": 75,
25        "median_forever": 60,
        "genres": ["Action", "Adventure"],
        "categories": ["Single-player"],
        "tags": ["Action", "Adventure", "Indie"]
    }
30    return GameAnalysisPipeline(user_game_data=user_game)

def test_pipeline_initialization(pipeline):
    """Тестовая инициализация всех необходимых анализаторов"""
    assert pipeline.analyzer is not None
35    assert pipeline.repository is not None
    assert pipeline.extractor is not None
    assert pipeline.clusterer is not None
    assert pipeline.competitiveness is not None
    assert pipeline.similarity is not None
40    assert pipeline.uniqueFeatures is not None
    assert pipeline.recommendations is not None

```

```

def test_rate_top_games(pipeline):
    """Тестовая оценка лучших игр"""
45     results = pipeline.rate_top_games(pipeline.user_game,
        top_n=5)

    assert isinstance(results, list)

    for result in results:
50         assert isinstance(result, dict)
        assert 'game_id' in result
        assert 'name' in result
        assert 'similarity_score' in result
        assert 'unique_features' in result
55         assert 'competitiveness_score' in result

def test_analyze_game(pipeline, sample_game):
    """Тестовая аналитика игры"""
    results = pipeline.analyze_game(sample_game)
60

    assert isinstance(results, dict)
    assert 'cluster_analysis' in results
    assert 'competitiveness_analysis' in results
    assert 'similarity_analysis' in results
65     assert 'unique_features' in results

def test_vectorize_game(pipeline, sample_game):
    """Тестовая векторизация игры"""
    vectorized = pipeline.vectorize_game(sample_game)
70     assert isinstance(vectorized, dict)

def test_empty_game_analysis(pipeline):
    """Тестовая обработка пустых игровых данных"""
    empty_game = {
75         "name": "",
        "description": "",
        "price": 0,
        "metacritic": 0,
        "median_forever": 0,
80         "genres": [],
        "categories": [],
        "tags": []
    }

85     results = pipeline.analyze_game(empty_game)

```



```

assert isinstance(results, dict)
assert all(key in results for key in ['cluster_analysis',
                                     'competitiveness_analysis',
                                     'similarity_analysis',
                                     'unique_features'
                                     ])

90 def test_pipeline_consistency(pipeline, sample_game):
    """Проверка того, что одни и те же входные данные дают соот-
        ветствующие результаты"""
    results1 = pipeline.analyze_game(sample_game)
    results2 = pipeline.analyze_game(sample_game)

95     assert results1['cluster_analysis'] == results2['
        cluster_analysis']
    assert results1['competitiveness_analysis'] == results2['
        competitiveness_analysis']
    assert results1['similarity_analysis'] == results2['
        similarity_analysis']
    assert results1['unique_features'] == results2['
        unique_features']

```

Программный код тестов модуля формирования рекомендаций игр

```

import pytest
from modules.GameRecommendationAnalyzer import
    GameRecommendationAnalyzer

5 @pytest.fixture
def analyzer():
    return GameRecommendationAnalyzer()

@pytest.fixture
10 def sample_user_game():
    return {
        "name": "Test Game",
        "description": "An epic RPG with dragons!",
        "price": 59.99,
15         "metacritic": 85,
        "median_forever": 120,
        "genres": ["RPG", "Action", "Adventure"],
        "categories": ["Single-player", "Multi-player"],
        "tags": ["RPG", "Action", "Fantasy"]
20     }

@pytest.fixture
def sample_similar_data():
    return {
25         "all_genres": ["RPG", "Action", "Strategy"],
        "all_tags": ["RPG", "Action", "Fantasy", "Open World"],
        "all_categories": ["Single-player", "Multi-player", "Co-op"],
        "unique_in_target_genres": ["Adventure"],
        "unique_in_target_tags": ["Dragons"],
30         "unique_in_target_categories": [],
        "intersection_genres": ["RPG", "Action"],
        "intersection_tags": ["RPG", "Action", "Fantasy"],
        "intersection_categories": ["Single-player", "Multi-
35         player"],
        "avg_price_diff": 10.0,
        "description_analysis": {
            "similarity_score": 0.75,
    
```

```

        "common_phrases": ["epic adventure", "fantasy
                           world", "unique combat"],
        "unique_phrases": ["dragon companions", "magic
                           system", "ancient ruins"]
    }
40 }

@pytest.fixture
def sample_dissimilar_data():
    return {
45     "all_genres": ["Strategy", "Simulation", "Sports"],
        "all_tags": ["Strategy", "Simulation", "Sports", "
                    Multiplayer"],
        "all_categories": ["Multi-player", "Competitive", "
                    Team-based"],
        "avg_price_diff": -5.0
    }
50

def test_feature_importance_calculation(analyzer,
    sample_similar_data, sample_dissimilar_data):
    """Test calculation of feature importance"""

    importance = analyzer._calculate_feature_importance("
        Adventure", sample_similar_data, sample_dissimilar_data
    )
55 assert importance == 0.7

    importance = analyzer._calculate_feature_importance("RPG",
        sample_similar_data, sample_dissimilar_data)
    assert importance == 0.3

60    importance = analyzer._calculate_feature_importance("
        NonExistent", sample_similar_data,
        sample_dissimilar_data)
    assert importance == 0.1

def test_price_positioning_analysis(analyzer, sample_user_game
    , sample_similar_data, sample_dissimilar_data):
    """Test price positioning analysis"""
65

    sample_user_game["price"] = 29.99
    result = analyzer._analyze_price_positioning(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert "ниже среднерыночной" in result

```

```

70 sample_user_game["price"] = 89.99
result = analyzer._analyze_price_positioning(
    sample_user_game, sample_similar_data,
    sample_dissimilar_data)
assert "выше среднерыночной" in result

sample_user_game["price"] = 69.99
75 result = analyzer._analyze_price_positioning(
    sample_user_game, sample_similar_data,
    sample_dissimilar_data)
assert "оптимальном диапазоне" in result

def test_feature_combinations_analysis(analyzer,
sample_user_game, sample_similar_data):
    """Test analysis of feature combinations"""
80 combinations = analyzer._analyze_feature_combinations(
    sample_user_game, sample_similar_data)
assert isinstance(combinations, list)

sample_user_game["genres"] = ["RPG", "Strategy", "
Simulation"]
combinations = analyzer._analyze_feature_combinations(
    sample_user_game, sample_similar_data)
85 assert any("Уникальная комбинация жанров" in comb for comb
in combinations)

def test_description_analysis(analyzer, sample_user_game,
sample_similar_data, sample_dissimilar_data):
    """Test description analysis"""
    recommendations = analyzer._analyze_description(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
90 assert isinstance(recommendations, list)

sample_similar_data["description_analysis"]["
similarity_score"] = 0.9
recommendations = analyzer._analyze_description(
    sample_user_game, sample_similar_data,
    sample_dissimilar_data)
assert any("очень похоже" in rec for rec in
recommendations)

95 sample_similar_data["description_analysis"]["
similarity_score"] = 0.2

```

```

        recommendations = analyzer._analyze_description(
            sample_user_game, sample_similar_data,
            sample_dissimilar_data)
        assert any("значительно отличается" in rec for rec in
            recommendations)

100 def test_market_positioning_analysis(analyzer,
    sample_user_game, sample_similar_data,
    sample_dissimilar_data):
    """Test market positioning analysis"""
    recommendations = analyzer._analyze_market_positioning(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert isinstance(recommendations, list)

105 sample_user_game["genres"] = ["RPG", "Strategy"]
    recommendations = analyzer._analyze_market_positioning(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert any("нишевые жанры" in rec for rec in
        recommendations)

def test_competitive_advantages_analysis(analyzer,
    sample_user_game, sample_similar_data,
    sample_dissimilar_data):
110     """Test competitive advantages analysis"""
    advantages = analyzer._analyze_competitive_advantages(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert isinstance(advantages, list)

    sample_user_game["tags"] = ["UniqueTag1", "UniqueTag2"]
115 advantages = analyzer._analyze_competitive_advantages(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert any("Уникальные теги" in adv for adv in advantages)

def test_potential_improvements_analysis(analyzer,
    sample_user_game, sample_similar_data,
    sample_dissimilar_data):
    """Test potential improvements analysis"""
120 improvements = analyzer._analyze_potential_improvements(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
    assert isinstance(improvements, list)

```

```

sample_user_game["tags"] = []
improvements = analyzer._analyze_potential_improvements(
    sample_user_game, sample_similar_data,
    sample_dissimilar_data)
125 assert any("Популярные теги" in imp for imp in
    improvements)

def test_generate_recommendations(analyzer, sample_user_game,
    sample_similar_data, sample_dissimilar_data):
    """Test full recommendations generation"""
    recommendations = analyzer.generate_recommendations(
        sample_user_game, sample_similar_data,
        sample_dissimilar_data)
130 assert isinstance(recommendations, list)
    assert len(recommendations) > 0

    sections = ["Позиционирование на рынке", "Уникальные особе
        нности",
                "Ценовое позиционирование", "Конкурентные преи
                мущества",
135 "Рекомендации по описанию", "Потенциальные улу
                чшения"]

    for section in sections:
        assert any(section in rec for rec in recommendations)

140 def test_empty_data_handling(analyzer):
    """Test handling of empty data"""
    empty_game = {
        "name": "",
        "description": "",
145 "price": 0,
        "metacritic": 0,
        "median_forever": 0,
        "genres": [],
        "categories": [],
150 "tags": []
    }

    empty_similar_data = {
        "all_genres": [],
155 "all_tags": [],
        "all_categories": [],
        "unique_in_target_genres": [],

```

```

        "unique_in_target_tags": [],
        "unique_in_target_categories": [],
160     "intersection_genres": [],
        "intersection_tags": [],
        "intersection_categories": [],
        "avg_price_diff": 0,
        "description_analysis": {
165             "similarity_score": 0,
            "common_phrases": [],
            "unique_phrases": []
        }
    }

170
    empty_dissimilar_data = {
        "all_genres": [],
        "all_tags": [],
        "all_categories": [],
175     "avg_price_diff": 0
    }

    recommendations = analyzer.generate_recommendations(
        empty_game, empty_similar_data, empty_dissimilar_data)
    assert isinstance(recommendations, list)
180    assert len(recommendations) > 0
    assert "общую рекомендацию" in recommendations[-1]

```

Программный код тестов компонента GameIdForm

```

import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-
  library/react';
import GameIdForm from '../frontend/src/components/forms/
  GameIdForm';
5
describe('GameIdForm Component', () => {
  const mockOnSubmit = jest.fn();
  const mockLoading = false;

  10  beforeEach(() => {
    global.fetch = jest.fn(() =>
      Promise.resolve({
        ok: true,
        json: () => Promise.resolve([
          { game_id: 1, name: 'Game 1' },
          { game_id: 2, name: 'Game 2' }
        ])
      })
    );
  });
  20  });

  test('renders form with autocomplete input', () => {
    render(<GameIdForm onSubmit={mockOnSubmit} loading={
      mockLoading} />);
    expect(screen.getByLabelText(/выберите игру/i)).
      toBeInTheDocument();
  25  });

  test('loads games on mount', async () => {
    render(<GameIdForm onSubmit={mockOnSubmit} loading={
      mockLoading} />);
    await waitFor(() => {
      30    expect(global.fetch).toHaveBeenCalledWith('http://
        localhost:8000/get_all_games');
    });
  });

  test('handles game selection and submission', async () => {

```



```

35     render(<GameIdForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);

    // Ждем загрузки игр
    await waitFor(() => {
        expect(screen.getByLabelText(/выберите игру/i)).
            toBeInTheDocument();
40    });

    // Открываем выпадающий список
    const input = screen.getByLabelText(/выберите игру/i);
    fireEvent.mouseDown(input);

45    // Выбираем игру
    const option = await screen.findByText('Game 1');
    fireEvent.click(option);

50    // Отправляем форму
    const submitButton = screen.getByText(/анализировать/i);
    fireEvent.click(submitButton);

    expect(mockOnSubmit).toHaveBeenCalledWith(1);
55 });

test('disables submit button when no game selected', () => {
    render(<GameIdForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);
    const submitButton = screen.getByText(/анализировать/i);
60    expect(submitButton).toBeDisabled();
});

test('shows loading state', () => {
    render(<GameIdForm onSubmit={mockOnSubmit} loading={true}
        />);
65    expect(screen.getByRole('progressbar')).toBeInTheDocument
        ();
});

test('handles fetch error gracefully', async () => {
    global.fetch = jest.fn(() => Promise.reject('API Error'));
70    render(<GameIdForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);

    await waitFor(() => {

```

```

        expect(screen.getByText(/failed to fetch games/i)).
            toBeInTheDocument();
    });
75 });

test('maintains form state on error', async () => {
    global.fetch = jest.fn()
        .mockRejectedValueOnce('API Error')
80     .mockResolvedValueOnce({
        ok: true,
        json: () => Promise.resolve([
            { game_id: 1, name: 'Game 1' }
        ])
85     });

    render(<GameIdForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);

    // Проверяем сообщение об ошибке
90    await waitFor(() => {
        expect(screen.getByText(/failed to fetch games/i)).
            toBeInTheDocument();
    });

    // Проверяем, что форма все еще интерактивна
95    const input = screen.getByLabelText(/выберите игру/i);
    expect(input).not.toBeDisabled();
    });
});

```

Программный код тестов компонента GameAnalysisForm

```

import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-
  library/react';
import GameAnalysisForm from '../frontend/src/components/forms
  /GameAnalysisForm';
5
describe('GameAnalysisForm Component', () => {
  const mockOnSubmit = jest.fn();
  const mockLoading = false;

10  beforeEach(() => {
    global.fetch = jest.fn(() =>
      Promise.resolve({
        ok: true,
        json: () => Promise.resolve({
15          genres: ['Action', 'RPG'],
          tags: ['Multiplayer', 'Single-player'],
          categories: ['Base Game', 'DLC']
        })
      })
    );
20  });

  test('renders all form fields', async () => {
    render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
      mockLoading} />);
25

    await waitFor(() => {
      expect(screen.getByLabelText(/название игры/i)).
        toBeInTheDocument();
      expect(screen.getByLabelText(/описание игры/i)).
        toBeInTheDocument();
      expect(screen.getByLabelText(/жанры/i)).
        toBeInTheDocument();
30      expect(screen.getByLabelText(/теги/i)).toBeInTheDocument
        ();
      expect(screen.getByLabelText(/категории/i)).
        toBeInTheDocument();
      expect(screen.getByLabelText(/стоимость/i)).
        toBeInTheDocument();
    });
  });
}

```

```

        expect(screen.getByLabelText(/средняя продолжительность/
            i)).toBeInTheDocument();
    });
35 });

test('loads schema data on mount', async () => {
    render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);
    await waitFor(() => {
40     expect(global.fetch).toHaveBeenCalledWith('http://
        localhost:8000/get_schema_data');
    });
});

test('handles form submission with valid data', async () =>
    {
45     render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);

    // Заполняем форму
    fireEvent.change(screen.getByLabelText(/название игры/i),
        {
            target: { value: 'Test Game' }
50     });

    fireEvent.change(screen.getByLabelText(/описание игры/i),
        {
            target: { value: 'Test Description' }
55     });

    // Выбираем жанры
    const genresInput = screen.getByLabelText(/жанры/i);
    fireEvent.mouseDown(genresInput);
    const actionOption = await screen.findByText('Action');
60     fireEvent.click(actionOption);

    // Заполняем числовые поля
    fireEvent.change(screen.getByLabelText(/стоимость/i), {
        target: { value: '29.99' }
65     });

    fireEvent.change(screen.getByLabelText(/средняя продолжите
        льность/i), {
        target: { value: '60' }
    });

```

```

70 // Отправляем форму
const submitButton = screen.getByText(/анализировать игру/
  i);
fireEvent.click(submitButton);

75 expect(mockOnSubmit).toHaveBeenCalledWith(expect.
  objectContaining({
    name: 'Test Game',
    description: 'Test Description',
    genres: ['Action'],
    price: 29.99,
80    median_forever: 60
  }));
});

test('validates required fields', async () => {
85   render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
    mockLoading} />);

    const submitButton = screen.getByText(/анализировать игру/
      i);
    fireEvent.click(submitButton);

90    await waitFor(() => {
      expect(screen.getAllByText(/обязательно/i)).toHaveLength
        (4); // name, description, price, median_forever
    });
  });

95 test('shows loading state', () => {
  render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
    true} />);
  expect(screen.getByRole('progressbar')).toBeInTheDocument
    ();
  });

100 test('handles schema data fetch error', async () => {
  global.fetch = jest.fn(() => Promise.reject('API Error'));
  render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
    mockLoading} />);

  await waitFor(() => {
105    expect(console.error).toHaveBeenCalledWith(
      'Error fetching schema data:',

```

```

        'API Error'
    );
    });
110 });

test('validates numeric input ranges', async () => {
    render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);

115     const priceInput = screen.getByLabelText(/стоимость/i);
    const durationInput = screen.getByLabelText(/средняя продо
        лжительность/i);

    // Проверяем отрицательные значения
    fireEvent.change(priceInput, { target: { value: '-10' } })
        ;
120     fireEvent.change(durationInput, { target: { value: '-20' }
        });

    await waitFor(() => {
        expect(priceInput).toHaveAttribute('min', '0');
        expect(durationInput).toHaveAttribute('min', '0');
125     });
    });

test('handles multiple selections in autocomplete fields',
    async () => {
    render(<GameAnalysisForm onSubmit={mockOnSubmit} loading={
        mockLoading} />);
130

    // Выбираем несколько жанров
    const genresInput = screen.getByLabelText(/жанры/i);
    fireEvent.mouseDown(genresInput);

135     const actionOption = await screen.findByText('Action');
    fireEvent.click(actionOption);

    const rpgOption = await screen.findByText('RPG');
    fireEvent.click(rpgOption);
140

    expect(screen.getByText('Action')).toBeInTheDocument();
    expect(screen.getByText('RPG')).toBeInTheDocument();
    });
});

```

Программный код тестов компонента ResultsDisplay

```

import React from 'react';
import { render, screen, fireEvent } from '@testing-library/
  react';
import ResultsDisplay from '../frontend/src/components/results
  /ResultsDisplay';
5
describe('ResultsDisplay Component', () => {
  const mockResults = {
    competitiveness_score: 0.75,
    similar_games: [
10      {
        Game_ID: 1,
        Name: 'Similar Game 1',
        Similarity_score: 0.8,
        Genres: ['Action', 'RPG'],
15        Price: 29.99,
        'Median Forever': 60,
        Metacritic: 85
      },
      {
20        Game_ID: 2,
        Name: 'Similar Game 2',
        Similarity_score: 0.6,
        Genres: ['Action'],
        Price: 19.99,
25        'Median Forever': 45,
        Metacritic: 75
      }
    ],
    recommendations: [
30      'Позиционирование на рынке: Рассмотрите нишевые жанры',
      'Уникальные особенности: Добавьте тег "Multiplayer" (важ
        ность: 0.8)',
      'Общие рекомендации по улучшению'
    ],
    competitiveness_scores: {
35      1: 0.8,
      2: 0.6
    },
  },
  cluster_info: [

```

```

40      {
        'Game ID': 1,
        Name: 'Cluster Game 1',
        Genres: ['Action'],
        Price: 29.99,
        'Median Forever': 60,
45      Metacritic: 85
      }
    ]
  };

50  test('renders competitiveness score section', () => {
    render(<ResultsDisplay results={mockResults} />);
    expect(screen.getByText(/оценка конкурентоспособности/i)).
      toBeInTheDocument();
    expect(screen.getByText('75.00%')).toBeInTheDocument();
  });

55  test('renders cluster information section', () => {
    render(<ResultsDisplay results={mockResults} />);
    expect(screen.getByText(/игры в кластере/i)).
      toBeInTheDocument();
    expect(screen.getByText('Cluster Game 1')).
      toBeInTheDocument();
60  });

  test('renders similar games section', () => {
    render(<ResultsDisplay results={mockResults} />);
    expect(screen.getByText(/топ конкурентоспособных похожих и
      rp/i)).toBeInTheDocument();
65    expect(screen.getByText('Similar Game 1')).
      toBeInTheDocument();
    expect(screen.getByText('Similar Game 2')).
      toBeInTheDocument();
  });

  test('renders recommendations section', () => {
70    render(<ResultsDisplay results={mockResults} />);
    expect(screen.getByText(/рекомендации/i)).
      toBeInTheDocument();
    expect(screen.getByText(/позиционирование на рынке/i)).
      toBeInTheDocument();
    expect(screen.getByText(/уникальные особенности/i)).
      toBeInTheDocument();
  });

```



```

75 test('handles missing data gracefully', () => {
    const incompleteResults = {
        competitiveness_score: 0.75,
        recommendations: []
80     };
    render(<ResultsDisplay results={incompleteResults} />);
    expect(screen.getByText(/оценка конкурентоспособности/i)).
        toBeInTheDocument();
    expect(screen.queryByText(/игры в кластере/i)).not.
        toBeInTheDocument();
    });

85 test('expands and collapses recommendation sections', () =>
    {
        render(<ResultsDisplay results={mockResults} />);

        // Проверяем начальное состояние
90 const marketPositioning = screen.getByText(/позиционирован
        ие на рынке/i);
        expect(marketPositioning).toBeInTheDocument();

        // Кликаем по заголовку секции
95 fireEvent.click(marketPositioning);

        // Проверяем, что контент отображается
        expect(screen.getByText(/рассмотрите нишевые жанры/i)).
            toBeInTheDocument();

        // Кликаем снова для сворачивания
100 fireEvent.click(marketPositioning);

        // Проверяем, что контент скрыт
        expect(screen.queryByText(/рассмотрите нишевые жанры/i)).
            not.toBeVisible();
    });

105 test('displays correct score indicators', () => {
    render(<ResultsDisplay results={mockResults} />);

    // Проверяем индикатор для основной оценки
110 const mainScoreBar = screen.getAllByRole('progressbar')
        [0];
    expect(mainScoreBar).toHaveStyle({ width: '75%' });

```

```

// Проверяем индикаторы для похожих игр
const gameScoreBars = screen.getAllByRole('progressbar').
  slice(1);
115 expect(gameScoreBars[0]).toHaveStyle({ width: '80%' });
    expect(gameScoreBars[1]).toHaveStyle({ width: '60%' });
  });

test('sorts similar games by competitiveness', () => {
120   render(<ResultsDisplay results={mockResults} />);

   const gameCards = screen.getAllByText(/similar game/i);
   expect(gameCards[0]).toHaveTextContent('Similar Game 1');
   expect(gameCards[1]).toHaveTextContent('Similar Game 2');
125 });

test('displays game details correctly', () => {
   render(<ResultsDisplay results={mockResults} />);

130   const game1 = screen.getByText('Similar Game 1').closest('
     div');
   expect(game1).toHaveTextContent('Action, RPG');
   expect(game1).toHaveTextContent('29.99 $');
   expect(game1).toHaveTextContent('60 мин. ');
   expect(game1).toHaveTextContent('Metacritic: 85');
135 });

test('handles empty results', () => {
   render(<ResultsDisplay results={null} />);
   expect(screen.queryByText(/оценка конкурентоспособности/i)
     ).not.toBeInTheDocument();
140 });
});

```

Программный код тестов компонента HomePage

```

import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-
  library/react';
import HomePage from '../frontend/src/pages/HomePage';
5
describe('HomePage Component', () => {
  beforeEach(() => {
    // Мокаем localStorage
    const localStorageMock = {
10      getItem: jest.fn(),
      setItem: jest.fn(),
      clear: jest.fn()
    };
    global.localStorage = localStorageMock;

15    // Мокаем window.matchMedia
    Object.defineProperty(window, 'matchMedia', {
      writable: true,
      value: jest.fn().mockImplementation(query => ({
20        matches: false,
        media: query,
        onchange: null,
        addListener: jest.fn(),
        removeListener: jest.fn(),
25        addEventListener: jest.fn(),
        removeEventListener: jest.fn(),
        dispatchEvent: jest.fn(),
      })),
    });
30  });

  test('renders main components', () => {
    render(<HomePage />);
    expect(screen.getByText(/game analyzer/i)).
      toBeInTheDocument();
35    expect(screen.getByText(/analyze by game id/i)).
      toBeInTheDocument();
    expect(screen.getByText(/analyze new game/i)).
      toBeInTheDocument();
  });
}

```

```

40 test('switches between tabs', () => {
    render(<HomePage />);

    // Проверяем начальное состояние
    expect(screen.getByText(/analyze by game id/i)).
      toHaveAttribute('aria-selected', 'true');

45 // Переключаемся на вторую вкладку
    fireEvent.click(screen.getByText(/analyze new game/i));
    expect(screen.getByText(/analyze new game/i)).
      toHaveAttribute('aria-selected', 'true');
  });

50 test('handles theme switching', () => {
    render(<HomePage />);

    // Проверяем начальную тему
    expect(document.body).toHaveClass('light-theme');

55 // Переключаем тему
    const themeButton = screen.getByText(/темная тема/i);
    fireEvent.click(themeButton);

60 // Проверяем изменение темы
    expect(document.body).toHaveClass('dark-theme');
    expect(localStorage.setItem).toHaveBeenCalledWith('theme',
      'dark');
  });

65 test('handles game analysis submission', async () => {
    global.fetch = jest.fn(() =>
      Promise.resolve({
        ok: true,
        json: () => Promise.resolve({
70     competitiveness_score: 0.75,
        recommendations: ['Test recommendation'],
        similar_games: []
      })
    })
  );

75 // Выбираем игру
    render(<HomePage />);

    // Выбираем игру

```

```

80  const gameInput = screen.getByLabelText(/выберите игру/i);
    fireEvent.mouseDown(gameInput);
    const option = await screen.findByText('Game 1');
    fireEvent.click(option);

85  // Отправляем форму
    const submitButton = screen.getByText(/анализировать/i);
    fireEvent.click(submitButton);

    // Проверяем загрузку результатов
90  await waitFor(() => {
        expect(screen.getByText(/оценка конкурентоспособности/i)
            ).toBeInTheDocument();
    });
});

95  test('handles new game analysis submission', async () => {
    global.fetch = jest.fn(() =>
        Promise.resolve({
            ok: true,
            json: () => Promise.resolve({
100             competitiveness_score: 0.75,
                recommendations: ['Test recommendation'],
                similar_games: []
            })
        })
    );

105  render(<HomePage />);

    // Переключаемся на вкладку анализа новой игры
110  fireEvent.click(screen.getByText(/analyze new game/i));

    // Заполняем форму
    fireEvent.change(screen.getByLabelText(/название игры/i),
        {
            target: { value: 'Test Game' }
115  });

    fireEvent.change(screen.getByLabelText(/описание игры/i),
        {
            target: { value: 'Test Description' }
120  });

    // Отправляем форму

```

```

const submitButton = screen.getByText(/анализировать игру/
  i);
fireEvent.click(submitButton);

125 // Проверяем загрузку результатов
await waitFor(() => {
  expect(screen.getByText(/оценка конкурентоспособности/i)
    ).toBeInTheDocument();
});
});
130

test('handles API errors gracefully', async () => {
  global.fetch = jest.fn(() => Promise.reject('API Error'));

  render(<HomePage />);

135 // Выбираем игру
const gameInput = screen.getByLabelText(/выберите игру/i);
fireEvent.mouseDown(gameInput);
const option = await screen.findByText('Game 1');
140 fireEvent.click(option);

// Отправляем форму
const submitButton = screen.getByText(/анализировать/i);
fireEvent.click(submitButton);

145 // Проверяем отображение ошибки
await waitFor(() => {
  expect(screen.getByText(/an error occurred/i)).
    toBeInTheDocument();
});
150 });

test('handles PDF report generation', async () => {
  global.fetch = jest.fn()
    .mockImplementationOnce(() => Promise.resolve({
155     ok: true,
     json: () => Promise.resolve({
       competitiveness_score: 0.75,
       recommendations: ['Test recommendation'],
       similar_games: []
     })
    })
    .mockImplementationOnce(() => Promise.resolve({
160     ok: true,

```

```

        blob: () => Promise.resolve(new Blob(['test'], { type:
          'application/pdf' })))
165      }));

      render(<HomePage />);

      // Выбираем игру и отправляем форму
170      const gameInput = screen.getByLabelText(/выберите игру/i);
      fireEvent.mouseDown(gameInput);
      const option = await screen.findByText('Game 1');
      fireEvent.click(option);
      const submitButton = screen.getByText(/анализировать/i);
175      fireEvent.click(submitButton);

      // Ждем загрузки результатов
      await waitFor(() => {
        expect(screen.getByText(/оценка конкурентоспособности/i)
          ).toBeInTheDocument();
180      });

      // Генерируем PDF
      const downloadButton = screen.getByText(/скачать отчет/i);
      fireEvent.click(downloadButton);
185

      // Проверяем запрос на генерацию PDF
      await waitFor(() => {
        expect(global.fetch).toHaveBeenCalledWith(
          'http://localhost:8000/generate_pdf_report',
190          expect.any(Object)
        );
      });
    });
  });

195 test('maintains state between tab switches', async () => {
  render(<HomePage />);

  // Заполняем форму на первой вкладке
  const gameInput = screen.getByLabelText(/выберите игру/i);
200  fireEvent.mouseDown(gameInput);
  const option = await screen.findByText('Game 1');
  fireEvent.click(option);

  // Переключаемся на вторую вкладку и обратно
205  fireEvent.click(screen.getByText(/analyze new game/i));
  fireEvent.click(screen.getByText(/analyze by game id/i));

```

```

// Проверяем, что выбранная игра сохранилась
expect(screen.getByText('Game 1')).toBeInTheDocument();
210 });
});

```

П49.1 Файл сценариев тестирования

```

# language: ru
Функционал: Анализ игр
    Как пользователь системы анализа игр
5    Я хочу иметь возможность анализировать существующие и новые
        игры
    Чтобы получать рекомендации по улучшению и позиционированию

    Сценарий: Анализ существующей игры
        Дано я нахожусь на странице анализа игр
10        Когда я выбираю игру "Dota 2" из выпадающего списка
            И нажимаю кнопку "Анализировать"
            Тогда система должна отобразить результаты анализа

    Сценарий: Анализ новой игры
15        Дано я нахожусь на странице добавления новой игры
            Когда я заполняю форму следующими данными:
                | Поле                | Значение                |
                | name                 | New Fantasy RPG         |
                | description           | Epic fantasy adventure game |
20                | price                 | 59.99                   |
                | genres                | RPG, Action, Adventure  |
                | tags                  | Space, Sandbox, Mystery |
                | categories            | Single-player, Multi-player |
            И нажимаю кнопку "Добавить и проанализировать"
25        Тогда система должна добавить игру в базу данных

    Сценарий: Валидация формы анализа
        Дано я нахожусь на странице добавления новой игры
        Когда я оставляю обязательные поля пустыми
30        И нажимаю кнопку "Добавить и проанализировать"
            Тогда система должна показать сообщения об ошибках валидац
                ии

    Сценарий: Проверка кластеризации игр
        Дано я нахожусь на странице анализа группы игр
35        Когда я выбираю несколько похожих игр
            И запускаю анализ кластеризации

```


Тогда система должна сгруппировать игры по схожести характеристик

Сценарий: Генерация PDF-отчета

40 Дано я нахожусь на странице с результатами анализа

Когда я нажимаю кнопку "Скачать PDF"

Тогда система должна создать PDF-файл с результатами анализа

Сценарий: Анализ уникальных особенностей

45 Дано я нахожусь на странице анализа игры

Когда я анализирую игру с уникальными характеристиками

Тогда система должна выявить уникальные особенности

П49.2 Файл реализации шагов тестирования

```

from behave import given, when, then
import requests
import json
5 import time
import logging

# Настройка логирования
logging.basicConfig(level=logging.INFO)
10 logger = logging.getLogger(__name__)

API_URL = "http://localhost:8000"

def log_response(response):
15     """Логирует информацию о HTTP-ответе"""
    logger.info(f"Response status: {response.status_code}")
    try:
        logger.info(f"Response body: {json.dumps(response.json(), indent=2, ensure_ascii=False)}")
    except:
20     logger.info(f"Response body: {response.text}")

# Базовые шаги для работы с API
@given('я нахожусь на странице анализа игр')
def step_impl(context):
25     try:
        response = requests.get(f"{API_URL}/get_all_games")
        log_response(response)
        assert response.status_code == 200, f"Ожидался код 200, получен {response.status_code}"
    
```

```

    context.games = response.json()
30     logger.info(f"Получено {len(context.games)} игр")
except requests.exceptions.RequestException as e:
    logger.error(f"Ошибка при получении списка игр: {e}")
    raise

35 @given('я нахожусь на странице добавления новой игры')
def step_impl(context):
    try:
        response = requests.get(f"{API_URL}/get_schema_data")
        log_response(response)
40         assert response.status_code == 200, f"Ожидался код
            200, получен {response.status_code}"
        context.schema = response.json()
        logger.info("Схема данных получена успешно")
    except requests.exceptions.RequestException as e:
        logger.error(f"Ошибка при получении схемы данных: {e}"
            )
45         raise

@given('я нахожусь на странице анализа группы игр')
def step_impl(context):
    try:
50         response = requests.get(f"{API_URL}/get_all_games")
        log_response(response)
        assert response.status_code == 200, f"Ожидался код
            200, получен {response.status_code}"
        context.games = response.json()
        logger.info(f"Получено {len(context.games)} игр для гр
            уппового анализа")
55     except requests.exceptions.RequestException as e:
        logger.error(f"Ошибка при получении списка игр: {e}")
        raise

@given('я нахожусь на странице с результатами анализа')
60 def step_impl(context):
    try:
        # Получаем список игр
        response = requests.get(f"{API_URL}/get_all_games")
        log_response(response)
65         assert response.status_code == 200, f"Ожидался код
            200, получен {response.status_code}"
        games = response.json()

        if not games:

```

```

70         raise AssertionError("Список игр пуст")

    game_id = games[0]['game_id']
    logger.info(f"Выбрана игра с ID: {game_id}")

    # Получаем результаты анализа
75     response = requests.get(f"{API_URL}/run_game_analysis/{game_id}")
    log_response(response)
    assert response.status_code == 200, f"Ожидался код 200, получен {response.status_code}"
    context.analysis_result = response.json()
    logger.info("Результаты анализа получены успешно")
80 except requests.exceptions.RequestException as e:
    logger.error(f"Ошибка при получении результатов анализа: {e}")
    raise

@given('я нахожусь на странице анализа игры')
85 def step_impl(context):
    try:
        # Получаем список игр для анализа
        response = requests.get(f"{API_URL}/get_all_games")
        log_response(response)
90     assert response.status_code == 200, f"Ожидался код 200, получен {response.status_code}"
        context.games = response.json()

        if not context.games:
            raise AssertionError("Список игр пуст")
95
        # Выбираем первую игру для анализа
        context.game_id = context.games[0]['game_id']
        logger.info(f"Выбрана игра с ID: {context.game_id}")

100    except requests.exceptions.RequestException as e:
        logger.error(f"Ошибка при получении списка игр: {e}")
        raise

    # Шаги для работы с игрой
105 @when('я выбираю игру "{game_name}" из выпадающего списка')
    def step_impl(context, game_name):
        if not hasattr(context, 'games'):
            raise AssertionError("Список игр не был получен")

```

```

110     game = next((g for g in context.games if g['name'] ==
        game_name), None)
    if game is None:
        available_games = [g['name'] for g in context.games]
        logger.error(f"Игра '{game_name}' не найдена. Доступны
            е игры: {available_games}")
        raise AssertionError(f"Игра '{game_name}' не найдена")
115
    context.game_id = game['game_id']
    logger.info(f"Выбрана игра: {game_name} (ID: {game['
        game_id']})")

    @when('я заполняю форму следующими данными')
120 def step_impl(context):
    game_data = {}
    for row in context.table:
        field = row['Поле'].lower()
        value = row['Значение']

125
        try:
            if field == 'цена':
                value = float(value)
            elif field in ['жанры', 'категории']:
130                 value = [x.strip() for x in value.split(',')]

            game_data[field] = value
            logger.info(f"Поле '{field}' заполнено значением:
                {value}")
        except ValueError as e:
135             logger.error(f"Ошибка при преобразовании значения
                для поля '{field}': {e}")
            raise

    context.new_game = game_data
    logger.info(f"Форма заполнена данными: {json.dumps(
        game_data, indent=2, ensure_ascii=False)}")
140

    @when('я оставляю обязательные поля пустыми')
    def step_impl(context):
        context.new_game = {
            "name": "",
145             "description": "",
            "genres": [],
            "tags": [],
            "categories": [],

```

```

        "price": 0
150     }
        logger.info("Форма очищена")

    @when('я выбираю несколько похожих игр')
    def step_impl(context):
155         if not hasattr(context, 'games') or not context.games:
            raise AssertionError("Список игр пуст или не был получен")

        context.selected_games = context.games[:3]
        context.game_ids = [game['game_id'] for game in context.
            selected_games]
160         logger.info(f"Выбрано {len(context.selected_games)} игр для анализа")

    @when('я анализирую игру с уникальными характеристиками')
    def step_impl(context):
        unique_game = {
165             "name": "Unique Test Game",
            "description": "A game with unique features",
            "genres": ["Unique Genre"],
            "tags": ["Unique Tag"],
            "categories": ["Unique Category"],
170             "price": 29.99,
            "median_forever": 120
        }

        try:
175             response = requests.post(f"{API_URL}/analyze_game",
                                       json={"user_game_data":
                                             unique_game})

            log_response(response)
            context.response = response
            logger.info("Отправлен запрос на анализ уникальной игры")
180         except requests.exceptions.RequestException as e:
            logger.error(f"Ошибка при анализе уникальной игры: {e}")
            raise

    # Шаги для действий с кнопками
185 @when('нажимаю кнопку "{button_text}"')
    def step_impl(context, button_text):
        try:

```

```

190     if button_text == "Анализировать":
        if not hasattr(context, 'game_id'):
            raise AssertionError("ID игры не был установле
            н")
        response = requests.get(f"{API_URL}/
            run_game_analysis/{context.game_id}")
        logger.info(f"Отправлен запрос на анализ игры {
            context.game_id}")

195     elif button_text == "Добавить и проанализировать":
        if not hasattr(context, 'new_game'):
            raise AssertionError("Данные новой игры не был
            и установлены")
        response = requests.post(f"{API_URL}/analyze_game"
            ,
                                json={"user_game_data":
                                    context.new_game})
        logger.info("Отправлен запрос на добавление и анал
            из новой игры")

200     elif button_text == "Скачать PDF":
        if not hasattr(context, 'analysis_result'):
            raise AssertionError("Результаты анализа не бы
            ли получены")
        response = requests.post(
205             f"{API_URL}/generate_pdf_report",
            json={"analysis_result": context.
                analysis_result},
            headers={'Accept': 'application/pdf'})
        logger.info("Отправлен запрос на генерацию PDF-отч
            ета")

210     else:
        raise ValueError(f"Неизвестная кнопка: {
            button_text}")

    log_response(response)
215     context.response = response

except requests.exceptions.RequestException as e:
    logger.error(f"Ошибка при нажатии кнопки '{button_text
        }': {e}")
    raise
220

```

```

# Шаги для анализа
@when('запускаю анализ кластеризации')
def step_impl(context):
    if not hasattr(context, 'game_ids'):
225         raise AssertionError("Список ID игр не был установлен"
                                )

    context.cluster_results = []
    for game_id in context.game_ids:
        try:
230             response = requests.get(f"{API_URL}/
                                     run_game_analysis/{game_id}")
            log_response(response)
            assert response.status_code == 200, f"Ожидался код
                200, получен {response.status_code}"
            context.cluster_results.append(response.json())
            logger.info(f"Получены результаты анализа для игры
                {game_id}")
235         except requests.exceptions.RequestException as e:
            logger.error(f"Ошибка при анализе игры {game_id}:
                {e}")
            raise

@when('происходит сбой API')
240 def step_impl(context):
    try:
        response = requests.get(f"{API_URL}/run_game_analysis
                                /999999")
        log_response(response)
        context.response = response
245         logger.info("Симулирован сбой API")
    except requests.exceptions.RequestException as e:
        logger.error(f"Ошибка при симуляции сбоя API: {e}")
        raise

250 # Шаги для проверки результатов
@then('система должна отобразить результаты анализа')
def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был пол
        учен"
    assert context.response.status_code == 200, f"Ожидался код
        200, получен {context.response.status_code}"
255
    data = context.response.json()
    # Проверяем наличие всех необходимых полей в ответе

```

```

assert "similarity_analysis" in data, "Отсутствует анализ
    схожести"
assert "unique_features" in data, "Отсутствует анализ уник
    альных особенностей"
260 assert "competitiveness_analysis" in data, "Отсутствует ан
    ализ конкурентоспособности"
assert "cluster_analysis" in data, "Отсутствует анализ кла
    стеризации"

# Проверяем, что хотя бы один из анализов содержит данные
has_data = (
265     data["similarity_analysis"] or
        data["unique_features"] or
        data["competitiveness_analysis"] or
        data["cluster_analysis"]
)
270 assert has_data, "Все анализы пусты"

logger.info("Результаты анализа успешно проверены")

@then('система должна добавить игру в базу данных')
275 def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был пол
        учен"
    assert context.response.status_code == 200, f"Ожидался код
        200, получен {context.response.status_code}"

    data = context.response.json()
280 # Проверяем наличие всех необходимых полей в ответе
    assert "similarity_analysis" in data, "Отсутствует анализ
        схожести"
    assert "unique_features" in data, "Отсутствует анализ уник
        альных особенностей"
    assert "competitiveness_analysis" in data, "Отсутствует ан
        ализ конкурентоспособности"
    assert "cluster_analysis" in data, "Отсутствует анализ кла
        стеризации"
285

    logger.info("Игра успешно добавлена в базу данных")

@then('система должна показать сообщения об ошибках валидации')
    )
def step_impl(context):
290     try:
        response = requests.post(f"{API_URL}/analyze_game",

```



```

                json={"user_game_data": context
                    .new_game})

        log_response(response)
        assert response.status_code == 400, f"Ожидался код
            400, получен {response.status_code}"
295 data = response.json()
        assert "detail" in data, "Отсутствует сообщение об оши
            бке"
        logger.info("Получены ожидаемые сообщения об ошибках в
            ализации")
    except requests.exceptions.RequestException as e:
        logger.error(f"Ошибка при проверке валидации: {e}")
300 raise

@then('система должна сгруппировать игры по схожести характери
    стик')
def step_impl(context):
    assert hasattr(context, 'cluster_results'), "Результаты кл
        астеризации не были получены"
305
    for i, result in enumerate(context.cluster_results):
        assert "cluster_analysis" in result, f"Отсутствует ана
            лиз кластеризации для игры {i}"
        assert "similarity_analysis" in result, f"Отсутствует
            анализ схожести для игры {i}"

310 logger.info("Кластеризация игр успешно проверена")

@then('система должна создать PDF-файл с результатами анализа')
def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был пол
        учен"
315 assert context.response.status_code == 200, f"Ожидался код
        200, получен {context.response.status_code}"
    assert context.response.headers['content-type'] == '
        application/pdf', "Неверный тип контента"
    logger.info("PDF-файл успешно создан")

@then('система должна выявить уникальные особенности')
320 def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был пол
        учен"
    assert context.response.status_code == 200, f"Ожидался код
        200, получен {context.response.status_code}"

```

```

data = context.response.json()
325 assert "unique_features" in data, "Отсутствует анализ уникальных особенностей"
assert data["unique_features"], "Анализ уникальных особенностей пуст"

logger.info("Уникальные особенности успешно выявлены")

330 @then('система должна показать понятное сообщение об ошибке')
def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был получен"
    assert context.response.status_code == 404, f"Ожидался код 404, получен {context.response.status_code}"

335 data = context.response.json()
assert "detail" in data, "Отсутствует сообщение об ошибке"
assert "not found" in data["detail"].lower(), "Сообщение об ошибке не содержит ожидаемого текста"

logger.info("Получено понятное сообщение об ошибке")

340 @then('предложить варианты решения проблемы')
def step_impl(context):
    assert hasattr(context, 'response'), "Ответ API не был получен"

    data = context.response.json()
345 assert "detail" in data, "Отсутствует сообщение об ошибке"
assert "suggestions" in data, "Отсутствуют предложения по решению проблемы"

logger.info("Получены варианты решения проблемы")

```